

Systems Programming with C++

Memory, Operating Systems, Networking, Concurrency,
and Low-Level Software

Dr. Charles Dorner

2026

Systems Programming with C++

1. [About this expanded volume](#)
 1. [Technical scope and source boundaries](#)
2. [How to use this book](#)
 1. [Read for a decision, then trace the mechanism](#)
 2. [Build the supplied examples](#)
 3. [Keep the lab bounded](#)
 4. [Interpret output at the correct level](#)
 5. [The central operating rule](#)
3. [Chapter 1 — Understanding How C++ Interacts with Computer Hardware](#)
 1. [1.1 Understanding CPU Execution](#)
 1. [1.1.1 Understanding Instructions and Registers](#)
 2. [1.1.2 Understanding the Fetch-Decode-Execute Cycle](#)
 2. [1.2 Understanding the Operating-System Boundary](#)
 1. [1.2.1 Distinguishing User and Kernel Space](#)
 2. [1.2.2 Understanding System Calls](#)
 3. [Worked example — Separate pure computation from native process queries](#)
 1. [Trace the implementation](#)
 2. [Establish a small, inspectable execution contract](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — A correct answer from the wrong executable](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Build a layered evidence record](#)
 3. [Primary reference notes](#)
4. [Chapter 2 — Representing Data as Bits and Bytes](#)
 1. [2.1 Working with Binary and Hexadecimal Values](#)
 1. [2.1.1 Converting Between Number Bases](#)
 2. [2.1.2 Inspecting Raw Binary Representation](#)
 2. [2.2 Manipulating Individual Bits](#)
 1. [2.2.1 Creating Bit Masks](#)
 2. [2.2.2 Setting, Clearing, and Testing Bits](#)

3. [Worked example — Manipulate checked masks and inspect native bytes safely](#)
 1. [Trace the implementation](#)
 2. [Turn representation decisions into independent tests](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — The correct round trip that concealed an incompatible format](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Design a small status-field codec](#)
 3. [Primary reference notes](#)
5. [Chapter 3 — Understanding Process Memory Layout](#)
 1. [3.1 Understanding Executable Memory Regions](#)
 1. [3.1.1 Understanding Code and Read-Only Data](#)
 2. [3.1.2 Understanding Initialized and Uninitialized Data](#)
 2. [3.2 Understanding Dynamic Runtime Memory](#)
 1. [3.2.1 Understanding Stack Memory](#)
 2. [3.2.2 Understanding Heap Memory](#)
3. [Worked example — Observe Linux mappings without assuming a fixed address layout](#)
 1. [Trace the implementation](#)
 2. [Read the map without turning it into an ABI promise](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — “The address is still there”](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Separate semantic checks from environmental observations](#)
 3. [Primary reference notes](#)
6. [Chapter 4 — Understanding Stack Frames and Heap Allocation](#)
 1. [4.1 Examining Function Stack Frames](#)
 1. [4.1.1 Storing Parameters and Local Variables](#)
 2. [4.1.2 Understanding Return Addresses](#)
 2. [4.2 Understanding Heap Allocation](#)
 1. [4.2.1 Understanding Allocator Metadata](#)
 2. [4.2.2 Understanding Fragmentation](#)

3. [Worked example — Count scoped allocation requests without claiming total heap usage](#)
 1. [Trace the implementation](#)
 2. [Count the resource you actually control](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — A dangling allocator outlives its stack frame](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Compare allocation histories, not slogans](#)
 3. [Primary reference notes](#)
7. [Chapter 5 — Understanding C++ Object Representation](#)
 1. [5.1 Examining Object Size](#)
 1. [5.1.1 Accounting for Member Storage](#)
 2. [5.1.2 Recognizing Compiler-Inserted Padding](#)
 2. [5.2 Understanding Object Layout Rules](#)
 1. [5.2.1 Understanding Standard-Layout Types](#)
 2. [5.2.2 Understanding Trivially Copyable Types](#)
 3. [Worked example — Inspect layout while preserving value-based semantics](#)
 1. [Trace the implementation](#)
 2. [Verify values after a legal representation copy](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — The structure changed but the file format did not](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Separate a wire record from a domain record](#)
 3. [Primary reference notes](#)
8. [Chapter 6 — Controlling Alignment and Padding](#)
 1. [6.1 Understanding Alignment Requirements](#)
 1. [6.1.1 Using alignof](#)
 2. [6.1.2 Using alignas](#)
 2. [6.2 Designing Cache-Conscious Layouts](#)
 1. [6.2.1 Understanding Cache-Line Boundaries](#)
 2. [6.2.2 Avoiding Wasteful Padding](#)
 3. [Worked example — Verify requested alignment without assuming cache geometry](#)
 1. [Trace the implementation](#)
 2. [Verify the declaration before discussing the cache](#)

3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Padding the wrong object](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Evaluate compact, separated, and split layouts](#)
 3. [Primary reference notes](#)
9. [Chapter 7 — Performing Advanced Pointer Operations](#)
 1. [7.1 Navigating Raw Memory](#)
 1. [7.1.1 Performing Pointer Arithmetic](#)
 2. [7.1.2 Interpreting Raw Buffers](#)
 2. [7.2 Using Specialized Pointer Types](#)
 1. [7.2.1 Working with Function Pointers](#)
 2. [7.2.2 Working with Pointers to Members](#)
 3. [Worked example — Decode a bounded buffer and invoke typed callback forms](#)
 1. [Trace the implementation](#)
 2. [Trace the field before invoking any callback](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — A cast replaced four missing checks](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Build a typed, bounded command decoder](#)
 3. [Primary reference notes](#)
10. [Chapter 8 — Understanding Undefined and Implementation-Dependent Behavior](#)
 1. [8.1 Distinguishing Language Behavior Categories](#)
 1. [8.1.1 Understanding Undefined Behavior](#)
 2. [8.1.2 Understanding Unspecified and Implementation-Defined Behavior](#)
 2. [8.2 Detecting Dangerous Behavior](#)
 1. [8.2.1 Using Compiler Warnings](#)
 2. [8.2.2 Using Runtime Sanitizers](#)
 3. [Worked example — Check arithmetic before evaluation and isolate a negative sanitizer exercise](#)
 1. [Trace the implementation](#)
 2. [Prove the guard before evaluating the addition](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)

5. [Failure clinic — The check that contains the bug](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Classify, prevent, and diagnose](#)
 3. [Primary reference notes](#)
11. [Chapter 9 — Managing Operating-System Resources Safely](#)
 1. [9.1 Managing Native Resource Handles](#)
 1. [9.1.1 Managing File Descriptors](#)
 2. [9.1.2 Managing Socket Handles](#)
 2. [9.2 Wrapping Native Resources with RAII](#)
 1. [9.2.1 Creating Ownership Wrappers](#)
 2. [9.2.2 Guaranteeing Cleanup During Failure](#)
 3. [Worked example — Transfer a descriptor and verify exception-path cleanup](#)
 1. [Trace the implementation](#)
 2. [Trace ownership through the exceptional path](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Cleanup code that closes a new connection](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Specify and test a unique native owner](#)
 3. [Primary reference notes](#)
12. [Chapter 10 — Designing Custom Memory Management](#)
 1. [10.1 Understanding Allocator Behavior](#)
 1. [10.1.1 Requesting and Releasing Blocks](#)
 2. [10.1.2 Understanding Allocation Overhead](#)
 2. [10.2 Building Specialized Allocators](#)
 1. [10.2.1 Implementing Memory Arenas](#)
 2. [10.2.2 Implementing Object Pools](#)
 3. [Worked example — Use a bounded arena and reject stale pool handles](#)
 1. [Trace the implementation](#)
 2. [Match storage policy to two different lifetimes](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Reuse without a lifetime protocol](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Make phase and slot boundaries observable](#)

3. [Primary reference notes](#)
13. [Chapter 11 — Performing Low-Level File and Device I/O](#)
 1. [11.1 Working with File Descriptors](#)
 1. [11.1.1 Opening and Closing Files](#)
 2. [11.1.2 Reading and Writing Raw Bytes](#)
 2. [11.2 Mapping Files into Memory](#)
 1. [11.2.1 Understanding Memory-Mapped I/O](#)
 2. [11.2.2 Processing Large Files Efficiently](#)
 3. [Worked example — Read and map the same owned regular-file bytes](#)
 1. [Trace the implementation](#)
 2. [Trace one file through three access methods](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Correct bytes, wrong guarantee](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Preserve progress and classify termination](#)
 3. [Primary reference notes](#)
14. [Chapter 12 — Creating and Managing Processes](#)
 1. [12.1 Understanding Process Creation](#)
 1. [12.1.1 Creating Child Processes](#)
 2. [12.1.2 Replacing Process Images](#)
 2. [12.2 Controlling Process Execution](#)
 1. [12.2.1 Passing Environment Information](#)
 2. [12.2.2 Handling Exit Status](#)
 3. [Worked example — Launch a controlled child image with a minimal environment](#)
 1. [Trace the implementation](#)
 2. [Follow the launch and the result as separate phases](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — The child that survived the error path](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Define a complete launch outcome table](#)
 3. [Primary reference notes](#)

15. [Chapter 13 — Communicating Between Processes](#)
 1. [13.1 Exchanging Data Through Pipes](#)
 1. [13.1.1 Creating Anonymous Pipes](#)
 2. [13.1.2 Building Producer-Consumer Process Pipelines](#)
 2. [13.2 Sharing Data Between Processes](#)
 1. [13.2.1 Using Shared Memory](#)
 2. [13.2.2 Synchronizing Shared Access](#)
 3. [Worked example — Coordinate one shared value and close every unused pipe endpoint](#)
 1. [Trace the implementation](#)
 2. [Draw the copies before running the process branches](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Shared bytes without shared meaning](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Test ownership, progress, and terminal states](#)
 3. [Primary reference notes](#)
16. [Chapter 14 — Understanding Computer Networking](#)
 1. [14.1 Understanding Network Layering](#)
 1. [14.1.1 Understanding IP Addressing](#)
 2. [14.1.2 Understanding Ports and Transport Protocols](#)
 2. [14.2 Understanding Socket Communication](#)
 1. [14.2.1 Creating Network Endpoints](#)
 2. [14.2.2 Resolving Names and Addresses](#)
 3. [Worked example — Resolve a numeric endpoint and inspect a loopback listener](#)
 1. [Trace the implementation](#)
 2. [Verify the endpoint you actually created](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — The correct name, the wrong path](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Build an endpoint evidence sheet](#)
 3. [Primary reference notes](#)
17. [Chapter 15 — Building TCP and UDP Applications](#)
 1. [15.1 Building Reliable TCP Connections](#)
 1. [15.1.1 Implementing TCP Servers](#)

2. [15.1.2 Implementing TCP Clients](#)
2. [15.2 Building Datagram-Based UDP Systems](#)
 1. [15.2.1 Sending UDP Datagrams](#)
 2. [15.2.2 Handling Loss and Ordering at the Application Layer](#)
3. [Worked example — Exchange a binary TCP frame and one bounded UDP datagram](#)
 1. [Trace the implementation](#)
 2. [Preserve byte identity across the TCP exchange](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — A timeout that restarts forever](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Test fragmentation without depending on timing luck](#)
 3. [Primary reference notes](#)
18. [Chapter 16 — Creating Concurrent Programs with Threads](#)
 1. [16.1 Creating and Managing Threads](#)
 1. [16.1.1 Starting Work with std::thread](#)
 2. [16.1.2 Managing Lifetime with std::jthread](#)
 2. [16.2 Dividing Work Across Threads](#)
 1. [16.2.1 Partitioning Independent Work](#)
 2. [16.2.2 Understanding Scheduling Overhead](#)
 3. [Worked example — Partition a bounded sum and join before consuming partial results](#)
 1. [Trace the implementation](#)
 2. [Prove the partition before measuring it](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Automatic joining at the wrong point](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Separate partition proof from scheduling observations](#)
 3. [Primary reference notes](#)
19. [Chapter 17 — Protecting Shared Data with Mutexes and Locks](#)
 1. [17.1 Serializing Shared Access](#)
 1. [17.1.1 Protecting Critical Sections](#)
 2. [17.1.2 Using RAII Lock Types](#)
 2. [17.2 Preventing Deadlocks](#)
 1. [17.2.1 Understanding Circular Wait](#)

2. [17.2.2 Establishing Lock Ordering](#)
3. [Worked example — Preserve a two-object invariant under opposing transfers](#)
 1. [Trace the implementation](#)
 2. [Locate the transfer's commit point](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Thread-safe parts, unsafe composition](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Test rejection without partial mutation](#)
 3. [Primary reference notes](#)
20. [Chapter 18 — Coordinating Threads with Condition Variables](#)
 1. [18.1 Waiting for Shared State Changes](#)
 1. [18.1.1 Waiting with Predicates](#)
 2. [18.1.2 Handling Spurious Wakeups](#)
 2. [18.2 Implementing Producer-Consumer Systems](#)
 1. [18.2.1 Building Thread-Safe Queues](#)
 2. [18.2.2 Applying Backpressure](#)
 3. [Worked example — Drain a bounded queue and wake every waiter during closure](#)
 1. [Trace the implementation](#)
 2. [Trace startup, production, draining, and failure](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Race-free but permanently blocked](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Verify the queue as a state machine](#)
 3. [Primary reference notes](#)
21. [Chapter 19 — Building Task-Based Asynchronous Systems](#)
 1. [19.1 Communicating Results with Futures and Promises](#)
 1. [19.1.1 Producing Deferred Results](#)
 2. [19.1.2 Retrieving Asynchronous Results](#)
 2. [19.2 Launching Asynchronous Tasks](#)
 1. [19.2.1 Using `std::async`](#)
 2. [19.2.2 Understanding Task Execution Policies](#)
 3. [Worked example — Transfer task failures and distinguish deferred execution](#)
 1. [Trace the implementation](#)

2. [Trace three result channels, not three kinds of magic](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — Timed out, still running](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Separate readiness, cancellation, and cleanup](#)
 3. [Primary reference notes](#)
22. [Chapter 20 — Understanding Atomics and the C++ Memory Model](#)
 1. [20.1 Performing Atomic Operations](#)
 1. [20.1.1 Performing Atomic Loads and Stores](#)
 2. [20.1.2 Performing Read-Modify-Write Operations](#)
 2. [20.2 Understanding Memory Ordering](#)
 1. [20.2.1 Understanding Acquire and Release Semantics](#)
 2. [20.2.2 Understanding Sequential and Relaxed Ordering](#)
 3. [Worked example — Distinguish a relaxed counter from release-acquire publication](#)
 1. [Trace the implementation](#)
 2. [Two examples with two different synchronization needs](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)
 5. [Failure clinic — An atomic flag around a data race](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Write the happens-before argument](#)
 3. [Primary reference notes](#)
23. [Chapter 21 — Building a Concurrent Networked System](#)
 1. [21.1 Designing the Server Architecture](#)
 1. [21.1.1 Separating Networking from Application Logic](#)
 2. [21.1.2 Managing Client Connections](#)
 2. [21.2 Scaling Concurrent Work](#)
 1. [21.2.1 Implementing a Worker Pool](#)
 2. [21.2.2 Measuring Throughput and Latency](#)
 3. [Worked example — Build and exercise HarborRelay](#)
 1. [Trace the implementation](#)
 2. [Assemble the compact server without overstating it](#)
 3. [Complete worked source](#)
 4. [Run and verify](#)

5. [Failure clinic — Healthy parser, unhealthy service](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
 2. [Guided lab — Review the complete ownership and failure graph](#)
 3. [Primary reference notes](#)
24. [Appendix A — The Systems Programmer's Diagnostic Desk Reference](#)
 1. [Preserve the first useful evidence](#)
 2. [A resource incident, worked through](#)
 3. [A stalled request, worked through](#)
 4. [A race investigation, worked through](#)
 5. [The recovery question](#)
25. [Appendix B — Complete Systems Laboratories and Shared Support](#)
 1. [Laboratory B.1 — Descriptor identity and open-description state](#)
 2. [Laboratory B.2 — Framing under every bounded three-chunk split](#)
 3. [Laboratory B.3 — Accepted-item conservation and startup cleanup](#)
 4. [Laboratory B.4 — A reusable single-producer/single-consumer handoff](#)
 5. [Shared assertion helper](#)
 6. [Shared blocking queue used by Chapters 18 and 21](#)
 7. [Shared elementary Linux adapter](#)
 8. [Running the laboratories](#)
26. [Appendix C — HarborRelay: A Bounded, Cancellable Loopback Service](#)
 1. [C.1 Requirements and acceptance contract](#)
 2. [C.2 Independent byte framing](#)
 3. [C.3 Bounded admission queue](#)
 4. [C.4 Native ownership and operation state](#)
 5. [C.5 Server ownership and supervision](#)
 6. [C.6 Command-line application](#)
 7. [C.7 Complete regression implementation](#)
 8. [C.8 Acceptance review](#)
27. [Appendix D — Working Glossary and Final Engineering Review](#)
 1. [A final review before expanding the system](#)
28. [Primary reference directory](#)
 1. [ACCEPT — Linux accept\(2\)](#)
 2. [ASAN — Clang AddressSanitizer](#)
 3. [ATOMICS — Atomic operations and ordering](#)
 4. [BIT — Bit manipulation utilities](#)
 5. [CLANG — Clang C++ language support](#)
 6. [CLASS — Classes and special member functions](#)

7. [CONDDVAR — Condition variables](#)
8. [CONNECT — Linux connect\(2\)](#)
9. [FORK — Linux fork\(2\)](#)
10. [FSYNC — Linux fsync\(2\)](#)
11. [FUNC — Functions and declarations](#)
12. [FUTURE — Futures and promises](#)
13. [GCC — GCC C++ language support](#)
14. [GETADDRINFO — getaddrinfo\(3\)](#)
15. [GUIDELINES — C++ Core Guidelines](#)
16. [JTHREAD — C++ working draft — joining threads](#)
17. [LANG — C++ working draft](#)
18. [LAYOUT — C++ working draft — objects](#)
19. [LIFETIME — Object lifetime](#)
20. [MEMORY — Memory management utilities](#)
21. [MEMORYMODEL — Data races and happens-before](#)
22. [MMAP — Linux mmap\(2\)](#)
23. [NEW — C++ working draft — new expressions](#)
24. [NUMERIC — Fundamental types and numeric limits](#)
25. [OPEN — Linux open\(2\)](#)
26. [PIPE — pipe\(2\)](#)
27. [POLL — Linux poll\(2\)](#)
28. [POSIX2024CLOSE — The Open Group — close, Issue 8](#)
29. [POSIXCLOSE — Linux close\(2\)](#)
30. [POSIXIO — Linux read\(2\) and associated I/O interfaces](#)
31. [PROC — Linux /proc filesystem](#)
32. [SEMAPHORE — sem_init\(3\)](#)
33. [SHUTDOWN — Linux shutdown\(2\)](#)
34. [SIGNALSAFE — Linux signal-safety\(7\)](#)
35. [SOCKETRECV — Linux recv\(2\)](#)
36. [SOCKETS — socket\(2\)](#)
37. [SOCKETSEND — Linux send\(2\)](#)
38. [SPAN — Non-owning span](#)
39. [SPAWN — posix_spawn\(3\)](#)
40. [STOPTOKEN — C++ working draft — stop tokens](#)
41. [THREADS — Threads and concurrency](#)
42. [TSAN — Clang ThreadSanitizer](#)
43. [UBSAN — Clang UndefinedBehaviorSanitizer](#)
44. [WAIT — wait\(2\)](#)

45. [WRITE — Linux write\(2\)](#)

About this expanded volume

Systems Programming with C++

Memory, Operating Systems, Networking, Concurrency, and Low-Level Software

Dr. Charles Dorner

Book IV of *C++ From Beginner to Expert* · Expanded manuscript · 2026

A systems program can return the right number while leaking its descriptor. It can protect a variable with an atomic operation while leaving the surrounding object lifetime unsafe. It can accept a network request while retaining more work than it can process. Systems programming begins where a language-level operation meets a resource, timing, representation, or failure boundary that the application cannot ignore.

This volume follows the fictional HarborWorks engineering team as it develops tools and a local request service. The setting connects object representation, resource ownership, native I/O, processes, protocols, concurrency, and shutdown through recurring engineering questions. The examples are original educational scenarios, not descriptions of an employer's infrastructure or production software. No classified systems or confidential implementation details are used.

The twenty-one-chapter curriculum remains in its original order, including all forty-two sections and eighty-four subsections. The expansion adds sustained explanations, mechanism traces, failure clinics, guided laboratories, and solution reasoning. Four supplemental programs and the separate extended HarborRelay application bring the individual ideas together. Every complete source listing is assembled from the delivered source tree so the examples can be built rather than merely read.

Technical scope and source boundaries

The common language baseline is **C++20**. The standard-library chapters use facilities available in that baseline, while native examples are explicitly targeted at **Linux** or the marked POSIX interface. Linux-specific descriptor flags, process inspection, shared-memory setup, and socket behavior are not presented as portable Windows or macOS APIs. A portable class interface does not make its native implementation platform-independent.

The primary reference directory links to language working-draft clauses, compiler documentation, and operating-system manuals. Living documentation can describe later language revisions or newer platform behavior. Consult the installed compiler, library, and manual for the environment you actually use. New prose and laboratory designs are distinguished from the retained curriculum and

compact examples; references support mechanisms, while the fictional requirements and acceptance criteria are teaching choices.

The capstone is a bounded **loopback teaching service**, not a certified production daemon. It deliberately lacks peer authentication, TLS, internet exposure, durable transactions, deployment supervision, and a hard-real-time scheduling analysis. Its explicit request deadlines and queue bounds are useful engineering mechanisms within stated limits, not a guarantee that an ordinary operating system will meet every wall-clock deadline under arbitrary load.

How to use this book

Read for a decision, then trace the mechanism

Before entering a command or copying a function, identify the requirement it serves. Ask who owns each resource, which objects are borrowed, what operation can fail, and what evidence would establish completion. A successful allocation does not establish object initialization. A successful connection does not establish application acceptance. A stop request does not establish that a worker has exited.

Each chapter connects concepts to a complete worked implementation. Read the requirement and walkthrough before the listing, then compare the actual result with the stated acceptance criteria. The failure clinic explains a plausible misleading success or hidden boundary. The guided lab requires a change, a test, and an explanation of what the test does not prove. Review answers are intended to support reasoning rather than reward memorized terminology.

Build the supplied examples

Unpack the companion and open a terminal in its top-level directory. A C++20 compiler, CMake 3.24 or newer, and Python 3.10 or newer are required for the source tests. Native chapters require Linux. The default build fetches no third-party libraries and does not require root privileges.

```
cmake -S source -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j 2
ctest --test-dir build --output-on-failure
```

To work on one chapter, use the supplied runner:

```
python3 tools/run_chapter.py 4 11
```

The runner configures the local source tree, builds the selected executable, and runs its registered fixtures. It does not install a compiler or modify the operating system. A change of compiler should use a new build directory. The complete compiler matrix and publishing commands are documented in the package's README and PUBLISHING files.

Keep the lab bounded

Use an ordinary unprivileged account in a disposable development environment. Network examples bind IPv4 loopback and use selected local ports; they do not contact external services or change firewall policy. File fixtures use owned temporary locations or explicitly marked scratch resources. Process examples

launch their own controlled executable or perform a small single-threaded fork exercise.

The intentionally invalid sanitizer example is isolated under Chapter 8's optional directory and excluded from the ordinary build. Run it only as the documented negative diagnostic exercise. Do not execute a chapter by pasting all its command blocks as one script, and do not practice low-level writes against real devices merely because they also expose descriptors.

Interpret output at the correct level

Addresses, PIDs, selected ports, layout measurements, timings, and lock-free observations can vary. Tests assert the intended relationships rather than one particular machine's incidental values. A printed elapsed duration is not a benchmark conclusion unless the workload and measurement method support that conclusion. A sanitizer-clean execution is evidence about the paths executed, not exhaustive proof of safety.

The ZIP's `TEST_REPORT.md` records the checks actually performed for this edition and the limits of that verification. Source logs, publication checks, and a checksum manifest are included. An untested platform is not described as validated merely because the source contains conditional compilation for it.

The central operating rule

Establish ownership before use, validate before mutation, synchronize before observation, and make cleanup possible before starting concurrent work.

Carry that rule from the first descriptor wrapper to the final server. A system is understandable when a reviewer can trace both its successful request path and its failed startup, truncated input, cancellation, and shutdown paths without guessing who still owns the resources.

Chapter 1 — Understanding How C++ Interacts with Computer Hardware

A C++ expression is not executed directly by the processor as source text. The compiler and runtime turn it into machine operations, while the operating system controls access to protected resources. This chapter separates those layers. We will compute a small result in ordinary C++, query process information through POSIX interfaces, and observe the distinction between a library call and the system boundary it may use.

HarborWorks has a small command-line utility that adds readings and prints a report. A developer describes it as “four additions and one print.” An administrator sees file descriptors, executable mappings, dynamic-library loading, scheduling, and a write to an output destination. Both descriptions can be useful, but neither is complete. Systems programming begins by choosing the layer at which a question can actually be answered.

Keep three questions separate throughout this volume. **What does the language require? What does this platform implement? What did this particular execution demonstrate?** A correct answer to one is not automatically an answer to the others. The first worked program is intentionally modest: its output is simple enough that you can spend attention on how an executable reaches that result and on what evidence supports your explanation.

1.1 Understanding CPU Execution

1.1.1 Understanding Instructions and Registers

A processor executes instructions defined by its instruction-set architecture. Registers hold operands, addresses, and intermediate state close to execution units. A compiler maps C++ expressions to those instructions under the language’s observable-behavior rules; one source operation may become several instructions, be folded into another, or disappear when its result is unnecessary.

Do not infer exact register use from variable names. A local variable may live in a register, a stack slot, another representation, or no independent storage at all after optimization. Assembly inspection and debug information describe one compiled artifact under specific options. They do not replace the portable semantics that make the source program correct across implementations.

Consider `total += reading`. At the language level, the expression reads values, performs arithmetic according to their types, and stores a result. At the machine level, an optimized implementation might combine several such updates, use a vector instruction, or calculate a constant before the program starts. A disassembly

is therefore a translation to inspect, not a second source specification to memorize. Correct source code must permit the transformations the compiler is entitled to make. ([LANG](#))

An instruction-set architecture defines the instructions software can use. A microarchitecture is an implementation of that architecture: the execution units, queues, caches, prediction machinery, and other mechanisms that carry those instructions out. Two processors can run the same executable while taking different amounts of time. Conversely, two executables built from the same C++ can use different instructions because the compiler was given different target options. This distinction explains why “it is x86-64” is not a complete performance description.

Registers are not an extra addressable array that ordinary portable C++ can freely inspect. The compiler decides how to keep intermediate values, and the calling convention assigns roles to some machine state across function calls. A debugger may show a source variable as optimized out when there is no recoverable independent location for it. That does not mean the variable was forgotten; its contribution may already be represented in another value.

Use a concrete arithmetic trace before inspecting instructions. For the worked array, the running sums are 2, 6, 12, and 20. An implementation can produce 20 without preserving those four intermediate states unless the program makes them observable in a way the implementation must respect. Adding logging changes that requirement and changes the experiment. The logging version is not simply the original version with a transparent window attached.

When recording an observation, include the source revision, compiler, standard mode, optimization level, target architecture, and exact executable path. “The compiler uses a register” is weaker than “this function in this artifact uses these instructions under these options.” The latter statement can be reproduced or contradicted by another build. It also leaves room for the source program to remain correct when the implementation changes.

1.1.2 Understanding the Fetch-Decode-Execute Cycle

Fetch, decode, and execute provide a useful conceptual model for instruction processing. Modern processors may pipeline stages, execute multiple operations, predict branches, and reorder internal work while preserving the architecture’s required externally visible behavior. A simple serial cycle diagram is therefore a teaching abstraction rather than a literal timing schedule for every instruction.

The compiler also reorders and transforms work when permitted. Performance reasoning must consider both compiler transformations and hardware execution. Correct synchronization and defined behavior constrain what transformations are allowed; volatile is not a general way to freeze an entire computation or make shared-memory code safe.

The fetch-decode-execute model describes a dependency: instructions must be obtained, interpreted, and carried out. It does not require every instruction to complete all three stages before the next one begins. A pipeline overlaps stages for

different instructions. Multiple execution units can perform independent operations, and speculative work can begin before a branch is resolved. A simple diagram deliberately suppresses those details to make the basic flow understandable.

Data dependencies constrain that freedom. If one operation needs the result of another, the second cannot produce its correct result until the relevant value is available. Independent operations may offer more overlap. In a sum, each update to one accumulator depends on the preceding update; an optimizer can sometimes change the evaluation structure when the language permits the resulting arithmetic behavior. Integer overflow and floating-point reassociation rules matter here. A mathematically equivalent arrangement is not automatically an equivalent C++ program.

Separate instruction latency from throughput. Latency concerns how long a dependent result takes to become available. Throughput concerns how much work can complete over an interval when enough independent work exists. A system may have good throughput while an individual request experiences substantial delay. Later server chapters distinguish the same two ideas at application scale: requests per second and per-request latency answer different questions.

Branches introduce another source of variation. A processor can predict a direction and start useful work, but an incorrect prediction may require discarding speculative results. This does not give the program permission to violate its language-level behavior. It does mean that two inputs following different branch patterns can have different runtime costs even when their source-level loop counts are equal. Do not turn a count of C++ statements into an unsupported cycle estimate.

For a beginner's experiment, inspect rather than tune. Build a small function with debug information at both `-O0` and `-O2`, then compare its instructions. Observe whether a loop remains, whether a constant appears, and whether the call itself survives. Keep the input/output contract unchanged. The purpose is to learn that the compiler and processor both contribute to execution, not to decide that one optimization level is universally better for debugging, latency, or every workload. ([GCC](#), [CLANG](#))

Tracing Arithmetic from C++ Expression to CPU Instructions

Purpose

This makes the section concrete by giving the compiler a tiny arithmetic function whose values naturally move through registers and instructions.

Code or command

```
#include <stdint>
#include <iostream>

std::uint64_t transform(std::uint64_t x) {
    return (x + 5) * 4;
}

int main() {
    std::cout << transform(7) << "\n";
}
```

What the important parts do

`transform` has one input and one returned value, so generated assembly is easy to inspect. The addition and multiplication are the operations the compiler must implement; an optimized build may replace multiplication by 4 with a left shift. `main` provides an observable result so the calculation cannot simply disappear.

Expected behavior or output

Running the program prints `48`. Compiling with `g++ -O2 -S` also produces an assembly file in which the calculation can be related to machine instructions and registers.

Concrete experiment

Compile once with `-O0 -S` and once with `-O2 -S`. Compare the instruction sequences and identify which source-level operations remain visible after optimization.

1.2 Understanding the Operating-System Boundary

1.2.1 Distinguishing User and Kernel Space

Ordinary application code runs with user-mode privileges in a process address space. Kernel code manages protected operations such as scheduling, virtual memory, and device access. This separation is a protection mechanism, not simply a naming convention for folders or libraries. A process cannot legitimately access arbitrary kernel memory merely by forming a pointer value.

System calls provide controlled entry points for requested operations. The kernel validates arguments and permissions and reports results through the platform's interface. A failure can reflect invalid input, resource limits, permissions, or changing external state. Systems code must treat those failures as part of normal operation rather than assume a valid C++ call expression guarantees success.

A process address space gives a program a view of memory. It is not a promise that every numerically representable address refers to memory the program owns. A pointer value can be formed incorrectly, refer to an expired object, or identify a page without the required permission. The operating system's page protections and C++'s object rules are different boundaries. Passing a page-permission check does not establish a valid object lifetime or a legal typed access.

User mode and kernel mode describe privilege levels of execution, not the identity of the human running a command. A program started by the root account still executes ordinary application instructions in user mode and reaches privileged services through controlled interfaces. Elevated credentials can change whether a request is authorized; they do not convert every pointer dereference into a legitimate kernel-memory access.

The boundary also affects error interpretation. A failed open can mean that a path does not exist, access is denied, a descriptor limit was reached, or some other condition described by the native interface occurred. The C++ expression can be well-formed while the operating-system operation fails. This is why the book treats native errors as expected states to model, not merely as exceptional mistakes by the programmer. ([OPEN](#))

Think of a file descriptor as a process-local reference to a resource managed through the kernel. It is not the file's complete contents, a globally meaningful identifier, or a permanent serial number. Descriptor 5 in one process need not refer to the same resource as descriptor 5 in another. Even within one process, the number can be reused after release. The resource-wrapper chapters turn that observation into an ownership discipline.

Use the least privileged environment that can perform the lesson. The supplied examples do not need root, raw device access, or changes to kernel policy. A permission failure is a reason to inspect the requirement and environment before adding privilege. If a container restricts process tracing or socket creation, record that boundary and run the relevant lab in an ordinary development VM. Do not reinterpret a denied facility as a language defect or silently disable a protection mechanism just to make a test appear successful.

1.2.2 Understanding System Calls

POSIX functions such as read and write expose operating-system operations through a C library interface. The library may wrap a system call, provide additional logic, or in some cases use an optimized user-space mechanism. Counting source-level library calls is therefore not always identical to counting kernel transitions.

On Linux, `strace` can observe many system-call interactions for a process you are authorized to trace. Use it as evidence for the compiled program and environment at hand. A trace can change timing and may reveal sensitive paths or data, so keep experiments bounded and avoid treating traced performance as an unperturbed benchmark.

A system-call interface is a protocol between user-space code and the operating system. Arguments describe the requested operation, and the result describes what happened. For a write, the returned count can represent only a prefix of the requested bytes. The remaining suffix is still the caller's responsibility. The result is not a Boolean confirmation that the whole high-level intent was achieved. ([WRITE](#))

Walk a hypothetical five-byte output through three native results: two bytes, one byte, and two bytes. After the first call, the next operation must start at offset two and request three bytes. After the second, it must start at offset three and request two. Restarting at offset zero would duplicate data. Advancing by the requested size rather than the returned size would lose data. This offset accounting is the central invariant of many systems loops.

Error reporting conventions differ across interfaces. Many native calls return `-1` and set `errno`; others return an error number directly. `posix_spawn`, introduced later, uses the latter style. A reusable error helper must fit the called interface. Reading an old `errno` after an operation that did not report failure can produce a persuasive but irrelevant diagnosis. Save the error at the failing boundary before performing other operations that might change it.

A wrapper can also do useful work without crossing into the kernel on every invocation. Standard streams can buffer output; some platform facilities can use shared read-only data or user-space mechanisms. Therefore, a source call graph and a system-call trace are complementary. The trace describes observed native transitions, while the source describes higher-level logic. Neither alone tells you every instruction executed.

Keep retry policy local. Retrying read after an interrupted call that transferred no bytes is not the same decision as retrying `close`, whose Linux descriptor may already have been released. A generic “retry every interrupted syscall” macro hides the very contract you need to understand. Later chapters make the platform-specific `close` policy explicit and distinguish per-operation timeouts from an overall deadline. The first habit to establish is narrower: read the return contract before deciding what another call would mean. ([POSIXIO](#), [POSIXCLOSE](#))

Crossing from User Space into the Kernel with write

Purpose

This directly demonstrates the operating-system boundary by invoking the POSIX `write` system-call interface rather than using only C++ stream abstractions.

Code or command

```
#include <unistd.h>
#include <cstring>

int main() {
    const char* msg = "kernel boundary\n";
    const std::size_t n = std::strlen(msg);
    return ::write(STDOUT_FILENO, msg, n) ==
        static_cast<ssize_t>(n) ? 0 : 1;
}
```

What the important parts do

`STDOUT_FILENO` identifies an OS-managed file descriptor. The process passes a user-space buffer pointer and byte count to `write`; the kernel validates the descriptor and copies or consumes the requested bytes according to the underlying object.

Expected behavior or output

The program writes `kernel boundary` and exits with status 0 when all bytes are accepted by the call.

Concrete experiment

Run the program under `strace -e write` on Linux. Compare the system-call trace with the single `write` call in the source and note the user/kernel transition.

Worked example — Separate pure computation from native process queries

Trace the implementation

The sum uses the standard library and needs no explicit operating-system service. `getpid` and `sysconf` ask platform questions; the program checks that their results satisfy the limited expectations used by this exercise. It does not assume a fixed process ID or page size.

`write_all` handles partial progress and `EINTR` rather than equating one `write` call with one complete output operation. It reports a nonprogressing or failed write

instead of looping forever. The message is small, but implementing the contract correctly at this boundary prepares the same reasoning for files and sockets later.

On a Linux development machine with `strace` installed, trace only this executable and compare the observed calls with the source. Runtime startup will perform additional operations before `main`. Do not assume every line of C++ corresponds to one visible system call, or that an absent call means no platform-specific implementation was involved.

Establish a small, inspectable execution contract

The program has two kinds of assertions. The sum must equal a fixed mathematical result. The process identifier and page-size query need only satisfy the stated validity conditions; neither is required to equal a value copied from somebody else's machine. This distinction prevents environment observations from becoming brittle tests.

Its external output contract is also small: write `PASS` and a newline to standard output. Standard output may be a terminal, a redirected regular file, or a pipe. The descriptor number identifies the inherited output reference, not a guarantee about its destination. Do not test this lesson against a disk device or a privileged file.

```
C++ source -> compiler and linker -> executable artifact
                                   |
                                   runtime
                                   |
                        pure calculation + native queries
                                   |
                        checked write -> output destination
```

The diagram is a map of responsibilities, not a performance timeline. Runtime initialization can occur before `main`; buffered or native output can occur at different points depending on the chosen interface. Identify which boundary an observation belongs to before interpreting it.

Complete worked source

The current files for this example are under `source/book04/ch01/`. Listings are inserted from those files when this book is assembled.

`main.cpp`

```
#include "check.hpp"
#include <array>
#include <cerrno>
#include <numeric>
#include <string_view>
#include <system_error>
#include <unistd.h>

void write_all(int descriptor, std::string_view text) {
    while (!text.empty()) {
        const auto count = ::write(descriptor, text.data(), text.size());
        if (count < 0) {
            if (errno == EINTR) continue;
```

```

        throw std::system_error(errno, std::generic_category(), "write");
    }
    if (count == 0) throw std::runtime_error("write made no progress");
    text.remove_prefix(static_cast<std::size_t>(count));
}
}
int main() {
    const std::array<int, 4> values{2, 4, 6, 8};
    CHECK(std::accumulate(values.begin(), values.end(), 0) == 20);
    const auto process = ::getpid();
    const long page_size = ::sysconf(_SC_PAGESIZE);
    CHECK(process > 0 && page_size > 0);
    write_all(STDOUT_FILENO, "PASS\n");
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 1
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses POSIX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — A correct answer from the wrong executable

Suppose an edit breaks compilation, but the terminal still prints PASS. The likely explanation may be an older executable left in the build directory, not a compiler accepting invalid source. Chain the compile and execution steps with a success condition, or use the supplied runner, which checks subprocess return codes. Record the full artifact path when investigating a stale result.

A second misleading observation is a trace containing many opens, mappings, and reads that do not appear in the tiny source. Dynamic loading and runtime initialization are part of the executed process. Filter a trace only after understanding the unfiltered startup; otherwise you can mistake an intentionally narrow view for the complete execution.

A third failure is output that looks correct on the terminal but has extra bytes when redirected. The terminal is a renderer, not a byte-level oracle. Inspect a small file using a byte-oriented tool or compare it to a known fixture. Escape sequences, carriage returns, and missing final newlines can be visually deceptive.

These investigations share a method: first identify the artifact, then the boundary, then the observable contract. Do not change arithmetic to repair a build-path mistake, and do not rewrite the output loop merely because process startup performs additional native operations.

Practice and implementation lab

Compile an optimized and an unoptimized version, inspect the generated assembly for the sum, and record which artifact and flags you examined. Then trace the process and identify startup calls separately from the explicit output operation.

Redirect output to a regular file and verify the bytes. Do not use a privileged descriptor or a device file for this exercise.

Solution and review notes

The constant sum may be folded or otherwise transformed in an optimized build, while runtime startup remains outside the source-level loop. Those observations are implementation evidence, not changes to the function's required result.

The redirected file should contain exactly PASS followed by a newline. Checking the produced bytes verifies the output contract independently of what the terminal happened to display. Tracing is optional and its availability should be recorded rather than assumed.

Guided lab — Build a layered evidence record

Make a directory containing only this chapter's source and the supplied check header. Compile with explicit C++20 mode, warnings, and debug information. Run the executable, capture its return status, and redirect its output to a disposable file. Compare that file with the exact expected bytes. This establishes a functional baseline before introducing diagnostic tools.

Next compile at a different optimization level into a differently named output. Inspect the sum in the generated assembly. Write a short explanation of what changed and what remained required by the source. An acceptable explanation can say that no separate loop remains; it cannot conclude that “the CPU did not execute the program” because an optimizer removed an unnecessary runtime calculation.

On Linux with tracing available, observe the executable's native operations. Keep the trace private if it includes environment paths. Find the explicit output write, then identify one operation attributable to process startup. Explain why the number of trace lines is not the number of C++ statements or an unperturbed measure of runtime cost.

Finally, deliberately break a source declaration and confirm that your build/run procedure stops rather than executing an older artifact. Restore the declaration, rebuild, and retain the failure as evidence that your workflow detects compilation failure.

Review reasoning. Does root run ordinary code in kernel mode? No; credentials and CPU execution mode are distinct. Does a successful `write` necessarily transfer the requested count? No; inspect the returned count. Does one assembly listing define portable C++ semantics? No; it describes one translation. Why keep the task

small? Because an inspectable contract lets you learn the boundary without confusing it with application complexity.

Completion criterion. Your notebook should connect one source requirement, one compiled artifact, one native boundary, and one independent output check. That chain is more valuable than a screenshot showing that an executable happened to run.

Primary reference notes

Technical references: [GCC](#) [CLANG](#) [POSIXIO](#) [PROC](#)

Chapter 2 — Representing Data as Bits and Bytes

Systems interfaces often describe fields as bits, masks, and byte sequences. C++ provides tools for those representations, but signed arithmetic and object layout rules still apply. We will build checked bit masks, inspect an unsigned value's bytes without violating aliasing rules, and distinguish the value's mathematical bits from the order in which its bytes appear on one machine.

A monitoring message arrives as eight bytes, but the receiving program treats it as two native integers. It works on the developer's machine and fails when a different producer sends the same logical fields. The immediate temptation is to add a cast. The actual problem is an undocumented representation contract: field width, byte order, valid values, and buffer boundaries were never specified.

This chapter gives names to those choices. Use bits for flags, bytes for encoded storage, and typed values for computation. Moving between them should be an explicit operation with a testable rule. A useful low-level program does not merely expose memory; it explains what those bytes mean and what it will reject.

2.1 Working with Binary and Hexadecimal Values

2.1.1 Converting Between Number Bases

Binary and hexadecimal are not different numeric types; they are different ways to write or display values. A hexadecimal digit corresponds to four binary bits, making hex convenient for masks and byte-oriented diagnostics. Integer literals can use binary or hexadecimal notation while still participating in ordinary type and conversion rules.

Choose an unsigned type when manipulating a fixed-width bit pattern. Exact-width types such as `uint32_t` exist when the implementation supplies the corresponding representation. A literal's suffix and the surrounding expression type affect shifts and promotions. Make the intended width explicit rather than assuming an unsuffixed one has the right type for every bit position.

Write the value thirteen in three notations: decimal 13, binary `0b1101`, and hexadecimal `0xd`. The notations differ, but the value does not. Hexadecimal is convenient because each digit accounts for four bits, so the pattern `0xa5` can be expanded as `1010 0101` without performing a new numerical operation. The spacing is for human reading; it is not part of the stored value.

Now distinguish the value from the expression's type. A literal does not acquire every desired property from the notation alone. An unsuffixed hexadecimal literal follows the language's literal-type selection rules, and smaller integer types can

undergo promotion before arithmetic or shifting. For deliberate mask operations, use an explicitly chosen unsigned type and inspect the type of the promoted operation when the width matters. A left operand written as plain 1 can be an int even when the destination variable is much wider. ([LANG](#), [NUMERIC](#))

A C++ byte is the unit measured by `sizeof`; its bit count is described by `CHAR_BIT`. The network protocols in the later labs specify eight-bit octets and use a compile-time check for that assumption. Making the assumption explicit is more precise than saying that every conceivable C++ implementation must have eight-bit bytes. Similarly, exact-width integer typedefs exist only when the implementation supports the required representation.

For a field with eight usable bits, values range from zero through 255. A field of width three has eight possible patterns, but the domain may assign meanings to only five. Storage capacity and application validity are different questions. An input can fit the integer type and still encode a reserved command or an invalid device state.

Practice by expanding `0x3c` to binary and naming which positions are set when the least significant bit is position zero. Then reverse the exercise: combine positions zero, three, and seven into one value. Doing this manually once makes later masks readable. Do not turn the exercise into a dependence on one debugger's preferred display format; the interpretation should survive decimal, hex, or a structured field view.

2.1.2 Inspecting Raw Binary Representation

An object's representation is the sequence of bytes that stores it, while its value is the meaning those bytes represent under the type. `std::bit_cast` can copy a representation between equal-sized trivially copyable types under its requirements. Character and byte access facilities also support inspecting object bytes, but arbitrary reinterpretation as an unrelated typed object can violate aliasing and lifetime rules.

Endianness describes byte ordering for multi-byte values. It does not change which numeric bit is selected by an unsigned shift. A portable protocol should encode byte order explicitly rather than dumping native representation. The example prints native bytes for observation but verifies only size and value-level mask behavior, not one assumed host byte order.

A diagnostic byte dump answers “what representation is present here?” It does not by itself answer “is this a valid packet?” For example, the mathematical value `0x01020304` might appear in memory with its least significant byte first or with its most significant byte first. A protocol can choose one order regardless of the host. The decoder should implement that choice rather than infer it from the machine running the decoder.

For a four-octet big-endian field, the bytes `01 02 03 04` represent $1 * 2^{24} + 2 * 2^{16} + 3 * 2^8 + 4$. A little-endian field containing the same value uses the reverse byte sequence. These formulas define the format independently of native storage. Test a decoder against a known vector, not only against its own encoder:

two components containing the same reversed-order mistake can pass a round-trip test.

Representation inspection has type restrictions. `std::bit_cast` is useful for suitable equal-sized trivially copyable types, but it is not a general permission to manufacture arbitrary class objects from hostile input. Padding, invalid representations, and object-lifetime requirements still matter. An array of character-like bytes can be used to inspect representation without claiming that every byte sequence is a valid value of an unrelated type. ([BIT](#), [LAYOUT](#))

A byte dump can also contain information that is not part of the object's logical value. Padding bytes and implementation-specific layout make raw structure dumps unsuitable as a portable wire format. Pointers are especially inappropriate: their numerical representations do not give another process ownership of the pointed-to objects. Define fields, lengths, and encodings explicitly instead.

In a debugging notebook, label a dump with both its interpretation and its limits: “native representation of this unsigned value on this target,” or “four-octet big-endian length field from the protocol.” The former is an observation; the latter is a contract. Keeping those labels separate prevents an incidental machine layout from quietly becoming the specification that every future client must imitate.

Displaying One Value in Decimal, Hexadecimal, Binary, and Bytes

Purpose

The section covers number bases and raw representation, so the same 16-bit value is printed in several human-readable bases and then inspected byte by byte.

Code or command

```
#include <bitset>
#include <cstdint>
#include <iomanip>
#include <iostream>

int main() {
    std::uint16_t value = 0x2A3C;
    std::cout << std::dec << value << "\n";
    std::cout << "0x" << std::hex << value << "\n";
    std::cout << std::bitset<16>(value) << "\n";
    auto* bytes = reinterpret_cast<unsigned char*>(&value);
    std::cout << std::hex << +bytes[0] << ' ' << +bytes[1] << "\n";
}
```

What the important parts do

`0x2A3C` is only source notation; the object stores bits. `std::hex` changes formatting, `std::bitset<16>` exposes all sixteen bits, and the `unsigned char\*` view is the permitted byte-wise inspection mechanism for an object representation.

Expected behavior or output

The decimal, hexadecimal, and binary lines represent the same numeric value. The final byte order can differ by endianness, which is precisely the machine-level property being observed.

Concrete experiment

Change the value to `0x0102` and predict the two byte values before running. Use the result to identify the machine's byte order.

2.2 Manipulating Individual Bits

2.2.1 Creating Bit Masks

A mask selects one or more bit positions. Shifting an unsigned one left by an index creates a single-bit mask when the index is smaller than the type's width. A negative shift count or a count at least as large as the promoted operand width is invalid. Check a runtime bit index before shifting.

Use named masks for domain meanings such as enabled, dirty, or acknowledged. The numeric position is an implementation detail that should not be repeated throughout the program. When combining fields, document which bits are reserved and whether unknown incoming bits are rejected or preserved for forward compatibility.

A single-bit mask is a value with exactly one selected position. Before computing it, establish the width of the operation and validate the index. For an unsigned 32-bit operation, position 31 is the last valid single-bit position; position 32 is not a request for an empty result. A runtime check must precede the shift. Checking the result afterward is too late if the expression itself had invalid behavior.

Named masks turn numerical positions into domain concepts. Suppose a status field assigns bit zero to enabled, bit two to dirty, and bit five to acknowledged. Define those names in one place. Code that tests `dirty_mask` communicates intent better than code that repeats `0x04` across parsing, logging, and business logic. Central definitions also make reserved positions visible during review.

Fields spanning several bits need a second contract: how a logical value is positioned and extracted. A three-bit mode at offset four occupies positions four through six. To encode a mode, first check that its value is within the accepted domain, then shift and combine it with the surrounding field. To replace an existing mode, clear those positions before inserting the new bits. Otherwise, old set bits can survive and produce a different mode.

Watch the boundary when constructing an all-ones mask. An expression such as `(one << width) - one` is not valid when `width` equals the full width of the promoted left operand. Handle the full-width case explicitly or use the type's maximum value. A helper should state its accepted width interval and test zero, one, the last partial width, and full width according to that contract.

Reserved-bit policy is an application decision. A strict internal protocol might reject every unknown incoming bit to catch incompatible versions early. A forwarding gateway might preserve unknown bits without interpreting them. Neither policy should arise accidentally from a mask that silently discards information. Describe the policy, then test an input containing an otherwise-valid value plus one reserved bit. The test should demonstrate the chosen compatibility behavior, not simply that an integer was produced.

2.2.2 Setting, Clearing, and Testing Bits

Bitwise OR sets selected bits, AND with the complemented mask clears them, and AND followed by comparison with zero tests them. XOR toggles selected bits. Logical AND and OR operate on truth values and short-circuit evaluation; they are not substitutes for bitwise field manipulation.

A bit-field update on an ordinary shared integer is not automatically atomic. Two threads setting different bits can still race or lose updates. Later atomic operations such as `fetch_or` address that concurrency requirement under a defined memory-order contract. This chapter's example is single-threaded and focuses only on representation correctness.

Trace flag updates as state transitions. Starting from zero, setting enabled produces a state containing enabled. Setting dirty adds that flag without removing enabled. Clearing enabled leaves dirty. Toggling dirty twice restores the previous value. These small algebraic properties are useful tests because they remain meaningful for many starting patterns, not just a single demonstration constant.

Distinguish “any selected bit” from “every selected bit.” `(state & mask) != 0` asks whether the intersection is nonempty. `(state & mask) == mask` asks whether every bit in the mask is present. If mask contains enabled and acknowledged, those predicates have different meanings. An empty mask makes the second predicate true, so an API that rejects empty selections should validate that policy separately.

Logical operators answer another question. `a && b` converts operands to truth values and can skip evaluating `b`; `a & b` combines bits and evaluates both operands. Replacing one with the other can alter both the result and the side effects. In a flags API, make the intended operation clear enough that a reviewer need not infer it from a magic-number expression.

Ordinary read-modify-write code is not a synchronization primitive. If two threads read the same old integer, independently set different flags, and store their results, one update can overwrite the other. If the accesses are unsynchronized ordinary accesses, the program has a language-level data race as well as a possible lost-update story. An atomic `fetch_or` can provide an indivisible flag update, but publishing a separate payload requires its own ordering argument. Chapter 20 develops that distinction. ([ATOMICS](#), [MEMORYMODEL](#))

Keep this chapter’s tests single-threaded. First establish the representation algebra; then add concurrency with a defined contract. Combining both before either is understood makes failures harder to classify. A wrong mask remains wrong even when every update is atomic, and a correct mask remains unsafe when its shared object is accessed through a data race. The design must satisfy both layers rather than treating one as compensation for the other.

Setting, Clearing, Toggling, and Testing Status Bits

Purpose

Bit masks are useful when one integer stores several independent boolean states. This example performs each core mask operation on named status bits.

Code or command

```
#include <cstdint>
#include <iostream>

int main() {
    constexpr std::uint8_t Ready = 1u << 0;
    constexpr std::uint8_t Error = 1u << 2;
    std::uint8_t flags = 0;
    flags |= Ready;           // set bit 0
    flags |= Error;           // set bit 2
    flags &= static_cast<std::uint8_t>(~Error); // clear bit 2
    flags ^= Ready;           // toggle bit 0
    std::cout << ((flags & Ready) != 0) << "\n";
}
```

What the important parts do

`1u << n`` creates a mask with one selected bit. OR sets a bit without disturbing others, AND with an inverted mask clears a bit, XOR toggles a bit, and AND tests whether the selected bit is currently present.

Expected behavior or output

``Ready`` is ultimately toggled back off, so the program prints ``0``.

Concrete experiment

Remove the final XOR and predict the output. Then add a third flag at bit 7 and verify that manipulating it does not alter bits 0 or 2.

Worked example — Manipulate checked masks and inspect native bytes safely

Trace the implementation

The mask function validates its index before performing the shift and gives the left operand an explicit unsigned width. The boundary test rejects bit thirty-two, while the popcount check includes the highest valid bit. This avoids relying on a default int width for a high-bit operation.

The native-byte printout is intentionally observational. A little-endian and a big-endian implementation can print different orders while every correctness check

passes. A wire-format encoder would instead choose one order and write each byte according to that contract.

The source uses bitwise operations for flags and ordinary Boolean comparisons for tests. It does not claim that writing flags is synchronized between threads. Representation and concurrency are separate layers, even when both involve the same integer object.

Turn representation decisions into independent tests

The worked program deliberately separates a checked mask operation from native-byte observation. The mask tests have fixed expected results because they concern numeric bit positions. The byte display has target-dependent order because it describes native representation. A test should not accidentally promote the latter observation into a requirement for all hosts.

Before running, predict three results: the mask at position zero, the mask at the highest accepted position, and the response to the first invalid position. Then inspect where validation occurs. A guard after the shift would not establish safety. The rejected case belongs at the interface boundary, before an invalid expression is evaluated.

For a protocol extension, draw the field before coding:

bits 7..6 reserved	bit 5 acknowledged	bits 4..3 mode	bit 2 dirty	bits 1..0 controls
-----------------------	-----------------------	-------------------	----------------	-----------------------

This is an illustrative layout, not a hardware register specification. Assign meanings and permitted combinations before turning it into constants. The purpose of the drawing is to expose overlap and reserved regions, not to imply that every bit combination is valid.

Complete worked source

The current files for this example are under `source/book04/ch02/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <array>
#include <bit>
#include <cstdint>
#include <cstdint>
#include <iomanip>
#include <stdexcept>

std::uint32_t mask(unsigned bit) {
    if (bit >= 32) throw std::out_of_range("bit index");
    return std::uint32_t{1} << bit;
}

int main() {
    std::uint32_t flags = 0;
    flags |= mask(2) | mask(7);
    CHECK((flags & mask(2)) != 0 && (flags & mask(1)) == 0);
}
```

```

    flags &= ~mask(2);
    CHECK(flags == mask(7));
    flags ^= mask(7);
    CHECK(flags == 0);
    CHECK(std::popcount(mask(0) | mask(31)) == 2);
    const std::uint32_t value = 0x01020304U;
    const auto bytes = std::bit_cast<std::array<std::byte,
sizeof(value)>>(value);
    std::cout << "native bytes:";
    for (auto byte : bytes)
        std::cout << ' ' << std::hex << std::setw(2) << std::setfill('0') <<
std::to_integer<unsigned>(byte);
    std::cout << std::dec << '\n';
    bool rejected = false;
    try { (void)mask(32); } catch (const std::out_of_range&) { rejected =
true; }
    CHECK(rejected);
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 2
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — The correct round trip that concealed an incompatible format

An encoder writes the low byte first, and its paired decoder reads the low byte first. Their round trip passes. A specification, however, requires the high byte first. The test has demonstrated agreement between the two functions, not agreement with the external contract. Add a known encoded vector whose bytes were derived independently from the format definition.

Another defect appears only at the top bit: a helper shifts a plain signed one, then converts the result to an unsigned destination. The destination type does not retroactively change how the shift was evaluated. Repair the operation’s type and precondition, not just the receiving variable. Compile warnings and sanitizer checks can support the review, but the expression’s rules are the reason the repair is correct.

A third defect confuses clearing with toggling. XOR removes a flag only when it was already set; it adds the flag when it was absent. Use a case starting with the flag absent to distinguish “clear” from “toggle.” Minimal counterexamples are especially effective for bit manipulation because many wrong operations agree on convenient demonstration inputs.

Retain tests for the meanings of the operations, not only their current hexadecimal output. The same behavior should remain recognizable when an enum, wrapper type, or different display format replaces the original integer notation.

Practice and implementation lab

Implement extraction and replacement of a bounded contiguous bit field. Validate the starting bit, field width, and incoming value before constructing masks or shifting.

Write explicit big-endian and little-endian encoders and verify that each decoder reconstructs the original value independently of native byte order.

Solution and review notes

A full-width field requires special treatment because shifting one by the type width is invalid. Define width zero explicitly, reject ranges extending beyond the word, and ensure the incoming value fits the field before shifting it into place.

Protocol tests should compare exact encoded bytes, unlike the native-representation observation. Round-trip tests alone can miss a pair of encoders and decoders that share the same wrong byte-order assumption, so include known external byte vectors.

Guided lab — Design a small status-field codec

Define a one-octet teaching format with named flags, one small enumerated field, and at least one reserved bit. Write down whether unknown bits are rejected. Implement an encoder from domain values and a decoder returning either a valid structured result or an explicit error. Do not expose a partially decoded object as success.

Enumerate all 256 octet values on the eight-bit lab baseline. For each value, classify validity using a simple independent specification function. Compare the decoder's decision and fields with that classification. For every valid domain value, verify encode/decode agreement, but keep the independently specified byte cases as well. A round trip and an external-vector test protect different contracts.

Add property checks for set, clear, and toggle. Setting a flag twice should be idempotent; clearing twice should also be idempotent; toggling twice should restore the original state. Test “any” and “all” with masks containing several bits. Include a zero mask and document whether the public API accepts it.

Extend the format with a four-octet length in a specified byte order. Test zero, one, 255, 256, and a value with four different octets. Reject truncated input before indexing. Do not allocate a payload merely because the decoded integer fits; a separate maximum-size policy belongs to the framing layer.

Review reasoning. Why can two wrong functions pass a round trip? They can share the same incorrect convention. Why validate before shifting? The invalid operation cannot be repaired by inspecting its result. Why are a mask and a domain type different? A mask describes representation; the domain type describes permitted meanings. Why is atomicity not covered by a single-thread test? It concerns interactions and ordering absent from this experiment.

Completion criterion. Produce the field specification, known vectors, validity enumeration, and exact failure policy. Those artifacts make the representation understandable to a second implementation rather than merely reversible by your own code.

Primary reference notes

Technical references: [BIT](#) [LANG](#) [NUMERIC](#)

Chapter 3 — Understanding Process Memory Layout

The language describes storage duration and object lifetime; the operating system maps virtual address regions with permissions and backing relationships. These views overlap but are not identical. This Linux exercise inspects its own process mappings and prints addresses of representative objects without assuming a fixed order, fixed address, or universal correspondence between a C++ variable and a named executable section.

A process reports a large virtual-memory size, yet its resident footprint is much smaller. Another process releases a vector, but the operating system's resident-memory display does not immediately fall. Neither observation alone proves a leak. The language, allocator, virtual-memory manager, and physical-memory accounting describe different things.

HarborWorks needs to investigate memory without turning familiar labels into assumptions. This chapter builds a map of a running process while preserving an important boundary: the C++ standard defines objects and lifetimes, not a universal diagram with the heap always below the stack. Use the diagram as a way to ask questions, then obtain evidence from the actual platform.

3.1 Understanding Executable Memory Regions

3.1.1 Understanding Code and Read-Only Data

Native executable formats commonly separate machine code and read-only data into sections that the loader maps with suitable permissions. Shared libraries add their own mappings, and address-space randomization can change locations between executions. The exact layout depends on the platform, linker options, executable format, and loader behavior.

C++ `const` qualification is a language access rule, not a promise that every `const` object occupies a read-only page. A `const` local object can reside in ordinary stack storage, and optimization can eliminate independent storage. Conversely, attempting to modify read-only mapped memory is a platform protection fault rather than a valid way to test `const` correctness.

Executable code and read-only data often occupy differently protected mappings in a native process. The executable file contains information the loader uses to establish those mappings, along with metadata for linking and runtime initialization. The resulting process also contains mappings for shared libraries, anonymous memory, stacks, and platform facilities. One source file therefore does not correspond to one contiguous region in the running address space.

The word “text” has a historical meaning in executable inspection: it commonly refers to machine code, not prose. A string literal can be placed in read-only data, but the exact section names and placement are toolchain details. A `const` declaration expresses a language-level restriction through that interface; it does not, by itself, promise a specific page permission or file section. The example calls its constant a read-only candidate for that reason.

Read a Linux mapping as a range plus properties. The range describes virtual addresses. The permissions indicate allowed page-level operations. A pathname, when present, associates the mapping with a backing object; an anonymous mapping may have no ordinary pathname. These observations help classify regions, but they do not inventory every live C++ object inside them. A writable mapping can hold many objects with independent lifetimes. ([PROC](#))

Distinguish executable sections from loadable segments. Sections organize information for linking and other tools; segments describe groups the loader maps for execution. A debugger, object-file inspector, and `/proc` reader can therefore display different groupings without disagreeing. Start by asking which representation each tool reports. Do not force a section-oriented explanation onto every runtime mapping.

For the worked program, printing an object’s address is an observation, not an invitation to dereference neighboring bytes. Locate the mapping that contains the address only after validating the range comparison in an external inspection tool or careful parser. The fact that a page is readable does not authorize arbitrary typed reads across unrelated objects. Page protection is coarser than C++ ownership and lifetime, so an apparently accessible address can still be invalid for the operation being attempted.

3.1.2 Understanding Initialized and Uninitialized Data

Static-storage objects have initialization rules, including zero initialization before later applicable initialization stages. Executable formats often represent initialized writable data separately from zero-filled storage commonly associated with BSS. The loader can establish zero-filled memory without storing every zero byte in the executable file.

The phrase uninitialized data section can be misleading at the C++ level: a static `int` without an explicit initializer is zero-initialized, not an indeterminate local integer. Keep file-format terminology separate from language initialization semantics. Reading an ordinary uninitialized automatic `int` is not justified by the fact that some executable section is called BSS.

An initialized namespace-scope integer and a zero-initialized namespace-scope integer have different source declarations, but both have defined initial values before the relevant program use. The executable does not necessarily need to store a long sequence of zero bytes for every zero-initialized object. A loader can establish zero-filled storage as part of creating the process. This is why the size of an executable file and the amount of memory its initial image requires are different measurements.

The common `.data` and `.bss` labels are useful implementation vocabulary. They are not substitutes for the language's initialization rules. Constant initialization, zero initialization, and dynamic initialization describe how objects obtain values; section placement describes a particular binary organization. An object requiring runtime construction can involve both reserved storage and initialization code. A section name alone does not prove that an arbitrary global object is ready for use at a particular cross-translation-unit initialization point.

The worked program checks the zero-initialized integer's value, not its exact address. That is a portable semantic fact about the declaration. It prints addresses to connect those objects to the process map on the Linux lab machine. Treat those two activities differently in a test: one assertion protects a language requirement, while the other supplies observational evidence that can legitimately vary between executions.

Address-space layout randomization and position-independent builds can change where mappings appear. A test that expects a global at a copied numerical address is usually testing an accident of one launch rather than the program's contract. Even without randomization, a compiler, linker, library, or source change can alter layout. Use relationships supported by the language or inspect current ranges; do not rely on a previously observed address as a persistent identifier.

When investigating startup memory, separate static footprint from later demand. A large zero-filled region can reserve virtual space without every page being resident immediately. Touching pages can change residency. Conversely, a small executable can allocate a large amount at runtime. Record the measurement name, unit, time, and workload phase before comparing two numbers. "Memory increased" is not an actionable observation until you know which memory measure increased and what happened between samples.

Comparing Addresses of Code, Constants, Initialized Data, and BSS

Purpose

Executable images typically separate code, read-only constants, initialized writable data, and zero-initialized storage. Printing addresses gives an observable way to relate language objects to those regions.

Code or command

```
#include <iostream>

const char message[] = "read-only";
int initialized = 7;
int zero_initialized;

void function_code() {}

int main() {
    std::cout << reinterpret_cast<const void*>(&function_code) <<
    "\n";
    std::cout << static_cast<const void*>(message) << "\n";
    std::cout << &initialized << "\n";
    std::cout << &zero_initialized << "\n";
}
```

What the important parts do

`function_code` names executable code, `message` is immutable static storage, `initialized` requires an initial value in the image, and `zero_initialized` is eligible for zero-filled storage such as BSS. Exact addresses are loader-dependent.

Expected behavior or output

The program prints four addresses; their absolute values vary because of ASLR. Tools such as `readelf -S` or `nm` provide stronger evidence about the actual sections.

Concrete experiment

Run `readelf -S` and `nm -S` on the executable and locate the four symbols. Compare symbol sections with the address-only observation.

3.2 Understanding Dynamic Runtime Memory

3.2.1 Understanding Stack Memory

Many implementations use thread stacks for call frames and some automatic objects, but C++ specifies automatic storage duration rather than one universal

physical stack layout. Registers, inlining, tail-call transformations, and optimization can change where or whether an object is materialized. Each thread also has its own execution stack arrangement under the platform runtime.

A printed local address is evidence about that object during that execution. Do not subtract unrelated object pointers to infer a portable stack size or ordering. A pointer remains usable only while its object lifetime and access rules permit it, regardless of whether the numeric address later appears in a familiar mapping.

Automatic storage duration means an object's lifetime is tied to execution of its block or function according to language rules. A stack is a common implementation mechanism for many such objects, but the compiler may keep values in registers or eliminate independent storage. This distinction prevents the beginner's shortcut "every local variable is on the stack" from becoming a false rule.

A function call can require storage for saved state, spills, local objects, alignment, and other ABI-defined needs. Recursive calls can create additional active invocations, but the amount of stack consumed per invocation is not simply the sum of the source variables' sizes. Optimization can inline calls, reuse slots, or transform eligible recursion. Debug builds and instrumented builds can have substantially different stack behavior from release builds.

The lifetime boundary still applies when an address appears unchanged. Returning a pointer or reference to an ordinary local object does not extend that object's life. The bytes may remain visible for a while, but the program no longer has a valid object to access through that stale reference. A repeated successful read is not evidence that the reference is safe. The object-lifetime rule is the relevant contract. ([LIFETIME](#))

Threads commonly have separate stacks, while sharing much of the process address space. That does not mean a pointer into another thread's automatic objects is inherently safe or inherently forbidden. Safety depends on lifetime, synchronization, and access rules. Passing a pointer to a local buffer to a worker that outlives the creating scope is a lifetime defect even when the address remains mapped. Joining before destruction can establish the required lifetime relationship for a bounded design.

Do not test stack limits by unbounded recursion in a shared environment. Use a small call chain and inspect its frames with a debugger, then reason about growth separately. The purpose is to understand active invocations, not to crash the host. For algorithms with potentially deep input-dependent recursion, use explicit bounds or an iterative representation when appropriate. A production failure caused by stack exhaustion is not repaired merely by making the stack larger without examining the input and recursion contract.

3.2.2 Understanding Heap Memory

Dynamic allocation obtains storage through allocation functions and allocator implementations. On Linux, an allocator can use more than one underlying mechanism, including mappings that do not appear inside the region labeled heap.

A large allocation may have a different backing arrangement from a small one, and freed storage can remain reserved by the allocator.

Virtual size, resident memory, requested allocation bytes, and live object bytes are different measurements. A vector's capacity is not the same as the process's resident set. Inspect the appropriate layer before diagnosing a leak or memory shortage. The chapter's mapping view shows virtual regions, not a complete account of which application objects are alive.

Dynamic storage gives an object a lifetime that is not automatically the lifetime of the pointer variable naming it. A local `unique_ptr` can own a dynamically allocated object; moving the owner can change which scope eventually destroys the object without moving the object itself. Distinguish the owner, the allocation, the constructed object, and any non-owning references. They are related but not interchangeable.

An allocator may obtain larger blocks from the operating system and satisfy many small allocations from those blocks. Releasing one object can return its storage to the allocator without immediately returning pages to the operating system. Therefore, a stable or elevated resident-memory measurement after deallocation does not by itself prove that the object remains live. Conversely, a stable resident footprint does not prove the absence of logical leaks when freed pages or unused capacity hide growth.

The worked program's `make_unique<int>` creates one small owned integer. Its address is useful for observing a dynamic allocation on this run. Do not assume every dynamic allocation appears in the mapping labeled `[heap]`; native allocators can use additional anonymous mappings. The label is a platform observation, not a complete inventory of C++ dynamic storage. ([MEMORY](#), [PROC](#))

A more useful memory investigation tracks several quantities: live application objects, bytes requested through a particular allocator, container capacity, virtual mappings, and resident pages. Chapter 4 introduces a scoped allocation counter that measures one of those quantities honestly. It does not claim to measure allocator metadata, every library allocation, or physical memory. Naming the quantity prevents a narrow instrument from producing an overly broad conclusion.

Ownership remains the first defense against leaks. A container or RAII wrapper makes the release point explicit and handles ordinary exceptional exits. But RAII alone does not bound how much live data a program retains. A queue whose owner lives forever can grow forever while every allocation is correctly owned. Later chapters add capacity limits and backpressure because lifetime correctness and resource-budget correctness are separate requirements. A system needs both to remain operational under sustained input.

Contrasting Stack Lifetime with Dynamically Allocated Heap Storage

Purpose

This isolates the two common runtime storage cases discussed in the section: an automatic local object and dynamically allocated storage with explicit ownership.

Code or command

```
#include <iostream>
#include <memory>

int main() {
    int stack_value = 10;
    auto heap_value = std::make_unique<int>(20);
    std::cout << "stack=" << &stack_value << "\n";
    std::cout << "heap=" << heap_value.get() << "\n";
}
```

What the important parts do

`stack_value` has automatic storage duration and is destroyed when `main` exits. `make_unique<int>` requests dynamic storage and returns an owning RAII object; `get()` exposes the managed address without transferring ownership.

Expected behavior or output

Two implementation-dependent addresses are printed. Their values are not a portable proof of physical regions, but their lifetimes and ownership rules differ as described.

Concrete experiment

Wrap the heap allocation in an inner scope and add destructor-observable resource code. Confirm that the dynamic allocation is released at scope exit even without calling `delete`.

Worked example — Observe Linux mappings without assuming a fixed address layout

Trace the implementation

The source observes only its own process through `/proc/self/maps` and requires no elevated privileges in an ordinary Linux environment. A restricted environment may deny that interface; such a failure is a platform limitation to report, not proof that the C++ objects are incorrect.

The global without an explicit initializer is checked for zero, illustrating static initialization semantics. The program prints object addresses as const void pointers but performs no ordering or subtraction between unrelated objects. It therefore avoids turning a diagnostic observation into undefined pointer arithmetic.

Only the first three mapping lines are printed to keep output bounded, while the full file is counted and checked for a normal end. Those lines may describe executable or library mappings depending on the build and loader. The test does not assert a particular section address, mapping count, or heap location.

Read the map without turning it into an ABI promise

The program creates five observations: initialized static data, zero-initialized static data, a constant object, an automatic object, and a dynamically owned object. It then reads its own Linux mapping information. No assertion requires one category to appear numerically before another. That omission is deliberate: the lesson is to identify the environment, not to impose a textbook picture on it.

The stream loop also checks its termination state. End-of-file after successful reads is different from a bad input stream. A count of mappings is useful only if the file was read successfully. This is a small example of the same evidence discipline used for network streams later: distinguish complete input, truncated input, and I/O failure.

<pre>source declarations -> object lifetimes and values loader + allocator -> virtual mappings and storage current execution -> observed addresses and residency</pre>
--

These views overlap, but none can replace the others. A mapping can outlive the object formerly stored in it; a valid object can occupy a small part of a much larger allocation or mapping.

Complete worked source

The current files for this example are under `source/book04/ch03/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <fstream>
#include <memory>
#include <string>

int initialized_global = 17;
int zero_initialized_global;
const int read_only_candidate = 23;

int main() {
    int automatic = 5;
    auto dynamic = std::make_unique<int>(9);
    CHECK(zero_initialized_global == 0);
    std::cout << "global=" << static_cast<const void*>(&initialized_global)
              << " zero_global=" << static_cast<const
```

```

void*>(&zero_initialized_global)
    << " const_object=" << static_cast<const
void*>(&read_only_candidate)
    << " automatic=" << static_cast<const void*>(&automatic)
    << " dynamic=" << static_cast<const void*>(dynamic.get()) << '\n';
std::ifstream maps{"/proc/self/maps"};
CHECK(maps.is_open());
std::string line;
std::size_t lines = 0;
while (std::getline(maps, line)) {
    if (lines < 3) std::cout << line << '\n';
    ++lines;
}
CHECK(maps.eof() && !maps.bad() && lines > 0);
std::cout << "mapping_count=" << lines << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 3
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — “The address is still there”

A developer returns a pointer to a local buffer and tests it immediately. The expected text appears, so the function is declared correct. The test has observed residual bytes, not a continuing object lifetime. The repair is to return an owned value or arrange an explicit caller-owned buffer whose lifetime covers the use. Repeated execution cannot turn an expired object into a valid one.

Another report says that `delete` failed because resident memory did not drop. First verify whether the owner actually released its object. Then distinguish allocator retention from a live-object leak. A targeted allocation instrument and a leak detector answer different questions from the operating system’s process summary. Use more than one measure, but do not combine them as though they shared a definition.

A third failure comes from assuming that `/proc/self/maps` exists in every environment. It is a Linux interface and may be unavailable or restricted. The example should report that failure; it should not fabricate a generic map or silently claim that no mappings exist. The platform boundary is part of the lesson.

For every memory claim, record the object or region, the relevant lifetime, the tool, and the time of observation. This turns an ambiguous screenshot into a statement another engineer can investigate.

Practice and implementation lab

Use the platform's executable-inspection tool to examine sections in this exact binary and compare them with the runtime mappings. Record the compiler and linker flags, especially position-independent executable settings.

Allocate a bounded large buffer and touch its pages, then compare virtual and resident measurements. Do not infer a leak solely from memory not immediately returning to the operating system after deallocation.

Solution and review notes

Executable sections describe the file, while mappings describe loaded virtual regions and permissions. Several sections may share a mapping, and runtime allocations add regions not represented as static file sections.

Touching pages can change residency without changing the reserved virtual range. Allocators may retain freed blocks for reuse. To investigate a leak, combine live-allocation ownership evidence with process measurements over a repeatable workload rather than equating one retained mapping with an unreachable object.

Guided lab — Separate semantic checks from environmental observations

Run the mapping example twice and save both outputs privately. Mark each field as either expected to be stable by the program's contract or permitted to vary. The zero-initialized value has a fixed expectation. Numerical addresses and mapping counts are observations. Explain why a changed address is not a failed test.

Add a dynamically allocated vector with a documented bounded size. Record its data pointer, size, and capacity before and after a deliberate growth operation. Do not dereference the old pointer after a reallocation. The exercise should demonstrate invalidation by comparing the documented operation and current ownership, not by executing an invalid access and hoping for a crash.

Next add a function returning an owned `std::string`. Compare its lifetime story with a hypothetical function returning a pointer to a local character array. Keep the invalid version as written analysis only; it need not be executed. Identify the owner that survives the return and the point where the owned result is eventually destroyed.

On a Linux development VM, compare a process summary before and after touching a bounded allocated buffer. Explain why virtual size, resident size, and requested allocation size can differ. Do not promise that a particular memory tool will show an exact delta; runtime and allocator behavior contribute to the observation.

Review reasoning. Does a writable mapping contain a valid object of every type? No. Must dynamic storage live in one mapping labeled heap? No. Does `const` specify an exact executable section? No. Does unchanged residency prove a leak?

No. What does an expired local reference violate? The object's lifetime contract, even if its old page remains accessible.

Completion criterion. Produce a two-column report: semantic obligations on one side, observed implementation details on the other. The ability to keep those categories separate is the foundation for reliable native-memory diagnosis.

Primary reference notes

Technical references: [PROC](#) [LIFETIME](#) [GCC](#)

Chapter 4 — Understanding Stack Frames and Heap Allocation

Function calls and dynamic allocation have costs and limits that are not visible in a class diagram. A stack frame is an implementation mechanism for an active call; an allocator manages reusable storage and its own metadata. We will observe a bounded recursive call chain and count requests through a polymorphic memory resource, carefully distinguishing requested bytes from allocator overhead and operating-system memory usage.

HarborWorks replaces repeated allocations with a reserved vector and sees fewer allocation requests. A report then claims that the application's memory use has been cut in half. The measurement does not support that conclusion: it counted requests through one resource, not all live objects, allocator overhead, or resident pages.

This chapter connects call-stack reasoning with scoped heap accounting. Both require precise boundaries. A stack frame is an implementation object associated with execution; an allocator counter is an instrument covering selected requests. Neither should be mistaken for a complete description of every byte used by the process.

4.1 Examining Function Stack Frames

4.1.1 Storing Parameters and Local Variables

A calling convention specifies how a compiled function receives arguments, returns results, and preserves required machine state. Some arguments may be passed in registers and others in memory. A function can reserve stack space for locals or spills, but optimization may eliminate, combine, or relocate those objects.

Debug builds often make source-level variables easier to inspect, while optimized builds better represent production performance. Neither one defines the only legal layout. Use a debugger to inspect a particular frame and note when a variable is optimized out rather than inventing a location. A source function need not correspond to one distinct frame after inlining.

A function invocation needs a way to receive arguments, preserve required state, and resume its caller. A calling convention specifies parts of that arrangement for a particular ABI. Arguments may travel in registers, memory, or a combination. A compiler can also inline a function so that no separate machine-level call remains. The source signature describes the language interface, not an instruction-by-instruction transfer plan.

Within an active invocation, automatic objects have lifetimes governed by their declarations and control flow. A compiler may place some in a stack frame, reuse storage for objects whose lifetimes do not overlap, or keep values elsewhere. Taking an address can affect what storage must be materialized, but even then the implementation must be interpreted through the language's observable behavior rather than a universal source-to-stack formula.

Consider a function with a local buffer and a helper that borrows it. The borrow is safe only while the object exists and the access is otherwise valid. Calling the helper synchronously is different from enqueueing a callback that retains the pointer after the function returns. The latter changes the lifetime requirement even if the helper's parameter type stays identical. A raw pointer does not communicate that difference by itself.

Debuggers show frames using available unwind information and debug metadata. Optimized code can make source variables unavailable, combine instructions from several source lines, or eliminate a frame through inlining. These effects complicate inspection without changing the source contract. For a first investigation, use a debug build and a small call chain; then compare an optimized build while keeping the explanation modest.

In the worked function, each invocation receives a bounded nonnegative depth and contributes that value to a recursive sum. Draw the active calls for depth four before examining the debugger: four waits for three, three for two, and so on until zero returns. This trace describes logical invocations. Whether the compiler retains exactly those machine frames is a separate observation. A public function accepting arbitrary depth would require validation and a resource bound; the private teaching call does not establish a general recursion API.

4.1.2 Understanding Return Addresses

The execution mechanism must preserve enough information to continue after a call, often including a return address and saved state according to the architecture. That state is not a portable C++ object that application code should modify. Corrupting it through an out-of-bounds write is undefined behavior and a serious security defect.

A backtrace reconstructs call history using available unwind metadata, frame conventions, and debug information. Optimizations, omitted frames, corrupted memory, or missing symbols can limit the result. Treat a trace as evidence with those limitations, and preserve the matching binary and symbols when diagnosing a failure.

A return address is part of the machine's mechanism for resuming execution after a call. Its storage and manipulation are ABI- and architecture-specific. Ordinary C++ does not provide a portable entitlement to walk arbitrary stack memory until something resembles a code address. Use a debugger's stack trace or supported unwind facilities rather than inventing pointer arithmetic across implementation state.

Destruction during normal return is governed by C++ object lifetime. Exception propagation adds another path: successfully constructed automatic objects in exited scopes are destroyed while the exception moves toward a handler. This is the mechanism RAII relies on. A native resource wrapper should therefore release its owned handle during unwinding without throwing a second exception from its destructor. The presence of a return address does not itself perform that cleanup; language and runtime machinery cooperate.

Tail-call optimization illustrates the gap between logical calls and retained frames. An implementation may replace an eligible final call with a transfer that reuses the caller's execution storage. The source can still be understood as a call, while the debugger may not display every apparent invocation. Do not base a correctness argument on a particular stack-trace shape. In particular, absence of a frame is not evidence that the source function was never involved.

Security-sensitive diagnosis also needs boundaries. A corrupt return path can be a symptom of memory damage, but this book's lab does not overwrite stack state or demonstrate control-flow exploitation. The useful engineering exercise is to prevent the preceding invalid write, retain diagnostic builds, and identify the first violation. A crash at return often occurs after the original defect, so the last visible frame is not automatically the root cause.

For a reproducible report, retain the exact executable and matching debug information. A stack trace interpreted with symbols from a different build can assign convincing but wrong source locations. Source revision, optimization flags, architecture, and binary identity belong together. Treat the trace as evidence from that artifact, then connect it back to the ownership, bounds, and lifetime contracts that should have prevented the invalid state.

Observing Distinct Stack Frames During Nested Function Calls

Purpose

Nested calls give each active function its own automatic objects and call context, which is the practical behavior the stack-frame model explains.

Code or command

```
#include <iostream>

void inner(int value) {
    int local = value + 1;
    std::cout << "inner local @ " << &local << "\n";
}

void outer() {
    int local = 10;
    std::cout << "outer local @ " << &local << "\n";
    inner(local);
}

int main() { outer(); }
```

What the important parts do

``outer`` creates one local object, then calls ``inner``, which creates another. Their lifetimes overlap while ``inner`` executes. A debugger can also show the saved return path and call chain that lets control resume in ``outer``.

Expected behavior or output

The program prints two different addresses. Under a debugger, ``backtrace`` inside ``inner`` should show ``inner``, ``outer``, and ``main`` as active frames.

Concrete experiment

Set a breakpoint in ``inner`` and run ``bt`` plus ``info frame`` in GDB. Compare the debugger's frame information with the two local addresses.

4.2 Understanding Heap Allocation

4.2.1 Understanding Allocator Metadata

An allocator tracks free and in-use regions using implementation-specific metadata. A request for N bytes may cause additional bookkeeping, alignment padding, or a larger reserved block. Different allocation sizes may use different internal paths. The C++ allocation interface does not promise that requested bytes equal total process memory cost.

A `memory_resource` wrapper can count requests made through that resource, but it does not automatically observe every allocation in the process. Standard strings, libraries, and runtime facilities using other allocators remain outside its scope. Label the measurement precisely so a local counter is not mistaken for a whole-process heap profiler.

A request for a block of storage is not necessarily a request for one equally sized physical-memory allocation. A general-purpose allocator can divide arenas, round sizes, retain freed blocks, and maintain metadata. Its upstream interaction with the operating system can occur less frequently than application-level allocation. That layering explains why counting new expressions, allocator calls, and system calls produces different totals.

The chapter's `CountingResource` intercepts requests through one polymorphic memory resource. It forwards allocation to an upstream resource, then records the requested byte count and alignment-related operation. The counter describes successful requests it sees. It does not intercept every allocation in the process, and it does not know the upstream allocator's internal bookkeeping or page residency. Those limitations are not defects; they define the instrument.

The order of accounting matters. If upstream allocation throws, the resource must not record a successful live block that was never returned. If deallocation succeeds, its accounting should remove the corresponding requested bytes. For the bounded demonstration, the test compares live bytes and request counts before and after the vector's scope. A production accounting facility would also need policies for counter overflow, concurrency, and misuse of the deallocation contract.

Resource lifetime is an ownership dependency. A `pmr::vector` using a borrowed memory resource must not outlive that resource. Declaring the resource before the vector's scope makes the dependency visible: the vector destroys its elements and releases storage before the resource is destroyed. A pointer to a resource is not shared ownership. Moving a container or storing it elsewhere can invalidate an otherwise-correct local lifetime argument. ([MEMORY](#))

Nested allocation deserves separate attention. A container whose elements are ordinary `std::string` objects does not automatically route every character allocation through the outer container's PMR resource. Choose allocator-aware element types and understand propagation when the accounting contract requires nested allocations to be included. The integer-vector example avoids that extra layer so the observed counts can be explained without pretending they cover a more complex object graph.

4.2.2 Understanding Fragmentation

Internal fragmentation is unused space inside allocated blocks due to rounding or alignment; external fragmentation is free space split into pieces that may not satisfy a desired larger request under an allocator's policy. Live object layout, allocation lifetimes, and size distributions influence both.

A high resident set after freeing objects does not alone identify fragmentation. The allocator may cache reusable memory, the operating system may retain pages, and unrelated allocations may still be live. Build a repeatable workload and use suitable allocator or heap-profiler evidence before changing memory management. A custom allocator can improve one pattern while making another worse.

Fragmentation describes ways storage can be available yet inconveniently organized or partly unused. Internal fragmentation concerns space inside allocated units that the workload cannot use directly, such as size rounding. External fragmentation concerns free space divided into pieces that may not satisfy a larger request under an allocator's constraints. These are useful concepts, but a process-level memory chart does not directly separate them.

Container capacity is another form of retained space, not necessarily allocator fragmentation. A vector can contain ten elements while reserving room for many more. That capacity may be an intentional tradeoff to avoid future reallocations. Calling it a leak confuses retained ownership with lost ownership. Conversely, keeping excessive capacity across thousands of long-lived objects can violate a memory budget even though the implementation is memory-safe.

Compare three quantities in the worked example: current element count, allocated capacity expressed in element bytes, and peak requested bytes observed by the resource. They can differ without contradiction. During growth, a container may temporarily hold old and new storage while transferring elements; the peak can exceed the final allocation. Reserving once before a known bounded workload changes that allocation history, but it does not prove lower total resident memory for the entire application.

A useful allocation experiment varies one factor. Run the same bounded insertion workload with and without an initial reserve, using the same resource instrument and result checks. Compare the number of successful allocation requests and the peak requested bytes. Keep both results, even if one metric improves while another does not. “Fewer requests” and “smaller peak” are distinct claims.

Do not solve every fragmentation concern with a custom pool. Specialized allocation adds lifetime, alignment, exhaustion, and observability obligations. A pool can be appropriate for a measured workload with predictable object sizes and phases, but it can also retain large blocks indefinitely or hide stale references after reuse. Chapter 10 develops bounded arenas and generation-checked handles. Here the priority is measurement discipline: identify the allocation boundary and the workload before deciding that the default allocator is the bottleneck.

Creating Fragmentation Pressure with Uneven Heap Allocations

Purpose

The allocator must manage metadata and reuse freed regions even when requests have different sizes. Uneven allocation and selective release creates a simple fragmentation-shaped workload.

Code or command

```
#include <cstdint>
#include <iostream>
#include <memory>
#include <vector>

int main() {
    std::vector<std::unique_ptr<std::byte[]>> blocks;
    for (std::size_t i = 1; i <= 8; ++i)
        blocks.push_back(std::make_unique<std::byte[]>(i * 1024));
    for (std::size_t i = 1; i < blocks.size(); i += 2)
        blocks[i].reset();
    auto large = std::make_unique<std::byte[]>(12 * 1024);
    std::cout << (large != nullptr) << "\n";
}
```

What the important parts do

The vector owns several differently sized heap blocks. Resetting alternating owners returns those blocks to the allocator, leaving free regions between live allocations. The later larger request may require a different region despite ample total freed bytes.

Expected behavior or output

The program normally prints `1`; the important observation is not failure but the allocator state created by the pattern.

Concrete experiment

Run the program under a heap profiler or allocator statistics tool and compare it with a version that allocates eight equal-sized blocks and frees all of them together.

Worked example — Count scoped allocation requests without claiming total heap usage

Trace the implementation

The resource lives longer than the vector that uses it. This ordering is essential because the vector calls back into the resource during deallocation. The test checks

that all requests through this resource have been balanced after the vector's scope ends.

Counters are updated only after allocation succeeds. A failed allocation therefore does not falsely increase live bytes. The wrapper is single-threaded and assumes callers obey the `memory_resource` deallocation contract; it is not a concurrent global allocator or a hardened detector of invalid deallocations.

`call_chain` gives the debugger a small recursive computation to inspect in a debug build. An optimizer may transform it substantially, so the program does not assert a particular number of machine stack frames. The allocation output similarly reports requested bytes, not hidden allocator metadata, resident pages, or all allocations made by the executable.

Count the resource you actually control

The recursive calculation and the PMR vector are deliberately separate. The first offers a small call-chain exercise; the second offers allocation accounting. Do not infer stack usage from the PMR counters, because the resource only sees requests routed through its interface.

The vector reserves capacity, fills a known number of integers, and then leaves scope. The final assertions concern the resource's accounting: live requested bytes return to zero and successful allocations have corresponding deallocations. They do not demand that the operating system return pages or that a process monitor display zero memory.

vector operation -> selected PMR resource -> upstream allocator -> OS, when needed
counted here
not fully observed

Before running, predict which counters can increase during reserve, insertion within capacity, and destruction. Then compare the prediction to the actual log. A difference should trigger inspection of the measured boundary, not a convenient relabeling of the result.

Complete worked source

The current files for this example are under `source/book04/ch04/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <cstdlib>
#include <memory_resource>
#include <vector>

class CountingResource : public std::pmr::memory_resource {
public:
    std::size_t allocations{}, deallocations{}, live_bytes{}, peak_bytes{};
private:
    void* do_allocate(std::size_t bytes, std::size_t alignment) override {
        void* block = std::pmr::new_delete_resource()->allocate(bytes,
```

```

alignment);
    ++allocations;
    live_bytes += bytes;
    if (live_bytes > peak_bytes) peak_bytes = live_bytes;
    return block;
}
void do_deallocate(void* block, std::size_t bytes, std::size_t alignment)
override {
    std::pmr::new_delete_resource()->deallocate(block, bytes, alignment);
    ++deallocations;
    live_bytes -= bytes;
}
bool do_is_equal(const std::pmr::memory_resource& other) const noexcept
override {
    return this == &other;
}
};
int call_chain(int depth) {
    const int local = depth;
    if (depth == 0) return local;
    return local + call_chain(depth - 1);
}
int main() {
    CHECK(call_chain(4) == 10);
    CountingResource resource;
    {
        std::pmr::vector<int> values{&resource};
        values.reserve(128);
        for (int i = 0; i < 128; ++i) values.push_back(i);
        CHECK(resource.allocations >= 1);
        CHECK(resource.live_bytes >= values.capacity() * sizeof(int));
        CHECK(values.back() == 127);
    }
    CHECK(resource.live_bytes == 0);
    CHECK(resource.allocations == resource.deallocations);
    std::cout << "resource_requests=" << resource.allocations
                << " peak_requested_bytes=" << resource.peak_bytes << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 4
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — A dangling allocator outlives its stack frame

A helper constructs a local memory resource and returns a container that still refers to it. The container's element values look correct on return, but later allocation or destruction accesses a dead resource. The failure is a lifetime defect, not a mysterious allocator bug. Return an object that owns the resource appropriately, accept a caller-owned resource with a clear lifetime contract, or use a container whose allocator does not borrow local state.

Another misleading result comes from counting only container storage while ignoring element allocations. A vector of strings can report modest outer storage

even when character buffers dominate. Expand the instrument deliberately or state its narrower scope. Do not claim full-application accounting from a single allocator adapter.

A third investigation starts with a deep recursive crash. The last frame may be ordinary arithmetic; the actual issue may be unbounded active call depth. Add a depth bound or change the representation of pending work. Merely increasing the process stack limit can postpone the failure without making the algorithm safe for unbounded input.

The common repair is to state what must outlive what, what can grow, and what the instrument covers. Those relationships are more durable than a one-run memory number.

Practice and implementation lab

Compare resource request counts when appending with and without reserve. Keep logical output identical and report capacity changes separately from elapsed time.

Set a debugger breakpoint inside `call_chain` and inspect depth and local at several calls. Repeat with optimization and document any missing or transformed frames.

Solution and review notes

Reserve can reduce growth allocations when the eventual size is known, but the exact vector growth policy remains implementation-specific. Count actual requests instead of assuming a particular multiplier. Over-reserving can waste memory, so the requirement should justify the bound.

Debug information and optimized machine code may not provide a one-to-one source frame mapping. A missing frame is not automatically a debugger defect. Preserve the exact build options when interpreting the observation.

Guided lab — Compare allocation histories, not slogans

Create two bounded vector-building functions that return the same sequence. One reserves the final capacity before insertion; the other allows ordinary growth. Run each with a fresh `CountingResource`. Record successful allocation requests, deallocation requests, live requested bytes, and peak requested bytes. Verify sequence equality independently of the counters.

Then repeat with a workload that clears and refills one existing vector. Explain which storage can remain owned after `clear` and why that differs from destroying the vector. Do not assert that `shrink_to_fit` must release a particular number of bytes or return physical pages; use the facility's contract and report observed behavior rather than inventing a guarantee.

For the stack portion, inspect the bounded recursive call chain in a debugger. Note which source locals are visible at each frame in the debug build. Compare an optimized build and describe missing or transformed information without treating

it as a semantic failure. Keep the exact executable identities with both observations.

Finally, draw the lifetime dependency for a PMR container and its resource. Mark destruction order explicitly. Propose a safe interface for a caller that wants the container to survive a helper function. The answer must identify an owner, not simply move the dangling resource pointer into a different variable.

Review reasoning. Does a successful deallocation imply an immediate resident-memory decrease? No. Are capacity and size the same? No. Does a PMR resource automatically observe every nested allocation? Only when the relevant allocator-aware types and propagation actually route those requests through it. Does a source function require a distinct retained frame after optimization? Not universally.

Completion criterion. Supply the two equal results, the distinct allocation histories, and an explicit resource-lifetime diagram. A sound conclusion names the measured improvement precisely instead of declaring one allocation strategy universally superior.

Primary reference notes

Technical references: [MEMORY](#) [GCC](#) [CLANG](#) [LIFETIME](#)

Chapter 5 — Understanding C++ Object Representation

An object's fields describe its logical state, but its byte representation may include padding and implementation-specific layout. That distinction matters for serialization, shared memory, binary interfaces, and low-level diagnostics. We will inspect a standard-layout record, copy a trivially copyable object through bytes using the permitted mechanism, and avoid using raw object bytes as a stable external format.

A binary log writer uses `write(fd, &record, sizeof record)`. The reader is built later with a different toolchain, and its fields no longer line up. The writer did not define a format; it exported one compiler's object layout. The distinction is easy to miss because a structure declaration looks like a neatly ordered list of fields.

This chapter separates logical values, object representation, and external encoding. The worked structure is intentionally simple enough for legal representation copying, yet it still demonstrates why size, padding, alignment, and type traits require careful interpretation. The goal is not to avoid low-level inspection. It is to know exactly what such inspection proves.

5.1 Examining Object Size

5.1.1 Accounting for Member Storage

`sizeof` reports the size of an object representation in bytes, including any padding needed for its type. A class containing several members is not guaranteed to occupy exactly the sum of their sizes. Arrays also require each element to begin at a suitable alignment, so a type's size supports the stride between array elements.

The size of a pointer, long, or library class can vary across data models and implementations. `std::string`'s `sizeof` does not include all dynamically allocated character storage it may own. Measure the layer relevant to the question: inline object footprint, owned allocations, or total reachable graph. Treating one `sizeof` result as the entire memory cost of a complex object is misleading.

`sizeof(T)` reports the size of an object of type `T` in C++ bytes, including storage needed to satisfy layout and array-stride requirements. It is not necessarily the total memory retained by everything the object owns. A `std::string` object can own a separate character allocation; a vector object can own a separate element buffer. Summing `sizeof` over top-level owners can therefore substantially undercount a live object graph.

For the teaching Record, the members have a character tag, a fixed-width unsigned count, and a floating-point value. The sum of member sizes establishes a lower-bound intuition for ordinary non-overlapping members, but the actual complete-object size can include gaps and trailing space. The example checks a relationship and prints the actual size rather than asserting a particular ABI's answer.

Arrays explain why trailing padding can matter. Every element must begin at a suitable alignment for its type. The complete-object size determines the stride between adjacent elements, so it must support valid placement of each element. A layout with a member near the end may therefore include trailing padding even when no following member needs it inside the same object. The next array element is the relevant neighbor.

Empty classes and specialized layout features make simplistic formulas even less reliable. Complete objects need identities under the language rules, while base subobjects and members using particular attributes can have special relationships. Do not generalize from one ordinary record to every inheritance hierarchy or library wrapper. Use the appropriate type traits and ABI documentation for the operation you intend to perform. ([CLASS](#), [LAYOUT](#))

When estimating a data structure's footprint, specify whether the estimate covers inline object bytes, separately allocated payloads, allocator overhead, unused capacity, or total process residency. These are different accounting models. A practical report might say "one million records require this much element storage under this build, excluding container and allocator overhead." That bounded statement is more useful than an exact-looking total that silently omits the largest allocations.

5.1.2 Recognizing Compiler-Inserted Padding

Padding can appear between members or at the end of a class to satisfy alignment and layout requirements. Reordering members can change padding on a particular implementation, but it may also affect readability, ABI compatibility, or cache behavior. A smaller sizeof is not automatically a better public layout if compatibility or access patterns matter more.

Padding bytes do not represent meaningful application fields and should not be used for logical equality. Comparing entire object representations with memcmp can report differences between logically equal values because padding need not have a stable value. It can also mishandle types with multiple representations for equivalent values. Compare fields or use a defined value comparison.

Padding is storage that supports representation and alignment but does not necessarily carry a member's logical value. A compiler can insert it between members or at the end of the object. Two records with equal member values can have different padding bytes, so a raw byte comparison is not generally an appropriate implementation of value equality.

Consider two independently constructed records with the same tag, count, and value. A member-wise comparison asks the domain question: do the fields

represent equal values? `memcmp` asks whether every compared representation byte is equal. Those questions can differ because of padding and because some types have multiple representations with related value semantics. Floating-point positive and negative zero illustrate why even an apparently meaningful bit difference need not match the intended equality contract.

Padding also matters when exporting memory. A raw structure dump can include bytes that the application never intended to serialize. Even when the dump appears stable in a local test, it couples the format to compiler layout and risks disclosing incidental representation data. An explicit encoder writes only named fields in a specified order and width. That is both a compatibility improvement and a clearer information boundary.

Field reordering can reduce gaps on one ABI, but it is not free when layout is part of an interface. A structure shared with another library, mapped file, or binary protocol may have an established contract. Changing order to save a few bytes can break that contract. Prefer keeping external formats independent of private in-memory layout so internal optimization does not require a coordinated wire-format change.

Avoid applying packing pragmas as a universal repair. A packed layout can introduce unaligned fields and platform-specific access costs or restrictions. Even when the compiler supports packed accesses, taking a pointer to a packed member and using it as an ordinarily aligned typed pointer can create additional hazards. The safer beginner's approach is to decode external bytes into a normal domain object, then perform computation through that object's valid typed interface.

Measuring Member Storage and Compiler-Inserted Padding

Purpose

`sizeof` and `offsetof` expose the space occupied by members plus alignment padding, which is exactly what the section asks the reader to reason about.

Code or command

```
#include <cstdint>
#include <iostream>

struct Record {
    char tag;
    int count;
    char state;
};

int main() {
    std::cout << "sizeof=" << sizeof(Record) << "\n";
    std::cout << "count offset=" << offsetof(Record, count) <<
    "\n";
    std::cout << "state offset=" << offsetof(Record, state) <<
    "\n";
}
```

What the important parts do

`tag` consumes one byte, but `count` normally requires stronger alignment, so padding can appear before it. Additional tail padding may make arrays of `Record` keep every element correctly aligned.

Expected behavior or output

On many 64-bit systems this layout is 12 bytes with `count` at offset 4 and `state` at offset 8, but the standard permits implementation-specific results.

Concrete experiment

Reorder the members to `int, char, char` and compare `sizeof` and offsets. Explain why the new arrangement often uses less padding.

5.2 Understanding Object Layout Rules

5.2.1 Understanding Standard-Layout Types

Standard-layout is a language category with restrictions intended to support predictable structural relationships and interoperability uses. It is not a promise that every compiler gives the type the same offsets, size, or ABI. Facilities such as `offsetof` have defined applicability tied to layout rules; use them on appropriate types rather than arbitrary polymorphic classes.

A type containing virtual functions, certain inheritance arrangements, or incompatible member-layout properties may fail the standard-layout requirements. Check with type traits when an interface depends on the category. Still define a separate byte protocol for external storage, because standard layout does not specify endianness, numeric encoding, or cross-version compatibility.

Standard-layout is a language category with specific conditions, not a synonym for “simple-looking struct.” It supports certain low-level layout operations and relationships that are not available for arbitrary classes. The category can be queried with `std::is_standard_layout_v<T>`, which makes a required property explicit at compile time. Do not infer it from the absence of a constructor or from the use of the keyword `struct` rather than `class`.

The worked program uses `offsetof` for the teaching record after asserting standard-layout status. The reported offsets describe that compiled type on that implementation. They are useful for understanding padding and member placement. They do not define a universal binary format for all compilers, architectures, or future revisions of the record. ([CLASS](#))

A useful design question is why an operation needs layout knowledge at all. A normal algorithm can access `record.count` without knowing its byte offset. Layout knowledge becomes relevant when interfacing with an established ABI, a native API, a device specification, or an intentionally designed binary format. In those cases, the external contract must be stated separately and checked. A type trait alone cannot prove compatibility with a particular external structure.

Inheritance complicates intuitive drawings. Base subobjects, virtual functions, access-control arrangements, and other properties affect the applicable rules. A pointer to a base subobject can be adjusted relative to the complete object’s address. Do not assume that a cast preserves every numerical address or that an inherited field can be found by adding an offset copied from another hierarchy. Use language-supported conversions for objects and explicit encoding for external data.

For a maintainable low-level interface, keep the representation boundary narrow. Put ABI-facing declarations in one adapter, assert the properties the adapter actually requires, and convert to ordinary application types. Then a compiler upgrade or platform port has a small, reviewable surface. The rest of the application should not contain scattered assumptions about member offsets merely because one early debugging session printed convenient numbers.

5.2.2 Understanding Trivially Copyable Types

Trivially copyable types support certain representation-copy operations specified by the language. Copying their bytes into another existing object of the same type through `memcpy` can preserve the value under the applicable rules. This differs from copying the bytes of `std::string` or `unique_ptr`, whose invariants and ownership cannot be duplicated by raw memory copying.

Trivial copyability and standard layout are separate properties. A type can satisfy one without every property associated with the other. Neither category turns

pointers into persistent addresses or makes a struct a safe network packet. Object lifetime, valid representations, alignment, and the destination's interpretation remain part of the low-level contract.

Trivially copyable describes a class of types for which particular representation-copy operations are supported. It does not imply that every possible byte sequence is a valid object value, that the type is standard-layout, or that its representation is portable across machines. These categories answer different questions and should be tested separately when both matter.

The worked `memcpy` copies the representation of one live complete `Record` object into another live complete object of the same trivially copyable type. The destination is then checked through its members. The source bytes came from a valid object, and the operation is used under the type's requirements. That is fundamentally different from reading arbitrary network bytes into a class containing a string or owning pointer. ([MEMORY](#), [LIFETIME](#))

A string's object representation contains implementation state, not just the characters a user sees. Copying that representation into another string object with `memcpy` would bypass its ownership semantics and can create invalid state or duplicate ownership. Use the string's copy operation to obtain an independent value. A type trait helps prevent a low-level helper from accepting such an object merely because its address and size are available.

Potentially overlapping subobjects and lifetime transitions require further care. A rule supporting representation copying of suitable complete objects is not a blanket rule for copying arbitrary interior regions of a class hierarchy. Nor does raw storage automatically contain a fully initialized object of every type that fits. The later arena and raw-buffer chapters keep allocation, construction, and typed access separate for this reason.

An external decoder should validate fields rather than rely on a trivially-copyable label as input validation. A packet can encode a count beyond the application's limit, an unsupported mode, or a non-finite numerical value even when every individual storage operation is mechanically legal. Representation safety and domain validity are separate gates. The strongest interface converts validated external fields into a domain object whose invariants can then be relied on by ordinary code.

Checking Standard-Layout and Trivially-Copyable Properties

Purpose

The section distinguishes language-defined layout categories from assumptions about raw object bytes. Type traits let the program ask those questions directly.

Code or command

```
#include <iostream>
#include <type_traits>

struct PacketHeader {
    unsigned type;
    unsigned length;
};

struct Polymorphic {
    virtual ~Polymorphic() = default;
    unsigned value{};
};

int main() {
    std::cout << std::boolalpha;
    std::cout << std::is_standard_layout_v<PacketHeader> << ' '
                << std::is_trivially_copyable_v<PacketHeader> <<
    "\n";
    std::cout << std::is_standard_layout_v<Polymorphic> << "\n";
}
```

What the important parts do

`PacketHeader` has simple data members and no virtual machinery, so it commonly satisfies both properties. Adding a virtual destructor to `Polymorphic` introduces implementation-managed object metadata and disqualifies it from standard-layout status.

Expected behavior or output

The first line should normally print `true true`; the second prints `false` for a conforming implementation.

Concrete experiment

Add a user-defined copy constructor to `PacketHeader` and check which trait changes. Explain why standard layout and trivial copying answer different questions.

Worked example — Inspect layout while preserving value-based semantics

Trace the implementation

The static assertions establish the specific language properties used by the example. `memcpy` copies between two live `Record` objects, and the test compares meaningful fields afterward. It does not compare padding or write the record to disk as a portable representation.

Offsets and size are printed rather than asserted against one compiler's layout. That makes the program useful across implementations that satisfy the source requirements while choosing different padding. The lower-bound size check reflects the stored member objects, not a claim about exact packing.

The `std::string` trait rejection is a reminder that owning library objects require their constructors and assignment operations. Copying a string's bytes into another string would not create an independent valid owner. The safe `memcpy` example must not be generalized beyond the type and lifetime conditions that justify it.

Verify values after a legal representation copy

The compile-time assertions establish the properties used by the demonstration: the record is standard-layout and trivially copyable, while `std::string` is not trivially copyable. Those assertions are part of the lesson, not decorative checks. They state why the selected inspection and copying operations are appropriate for these types.

The runtime check compares the destination's members after copying. It does not compare padding bytes or demand a fixed total size. The printed size, alignment, and offsets are labeled implementation observations. Keep the output with the compiler and architecture that produced it.

```
valid source Record -- permitted representation copy --> live destination Record
                                                         |
                                                         member-value checks
```

The diagram does not authorize arbitrary input bytes, cross-version persistence, or copying an owning class. Its narrowness is the point: a low-level operation is safe because its preconditions are known, not because `memcpy` is either universally safe or universally dangerous.

Complete worked source

The current files for this example are under `source/book04/ch05/`. Listings are inserted from those files when this book is assembled.

`main.cpp`

```
#include "check.hpp"
#include <cstdint>
#include <cstdint>
```

```

#include <cstring>
#include <string>
#include <type_traits>

struct Record { char tag; std::uint32_t count; double value; };
static_assert(std::is_standard_layout_v<Record>);
static_assert(std::is_trivially_copyable_v<Record>);
static_assert(!std::is_trivially_copyable_v<std::string>);

int main() {
    const Record source{'A', 17, 2.5};
    Record destination{};
    std::memcpy(&destination, &source, sizeof(Record));
    CHECK(destination.tag == 'A' && destination.count == 17 && destination.value
    == 2.5);
    CHECK(sizeof(Record) >= sizeof(char) + sizeof(std::uint32_t) +
    sizeof(double));
    std::cout << "size=" << sizeof(Record) << " alignment=" << alignof(Record)
    << " offsets=" << offsetof(Record, tag) << ',' << offsetof(Record,
    count)
    << ',' << offsetof(Record, value) << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 5
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — The structure changed but the file format did not

A developer adds a field to an internal record and rebuilds the writer. Old readers still consume the same number of bytes and interpret them with the previous layout. The defect is not necessarily in the new field's value; it is the absence of an explicit versioned encoding contract. Repair the serialization boundary rather than asking every reader to guess which native layout produced a file.

Another failure appears in a test that uses `memcmp` for equality. Two semantically equal records compare unequal because representation bytes differ. Replace the test with field-level assertions appropriate to the domain. If exact byte identity is the actual requirement, define an explicit encoded representation and compare that instead of an incidental in-memory class layout.

A third shortcut copies a string-containing structure as raw bytes because the simple record example did so. The relevant difference is the type's ownership and copy semantics. A legal operation on one trivially copyable record does not generalize to arbitrary classes. Constrain a generic helper or avoid the representation-level operation entirely.

After each repair, preserve one regression that distinguishes the old assumption from the intended contract: a version mismatch, equal values with independent construction, or a type rejected at compile time. The regression should make the boundary visible to future maintainers.

Practice and implementation lab

Add a reordered record type and compare sizes and offsets on your target compiler. Explain why changing an exported library struct's layout can break clients even when all member names remain the same.

Design an explicit serialized representation for tag and count only, with a version byte and fixed byte order. Exclude the double until its encoding policy is specified.

Solution and review notes

Layout changes affect offsets, size, alignment, and code compiled by clients, so they can be ABI changes. A source-compatible field name is not enough for binary compatibility. Measure the new layout and version the interface appropriately.

A byte-level format should identify version, field width, and order, and reject unsupported versions or truncated input. Floating-point interchange needs an explicit representation and exceptional-value policy rather than assuming native double bytes are universally suitable.

Guided lab — Separate a wire record from a domain record

Define a domain record containing a tag, a bounded count, and an integer measurement. Choose an external format with explicit field widths and byte order. Use an integer measurement initially so floating-point interchange does not introduce an unrelated representation choice. Write a table describing offsets in the external format, not offsets in the C++ class.

Implement encode and decode functions using a byte buffer. The decoder should reject truncated input, unsupported tags, out-of-range counts, and trailing bytes if the chosen framing contract forbids them. It should produce a valid domain value only after all required checks succeed. Do not reinterpret the buffer as a pointer to the domain class.

Test one known external byte vector derived by hand. Test the round trip separately. Then reorder the private members of the domain class and confirm that the external encoding remains unchanged. This experiment demonstrates the benefit of separating private layout from persistent format without requiring a second architecture.

For representation inspection, compare two ordinary record layouts with the same logical fields in different orders. Print size, alignment, and supported offsets on the lab compiler. Explain the observed gaps without claiming that the result must be identical elsewhere. Preserve the source and build options with the observation.

Review reasoning. Does standard-layout imply portable serialization? No. Does trivially copyable mean arbitrary bytes are valid input? No. Does `sizeof(std::string)` include every owned character? No. Why compare members after the worked copy? Because the intended result is value preservation, not a claim about padding. Why keep a native adapter narrow? Because implementation assumptions then remain localized and reviewable.

Completion criterion. Deliver a stable external-format specification, a decoder with explicit rejection rules, and a layout observation report clearly marked as target-specific. The two artifacts should agree about values without depending on identical in-memory layouts.

Primary reference notes

Technical references: [CLASS](#) [LANG](#) [MEMORY](#)

Chapter 6 — Controlling Alignment and Padding

Alignment constrains where an object may begin, while padding helps satisfy those constraints within compound objects and arrays. These properties affect correctness before they affect speed. This chapter uses `alignof` and `alignas` to inspect and request alignment, then explains why a chosen sixty-four-byte boundary is an experimental layout choice rather than proof of the target processor's cache-line size.

Two worker threads update different counters. The program has no data race, yet its throughput is disappointing. The counters happen to occupy nearby storage that the processor's coherence machinery treats together. Padding might help, but it might also expand the working set enough to hurt a different workload.

This chapter separates language alignment, object layout, and cache behavior. `alignas` can request an object-placement property. It cannot certify that a particular processor has the cache geometry you imagined, and it cannot make a contended algorithm scalable by itself. The worked program checks the property it actually requests and leaves performance claims for a measured experiment.

6.1 Understanding Alignment Requirements

6.1.1 Using `alignof`

`alignof(T)` reports the alignment requirement of `T` in bytes. An object must be placed at an address suitable for its type under the applicable allocation and lifetime rules. A raw byte buffer is not automatically suitable for every over-aligned object merely because it contains enough bytes.

Standard allocation facilities account for the alignment of ordinary and supported over-aligned types under their specified interfaces. Custom arenas must do so explicitly. Misalignment can be undefined behavior even on hardware that appears to tolerate some unaligned accesses. Distinguish the processor's instruction capabilities from the language requirements of a typed access.

Alignment is a requirement on suitable addresses for objects of a type. `alignof(T)` reports that requirement for the implementation. It is not the same as `sizeof(T)`: a type can occupy many bytes while requiring a smaller alignment boundary. Arrays use a stride that allows every element to satisfy its own alignment requirement, which connects object size and alignment without making them equal.

An allocation interface must provide storage suitable for the intended type. Ordinary standard allocation and containers support the relevant language

requirements, including over-aligned types through the appropriate mechanisms. A custom allocator, memory arena, or raw-buffer adapter must preserve those requirements explicitly. Returning enough bytes is insufficient if the starting address is unsuitable for the object to be constructed there. ([MEMORY](#), [NEW](#))

Do not confuse address divisibility with a complete validity proof. An address numerically divisible by `alignof(T)` may still lack storage, a live object, a valid representation, or permission for the intended access. Alignment is one precondition among several. The pointer chapter collects those obligations so an apparently neat address calculation does not hide a lifetime or bounds error.

The worked example converts addresses to an available integer pointer representation only for observation. It does not modify those integers and reconstruct pointers for access. Its actual reads and writes use valid vector elements. This keeps the diagnostic separate from the language operations whose correctness is being tested.

When designing a raw-storage interface, document the requested byte count, alignment, lifetime, and release procedure together. An arena that remembers only byte count can mishandle alignment padding; an interface that remembers only alignment can still overrun capacity. Use checked arithmetic and a known remaining-space calculation before advancing a cursor. The result should state whether allocation succeeded, rather than returning a misaligned fallback that appears usable until a different architecture exposes the defect.

6.1.2 Using `alignas`

`alignas` requests an alignment for an object or type, subject to language and implementation constraints. It can strengthen an alignment requirement but cannot validly weaken a type's natural requirement. An aligned type's size also accommodates array-element alignment, so requesting a large alignment can substantially increase the footprint of small objects.

Use alignment for a justified requirement: an ABI, a device interface, a SIMD operation, or a measured separation of frequently written data. Do not add large alignments everywhere as a generic optimization. They consume memory and can change layout compatibility. Validate actual addresses when testing a custom allocation path rather than assuming the declaration repairs an incorrectly implemented allocator.

`alignas` expresses an alignment requirement on a declaration where the language permits it. It can request stricter alignment; it is not a way to weaken a type's natural requirement so an unsuitable address becomes valid. Unsupported or inconsistent requests are not portable optimization recipes. Keep alignment choices in a small representation layer and let compilation check whether the target accepts them.

The teaching `Separated` type requests alignment 64. The program asserts that the resulting alignment is at least that request and that complete-object size is compatible with the stride needed for arrays. Those are language/layout checks for

the built type. The number 64 is an explicit experiment parameter, not a discovered fact about every deployment processor.

A vector of that type must place its elements appropriately for the type. Checking each current element address helps demonstrate the allocator/container relationship on the lab implementation. It does not establish that another custom container or placement-new operation will automatically do the same. If you bypass the standard allocation path, the obligation returns to your code.

Remember that increasing alignment can increase complete-object size. A single eight-byte counter wrapped in an over-aligned type may consume substantially more element storage in an array. This can be a sensible tradeoff for frequently modified per-thread state and a poor tradeoff for millions of read-mostly records. The declaration specifies a layout constraint, not the workload that makes the constraint worthwhile.

Treat alignment as part of a public type's ABI when the type crosses a binary boundary. Changing it can affect size, array stride, allocation requirements, and calling conventions. A private implementation detail can be revised with ordinary rebuilds; a separately distributed binary interface needs compatibility planning. Keep performance experiments private until their consequences are understood. The later professional volume develops ABI management; this chapter establishes why an apparently local `alignas` edit can have wider effects.

Requesting and Verifying Explicit Alignment

Purpose

``alignof`` reports a type requirement while ``alignas`` requests a stronger one. The address modulus verifies that a concrete object satisfies the declared boundary.

Code or command

```
#include <cstdint>
#include <iostream>

struct alignas(64) CacheAligned {
    std::uint64_t value;
};

int main() {
    CacheAligned x{};
    auto address = reinterpret_cast<std::uintptr_t>(&x);
    std::cout << "alignof=" << alignof(CacheAligned) << "\n";
    std::cout << "address_mod_64=" << (address % 64) << "\n";
}
```

What the important parts do

``alignas(64)`` raises the type alignment to at least 64 bytes. Converting the address to ``uintptr_t`` permits the simple divisibility check; a correctly aligned address has remainder zero.

Expected behavior or output

The program should report an alignment of at least 64 and ``address_mod_64=0``.

Concrete experiment

Change ``alignas(64)`` to ``alignas(32)`` and compare ``alignof``, ``sizeof``, and the address remainder for several array elements.

6.2 Designing Cache-Conscious Layouts

6.2.1 Understanding Cache-Line Boundaries

Processors transfer and coordinate cache data in implementation-specific units commonly called cache lines. Independent threads writing separate objects that share one coherence line can interfere through false sharing even when no C++ data race exists. Separating hot write locations can reduce that interaction for an appropriate workload.

A hardcoded alignment such as sixty-four bytes is not a portable discovery mechanism for cache geometry. Standard interference-size constants, when available and appropriate, are implementation guidance rather than a universal runtime hardware probe. Measure on the deployment target and include adjacent object layout, array stride, and allocator behavior in the analysis.

Caches move and coordinate data in units that need not match C++ objects. When independent threads repeatedly modify distinct objects that fall in the same coherence unit, their updates can generate traffic even though the language considers the objects separate. This phenomenon is commonly called false sharing. It is a performance interaction, not automatically a C++ data race.

True contention is different: several threads intentionally access the same logical object or lock. Padding neighboring counters does not remove that shared dependency. Before changing layout, identify which object each thread reads and writes. A single global counter remains a shared update point even when it occupies a padded structure by itself.

A correct experiment assigns one counter per worker, keeps the counter container stable while workers run, and reads final values only after synchronization. Ordinary distinct scalar elements can be independently modified, but specialized representations such as packed proxy containers require separate analysis. Do not assume that “different indices” universally means different independently writable memory locations. The type and representation matter. ([THREADS](#), [MEMORYMODEL](#))

Performance also depends on placement, scheduling, compiler optimization, update frequency, and surrounding work. A loop that repeatedly writes a private ordinary counter may be optimized into fewer stores, changing what the benchmark measures. A diagnostic design must ensure that the measured operation actually exists without introducing an unrelated synchronization bottleneck. Atomic per-thread counters can make updates observable for a specific experiment, but they also change the workload and must be described honestly.

The book’s default alignment program deliberately avoids timing. It proves requested alignment and correct element access, then prints footprint information. A separate benchmark can compare compact and separated layouts under a stated workload. The acceptance test should verify final counts and report timing; it should not fail merely because one layout is not faster on a particular shared machine. Correctness is a stable contract here, while speed is an empirical result to interpret.

6.2.2 Avoiding Wasteful Padding

Padding can improve separation or satisfy alignment, but it also expands working sets. Reordering fields by alignment requirements may reduce internal gaps on a particular ABI. Separating hot and cold fields can reduce bytes touched during a critical loop, while padding every object can make a read-heavy workload slower by increasing cache pressure.

Choose layout based on access patterns and compatibility requirements. A public struct's layout is an interface concern, so a private performance experiment should not silently become an ABI change. Record both sizeof and the measured workload result. Smaller and faster are related only under a particular operation and environment, not by definition.

Padding solves some placement problems by spending storage. It can also waste bandwidth and cache capacity when the program scans fields that do not need separation. A design with one padded object per reading might reduce interference during updates but force a reporting pass to touch many more cache lines. The correct choice depends on the dominant operations and their frequency.

Field grouping offers another approach. Separate frequently updated counters from infrequently read metadata, or use a structure of arrays when operations process one field across many records. This can reduce unnecessary bytes touched without padding every complete record. It can also make whole-record operations less convenient, so the representation should follow measured access patterns rather than a universal preference for one layout style.

Compare a compact array, a padded array, and split hot/cold storage using the same logical dataset. Count element storage and verify identical final results before examining timing. If one design retains additional copies or changes the algorithm, disclose that difference. A benchmark that compares different amounts of useful work cannot isolate layout effects.

Alignment and field order are not the only influences on locality. Pointer-heavy indirection, allocation scatter, iteration order, and working-set size can dominate. A linked structure with individually padded nodes may still spend most of its time following unpredictable addresses. Conversely, a compact contiguous representation can perform well without explicit cache-line tuning. Begin with simple ownership and contiguous storage where they fit the problem, then use measurements to justify specialized changes.

Document the reason for every non-obvious padding choice. State the workload, target class, measurement procedure, and compatibility cost. Without that record, a future maintainer may remove necessary separation as “wasted space” or replicate it where it hurts. The useful deliverable is not the padding constant alone; it is the evidence that connects a specific access pattern to a specific representation decision and the conditions under which that decision should be reevaluated.

Separating Frequently Updated Counters onto Cache-Line-Sized Slots

Purpose

Cache-conscious layout is about where frequently accessed data lands, not merely about shrinking structs. This creates separate aligned slots suitable for counters updated by different workers.

Code or command

```
#include <cstdint>
#include <iostream>

struct alignas(64) CounterSlot {
    long long value{};
};

int main() {
    CounterSlot slots[2];
    auto distance = reinterpret_cast<char*>(&slots[1]) -
        reinterpret_cast<char*>(&slots[0]);
    std::cout << "slot bytes=" << sizeof(CounterSlot) << "
        distance=" << distance << "\n";
}
```

What the important parts do

`alignas(64)` requests a cache-line-sized alignment on common processors. Arrays preserve each element's alignment, so the distance between adjacent slots is at least the padded object size.

Expected behavior or output

The printed size and distance should both be at least 64 on an implementation honoring the requested alignment.

Concrete experiment

Compare a two-thread increment benchmark using adjacent plain `long long` values versus two `CounterSlot` values. Treat timing as machine-specific evidence, not a universal constant.

Worked example — Verify requested alignment without assuming cache geometry

Trace the implementation

The vector allocates objects of the over-aligned Separated type. Each element's address is checked against that type's alignment requirement, not against a

guessed cache-line address rule. The static size-multiple assertion reflects the array-stride requirement for aligned objects.

The output compares footprint with a compact single-counter type. It does not time concurrent writes or claim false sharing has been eliminated on every machine. A cache line larger than the chosen separation, other neighboring metadata, or a different workload could change the effect.

`uintptr_t` is used for an observational integer representation of addresses on implementations that provide it. The code does not reconstruct pointers from modified integers or use arithmetic to access unrelated objects. The actual field accesses remain ordinary valid accesses through the vector's elements.

Verify the declaration before discussing the cache

The source creates four over-aligned elements and checks the address of each against the type's actual alignment. It writes through the vector's normal element access and verifies a stored value. The test therefore exercises construction, allocation suitability, stride, and ordinary use without requiring assumptions about processor cache size.

The printed compact and separated sizes are observations. Record them with the build target. A larger separated size is not a failed optimization; it is the cost side of a tradeoff whose benefit has not yet been measured in this program.

```
requested alignment -> type layout -> allocator suitability -> valid element
access
|
+-- does not by itself prove cache-line isolation or faster execution
```

This separation is important because a strong-sounding statement such as “64-byte aligned” can distract from an untested performance claim. Verify the mechanical property first, then design a workload capable of testing the proposed benefit.

Complete worked source

The current files for this example are under `source/book04/ch06/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <cstdint>
#include <cstdint>
#include <vector>

struct Compact { std::uint64_t value{}; };
struct alignas(64) Separated { std::uint64_t value{}; };
static_assert(alignof(Separated) >= 64);
static_assert(sizeof(Separated) % alignof(Separated) == 0);

int main() {
    std::vector<Separated> counters(4);
    for (std::size_t i = 0; i < counters.size(); ++i) {
        const auto address = reinterpret_cast<std::uintptr_t>(&counters[i]);
```

```

        CHECK(address % alignof(Separated) == 0);
        counters[i].value = i + 1;
    }
    CHECK(counters[3].value == 4);
    std::cout << "compact_size=" << sizeof(Compact)
              << " separated_size=" << sizeof(Separated)
              << " requested_alignment=" << alignof(Separated) << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 6
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — Padding the wrong object

A throughput experiment pads the structure containing a pointer, while all threads still update one pointed-to counter. The representation change separates owners, not the shared data being modified. Draw the actual access path and identify the final memory location. The repair may require partitioning work or reducing contention rather than adding more padding.

Another experiment moves each counter apart but grows the vector while workers hold references to its elements. The resulting invalidation is a lifetime/storage bug, not a cache problem. Establish stable storage before launching workers and keep it stable until they have completed.

A third report declares a speedup from one short run. Background scheduling, startup overhead, and measurement granularity may dominate. Repeat the experiment, preserve all samples, and separate warm-up from the measured phase when appropriate. Do not remove inconvenient observations or turn a variable timing result into a correctness assertion.

A layout change should survive two reviews: first, that all accesses remain valid and race-free; second, that the measured workload supports the claimed performance benefit. Passing only one is not enough.

Practice and implementation lab

Measure independent per-thread counters with compact and separated layouts using the same number of updates. Ensure each thread has its own element and combine results only after joining.

Compare a padded array's memory footprint with a structure-of-arrays design that separates hot counters from cold metadata.

Solution and review notes

Distinct elements avoid a language-level race, while cache coherence may still affect throughput. Keep thread count, placement, work, and result verification identical. Do not treat one run as a universal speedup, and distinguish false sharing from true contention on one shared counter.

Separating hot fields can improve the bytes used per scan without padding every full record. The best layout depends on whether operations read whole records or update one field repeatedly. Preserve the simpler layout unless measurements justify the extra representation complexity.

Guided lab — Evaluate compact, separated, and split layouts

Build three representations of a bounded set of worker statistics. The compact version stores small records contiguously. The separated version uses a documented alignment request. The split version keeps frequently updated counters apart from descriptive metadata. Keep the logical data and number of updates identical.

Before measuring, state which thread owns each writable location. Allocate all storage before launching workers, avoid container growth while they run, and join before aggregating results. Add assertions for every final count and for the aggregate. A fast result with missing updates is a failed program, not a successful optimization.

Measure both the update workload and a read-only reporting pass. Record storage footprint as well as elapsed time. A design may help one phase and hurt the other; weight the phases according to an explicitly stated application scenario rather than selecting only the flattering result.

Repeat on the same machine with controlled build settings and record variability. Do not claim hard-real-time bounds or universal cache geometry from the experiment. If thread placement is not controlled, say so. If the execution environment is shared, describe timing as exploratory evidence.

Review reasoning. Is false sharing necessarily a data race? No; distinct valid objects can interact through cache coherence. Does `alignas(64)` discover a cache-line size? No; it requests alignment. Does padding fix true contention on one shared counter? No. Can a smaller record be slower for one workload? Yes; size alone does not determine access cost. What must remain stable during worker access? The lifetime and storage identity of the objects being referenced.

Completion criterion. Deliver a correctness record, a footprint comparison, and a workload-specific interpretation of repeated timing results. Keep the simplest layout unless the measured benefit justifies the added space and maintenance cost.

Primary reference notes

Technical references: [LANG MEMORY THREADS](#)

Chapter 7 — Performing Advanced Pointer Operations

Low-level interfaces often provide raw buffers and callable addresses. Correct code must distinguish navigating existing objects, decoding bytes, and invoking different pointer categories. This chapter decodes a little-endian field without inventing an unaligned `uint32_t` object in a byte buffer, then compares an ordinary function pointer with pointers to data members and member functions.

A packet decoder crashes only when its input starts at an odd address. Another callback works until the object that supplied its state is destroyed. Both defects can be hidden behind syntactically valid pointer operations. The missing information is not the address alone: it is the bounds, type, alignment, lifetime, and ownership contract associated with that address.

HarborWorks will keep raw-pointer boundaries narrow. Inside the application, spans and ordinary typed interfaces carry enough structure to reason about access. At the boundary, explicit decoding and typed callback registration make assumptions visible. The objective is not to avoid every pointer, but to stop asking a pointer value to prove facts it does not contain.

7.1 Navigating Raw Memory

7.1.1 Performing Pointer Arithmetic

Pointer arithmetic is defined in relation to an array object and its permitted one-past boundary. Adding an element count is not arbitrary address arithmetic across every mapped page. A span packages a pointer and count for a borrowed contiguous sequence and makes bounds available to an interface, though `operator[]` still relies on the caller's valid index in the C++20 baseline.

Subtraction between pointers is meaningful only under the language's related-array conditions. An operating-system mapping covering both addresses does not make two unrelated allocations one array. For byte protocols, keep offsets within a validated buffer and check that the entire field fits before reading any part of it.

An ordinary pointer identifies a location in relation to an object; it does not carry an automatically checked length. Pointer arithmetic is defined within the language's permitted array relationships, including a one-past endpoint that can be formed but not dereferenced. Numerically adjacent allocations do not become one array merely because an allocator placed them close together. The language boundary is about objects, not just mapped addresses.

For a range of n elements, reason with the half-open interval $[0, n)$. An index inside that interval can select an element; n names the end position. This notation

scales to subranges and avoids the ambiguity of an “inclusive last index” when the range is empty. A loop using `i < n` maps directly to the interval, while `i <= n` includes the non-dereferenceable endpoint.

A span packages a pointer with an extent, but its ordinary C++20 indexing operation is not a universal runtime bounds check. The caller still needs a valid index and a live underlying range. A span obtained from a vector becomes invalid when the relevant storage is invalidated. It is a view, not a mechanism that prolongs the vector’s lifetime or pins its allocation. ([SPAN](#))

Validate offset and length without overflowing their sum. First require `offset <= size`; then require `field_length <= size - offset`. Only after those checks should the code form the subrange or access its elements. This subtraction-based structure protects against a very large offset wrapping an unchecked `offset + field_length` expression into an apparently small number.

Pointer subtraction also has a domain: it is not a general distance operation across unrelated allocations. When the problem is an encoded buffer, retain offsets as checked sizes and derive pointers only at validated access points. This keeps the proof in one numerical coordinate system. Mixing absolute address integers, element counts, and byte offsets in the same expression makes review harder and can hide unit mistakes even before lifetime or aliasing rules are considered.

7.1.2 Interpreting Raw Buffers

A buffer of bytes does not automatically contain a live object of whichever type a cast names. Reinterpreting its address as a typed pointer can violate alignment, lifetime, aliasing, or representation requirements. Decode protocol fields explicitly or use an appropriate representation-copy operation only when its type and lifetime conditions are established.

Explicit decoding also handles byte order and malformed input. A four-byte unsigned field can be reconstructed by converting bytes to a suitable unsigned type and shifting them into defined positions. Check length before accessing bytes, and define what trailing data means at the framing layer. A successful pointer cast is not validation.

A raw buffer is a sequence of storage units. It becomes meaningful only through an encoding contract. The worked decoder interprets four bytes as a little-endian unsigned field by converting each byte to an unsigned value and combining it at the specified shift position. It never invents a `uint32_t` object at the buffer’s starting address, so it does not need the buffer to satisfy that type’s alignment for a typed load.

This is different from a cast such as “treat these bytes as a pointer to the desired structure.” A cast can change the static type of an expression without establishing object lifetime, suitable alignment, legal aliasing, sufficient extent, or domain validity. A successful compilation is not evidence that those preconditions hold. Keep each obligation explicit when a low-level interface truly requires typed storage access. ([LIFETIME](#), [LANG](#))

The worked function accepts a span containing at least four bytes and decodes its first field. It is not a complete frame parser and does not reject every trailing byte. That is a deliberate layering decision: a field decoder and a message decoder have different contracts. When combining fields, the message layer must track how much input was consumed and decide whether extra data belongs to another field, another frame, or an error.

Malformed input should not create a partially trusted domain object. Decode bounded primitive fields, validate lengths and permitted values, and only then construct the object whose invariants the rest of the application will assume. A decoded length that fits `uint32_t` can still exceed the application's memory budget. Type-range validity is not a resource authorization.

Test the buffer boundary independently from the value calculation. Use a known four-byte vector, each shorter prefix, an offset at the last valid position, and an offset just beyond it. Include embedded zero bytes so the implementation cannot accidentally rely on C-string termination. The correct abstraction is a byte sequence with an explicit length, not text that happens to contain printable characters in the demonstration.

Walking a Raw Byte Buffer with Explicit Offsets

Purpose

Pointer arithmetic is safest when performed in a byte-oriented buffer with validated offsets. `memcpy` avoids alignment and aliasing violations while moving object representation bytes.

Code or command

```
#include <cstdint>
#include <cstring>
#include <iostream>

int main() {
    std::uint8_t buffer[8]{};
    std::uint32_t value = 0x11223344;
    std::memcpy(buffer + 2, &value, sizeof value);
    std::uint32_t decoded{};
    std::memcpy(&decoded, buffer + 2, sizeof decoded);
    std::cout << std::hex << decoded << "\n";
}
```

What the important parts do

`buffer + 2` advances two bytes, not two 32-bit integers. `memcpy` copies exactly `sizeof(value)` bytes into and back out of the buffer, so no misaligned `uint32_t*` dereference is required.

Expected behavior or output

On the same process the round trip prints `11223344` in hexadecimal.

Concrete experiment

Change the insertion offset from 2 to 4, then calculate the last byte touched. Add an explicit bounds check that rejects any offset for which `offset + sizeof(value) > sizeof(buffer)`.

7.2 Using Specialized Pointer Types

7.2.1 Working with Function Pointers

A function pointer identifies a compatible free or static function. Its type includes the function's parameter and return types and relevant qualifiers. Calling through an incompatible signature is not a valid shortcut for generic dispatch. A null function pointer must be checked before invocation when absence is permitted.

Function pointers are useful for small dispatch tables or C interfaces, but they do not own captured state. A capturing lambda cannot generally convert to an

ordinary function pointer because its closure carries an object. Use a suitable callable abstraction when stateful behavior is required, and document callback lifetime and exception boundaries.

A function pointer's type states how the function can be called. Parameters, return type, and relevant qualifiers must agree with the actual function. Casting incompatible functions into one apparent signature does not make their calling conventions or argument interpretations compatible. A dispatch table should store functions sharing a deliberate interface rather than erase mismatches through casts.

A noncapturing lambda can often be used where a compatible ordinary function pointer is required. A capturing lambda carries state in a closure object and generally cannot be represented by a function address alone. This is a useful reminder that behavior and state can have separate lifetimes. `std::function`, templates, or an explicit function-plus-context design may be more appropriate when the callback needs captured data.

C-style APIs frequently pair a callback with an opaque context pointer. The pointer does not own the context. The application must ensure that the object survives every possible callback invocation, including callbacks racing with shutdown or deregistration. A safe registration object can make the subscription lifetime explicit, but its destructor must coordinate with in-flight callbacks according to the API's rules rather than simply delete the context and hope no callback remains.

Exception boundaries deserve similar care. A callback invoked through a C interface or native event mechanism may not permit a C++ exception to escape. Catch failures at the adapter boundary and translate them into the interface's supported error channel. Do not add a `noexcept` declaration merely to suppress concern: an escaping exception from a nonthrowing function leads to termination rather than an ordinary error result.

The worked square callback is intentionally bounded by its test inputs. A reusable numerical dispatch API would need an overflow policy for arbitrary integers. Type-compatible invocation establishes that the call is well-formed, not that every operation performed inside the callee is safe for every possible argument. Keep callback validity, argument validation, and numerical correctness as separate review items.

7.2.2 Working with Pointers to Members

A pointer to a data member describes a member relative to an object of a class, not an ordinary absolute address. A pointer to a member function similarly needs an object on which to invoke it. The `.*` and `->*` operators apply these pointers, and `std::invoke` provides a uniform invocation interface for several callable forms.

Member pointers can have representations unlike ordinary pointers, especially with inheritance and virtual dispatch. Do not serialize them as addresses or assume equal sizes. Their type encodes the associated class and member signature. Use them for projections or controlled dispatch when the object lifetime is already valid.

A pointer to a member is not simply the member's absolute address. It describes how to select or invoke a member relative to an appropriate object. A data-member pointer needs an object for `.*` or `->*`; a member-function pointer needs an object plus its ordinary arguments. The object's lifetime and type relationship remain part of the operation.

This distinction becomes important with inheritance. Applying a base-class member pointer to an appropriate derived object can require adjustment according to the language and ABI. The representation may contain more than one ordinary address-sized quantity. Do not assume that member pointers fit in `void*`, serialize them as integers, or compare their raw bytes as portable identifiers.

`std::invoke` provides a consistent calling interface for ordinary callables and member-pointer forms while retaining their type requirements. It does not erase ownership or validate that an object is still alive. A dangling object reference remains invalid even when the invocation syntax is elegant. Use it to simplify correct typed dispatch, not to conceal missing lifetime reasoning. ([FUNC](#))

Member pointers can be useful as projections. A generic operation can select the same field from many live objects without embedding a field-specific lambda everywhere. This is a design convenience, not permission to mutate through a `const` object or to assume every class has the same layout. The constness of the object and the member function's qualifiers still govern allowed operations.

In the worked example, the data-member pointer reads from a live `const` sensor, and the member-function pointer invokes a `const` operation on that same sensor. The ordinary function pointer has a different type and no associated object. Draw those three signatures explicitly. Understanding why they differ is more important than memorizing punctuation: each signature states what information must exist at the call site and which object, if any, supplies the operation's state.

Calling Through a Function Pointer and a Pointer-to-Member

Purpose

The two specialized pointer forms encode different calling contexts: a free function address versus a member function that must be invoked with an object.

Code or command

```
#include <iostream>

int add(int a, int b) { return a + b; }

struct Scale {
    int factor{3};
    int apply(int x) const { return x * factor; }
};

int main() {
    int (*operation)(int,int) = &add;
    int (Scale::*member)(int) const = &Scale::apply;
    Scale s;
    std::cout << operation(2,4) << ' ' << (s.*member)(5) << "\n";
}
```

What the important parts do

`operation` can be called directly because it points to an ordinary function. `member` has a pointer-to-member type and is applied with `s.*member`, which supplies the object whose `factor` participates in the call.

Expected behavior or output

The program prints `6 15`.

Concrete experiment

Change `factor` to 7 and confirm only the member-pointer result changes. Then store two free-function pointers in an array and select one at runtime.

Worked example — Decode a bounded buffer and invoke typed callback forms

Trace the implementation

The decoder checks the complete field size before indexing and reconstructs a value with explicit little-endian shifts. The byte array's alignment is irrelevant to a `uint32_t` typed load because no such load is performed. The known byte vector

provides a stronger test than a round trip through two potentially matching mistakes.

The ordinary callback and the member-function pointer have different types and invocation requirements. `std::invoke` handles the syntax while preserving those requirements; it does not erase type checking. The data-member pointer is applied to a live `const Sensor`, producing read-only access through that object.

The arithmetic in `square` and `doubled` is exercised only with small bounded values. A public numerical callback accepting arbitrary `int` would need an overflow policy. Low-level pointer correctness does not automatically validate the arithmetic performed by the function being called.

Trace the field before invoking any callback

The known bytes `04 03 02 01` should decode to the chosen little-endian value. That expected result is derived from the encoding rule, not from a second function containing the same shifts. The truncated-prefix case then tests a different property: no byte access occurs until the complete field is known to fit.

The callback portion demonstrates three categories side by side. One function takes an integer directly. One member pointer selects data from a sensor. One member-function pointer invokes behavior on a sensor. The source does not convert them into a common raw address representation.

bounded bytes -> explicit unsigned decoding -> numeric value
function pointer + argument -> result
member pointer + live object -> member access or invocation

Keep the two halves conceptually separate. A decoded number might later choose an operation, but the parser still needs an explicit unknown-operation policy and numerical bounds before dispatch. Successful decoding is not automatic authorization to call arbitrary behavior.

Complete worked source

The current files for this example are under `source/book04/ch07/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <array>
#include <cstdint>
#include <cstdlib>
#include <functional>
#include <span>
#include <stdexcept>

std::uint32_t little_u32(std::span<const std::byte> bytes) {
    if (bytes.size() < 4) throw std::invalid_argument("truncated field");
    std::uint32_t result = 0;
    for (unsigned i = 0; i < 4; ++i)
        result |= std::uint32_t{std::to_integer<unsigned char>(bytes[i])} << (8
* i);
```

```

    return result;
}
int square(int value) { return value * value; }
struct Sensor {
    int reading;
    int doubled() const { return reading * 2; }
};
int main() {
    const std::array<std::byte, 4> encoded{std::byte{0x04}, std::byte{0x03},
std::byte{0x02}, std::byte{0x01}};
    CHECK(little_u32(encoded) == 0x01020304U);
    bool rejected = false;
    try { (void)little_u32(std::span{encoded}.first<3>()); }
    catch (const std::invalid_argument&) { rejected = true; }
    CHECK(rejected);
    int (*operation)(int) = &square;
    CHECK(operation && std::invoke(operation, 5) == 25);
    const Sensor sensor{7};
    int Sensor::* field = &Sensor::reading;
    int (Sensor::* method)() const = &Sensor::doubled;
    CHECK(sensor.*field == 7);
    CHECK(std::invoke(method, sensor) == 14);
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 7
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — A cast replaced four missing checks

A decoder casts a buffer start to a structure pointer and dereferences it. When it fails, adding alignment padding may fix one symptom while leaving length, lifetime, byte-order, and domain-validation errors intact. List the missing preconditions instead of treating the crash as a single alignment puzzle. Explicit field decoding often removes several unnecessary assumptions at once.

A callback defect can look equally intermittent. The registering function returns, its local context dies, and the callback later reads residual state. The repair is a lifetime protocol: owned registration state, synchronized deregistration, or a caller-owned context guaranteed to outlive use. Changing the callback's parameter to `void*` does not extend the object.

Another failure comes from using an unknown operation name as a null function pointer and invoking it without checking. Return an explicit unsupported-operation result. A default operation is acceptable only when the protocol actually defines it; silently choosing one can turn malformed input into unintended work.

After correction, preserve minimal tests for each boundary. A one-byte-short field, an unknown operation, and a completed deregistration with no later callbacks

protect distinct contracts. Do not replace them with one large happy-path demonstration that cannot identify which assumption failed.

Practice and implementation lab

Add a checked field offset to `little_u32` and reject offsets that leave fewer than four bytes without overflowing offset plus four. Test the exact final valid position and one byte beyond it.

Build a typed dispatch table for two bounded operations, with an explicit result for an unknown operation name.

Solution and review notes

Check offset no greater than buffer size, then compare four with size minus offset. Only form the subspan after those checks. This subtraction-based validation avoids overflowing an unchecked offset-plus-length expression.

A dispatch table can store compatible function pointers, but input parsing and numerical bounds remain separate. Unknown names should not produce a null call or silently select a default operation. Return a structured error or optional callable according to the interface contract.

Guided lab — Build a typed, bounded command decoder

Extend the little-endian decoder with a field offset. Use subtraction-based range validation and test the last valid starting position, one position beyond it, and a very large offset. The function must reject invalid input before forming an invalid subspan or reading a byte.

Define a small command format containing an operation identifier and a bounded integer argument. Map accepted identifiers to compatible functions. Return an explicit error for unknown identifiers and out-of-domain arguments. Keep parsing separate from dispatch so each can be tested independently. The numerical functions should use a stated safe domain rather than rely on demonstration-sized inputs by accident.

Add a projection exercise using a pointer to a data member of a simple record. Apply it only to live objects in a stable container. Then explain what would change if the container reallocated while another component retained references. The exercise should identify invalidation without intentionally dereferencing stale references.

Finally, design a callback-registration interface on paper. State who owns the context, when callbacks may occur, how deregistration coordinates with in-flight work, and how exceptions are translated. A complete answer includes shutdown behavior, not merely the callback's function signature.

Review reasoning. Does a span own its elements? No. Does a pointer cast validate a packet? No. Why compare field length with `size - offset` after validating the offset? To avoid overflowing an unchecked sum. Why can a capturing lambda need

more than a function address? Its closure contains state. Does `std::invoke` make a dead object usable? No.

Completion criterion. Deliver a decoder with independent known-vector and boundary tests, a typed dispatch table, and a callback-lifetime contract. Every address in the design should have an identifiable extent, interpretation, and lifetime rather than relying on its numerical value alone.

Primary reference notes

Technical references: [LANG](#) [SPAN](#) [BIT](#) [FUNC](#)

Chapter 8 — Understanding Undefined and Implementation-Dependent Behavior

Systems programming requires knowing which assumptions the language actually permits. Undefined behavior, unspecified choices, and implementation-defined properties are not interchangeable categories. This chapter keeps the normal executable fully defined and places a deliberately invalid sanitizer experiment in a separate optional file. A clean run is useful evidence, but the central goal is to remove invalid assumptions rather than depend on one machine's tolerance.

A program appears reliable in a debug build and fails after optimization. It is tempting to blame the optimizer before checking whether the source gave the implementation a valid program to translate. Undefined behavior can make apparently reasonable source-level expectations inapplicable, while implementation-defined behavior can make a correct program depend on a documented target choice.

HarborWorks needs a vocabulary that distinguishes those cases. This chapter classifies behavior, prevents a concrete arithmetic defect before it occurs, and treats diagnostic tools as evidence with a stated scope. The optional invalid program is a controlled diagnostic exercise, not part of the passing application suite.

8.1 Distinguishing Language Behavior Categories

8.1.1 Understanding Undefined Behavior

Undefined behavior means the language imposes no requirements on that execution after the relevant violation. Examples include certain out-of-bounds accesses, signed overflow, invalid object-lifetime use, and unsynchronized conflicting ordinary accesses. It is not a guaranteed exception or a predictable wraparound rule.

Compilers optimize under the assumption that a well-formed program follows applicable semantic requirements. Consequently, undefined behavior can affect surrounding code in surprising ways, not merely the single instruction a programmer expected to fail. Repair the source contract and validate inputs before dangerous operations rather than using observed output as proof that the violation is harmless.

Undefined behavior means the language imposes no requirements on the result of the invalid execution. It is not a standardized exception, a guaranteed crash, or a promise that the program will continue with a wrapped value. A test that happens

to print the expected result does not establish a valid contract for the same source under another build or input.

Signed integer overflow is a useful example because the invalid operation can look like ordinary arithmetic. Computing $a + b$ and then checking whether the result exceeded the type's limit does not prevent overflow: the addition has already been evaluated. The worked helper instead compares operands against safe boundary expressions before adding them. Prevention must occur before the operation whose validity is in question. ([LANG](#), [NUMERIC](#))

Lifetime and bounds defects have the same ordering problem. Checking a pointer after dereferencing it does not make the dereference safe. Catching a C++ exception around an invalid access does not turn arbitrary native memory faults into ordinary exceptions. A pointer can be nonnull yet refer to an expired object or insufficient range. Validation must establish the actual preconditions, not merely the easiest property to test.

Undefined behavior also affects optimization reasoning. The compiler can transform code under the assumption that a valid execution obeys the language's rules. A programmer who relies on a signed overflow check that itself overflows may find the check removed or rearranged in a way that seems surprising. The repair is to express a valid calculation, not to demand that optimization preserve an invalid intermediate result.

Avoid executing an invalid version merely to “see what happens” when inspection already identifies the defect. Controlled diagnostic demonstrations can be educational, but they should be isolated, clearly labeled, and excluded from ordinary tests. The normal companion programs are written to satisfy their stated bounds and ownership contracts. A negative sanitizer run is evidence that a particular diagnostic detected a particular exercised defect, not a license to use undefined behavior elsewhere.

8.1.2 Understanding Unspecified and Implementation-Defined Behavior

Unspecified behavior allows one of several permitted choices without requiring the implementation to document which choice occurs. Implementation-defined behavior requires the implementation to document its choice. Properties such as the signedness of plain `char` depend on the implementation, while some evaluation-order choices remain unspecified under applicable language rules.

These categories are not automatically bugs when the program remains correct for every permitted outcome it relies on. Portability fails when code assumes one choice without stating or checking it. Record platform-dependent requirements with traits, static assertions, or configuration checks, and avoid presenting one compiler's observed result as a universal language rule.

Implementation-defined behavior involves a choice the implementation documents. The signedness of plain `char` is a familiar example. A program can inspect that property and adapt, or avoid depending on it by choosing an explicitly signed or unsigned type where the distinction matters. The important point is that

the choice is not arbitrary on every execution; it belongs to the implementation contract.

Unspecified behavior permits more than one allowed result without requiring the implementation to document one fixed choice. Evaluation order can expose such freedom in otherwise-valid code. When independent function arguments have observable effects, their allowed ordering may not be the order a reader assumes from left-to-right typography. Write separate statements when a particular order is part of the intended behavior rather than relying on a convenient observation.

Neither category is automatically a bug. A program can be correct when all permitted outcomes satisfy its requirements. The defect arises when the application silently assumes a narrower outcome than the language or platform guarantees. A test that accepts either allowed ordering may be appropriate for demonstrating the language rule; an application requiring one ordering should express it explicitly.

Keep defined unsigned wrap separate from both categories. Unsigned arithmetic has modular behavior under its type's rules, but that does not make every wrapping calculation sensible for a resource limit. A length that wraps to a small allocation followed by a large copy can still cause a serious bounds defect. Defined arithmetic is not the same as a correct application algorithm.

A useful portability record has three columns: required invariant, implementation choice relied upon, and evidence for that choice. For example, an octet protocol can require eight-bit bytes, check the lab assumption, and isolate it in the codec. A native file-descriptor wrapper can explicitly target Linux close behavior. This is more honest than labeling code portable while embedding undocumented assumptions throughout it. The goal is controlled dependence, not the impossible elimination of every platform-specific decision from systems software.

Separating Implementation-Defined Results from Undefined Operations

Purpose

The safest way to learn behavior categories is to observe values the implementation is required to document while avoiding deliberately executing undefined behavior.

Code or command

```
#include <climits>
#include <iostream>
#include <limits>

int main() {
    std::cout << "char bits=" << CHAR_BIT << "\n";
    std::cout << "int digits=" << std::numeric_limits<int>::digits
    << "\n";
    unsigned x = std::numeric_limits<unsigned>::max();
    std::cout << "unsigned wrap=" << (x + 1u) << "\n";
}
```

What the important parts do

`CHAR\_BIT` and integer widths are implementation properties. Unsigned overflow, however, is fully defined modulo 2^N , so adding one to the maximum unsigned value must wrap to zero. Signed overflow would not be a valid comparison because it is undefined behavior.

Expected behavior or output

The exact width values depend on the implementation; the final line must show `0` for the unsigned wrap.

Concrete experiment

Print `sizeof(long)` and `std::numeric\_limits<long>::digits` on two different platforms. Classify those differences as implementation properties rather than undefined behavior.

8.2 Detecting Dangerous Behavior

8.2.1 Using Compiler Warnings

Strong warnings can expose narrowing, suspicious comparisons, shadowing, missing returns, and other likely defects. Warning sets and diagnostic quality vary by compiler and version. Enable a useful baseline and review each finding rather than disabling a broad category to make a build appear clean.

A warning-free program can still contain serious undefined behavior, and some warnings identify a risk rather than a proven defect. Treat diagnostics as engineering signals that require interpretation. Keep intentional conversions explicit and justify their range at the boundary, so the code communicates why an operation is safe instead of merely silencing the warning.

Warnings are compiler observations about suspicious or nonconforming code, not a complete proof system. Enable strong warnings early so newly introduced issues are visible against a clean baseline. A warning-free build is useful evidence, but it does not establish the absence of lifetime defects, data races, protocol errors, or invalid input paths that the compiler cannot infer.

Read the diagnostic's location and reasoning. A conversion warning may reveal a real loss of range, or it may identify a deliberate bounded conversion whose precondition should be made explicit. Do not silence every warning with a cast. A cast changes the expression's type treatment; it does not prove the runtime value fits. Validate or redesign the interface, then use a cast only where the checked relationship is clear.

Different compilers can diagnose different aspects of the same program. Building with GCC and Clang provides two front-end perspectives, but both can use the same standard-library implementation. Record that distinction in verification reports. Two compiler names do not imply two independent libraries, operating systems, or architectures. A portability claim must describe the actual matrix rather than count tool names as interchangeable evidence.

Treat warnings in generated or third-party code deliberately. A project can enforce strict warnings on its own targets while managing external code through separate policies. Global suppression can hide new defects in application code; global error treatment can make dependency upgrades difficult for reasons unrelated to the application's correctness. Target-based build settings make those choices visible.

The companion's checks remain active in release builds. This matters because ordinary `assert` expressions can disappear when `NDEBUG` is defined. A test whose verification is compiled out can report success without checking its result. Inspect the test helper and verify both debug and release configurations. The build system and test harness are part of the evidence chain, so their behavior deserves the same care as the application code they evaluate. ([GCC](#), [CLANG](#))

8.2.2 Using Runtime Sanitizers

`AddressSanitizer` instruments memory accesses and related allocation behavior to detect many invalid accesses; `UndefinedBehaviorSanitizer` checks selected undefined-behavior classes. They cover different mechanisms and have configuration limits. Compile and link consistently with the chosen instrumentation, retain useful debug information, and run representative tests.

Sanitizers observe executed paths, not every possible path. They do not prove race freedom or all lifetime correctness, and a successful run does not validate an untested platform. `ThreadSanitizer` is a separate tool for data-race detection and is

typically used in a separate build rather than combined indiscriminately with AddressSanitizer. Record which tool and tests actually ran.

Runtime sanitizers add instrumentation to selected operations and report classes of defects when the relevant paths execute. AddressSanitizer targets many invalid-memory-access and lifetime-related errors. UndefinedBehaviorSanitizer checks selected undefined or suspicious operations according to its enabled checks. Neither explores every possible input or scheduling history automatically. A clean run means the exercised instrumented execution produced no reported violation within that tool's scope. ([ASAN](#), [UBSAN](#))

Instrumentation changes execution cost, memory layout, and sometimes timing. Use sanitized builds for correctness investigation, not as direct evidence of production throughput. A performance comparison should use an appropriate optimized configuration and retain the functional checks that ensure both implementations performed equivalent work.

ThreadSanitizer addresses a different concern: unsynchronized concurrent accesses and related races in supported environments. It normally belongs in a separate build from AddressSanitizer. A clean ASan/UBSan run is not a substitute for race analysis, and even a clean TSan run does not prove freedom from deadlock, starvation, or every possible race schedule. The concurrency chapters require explicit ownership and happens-before arguments alongside tests. ([TSAN](#))

When a diagnostic occurs, begin with the first reported violation and a small reproducer. Later crashes may be consequences of earlier memory damage. Preserve the source revision, compiler options, input, and full diagnostic rather than only a screenshot of the final stack frame. Reduce the case without removing the operation that causes the violation.

The optional unsafe program intentionally indexes beyond a small allocation. It is excluded from the default build and passing test count. Its expected result is a diagnostic failure under a supported configuration, not PASS. The verification report separately states whether that negative exercise was executed. Keeping negative demonstrations separate prevents an expected failure from being disguised as a broken ordinary test or, worse, from being shipped as normal application logic.

Triggering AddressSanitizer with a Controlled Out-of-Bounds Access

Purpose

Warnings diagnose suspicious source patterns, while runtime sanitizers instrument executed code to catch errors such as out-of-bounds memory access. This intentionally invalid program is suitable only as a bounded diagnostic exercise.

Code or command

```
#include <vector>

int main() {
    std::vector<int> values{1,2,3};
    volatile int x = values[3]; // intentional bug for sanitizer
    exercise
    (void)x;
}
```

What the important parts do

`values` contains indices 0 through 2. Accessing index 3 is outside the vector's element range. Compiling with AddressSanitizer adds checks that should report the invalid access when this line executes.

Expected behavior or output

With `-fsanitize=address -fno-omit-frame-pointer`, execution should terminate with an AddressSanitizer diagnostic instead of being treated as a valid program result.

Concrete experiment

Change the access to `values.at(3)` and compare behavior: the bounds-checked member should throw `std::out\_of\_range` even without AddressSanitizer.

Worked example — Check arithmetic before evaluation and isolate a negative sanitizer exercise

Trace the implementation

checked_add rearranges the overflow conditions so that the checks themselves remain representable. Only after both checks pass does it evaluate a plus b. The test covers both signed extremes and distinguishes them from unsigned arithmetic's defined modulo behavior.

Plain char signedness is printed as an implementation property, not asserted against one expected platform. The normal program has no intentional undefined

behavior. The optional source is separate, excluded from the build, and labeled as a diagnostic experiment whose successful purpose is a reported failure.

Run sanitizer builds of the ordinary examples first. A negative demonstration can confirm that an instrumentation setup detects its selected defect, but it does not prove that the same tool detects every possible memory error. Never mix the expected nonzero exit of an intentionally broken example with a claim that the application's correctness suite passed.

Prove the guard before evaluating the addition

For a positive second operand, the helper checks whether the first operand exceeds `maximum - b`. For a negative second operand, it checks whether the first falls below `minimum - b`. The branch conditions make those boundary expressions safe within the integer domain. Only after both relevant checks does the function evaluate the addition.

The result is optional because overflow is a rejected input combination, not a fabricated numerical answer. The tests distinguish accepted boundary values from the first invalid step beyond each boundary. The unsigned-wrap check demonstrates a different rule and is not used as a replacement for the signed helper.

Before running, trace `maximum plus one`, `minimum minus one`, `minimum plus zero`, and an ordinary positive pair. Explain which branch applies and why the returned optional state is correct. A proof about the guard is stronger than a test that computes an overflowing sum and hopes a later comparison notices it.

Complete worked source

The current files for this example are under `source/book04/ch08/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <limits>
#include <optional>

std::optional<int> checked_add(int a, int b) {
    const int maximum = std::numeric_limits<int>::max();
    const int minimum = std::numeric_limits<int>::min();
    if ((b > 0 && a > maximum - b) || (b < 0 && a < minimum - b))
        return std::nullopt;
    return a + b;
}

int main() {
    CHECK(checked_add(20, 22) == 42);
    CHECK(!checked_add(std::numeric_limits<int>::max(), 1));
    CHECK(!checked_add(std::numeric_limits<int>::min(), -1));
    CHECK(checked_add(std::numeric_limits<int>::min(), 0) ==
std::numeric_limits<int>::min());
    const unsigned wrapped = std::numeric_limits<unsigned>::max() + 1U;
    CHECK(wrapped == 0);
    std::cout << "plain_char_signed=" << std::numeric_limits<char>::is_signed <<
```

```
"\nPASS\n";  
}
```

optional/unsafe_demo.cpp

```
// INTENTIONALLY INVALID. Not part of the default build or passing test suite.  
// Compile with AddressSanitizer in an isolated development process.  
#include <memory>  
int main(int argc, char**) {  
    auto data = std::make_unique<int[]>(4);  
    volatile int index = argc + 3; // With no extra arguments, index is 4.  
    return data[index];           // Deliberate heap out-of-bounds read.  
}
```

optional/README.txt

```
This file intentionally executes undefined behavior to demonstrate a diagnostic.  
It is excluded from the normal CMake targets. Do not use it as application code.  
From this chapter directory, on a supported Clang/GCC development environment:  
  c++ -std=c++20 -g -O1 -fsanitize=address -fno-omit-frame-pointer optional/  
  unsafe_demo.cpp -o unsafe_demo  
  ./unsafe_demo  
Expected outcome: a nonzero diagnostic run, not PASS. Exact output depends on  
the tool.  
The shipped verification report states whether this negative exercise was run.
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 8
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — The check that contains the bug

A helper calculates a signed sum, compares it with an operand, and declares overflow if the sum became smaller. That reasoning borrows an unsigned-wrap intuition for an operation whose signed overflow is undefined. The diagnostic may disappear under optimization because the valid-program assumptions differ from the programmer’s intended machine behavior.

Repair the precondition or use an explicitly supported checked-arithmetic facility with a documented contract. Do not merely mark the result volatile. Volatile is not a general optimization-disable switch and does not redefine signed arithmetic.

Another failure is a sanitizer report dismissed because the uninstrumented program “works.” The report identifies an exercised violation under the tool’s model; ordinary output does not refute it. Reduce the case, inspect the access and lifetime, and fix the invalid operation before comparing builds again.

A third false confidence comes from an empty test body after release preprocessing. Verify that checks remain executable in the optimized

configuration. The absence of a diagnostic is meaningful only when the relevant code and assertions actually ran.

Practice and implementation lab

Add checked subtraction and multiplication with boundary tests that avoid performing the invalid operation in the test oracle. Compare warnings and sanitizer reports from the two supported compilers available in your environment.

Run the optional diagnostic only in a separate local development process and retain the exact command and exit status.

Solution and review notes

Derive conditions using division or range comparisons that remain valid at the minimum signed value; negating that minimum is itself unsafe. A wider type can help only when its representable range is proven sufficient for every intermediate product.

The report should separate normal tests, negative diagnostic tests, and unavailable tools. A sanitizer finding is evidence of a defect on an executed path; lack of a finding is not a certificate of complete safety.

Guided lab — Classify, prevent, and diagnose

Create a short behavior ledger containing examples of defined behavior, implementation-defined choices, unspecified allowed outcomes, and invalid operations that would invoke undefined behavior. Keep invalid examples as written analysis unless they belong to the isolated diagnostic exercise. For each entry, name the relevant rule and the application's required outcome.

Extend the checked-add tests with the minimum and maximum values paired with zero, one, and minus one where appropriate. Add a bounded exhaustive domain such as all pairs from minus one hundred through one hundred, comparing against a wider reference calculation whose range is demonstrably safe. That finite oracle supports the implementation without pretending to cover every possible integer pair.

Build the normal program with warnings and with the supported sanitizer configuration. Retain exact commands and outputs. Separately compile the optional invalid program in a disposable directory and observe the diagnostic only when the environment supports it. Record its nonzero result as an expected negative demonstration, not as an ordinary passing test.

Finally, introduce a harmless source warning in an exercise copy, read it, and repair the underlying issue rather than adding an unexplained cast. Compare GCC and Clang diagnostics and note any differences. Do not treat disagreement as proof that one compiler is wrong; first inspect the rule and the configured warning scope.

Review reasoning. Can an overflow check safely follow an already-overflowing signed addition? No. Is defined unsigned wrap always appropriate for lengths? No. Does a clean sanitizer run prove all inputs safe? No. Does plain char signedness have to match every target? No. Why separate negative tests? Their expected failure serves a different purpose from the application contract.

Completion criterion. Provide the behavior ledger, the pre-operation arithmetic guard, a boundary test set, and clearly separated normal and negative diagnostic records. The result should explain why the program is valid, not only that one build happened to finish.

Primary reference notes

Technical references: [LANG](#) [ASAN](#) [UBSAN](#) [GCC](#) [CLANG](#)

Chapter 9 — Managing Operating-System Resources Safely

A native handle is often a small integer or opaque value, but it represents an external resource with ownership and failure rules. Copying the number does not necessarily create another owned resource. This chapter wraps a Linux file descriptor in a move-only RAII type, verifies cleanup after an exception, and distinguishes destructor cleanup from an explicit checked close when an application needs to observe release errors.

A server closes a connection, stores the old descriptor number in a work item, and later closes that number again during cleanup. By then the operating system has reused it for another connection. The second close is not harmless tidying; it releases somebody else's resource. The integer survived, but the ownership relationship did not.

This chapter turns native handles into explicit lifetime contracts. The wrapper is small, yet its design addresses copying, transfer, error reporting, exception paths, and platform-specific release behavior. Resource safety is not achieved by adding a destructor alone. It requires knowing which object is responsible for which handle at every point where control can leave the function.

9.1 Managing Native Resource Handles

9.1.1 Managing File Descriptors

A POSIX-style file descriptor is a process-local integer referring to an open resource. Zero is a valid descriptor, so negative one is a useful empty sentinel for an owning wrapper. The descriptor can refer to a regular file, pipe, socket, or another supported object, and operations depend on the resource kind.

Copying the integer creates another observer of the same descriptor number, not another independent owner. Closing through either copied number releases that descriptor, and the operating system can reuse the number for a later resource. Use `dup` or another appropriate API when a separate descriptor is intended, and define ownership of every returned handle immediately.

A file descriptor is a nonnegative integer in a process's descriptor table. Zero is a valid descriptor, so a check such as `fd > 0` is wrong for general acquisition. Native APIs commonly use a negative result to report failure. Store the successful result in an owner promptly, and keep the failure sentinel distinct from every valid descriptor value.

The descriptor refers to an open file description, which can hold state such as the current file offset and status flags. Duplicated descriptors can refer to the same

underlying open description. They are separate descriptor-table entries with related underlying state, not necessarily independent file positions. This matters when two components assume that seeking through one handle cannot affect reads through another. ([OPEN](#))

A borrowed descriptor returned by `get ( )` is not a new ownership share. It permits an operation while the owning wrapper remains alive and while no competing action invalidates the handle. A callee receiving a borrowed integer must not close it unless ownership transfer is explicitly part of the interface. Conversely, an adopting constructor assumes that the caller is handing over release responsibility rather than merely lending an integer.

Descriptor reuse makes stale references particularly deceptive. A pointer to freed memory may fail dramatically; a stale descriptor can successfully operate on a new resource with the same number. Therefore, “the system call succeeded” does not prove that the intended resource was used. Ownership and synchronization must prevent the stale operation in the first place.

Inheritance across process creation is another resource boundary. Close-on-exec flags prevent unintended descriptors from surviving a successful program-image replacement. On Linux, acquisition flags such as `O_CLOEXEC` avoid the race involved in creating a descriptor and setting the flag later in a multithreaded process. The flag is not a substitute for deciding which descriptors the child actually needs; process-launch code still needs an explicit inheritance plan.

9.1.2 Managing Socket Handles

On POSIX systems, sockets also use file descriptors, while other platforms expose different handle types and cleanup functions. A wrapper must match its platform’s invalid sentinel and release operation. A Windows socket cannot be treated as an ordinary POSIX descriptor simply because both are represented by small values in some builds.

Socket lifetime includes protocol concerns beyond closing a handle: pending output, half-close behavior, timeouts, and peer expectations may matter. RAII provides eventual resource cleanup, but a graceful application shutdown may need an explicit sequence before destruction. The network chapters build that protocol layer separately.

Sockets use descriptors on the Linux baseline, but their behavior is not identical to regular files. A stream socket has connection state, peer behavior, buffering, and directional shutdown. Releasing the local descriptor and announcing that no more bytes will be sent are different operations. `shutdown` changes communication directions; `close` releases the descriptor reference. A design should choose the operation that matches its protocol state. ([SHUTDOWN](#), [SOCKETS](#))

One clear ownership pattern gives each accepted connection to one worker. The listener creates the connection, moves its owner into a queue, and relinquishes responsibility. The worker owns the descriptor through receive, processing, response, and cleanup. Other components may observe metrics, but they do not

independently close the same descriptor number. This pattern appears in the capstone because it simplifies both failure handling and shutdown reasoning.

Sharing a descriptor among threads requires a stronger protocol. A thread blocked in I/O may still be using the underlying open description while another closes its descriptor-table entry. Linux behavior does not provide a general guarantee that such a close immediately interrupts the blocked operation. Reuse of the number adds further hazards. Prefer explicit cancellation, bounded waits, and ownership coordination over closing a borrowed handle from an unrelated thread.

[\(POSIXCLOSE\)](#)

Socket errors also belong to the transport layer. A peer can disconnect after a request is received but before a response is sent. Cleanup must still release the local resource, while the application records that response delivery was not confirmed. RAII prevents a leaked descriptor; it does not make the remote operation transactional or prove that the peer consumed the response.

Treat platform differences explicitly. Windows socket handles and release functions are not interchangeable with POSIX integer descriptors and `close`. A cross-platform application should provide native adapters with their own invalid-handle values, cleanup operations, and error translations. The common application layer can then depend on a stable ownership interface without pretending the underlying operating systems expose identical handle semantics.

Opening, Using, and Closing a Native File Descriptor

Purpose

A file descriptor is an integer handle owned by the process, not the file contents themselves. This example shows the explicit acquire-use-release lifecycle of a native resource.

Code or command

```
#include <fcntl.h>
#include <unistd.h>
#include <cstring>

int main() {
    int fd = ::open("handle_demo.txt", O_CREAT|O_TRUNC|O_WRONLY,
0600);
    if (fd < 0) return 1;
    const char* text = "native handle\n";
    const auto n = ::write(fd, text, std::strlen(text));
    const int close_result = ::close(fd);
    return n > 0 && close_result == 0 ? 0 : 2;
}
```

What the important parts do

`open` returns a descriptor or -1. `write` uses that descriptor to identify the kernel object. `close` releases the process's reference; failing to call it on every path leaks a finite native resource.

Expected behavior or output

On success the program creates `handle_demo.txt`, writes one line, closes the descriptor, and exits 0.

Concrete experiment

Add an early return immediately after `open` and run the program repeatedly under a descriptor-counting harness. Then compare with the RAII version in the next section.

9.2 Wrapping Native Resources with RAII

9.2.1 Creating Ownership Wrappers

A move-only wrapper owns one valid descriptor or an empty state. Its destructor releases a valid descriptor without throwing. Move construction transfers the integer and empties the source; move assignment first releases the destination's current resource, then transfers ownership. A `get` function exposes a borrowed handle for a call, while `release` explicitly gives ownership back to the caller.

Avoid implicit conversion to `int` when it would make ownership boundaries unclear. A raw descriptor returned by `get` must not be closed by a borrower. If a function adopts ownership, name that behavior and pass an owning object or a deliberately released handle. Type-level ownership reduces double-close mistakes that can otherwise affect a newly reused descriptor number.

A unique handle wrapper needs a small state machine: empty, owning, and transferred. Copying is disabled because copying the integer would create two apparent owners of one descriptor-table entry. Moving transfers the integer and leaves the source empty. Destruction releases only an owned handle. The operations should make it difficult to express duplicate ownership accidentally.

The worked move assignment first releases the destination's previous resource, then takes the source's handle. A self-move guard prevents a wrapper from closing its own handle and then trying to acquire the same invalidated state. The exact moved-from representation is an implementation detail of the wrapper, but its public behavior should clearly describe an empty non-owning source.

`release()` transfers responsibility without closing the resource. It is useful when handing a handle to another owner, but it is also an escape hatch that can leak if the caller loses the returned integer. Use it only at a visible adoption boundary. A normal borrowed operation should call `get()` instead. The distinction between `release` and `get` is an ownership distinction, not merely a naming convention.

Acquisition needs the same discipline. Obtain the raw result, check the native failure convention, and establish ownership before performing unrelated operations that can throw. If a factory performs several acquisitions, wrap each successful one before starting the next. Then partial construction failures naturally release earlier resources through their owners.

Keep resource cleanup separate from application success. A destructor should normally be nonthrowing and best-effort because it may run during exception unwinding. An explicit checked-close operation can report errors when the caller still has an error channel. That separation lets the application distinguish “the resource was released” from “all output obligations were confirmed.” It also prevents a second exception during cleanup from obscuring the original failure. ([GUIDELINES](#))

9.2.2 Guaranteeing Cleanup During Failure

When a scope exits through an exception, completed local owners are destroyed. This allows a function to acquire several resources sequentially without manually closing every earlier acquisition in every error branch. A constructor that fails must still arrange cleanup for any raw resource acquired before ownership was established.

Destructors cannot conveniently report every close failure while unwinding. Provide an explicit checked close or commit operation when the caller must know whether a write-related release succeeded. Linux close error behavior also makes blindly retrying close dangerous because the descriptor number may already have

been released. Follow the target platform's documented policy rather than a generic retry-every-EINTR rule.

Linux close behavior deserves an explicit adapter policy. The descriptor number is released early in the close operation, and a later error does not make retrying that number safe. Retrying can close a different resource that another thread acquired under the reused number. The worked wrapper invalidates its local ownership before the native close and does not retry on Linux. This is intentionally not advertised as a universal POSIX retry rule. ([POSIXCLOSE](#))

The distinction matters because POSIX.1-2024 specifies interrupted-close behavior differently from Linux. A portability layer must consult its target contract rather than copy an unconditional EINTR loop from read/write code. The reference directory identifies both the Linux manual and the relevant standards reference; the executable wrapper in this package follows Linux behavior.

A destructor cannot report every late I/O error through an exception. The wrapper therefore offers `close_checked()` for an explicit boundary where the caller can handle a failure. It releases its local ownership first, so a thrown error does not leave the destructor poised to retry the old number. This design reports the error without pretending that the close can be rolled back.

Durability is another separate obligation. A successful close does not by itself prove that file data and relevant metadata have reached durable storage. Applications with such requirements need an explicit synchronization and recovery protocol, including the storage system's guarantees. A generic descriptor destructor cannot infer whether the resource is a regular file, a pipe, a socket, or a file participating in a multi-step commit. ([FSYNC](#))

The worked exception test deliberately avoids concurrent descriptor reuse while checking that the old number is invalid after scope exit. That makes the observation meaningful for the fixture. In a real multithreaded process, asking whether an old number is currently valid can find a newly reused descriptor and cannot identify the original resource. Tests must control that history; application correctness should rely on ownership, not on probing a stale integer after the fact.

Guaranteeing Descriptor Cleanup with an RAII Owner

Purpose

RAII turns native-handle ownership into an object invariant so cleanup occurs during normal scope exit and exception unwinding.

Code or command

```
#include <fcntl.h>
#include <unistd.h>
#include <stdexcept>

class UniqueFd {
    int fd_{-1};
public:
    explicit UniqueFd(int fd) : fd_(fd) {}
    ~UniqueFd() { if (fd_ >= 0) ::close(fd_); }
    UniqueFd(const UniqueFd&) = delete;
    UniqueFd& operator=(const UniqueFd&) = delete;
    int get() const { return fd_; }
};

int main() {
    UniqueFd fd(::open("raii.txt", O_CREAT|O_WRONLY, 0600));
    if (fd.get() < 0) return 1;
    throw std::runtime_error("simulated failure");
}
```

What the important parts do

`UniqueFd` stores the descriptor, deletes copying to prevent two owners from closing the same handle, and closes in its destructor. Throwing after acquisition demonstrates that cleanup is tied to stack unwinding rather than to manually reaching a final `close` statement.

Expected behavior or output

The program terminates because the exception is uncaught, but the `UniqueFd` destructor runs while unwinding and closes the descriptor first.

Concrete experiment

Catch the exception in `main`, then add a move constructor and move assignment so ownership can be transferred safely without duplicating the handle.

Worked example — Transfer a descriptor and verify exception-path cleanup

Trace the implementation

The descriptor is placed into an owner immediately after open, and its validity is checked before use. Move construction empties original. Throwing afterward destroys the moved owner during unwinding, and a later `fcntl` check confirms that the descriptor is no longer open in this single-threaded test.

The check occurs before another open can reuse the released number. In a concurrent process, observing a raw integer after close would not reliably identify the old resource because another thread could obtain the same number. This fixture's sequencing is part of what makes its evidence meaningful.

The explicit `close_checked` path releases ownership before calling close and reports an error without attempting a second close. Destructor reset preserves `errno` and does not throw. This wrapper follows Linux's documented release behavior; portable resource abstractions must review their target's semantics rather than copying this policy blindly.

Trace ownership through the exceptional path

The test opens a harmless native resource, records its number, moves the owner, and throws a deliberately local exception. The moved-from wrapper is empty. The moved-to wrapper is destroyed during unwinding and performs the release. After the handler, a controlled descriptor query verifies the expected state before any new acquisition can reuse the number.

The second part exercises explicit checked close. The wrapper becomes empty regardless of whether a late close error would need reporting. On the successful fixture path, a later destructor has nothing left to release.

```
open success -> original owner -> move -> destination owner
                  empty                |
                                      scope exit / exception
                                      |
                                      one release attempt
```

The arrow means transfer of responsibility, not duplication of the underlying resource. Read every `get`, `release`, `move`, and `close` in the source against that diagram. A correct wrapper is small enough that its ownership transitions can be audited in one pass.

Complete worked source

The current files for this example are under `source/book04/ch09/`. Listings are inserted from those files when this book is assembled.

fd.hpp

```

#ifndef HARBOR_CHAPTER_FD_HPP
#define HARBOR_CHAPTER_FD_HPP
#include <cerrno>
#include <system_error>
#include <utility>
#include <unistd.h>

class FileDescriptor {
    int value_{-1};
public:
    explicit FileDescriptor(int value = -1) noexcept : value_(value) {}
    FileDescriptor(const FileDescriptor&) = delete;
    FileDescriptor& operator=(const FileDescriptor&) = delete;
    FileDescriptor(FileDescriptor&& other) noexcept : value_(other.release()) {}
    FileDescriptor& operator=(FileDescriptor&& other) noexcept {
        if (this != &other) { reset(); value_ = other.release(); }
        return *this;
    }
    ~FileDescriptor() { reset(); }
    int get() const noexcept { return value_; }
    explicit operator bool() const noexcept { return value_ >= 0; }
    int release() noexcept { return std::exchange(value_, -1); }
    void reset() noexcept {
        const int saved_errno = errno;
        if (value_ >= 0) (void)::close(release()); // Linux: do not retry close.
        errno = saved_errno;
    }
    void close_checked() {
        if (value_ < 0) return;
        const int descriptor = release();
        if (::close(descriptor) < 0)
            throw std::system_error(errno, std::generic_category(), "close");
    }
};
#endif

```

main.cpp

```

#include "check.hpp"
#include "fd.hpp"
#include <cerrno>
#include <fcntl.h>
#include <stdexcept>

int main() {
    struct SimulatedFailure {};
    int released_number = -1;
    try {
        FileDescriptor original{::open("/dev/null", O_RDONLY | O_CLOEXEC)};
        CHECK(static_cast<bool>(original));
        released_number = original.get();
        FileDescriptor moved{std::move(original)};
        CHECK(!original && moved.get() == released_number);
        throw SimulatedFailure{};
    } catch (const SimulatedFailure&) {}
    errno = 0;
    CHECK(::fcntl(released_number, F_GETFD) == -1 && errno == EBADF);
    FileDescriptor file{::open("/dev/null", O_RDONLY | O_CLOEXEC)};
    CHECK(static_cast<bool>(file));
    file.close_checked();
    CHECK(!file);
}

```

```
std::cout << "PASS\n";  
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 9
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — Cleanup code that closes a new connection

A worker stores a borrowed descriptor in a cleanup list. The real owner closes it, a later accept reuses the number, and the cleanup list eventually closes the new connection. Adding a validity probe does not solve the problem: the reused number is valid, but for the wrong resource. Remove duplicate release responsibility and make transfer explicit.

Another defect arises when a destructor throws while another exception is already unwinding. The process can terminate instead of reaching the intended handler. Keep destructors nonthrowing and expose an explicit completion operation when errors must be reported. Log or collect cleanup failures through a carefully designed nonthrowing channel only when that channel is itself safe in the relevant context.

A third shortcut retries every failed close with the same loop used for read. On Linux, that can act on a reused descriptor. The repair is a platform-specific release contract, not a larger retry count. Similar-looking error names do not imply identical recovery semantics across operations.

Regression tests should cover empty destruction, move construction, move assignment over an existing resource, repeated explicit close of an empty wrapper, and exception-path release. They should also document which failure paths cannot be deterministically injected through the real operating system and are instead checked with a controlled test adapter.

Practice and implementation lab

Add a function that duplicates a descriptor and returns a new `FileDescriptor` owner. Test that closing one duplicate does not close the other descriptor, while noting that they may share an underlying open-file description.

Design an error-injected resource wrapper test without depending on descriptor-number reuse timing.

Solution and review notes

Call `dup` or an appropriate `close-on-exec` duplication API, check its return, and immediately adopt the new descriptor. The two descriptor entries have separate close lifetimes but can share file offset and other open-file-description state.

A fake release function can record which handles were released and return controlled failures. Such a unit test isolates wrapper ownership from the operating system's allocation timing. Keep real-resource integration tests as a separate layer.

Guided lab — Specify and test a unique native owner

Write a state table for the wrapper before modifying code. Include empty construction, successful adoption, move construction, move assignment, borrowing, release, checked close, and destruction. For each operation, identify the owner before and after it and whether the native resource remains open.

Add a move-assignment test using two distinct disposable resources. Verify that the destination's old resource is released and the source's resource remains owned by the destination. Keep the test single-threaded so descriptor reuse cannot make the observations ambiguous. Test self-move according to the wrapper's documented policy.

Build a small fake release adapter that records calls and can simulate an error. Use it to verify that an explicit close invalidates ownership before reporting the error and that later destruction does not issue another release. The fake tests wrapper control flow; the native tests verify integration with Linux. Neither replaces the other.

Inspect a process-launch design and list which handles should survive `exec`. Use `close-on-exec` acquisition where appropriate and explicit spawn file actions for intended inheritance. Do not open sensitive files merely to test inheritance; a disposable pipe or temporary file is sufficient.

Review reasoning. Is descriptor zero a failure? No. Does `get ( )` transfer ownership? No. Does `release ( )` close the handle? No. Why not retry Linux `close` after an error? The number may already be available for reuse. Does a destructor prove durable output? No. What makes the post-unwind validity check meaningful here? The fixture controls subsequent acquisitions and therefore the descriptor history.

Completion criterion. Deliver the ownership state table, native lifetime tests, and a simulated-error test proving that cleanup is attempted only according to the platform policy. The wrapper should make release responsibility visible on both success and failure paths.

Primary reference notes

Technical references: [POSIXCLOSE](#) [POSIXIO](#) [GUIDELINES](#)

Chapter 10 — Designing Custom Memory Management

Custom memory management is justified by a workload contract, not by the belief that ordinary allocation is inherently unacceptable. A monotonic arena can serve a bounded phase and release storage together; a pool can reuse a fixed number of object slots. This chapter builds both with explicit exhaustion and lifetime rules, including generation-tagged handles that reject a stale reference after a slot is reused.

A request-processing service allocates thousands of short-lived objects, then discards them together. A phase-based arena may fit that lifetime pattern. Another service repeatedly reuses a fixed number of slots, but clients retain identifiers after release. A generation-checked pool may fit that identity problem. These are different requirements, even though both are often described as “custom memory management.”

This chapter starts with the workload and lifetime model before choosing an allocator. The examples deliberately use standard facilities where possible: a PMR monotonic resource for bounded phase storage and ordinary optional objects for a small slot pool. The goal is to make exhaustion and reuse explicit without introducing unnecessary raw-storage hazards.

10.1 Understanding Allocator Behavior

10.1.1 Requesting and Releasing Blocks

An allocation interface receives a byte count and alignment requirement and returns suitable storage or reports failure. Object construction is a related but separate step. An allocator that returns enough bytes at the wrong alignment is incorrect even before any performance measurement. Deallocation must match the allocation contract and resource lifetime.

Polymorphic memory resources separate allocation policy from compatible containers through a runtime interface. A pmr container still owns its elements, but the resource supplying its storage must outlive the container. Releasing an arena while objects still refer to its storage invalidates their backing memory regardless of whether their pointers remain numerically unchanged.

Allocation obtains storage; construction creates an object in suitable storage. Destruction ends an object’s lifetime; deallocation releases storage according to the allocation interface. A new expression normally combines allocation and initialization, while lower-level facilities can separate them. Keeping the

operations distinct is essential when an arena retains storage after individual objects are destroyed. ([NEW](#), [LIFETIME](#))

A specialized allocator should state its alignment contract, capacity policy, failure behavior, and release rules. A fixed buffer can provide a hard bound on the storage routed through it, but only when it does not silently fall back to an unbounded upstream allocator. The worked monotonic resource uses `null_memory_resource` as its upstream, so exhaustion reports failure rather than obtaining additional storage elsewhere.

An allocation request can fail even when the application's object count looks modest. Alignment padding, container capacity, earlier unreclaimed arena requests, and element-owned allocations can all affect the budget. A capacity specified in bytes is not the same as a capacity specified in logical objects. Tests should exercise both the expected workload and deliberate exhaustion.

For manual cursor arithmetic, validate available space before advancing. An expression such as `used + bytes + padding` can overflow if computed without bounds. A safer design reasons about the remaining region and uses checked alignment adjustment. Standard memory-resource facilities reduce the amount of such arithmetic you must implement yourself, but their lifetime and upstream policies still need to be understood.

Deallocation must match the allocation contract. Passing the wrong resource, size, or alignment to a low-level deallocation interface can violate its preconditions. A custom wrapper should retain enough information to perform the matching release rather than ask callers to reconstruct it from memory. When a simpler owning container already satisfies the requirement, prefer it over a custom allocator whose correctness obligations are larger than the performance benefit being sought.

10.1.2 Understanding Allocation Overhead

Allocation can involve bookkeeping, synchronization, alignment, cache effects, and calls into a broader memory manager. The cost varies by size, contention, lifetime pattern, and implementation. Measure the actual path before replacing it with a specialized allocator that introduces new restrictions.

A custom policy often trades flexibility for predictability: fixed capacity, phase-based lifetime, no individual reclamation, or limited object types. Document those restrictions as part of the API. Counting fewer calls is useful evidence, but it does not alone prove lower end-to-end latency or lower total memory use.

Allocation overhead includes more than the cost of obtaining a pointer. It can involve metadata, synchronization, size-class selection, page management, and cache effects. The workload also pays for construction, initialization, destruction, and any data movement caused by container growth. Measuring only the allocator call can miss the dominant cost of the complete operation.

A monotonic arena often makes individual deallocation cheap because storage is reclaimed as a phase. That does not make every workload cheaper. If long-lived

and short-lived objects share one arena, the long-lived objects can delay reclamation of a large amount of otherwise-dead storage. The lifetime grouping must match the application's phases, not merely the convenience of passing one resource pointer everywhere.

Container growth can consume multiple arena blocks over a phase. Destroying an old buffer may not recover its space for reuse within a monotonic resource. Reserving a known upper bound can therefore change peak arena demand substantially. However, reserving an unrealistic maximum for every request can waste more space than controlled growth. Compare the actual allocation history under representative input distributions.

A useful cost model includes peak live storage, peak retained storage, number of upstream requests, and latency distribution for allocation-heavy operations. These measures answer different questions. A pool can reduce average allocation work while still suffering occasional expensive initialization or contention. A real-time claim needs a timing argument that covers those paths, not simply a benchmark with a lower mean.

Do not let a faster allocator hide an unbounded queue or missing resource limit. If the application accepts work indefinitely, improving allocation speed may only accelerate memory exhaustion. The allocator is one layer in a resource policy that also includes admission, capacity, ownership, and reclamation. Later chapters connect those layers through bounded queues and explicit request deadlines. Here, the first engineering decision is whether the workload truly has a phase or slot structure that justifies specialized management.

Measuring Requested Size Versus Usable Allocator Size

Purpose

Allocators often reserve more space than the exact request because of size classes, alignment, and metadata. The glibc-specific observation makes that overhead visible without assuming it is portable.

Code or command

```
#include <cstdlib>
#include <iostream>
#if defined(__GLIBC__)
#include <malloc.h>
#endif

int main() {
    void* p = std::malloc(13);
    if (!p) return 1;
    std::cout << "requested=13";
#if defined(__GLIBC__)
    std::cout << " usable=" << malloc_usable_size(p);
#endif
    std::cout << "\n";
    std::free(p);
}
```

What the important parts do

``malloc(13)`` requests at least 13 usable bytes. On glibc, ``malloc_usable_size`` reports the allocator's usable block size, which may be larger. The block still must be released with ``free``.

Expected behavior or output

Every implementation prints ``requested=13``; glibc builds also print a usable size that is commonly greater than or equal to 13.

Concrete experiment

Run requests for sizes 1 through 128 and record usable sizes on glibc. Identify repeated size classes and explain why those classes can trade internal fragmentation for allocation speed.

10.2 Building Specialized Allocators

10.2.1 Implementing Memory Arenas

A monotonic arena allocates successive regions from backing storage and normally reclaims them together. Individual deallocation requests do not return each region for immediate reuse under the basic policy. This fits a phase where

many objects share a lifetime boundary, such as parsing one bounded document or constructing one frame's temporary data.

The example uses a fixed aligned byte buffer and `null_memory_resource` as the upstream resource, so exhaustion throws rather than silently allocating from the general heap. All arena-using containers are destroyed before release. A fast reset is safe only when every object and observer depending on the old phase has ended its use.

A memory arena groups allocations under a shared lifetime policy. In a monotonic resource, storage grows through a sequence of allocations and is reclaimed together by releasing the resource or ending its lifetime. Individual object destruction does not necessarily make earlier arena space available for subsequent requests. This behavior is useful when a complete request or batch has a clear end.

The worked example declares aligned backing storage, constructs a monotonic resource over it, and gives the resource a null upstream. A vector uses that resource inside a nested scope. The scope ends before `arena.release()`, ensuring that no vector still depends on the previous phase's storage. That order is the central safety argument, not the mere presence of a release call.

Releasing an arena does not call destructors for arbitrary objects previously created in its storage. Their lifetimes and cleanup obligations remain the program's responsibility. A resource that contains objects owning files or other allocations cannot replace their destruction with one raw-storage reset. Destroy the objects first, then reclaim the storage when no live users or valid borrows remain. ([MEMORY](#), [LIFETIME](#))

A phase boundary should be visible in the API. One design passes a resource into a function that returns only an owned summary, ensuring no arena-backed view escapes. Another keeps the arena inside an owner whose lifetime encloses all dependent objects. Returning a span or string view into released arena storage defeats the design even when the summary itself is small.

Test exhaustion deliberately. The example asks for a block larger than the bounded backing storage can satisfy and expects `bad_alloc`. This is a controlled failure contract, not a crash test. A real service should translate such failure into an appropriate rejected request or degraded operation while preserving cleanup. Do not silently switch to an unbounded upstream during an incident unless that fallback is a documented policy with its own memory and latency consequences.

10.2.2 Implementing Object Pools

An object pool manages a bounded set of reusable slots. It must track which objects are alive, construct and destroy them correctly, and reject allocation when no slot is available. A slot index alone can become stale after release and reuse, causing an old handle to refer to a different object.

Pairing an index with a generation counter lets the pool distinguish successive occupants. Access succeeds only when the slot is live and its generation matches.

Counter wraparound needs a policy; this example retires a slot before generation reuse rather than silently accepting an ancient handle. Pool capacity and stale-handle errors remain explicit outcomes.

An object pool reuses a collection of slots. Reuse creates an identity problem: slot three can hold one object now and a different object later. A bare index identifies a location, not a lifetime instance. If a client retains the old index, it can accidentally act on the replacement object. A generation value distinguishes successive uses of the same slot.

The teaching handle contains an index and a generation. Lookup first validates the index, then requires an occupied slot with a matching generation. Release clears the object and advances the generation. A stale handle therefore fails lookup after release, even if the same slot is immediately reused. This is a logical identity check, not a replacement for ordinary C++ pointer lifetime rules.

Generation wrap needs a policy. The worked pool refuses to acquire a slot whose generation has reached the maximum value. It retires that slot rather than wrapping to an old generation that could match a stale handle. The small fixed pool is intentionally simple; a larger implementation might use a different identity scheme, but it must explain what happens at its counter boundary.

The returned pointer from `get ( )` is still borrowed. Once the slot is released, that pointer must not be used, even though a later handle lookup would correctly reject the stale generation. Generation checks protect accesses made through the handle API; they cannot retroactively validate a raw pointer already handed out. Document the borrow lifetime and avoid retaining it across pool mutations.

The pool also bounds slot count, not necessarily all payload memory. Each stored string can own dynamic character storage, and constructing an input string can allocate before acquisition. A service requiring a strict byte budget needs bounded payloads or an allocator policy covering those allocations too. Finally, the teaching pool is not thread-safe. Concurrent acquire, release, or lookup requires external synchronization or a separately proved concurrent design; adding an atomic generation counter alone would not protect the object's lifetime and contents.

Allocating Fixed-Size Objects from a Bounded Arena

Purpose

A simple arena makes allocation policy explicit: storage is reserved once, then handed out by advancing an offset until capacity is exhausted.

Code or command

```
#include <cstdint>
#include <iostream>
#include <new>

class Arena {
    alignas(std::max_align_t) std::byte storage_[128]{};
    std::size_t used_{};
public:
    void* allocate(std::size_t n) {
        if (used_ + n > sizeof storage_) throw std::bad_alloc{};
        void* p = storage_ + used_;
        used_ += n;
        return p;
    }
};

int main() {
    Arena a;
    using Int = int;
    auto* x = new (a.allocate(sizeof(Int))) Int(42);
    std::cout << *x << "\n";
    x->~Int();
}
```

What the important parts do

`storage\_` owns a fixed 128-byte region. `allocate` performs only a bounds check and pointer bump. Placement new constructs an `int` in returned storage; because the arena does not track individual frees, object destruction and arena lifetime are separate concerns.

Expected behavior or output

The program prints `42`. Repeated allocations eventually throw `std::bad\_alloc` when the fixed arena is exhausted.

Concrete experiment

Add proper alignment adjustment before each allocation and then allocate alternating `char` and `double` objects. Verify every returned address satisfies the requested alignment.

Worked example — Use a bounded arena and reject stale pool handles

Trace the implementation

The arena is backed by a local buffer and cannot fall back to the heap. Its vector is destroyed before release, so no live vector capacity points into a reset phase. The oversized request intentionally tests the documented exhaustion path without consuming the machine's memory.

The pool stores optional<string> objects, which manage construction and destruction for each slot. Releasing the first handle destroys its string and increments the generation. A new occupant can reuse the index, but the old generation no longer matches, so the stale handle is rejected.

This pool bounds the number of string objects, not necessarily all character allocations: each std::string may allocate internally. A truly allocation-free string pool would also need bounded character storage or an appropriate allocator policy. That distinction prevents a fixed slot count from being misrepresented as a complete no-allocation guarantee.

Match storage policy to two different lifetimes

The arena demonstration has a phase boundary: all vector users disappear before release. The pool demonstration has a reuse boundary: a handle becomes stale when its slot is released. The two examples should not be blended into a vague claim that “memory is reused safely.” They prove different properties through different mechanisms.

For the arena, predict that the bounded allocation succeeds, that its users leave scope, and that an oversized request fails through the null upstream. For the pool, predict two successful acquisitions, one capacity rejection, release of the first handle, reuse of its index, and rejection of the old generation.

```
arena: acquire storage -> construct/use objects -> destroy users -> release  
phase  
pool:  acquire slot(g) -> use through handle   -> release       -> slot(g+1)
```

A stale pointer bypasses the second line's lookup checks. An escaping arena-backed view bypasses the first line's lifetime boundary. The tests should make those limits visible rather than imply that either mechanism can rescue arbitrary retained references.

Complete worked source

The current files for this example are under source/book04/ch10/. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"  
#include <array>
```

```

#include <cstddef>
#include <cstdint>
#include <limits>
#include <memory_resource>
#include <optional>
#include <string>
#include <vector>

struct Handle { std::size_t index; std::uint64_t generation; };
class StringPool {
    struct Slot { std::optional<std::string> value; std::uint64_t
generation{}; };
    std::array<Slot, 2> slots_{};
public:
    std::optional<Handle> acquire(std::string value) {
        for (std::size_t i = 0; i < slots_.size(); ++i) {
            auto& slot = slots_[i];
            if (!slot.value && slot.generation !=
std::numeric_limits<std::uint64_t>::max()) {
                slot.value.emplace(std::move(value));
                return Handle{i, slot.generation};
            }
        }
        return std::nullopt;
    }
    const std::string* get(Handle handle) const {
        if (handle.index >= slots_.size()) return nullptr;
        const auto& slot = slots_[handle.index];
        return slot.value && slot.generation == handle.generation ?
&*slot.value : nullptr;
    }
    bool release(Handle handle) {
        if (!get(handle)) return false;
        auto& slot = slots_[handle.index];
        slot.value.reset(); ++slot.generation;
        return true;
    }
};

int main() {
    alignas(std::max_align_t) std::array<std::byte, 2048> storage{};
    std::pmr::monotonic_buffer_resource arena{storage.data(), storage.size(),
std::pmr::null_memory_resource()};
    {
        std::pmr::vector<int> values{&arena};
        values.reserve(128);
        for (int i = 0; i < 128; ++i) values.push_back(i);
        CHECK(values.back() == 127);
    }
    arena.release(); // All users of the previous phase have been destroyed.
    bool exhausted = false;
    try { (void)arena.allocate(8192, alignof(std::max_align_t)); }
    catch (const std::bad_alloc&) { exhausted = true; }
    CHECK(exhausted);
    StringPool pool;
    const auto first = pool.acquire("web1"); const auto second =
pool.acquire("web2");
    CHECK(first && second && !pool.acquire("web3"));
    CHECK(*pool.get(*first) == "web1");
    CHECK(pool.release(*first) && !pool.get(*first));
    const auto replacement = pool.acquire("web3");
    CHECK(replacement && replacement->index == first->index);
    CHECK(!pool.get(*first) && *pool.get(*replacement) == "web3");
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 10
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — Reuse without a lifetime protocol

A request resets its arena while a queued response still contains a view into that storage. The response may appear correct until the next request overwrites the same bytes. The repair is to keep the arena alive through response consumption or copy the required result into an independent owner before release. Resetting later by a guessed delay is not a lifetime protocol.

A pool bug has a similar shape: an old index selects a new occupant. Add an identity generation and validate it at each handle-based access. Then review raw pointers returned by the pool, because they can outlive the generation check that originally produced them.

Another apparent success occurs when a supposedly fixed-budget resource quietly uses its default upstream after the local buffer fills. The test never sees exhaustion, but the claimed bound was never enforced. Select the upstream policy explicitly and include an oversized-request test.

A final misconception treats pool capacity as a complete memory bound while storing unbounded strings. Count both slots and payload storage. Resource limits should describe what they actually cover, not inherit reassuring names from the allocator abstraction.

Practice and implementation lab

Replace string with a fixed-capacity text type and measure whether the whole pool path avoids upstream allocations. Add double-release and invalid-index tests.

Design a phase owner that prevents arena reset while client objects remain alive. Explain whether that guarantee is enforced by types, runtime checks, or an external convention.

Solution and review notes

The fixed text type must reject excess length before copying and preserve its invariant after failure. The pool should return false for a stale double release and null for an out-of-range handle, without touching another slot.

An owner can encapsulate arena and clients in one scope or expose a phase object whose destructor ends all dependent uses. A raw reset method callable at any time relies on a fragile convention. State the enforcement mechanism instead of calling the design safe solely because it uses RAII internally.

Guided lab — Make phase and slot boundaries observable

Build a bounded batch processor using a caller-supplied monotonic resource. Its temporary data must remain inside the batch function, and its returned result must own any information that survives. Test an ordinary batch and a batch that exhausts the supplied storage. Verify that failure leaves no live object depending on a released resource.

Add a small object with an observable destructor counter to a separate lifetime exercise. Construct several objects, destroy them, and only then release the arena. Explain why arena release alone would not satisfy the destructor contract. Keep the counter's own lifetime outside the arena so the observation remains valid.

Extend the slot-pool tests with invalid indices, repeated release, stale lookup after reuse, and capacity exhaustion. Use handles for identity checks and avoid dereferencing previously borrowed pointers after mutation. To test generation exhaustion without billions of iterations, provide a narrowly scoped test seam or a reduced-width model; do not alter production state through undefined representation tricks.

Compare a reserved vector using the arena with an incrementally growing one under the same byte budget. Record which workload fits and why. The result is about allocation history under this resource, not a universal statement that reserve is always optimal.

Review reasoning. Does arena release destroy arbitrary objects? No. Does a generation check keep an already-returned raw pointer valid? No. Does a two-slot string pool necessarily bound all bytes? No. Why use a null upstream in the lab? To make exhaustion explicit. Why retire a generation-exhausted slot? To avoid making an ancient handle valid again through wraparound.

Completion criterion. Deliver one phase-lifetime diagram, one slot-identity state machine, and tests for both ordinary use and exhaustion. The implementation should fail within its stated policy rather than silently escaping the resource budget or reusing an identity ambiguously.

Primary reference notes

Technical references: [MEMORY LIFETIME GUIDELINES](#)

Chapter 11 — Performing Low-Level File and Device I/O

Low-level I/O reports byte counts and errors rather than automatically delivering complete records. A file may return fewer bytes than requested, and a successful write is not the same as durable storage. This Linux chapter writes a small private fixture, reads it through a descriptor, and maps it read-only. It does not access hardware device registers or arbitrary device files; those require separate platform-specific interfaces and authorization.

A log processor opens a file, reads a block, and treats a short read as a complete record. It passes its small demonstration because the file fits in one block. When the same parser is placed behind a pipe, records arrive in pieces and the interpretation fails. The defect is not that Linux lost bytes. The program confused one transfer operation with one application message.

This chapter separates object ownership, byte transfer, record interpretation, and storage persistence. A descriptor tells you which open resource an operation addresses. A returned count tells you how much progress occurred. A parser decides whether the accumulated bytes form a complete object. A durability policy decides when acknowledged output is expected to survive a failure. None of these responsibilities disappears when the interface is reduced to a short function call.

11.1 Working with File Descriptors

11.1.1 Opening and Closing Files

`open` selects access mode and behavior flags and returns a descriptor or an error. Creation requires an explicit permission mode, and flags such as exclusive creation can prevent accidental replacement of an existing file. `Close-on-exec` behavior limits unintended descriptor inheritance into a later process image. Adopt a successful descriptor into an owner immediately.

A path is resolved at the time of the operation and can be affected by links, directories, and concurrent changes. An earlier exists check does not make a later open exclusive. The fixture uses a known filename only inside the test runner's private temporary directory and requests exclusive creation, then unlinks that exact file while its descriptor keeps the open resource usable.

Opening a file establishes an open file description and returns a process-local descriptor referring to it. The description contains state such as the current offset and access mode; the descriptor has its own relevant flags, including `close-on-exec`. Duplicated descriptors can refer to the same open description and therefore share

an offset. Copying an integer does not create an independent open operation or an independent resource owner. ([OPEN](#))

Select access and creation flags deliberately. A reader generally does not need write access. Creating a new sensitive file calls for an explicit mode, with the understanding that the process umask can further restrict that mode. The worked example uses exclusive creation so an existing path is not silently truncated. Its fixture runner supplies a fresh temporary working directory; the filename alone would not establish a safe location in an arbitrary production directory.

Use close-on-exec at acquisition where the platform supports it. Opening first and setting the flag later leaves a window in which another thread could create a child process that inherits the resource. Linux's `O_CLOEXEC` and corresponding socket or pipe facilities avoid that specific acquisition race. The choice protects the intended process boundary; it does not automatically close copies deliberately inherited through other mechanisms.

A pathname can change after a descriptor is opened. The demonstration unlinks its temporary name while retaining the open descriptor. Subsequent descriptor operations still address the open file. This is useful for a scoped scratch file, but it should not be mistaken for secure deletion, a portable replacement strategy, or permission to remove another application's pathname. The fixture owns both the directory and the created file.

Closing releases the descriptor reference. On Linux, a close error must not be handled by blindly retrying the same integer, because the number may already have been released and reused. The supplied owner invalidates its stored descriptor before performing a checked close. It can report the error without later accidentally closing another resource. These are Linux-specific semantics; review another platform's contract before transplanting the wrapper. ([POSIXCLOSE](#))

11.1.2 Reading and Writing Raw Bytes

read and write return the number of bytes transferred, which can be smaller than requested. A loop must advance by actual progress and distinguish interruption, end-of-file, and other errors. A zero read indicates end-of-file for an ordinary stream-like file interface; a partial record followed by EOF is different from a clean end before any new record.

A successful write means bytes were accepted by the interface, not necessarily committed to durable storage. Filesystems, caches, and devices add persistence semantics. When durability is required, use the appropriate synchronization and error-checking protocol. Raw device I/O can impose additional alignment, ordering, and safety constraints that this regular-file example does not establish.

A successful read or write can transfer fewer bytes than requested. A correct loop advances by the returned count and works only on the remaining region. Returning the number of bytes actually transferred is not an inconvenience to hide: it is how the interface communicates progress across files, pipes, sockets, and other resources. The maximum request must also fit the interface's representable

count and platform limits. The book uses deliberately small bounded buffers. ([POSIXIO](#), [WRITE](#))

Distinguish an empty result from a failed operation. For a nonzero read request on a regular file or stream, zero conventionally signals end-of-file. A negative return reports an error through `errno`. Capture that error before unrelated calls can replace it. An interrupted call with no progress can often be retried, but a successful partial transfer has already changed the operation's state and must be accounted for rather than replayed from the beginning.

An exact-read helper needs more than a Boolean success notion internally. Clean end-of-stream before a new record is different from end-of-stream halfway through a header or body. The shared helper returns false when no requested bytes were obtained before EOF, and throws for a truncated partial read. Its callers must decide whether that initial EOF is expected at that protocol point. A zero-length request is already complete; it should not be interpreted as evidence that the peer closed.

Blocking behavior belongs in the contract. A loop that correctly handles partial progress can still wait forever for the remainder. A per-call timeout can be repeatedly renewed by a peer that makes tiny progress. A total deadline requires carrying one absolute end time through all the reads and waits that constitute the operation. The extended server appendix makes this distinction executable rather than claiming that the elementary helper already solves it.

Writes also have an external-effect boundary. Once a prefix has been accepted by the kernel, throwing an exception cannot retract it. A retry at a higher layer can duplicate output unless the protocol supplies identifiers, acknowledgement rules, or idempotent operations. For a file, an accepted write does not by itself prove crash durability. Define whether the application promises merely to submit bytes, to finish a logical record, or to complete a persistence protocol.

Reading and Writing Exact Byte Counts with File Descriptors

Purpose

Raw descriptor I/O operates on bytes and returns counts that must be checked. This demonstrates acquisition, exact-count write, repositioning, exact-count read, and release.

Code or command

```
#include <fcntl.h>
#include <unistd.h>
#include <array>
#include <iostream>

int main() {
    int fd = ::open("bytes.bin", O_CREAT|O_TRUNC|O_RDWR, 0600);
    if (fd < 0) return 1;
    std::array<unsigned char,4> out{1,2,3,4}, in{};
    if (::write(fd, out.data(), out.size()) != 4) return 2;
    ::lseek(fd, 0, SEEK_SET);
    if (::read(fd, in.data(), in.size()) != 4) return 3;
    ::close(fd);
    std::cout << +in[0] << ' ' << +in[3] << "\n";
}
```

What the important parts do

`write` and `read` return signed byte counts rather than C++ container sizes. `lseek` resets the shared file offset so the following read starts at byte zero. The unary plus prints byte values numerically instead of as characters.

Expected behavior or output

The program should print `1 4` and create a four-byte file.

Concrete experiment

Replace the regular file with a pipe and observe that seeking is no longer available. Explain why descriptors provide a common handle interface but not identical capabilities.

11.2 Mapping Files into Memory

11.2.1 Understanding Memory-Mapped I/O

`mmap` can map a file region into a process address space so ordinary memory accesses read or modify the mapping under its permissions and sharing mode. Mapping offsets and lengths have platform requirements, and failure is

represented by `MAP_FAILED` rather than a null pointer. A mapping has its own lifetime and must be released with `munmap`.

File-backed mappings are not the same as device memory-mapped registers. Device mappings can require special access semantics and kernel-driver coordination. Even ordinary file mappings can fault if the underlying file is truncated by another actor. Establish ownership and mutation constraints before treating a mapped region as a stable byte sequence.

Memory mapping associates a region of virtual address space with a file or other supported object. A read-only mapping lets ordinary loads access the mapped bytes, but it does not turn arbitrary bytes into arbitrary C++ objects. Alignment, object lifetime, valid representations, endianness, and format constraints still apply. Treat mapped file content as bytes and decode the file format explicitly unless a much narrower representation contract has been established. ([MMAP, LIFETIME](#))

A mapping owns a different resource from the descriptor used to create it. Closing the descriptor after a successful mapping does not normally remove the mapping on Linux, and destroying the mapping does not close an unrelated descriptor. Use separate owners with explicit lifetime relationships. Any span or string view into mapped storage is borrowed and becomes unusable when the mapping is removed, regardless of whether its pointer value still looks plausible.

Length and offset require validation. An empty file should be handled explicitly rather than passed as a zero-length mapping request. Mapping offsets must satisfy the platform's alignment requirements. A window into a large file may therefore map from an earlier aligned offset and expose only the requested subrange. Check the arithmetic that combines the aligned offset, leading adjustment, and requested length before requesting address space.

The file can change independently of your pointer. If another actor truncates a mapped file, later accesses can generate a fault such as `SIGBUS` on Linux. Bounds checked against the original size do not protect against every concurrent filesystem mutation. Choose a stability policy: a private immutable input, coordination with writers, versioned files, or a copied snapshot. The teaching example controls its own scratch file and performs no concurrent truncation.

`MAP_PRIVATE` does not mean that the application receives a permanently frozen snapshot of every future external modification. It selects private copy-on-write behavior for writes through the mapping; broader visibility and stability questions still need a documented file-sharing contract. Likewise, unmapping releases an address-space resource, not a durability acknowledgement. A writable mapping requires a separately designed synchronization and persistence policy.

11.2.2 Processing Large Files Efficiently

Mapping can avoid an explicit user-space copy in some access patterns, but page faults, locality, storage speed, and working-set size still determine performance. It is not a universal zero-cost replacement for buffered reads. Sequential streaming, random access, and repeated scans can favor different strategies.

For large files, map bounded windows or use explicit reads when that better controls address-space use and failure handling. Check file size and arithmetic before constructing offsets or spans. The small example owns an unlinked fixture with no competing writer, so its mapping remains stable for the test; production readers need an equally clear consistency model.

Mapping can simplify traversal of a large random-access input because the program indexes bytes without issuing a read call for each region. The operating system still services page faults and performs storage work. A pointer-based loop can therefore block or incur variable latency at an ordinary load. Eliminating explicit read calls from source does not eliminate I/O, errors, or the need to measure the actual workload.

Compare strategies using access pattern, retained working set, failure handling, and concurrency requirements. Sequential buffered reads provide an explicit place to handle partial transfers and impose an input-buffer bound. Mapping can make random lookup convenient but introduces mapping lifetime and file-stability requirements. A windowed mapping limits address-space exposure while adding alignment and remapping logic. None is universally the fastest or simplest for all files.

For a large-file parser, record offsets or owned results rather than retaining borrowed pointers into a window that will be replaced. An offset is meaningful only with the identity of the file version and validation of the referenced extent. An offset alone does not prove that the bytes still exist, are unchanged, or satisfy the intended record format. Treat file identity and interpretation as part of the application model.

A realistic comparison includes cold and warm cache states, input size, storage type, parser work, and peak memory. The operating-system page cache can make a repeated benchmark measure memory access rather than storage throughput. Report the workload and measurement conditions instead of labeling one interface intrinsically faster. Do not flush shared production caches to manufacture a benchmark condition.

Device I/O deserves an additional boundary. Device nodes may require special alignment, command protocols, privileges, or timing behavior, and a read or write can have effects unrelated to ordinary file storage. This chapter does not direct you to experiment on block devices or hardware control interfaces. Practice with the owned scratch file first, then consult the specific device interface and organizational controls before adapting the pattern to hardware.

Summing a File Through a Read-Only Memory Mapping

Purpose

Memory mapping exposes file bytes through virtual memory, which can simplify large-file traversal while leaving paging decisions to the operating system.

Code or command

```
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#include <cstdint>
#include <iostream>

int main() {
    int fd = ::open("mapped.bin", O_RDONLY);
    if (fd < 0) return 1;
    struct stat st{}; if (::fstat(fd, &st) != 0 || st.st_size == 0)
return 2;
    auto* p = static_cast<const unsigned char*>(::mmap(nullptr,
st.st_size, PROT_READ, MAP_PRIVATE, fd, 0));
    if (p == MAP_FAILED) return 3;
    unsigned long long sum=0; for (off_t i=0;i<st.st_size;++i) sum
+= p[i];
    ::munmap(const_cast<unsigned char*>(p),
st.st_size); ::close(fd);
    std::cout << sum << "\n";
}
```

What the important parts do

`fstat` obtains the file length, `mmap` creates a read-only private mapping, and the loop treats the mapped region as bytes. `munmap` releases the virtual mapping separately from `close`, which releases the descriptor.

Expected behavior or output

For a nonempty `mapped.bin`, the program prints the arithmetic sum of all byte values.

Concrete experiment

Create a 100 MB file and compare this traversal with a buffered `read` loop. Measure both only after dropping any claim that one result generalizes across filesystems and cache states.

Worked example — Read and map the same owned regular-file bytes

Trace the implementation

The shared Linux support header supplies the move-only descriptor owner and exact-progress read/write helpers. Its full source is included in the companion support directory. The regular file is exclusively created in the fixture directory and immediately unlinked, so the operating system can reclaim it when the remaining open references are released even if a later check throws.

The read path resets the file offset before requesting exactly the message length. The mapping path uses a separate mapping lifetime and read-only permissions. Destruction order releases the mapping before the descriptor owner, though a successful mapping can retain its backing relationship independently under the operating system's rules.

No fsync claim is made: this is a round-trip and mapping test, not a durability test. The file is private to the process, so there is no external truncation race in the fixture. Applying the same code to a mutable shared production file would require a separate consistency and fault-handling design.

Trace one file through three access methods

The example creates a new scratch file in the fixture's temporary directory and immediately removes its directory entry. Its descriptor remains owned. It then writes the exact text Harbor file bytes, resets the file offset, reads the same byte count, and verifies equality. Finally, it creates a read-only mapping of the known length and compares that view with the same expected bytes.

These operations establish three different facts: the write loop submitted the chosen bytes, the read loop retrieved them through descriptor I/O, and the mapping exposed equal bytes while its owner remained alive. They do not establish persistence after power loss, performance on a large dataset, or safety under concurrent truncation. The file is intentionally temporary and disappears when the remaining references are released.

```
owned directory -> exclusive create -> descriptor owner
|
+--> unlink name; open file remains
+--> write -> seek -> read -> compare
+--> map -> compare -> unmap
+--> close descriptor
```

Inspect construction order. If mapping construction throws, the already-constructed descriptor owner still runs its destructor. If a comparison fails, both completed owners clean up during unwinding. No success path needs an explicit sequence of manual closes scattered through branches.

The expected text contains seventeen bytes without an added newline. Lengths refer to encoded bytes, not visual character count in a general Unicode document.

Keeping this fixture ASCII makes the byte contract explicit; it is not a text-encoding design for arbitrary input.

Complete worked source

The current files for this example are under `source/book04/ch11/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include "linux_support.hpp"
#include <algorithm>
#include <array>
#include <cstdint>
#include <span>
#include <string_view>
#include <sys/mman.h>

class ReadMapping {
    void* address_{MAP_FAILED};
    std::size_t size_{};
public:
    ReadMapping(int fd, std::size_t size) : size_(size) {
        if (size == 0) throw std::invalid_argument("empty mapping");
        address_ = ::mmap(nullptr, size, PROT_READ, MAP_PRIVATE, fd, 0);
        if (address_ == MAP_FAILED) harbor_linux::fail("mmap");
    }
    ReadMapping(const ReadMapping&) = delete;
    ReadMapping& operator=(const ReadMapping&) = delete;
    ~ReadMapping() { if (address_ != MAP_FAILED) (void)::munmap(address_,
size_); }
    std::span<const std::byte> bytes() const {
        return {static_cast<const std::byte*>(address_), size_};
    }
};

int main() {
    using namespace harbor_linux;
    auto file = checked_fd(::open("chapter11.bin", O_CREAT | O_EXCL | O_RDWR |
O_CLOEXEC, 0600), "open");
    if (::unlink("chapter11.bin") < 0) fail("unlink fixture");
    const std::string_view message = "Harbor file bytes";
    write_all(file.get(), std::as_bytes(std::span{message.data(),
message.size()}));
    if (::lseek(file.get(), 0, SEEK_SET) < 0) fail("lseek");
    std::array<std::byte, 17> readback{};
    CHECK(message.size() == readback.size());
    CHECK(read_exact(file.get(), readback));
    const auto original = std::as_bytes(std::span{message.data(),
message.size()});
    CHECK(std::equal(readback.begin(), readback.end(), original.begin(),
original.end()));
    ReadMapping mapping{file.get(), message.size()};
    CHECK(std::equal(mapping.bytes().begin(), mapping.bytes().end(),
original.begin(), original.end()));
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root:

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — Correct bytes, wrong guarantee

A service writes a response file, closes it, and reports “durable.” A subsequent crash loses the update. The repair is not to call close twice. Persistence requires a filesystem-appropriate protocol that can include checking write errors, syncing file contents, performing the intended namespace operation, and syncing the relevant directory when namespace durability is required. The exact guarantee depends on the filesystem, storage stack, and failure model. ([FSYNC](#))

Another application maps a file and returns a string view while the local mapping owner is destroyed. The view is already invalid when the caller receives it. Returning an owned string or an object that retains the mapping owner makes the lifetime explicit. Merely documenting that the caller should use the view quickly does not repair the defect.

A third failure appears only on a pipe: one read returns half a logical record and the parser treats the missing fields as zero. Repair the framing and accumulation logic. Do not demand that the operating system group bytes according to the application’s struct size.

Finally, an input reader spins after EOF because its loop retries every non-full result. Zero for a nonzero request is a terminal stream condition, not a request to poll faster. Preserve the distinction between progress, EOF, recoverable interruption, temporary unavailability, and permanent error in both code and logs.

Practice and implementation lab

Add a bounded reader that distinguishes clean EOF from a truncated fixed-size record. Test both with private fixtures. Add explicit error reporting for a failed mapping without using an arbitrary enormous allocation.

Compare buffered reads and mapping for a controlled read-only dataset, including setup cost and repeated-use behavior in separate measurements.

Solution and review notes

A clean EOF before a new record can return absence; EOF after some record bytes is a format error. The shared helper already models this distinction, but the caller must interpret it according to its record contract.

A zero-length mapping request is a deterministic invalid-input test, while permission or descriptor errors can test platform failures. Performance

measurements should separate one-time mapping setup, page-fault behavior, and repeated scans, and verify identical output before comparing durations.

Guided lab — Preserve progress and classify termination

Write a small byte-reader test seam that returns a scripted sequence of chunks. Supply the same logical input as one chunk, single-byte chunks, and mixed sizes. The reconstructed result must be identical. Then end the sequence before any byte, halfway through a header, and halfway through a body. Give each termination a documented outcome rather than allowing default-initialized fields to hide truncation.

Extend the real scratch-file example with an empty-file path. The ordinary read should establish EOF; the mapping path should intentionally skip mapping and return an empty owned result. Confirm that no zero-length `mmap` call is used as the empty-file policy.

Add a mapping wrapper move test. Transfer ownership to a new wrapper, verify that the old wrapper no longer owns a region, and read only through the new owner while it is alive. Do not test invalidation by dereferencing the old view after unmapping; that would introduce undefined or faulting behavior rather than an ordinary correctness assertion.

For a design exercise, compare three requirements: temporary scratch data, a replaceable configuration file, and an append-only audit record. State the creation flags, ownership, concurrency policy, completion acknowledgement, and durability expectation for each. The scratch implementation is not automatically sufficient for either persistent case.

Review reasoning. Does one successful read imply a complete application message? No. Does unlinking an owned temporary filename immediately invalidate its open descriptor? No. Does a mapped pointer outlive its mapping owner? No. Does a successful write establish survival after power loss? Not by itself. Why preserve partial progress? Because accepted bytes are an external effect that cannot safely be replayed as though nothing happened.

Completion criterion. Deliver a transfer-state table, a tested empty-input policy, and a mapping lifetime diagram. A reviewer should be able to locate the exact point where each resource is acquired, each byte count is accepted, each borrow expires, and each promised completion guarantee is established.

Primary reference notes

Technical references: [POSIXIO](#) [MMAP](#) [POSIXCLOSE](#)

Chapter 12 — Creating and Managing Processes

A process has its own execution context and resource relationships. Starting one is not the same as calling a function in another thread. This chapter launches a controlled child image with `posix_spawn`, passes a minimal explicit environment, and interprets its exit status. The child is the same executable in a separate mode, so the test does not depend on a shell command string or interpolate untrusted text.

A maintenance tool launches another executable and sees a zero return from the launch function. It reports success before the child has read its configuration. Seconds later the child exits with an error, leaving the operator with a misleading success message and no recorded exit status. Starting work and completing work are different events, even when the interface makes launch look like a single operation.

Process management adds identity, inherited resources, environment, executable selection, and reaping to the lifetime reasoning developed for ordinary C++ objects. The parent and child do not share the same control flow after an image replacement. A parent must decide which failures it reports immediately, which results require waiting, and how the child is stopped and collected when the surrounding operation fails.

12.1 Understanding Process Creation

12.1.1 Creating Child Processes

POSIX systems provide several process-creation mechanisms. `fork` creates a child with a related copy of the parent's process state, while `spawn` interfaces combine creation with launching a selected program under specified file and attribute actions. The choice affects inherited resources, failure handling, and what code can safely run in the child before a new image is installed.

After `fork` in a multithreaded program, the child has a particularly restricted safe path before `exec` because locks held by vanished sibling threads may remain represented in memory. Avoid casually invoking complex C++ library code in that interval. `posix_spawn` is often a useful higher-level boundary when the goal is simply to launch another executable with controlled arguments and environment.

A process is an operating-system execution context with an address space, threads, credentials, open resources, and other state. Linux `fork` creates a child process whose initial memory reflects the parent, but the two normally evolve in separate address spaces. Implementation techniques such as copy-on-write can avoid

immediately copying every page; they do not make ordinary heap variables into a synchronized cross-process communication channel. ([FORK](#))

The return value distinguishes the two branches. A positive result identifies the child to the parent; zero identifies the child branch; failure is reported in the parent with an error. Each branch must close descriptor copies it does not need. Forgetting a pipe write-end copy in either process can keep the reader from observing EOF long after the intended producer has finished.

Forking a multithreaded process adds a serious restriction. Only the calling thread exists in the child, while inherited memory can reflect locks held by threads that no longer exist there. Calling ordinary allocation, C++ stream, or mutex code in that state can deadlock or violate the required post-fork constraints. Between fork and successful exec in such a child, use only operations permitted by the relevant async-signal-safety rules. A normal-looking helper can call unsafe code internally. ([SIGNALSAFE](#))

For launching an independent executable, `posix_spawn` can express the intent more directly. It takes an executable path, argument and environment vectors, and optional file actions and attributes. It still requires deliberate choices about inherited state; it is not a sandbox or a substitute for validating which program is being launched. The worked example uses a fixed program and a small explicit environment.

The parent owns a lifecycle responsibility, not merely a numeric PID. If an exception occurs after successful creation, the program must still arrange the required waiting or termination. A child can remain running after the parent abandons its local variable. Design a process owner or supervisory scope whose cleanup policy is documented; do not assume that a C++ destructor for a PID integer has any operating-system effect.

12.1.2 Replacing Process Images

An `exec-family` operation replaces the current process image with a new program on success; it does not return to the old code as an ordinary successful function call. The process identity and selected resource relationships follow platform rules, while the address space and program state are replaced. A failed exec returns an error to the old image.

Descriptor inheritance matters. Close-on-exec flags or explicit spawn file actions can prevent leaking unrelated open files and sockets into the new image. Arguments are an array of strings, not automatically a shell expression. Calling a shell adds another parsing and security boundary that should be deliberate rather than an accidental convenience.

An `exec-family` operation replaces the current process image with another program when it succeeds. It does not create an additional process by itself. The successful call does not return to the old program's next statement, because that program image has been replaced. Failure returns an error within the still-existing old image, and the child must take an explicit failure path rather than accidentally continuing parent logic.

Arguments are a vector of strings terminated by a null pointer, not one shell command line. Spaces inside an argument remain part of that argument. Shell expansion, pipelines, wildcard processing, and redirection do not occur merely because a string contains shell punctuation. Keeping arguments separate avoids a large class of quoting and injection errors, but the invoked program still interprets its own options and operands.

Executable selection needs a trust boundary. An explicit reviewed path avoids accidentally selecting a same-named program from an untrusted search directory. Search-based variants follow a path policy that must be understood. Do not construct a privileged launch by concatenating uncontrolled input into a shell command. Prefer a fixed executable and a validated argument vector, with - - where that particular tool supports an end-of-options marker.

Open descriptors survive image replacement unless close-on-exec or explicit file actions remove them. An inherited listening socket, temporary file, or secret-bearing pipe can keep resources alive or expand the child's capabilities. Inventory what the new program needs and pass only those resources deliberately. The acquisition-time close-on-exec policies from earlier chapters help make default inheritance narrower.

A launched program can fail after process creation but before performing its intended operation. POSIX spawn semantics distinguish errors reported by the parent-side call from some failures reported through the child's termination status. Therefore, "spawn returned success" is not the same as "the requested maintenance action completed." The book's example waits and decodes the status before declaring the known child outcome. ([SPAWN](#))

Forking a Child and Replacing Its Process Image

Purpose

POSIX process creation and program loading are separate operations: `fork` creates a child process context, then an `exec` function replaces that child's program image.

Code or command

```
#include <sys/wait.h>
#include <unistd.h>
#include <iostream>

int main() {
    pid_t pid = ::fork();
    if (pid == 0) {
        ::execlp("printf", "printf", "child image\\n",
static_cast<char*>(nullptr));
        _exit(127);
    }
    if (pid < 0) return 1;
    int status{}; ::waitpid(pid, &status, 0);
    std::cout << "child exited=" << WIFEXITED(status) << " code="
<< WEXITSTATUS(status) << "\\n";
}
```

What the important parts do

After `fork`, both processes continue from the same source location with different return values. The child calls `execlp`; success never returns because the image is replaced. `_exit(127)` handles only exec failure without running parent-side buffered cleanup.

Expected behavior or output

The child prints `child image`, and the parent reports a normal exit code of 0 when `printf` succeeds.

Concrete experiment

Change the executable name to one that does not exist. Observe exit code 127 and explain why the explicit failure path is necessary after `exec` returns.

12.2 Controlling Process Execution

12.2.1 Passing Environment Information

A process environment is a collection of name-value strings made available to the new program. Passing an explicit environment can avoid inheriting unrelated secrets, locale settings, or library-control variables. It also means the child does not

automatically receive PATH or other settings it might need, so use a known executable path and document required variables.

Environment data is input. Validate values before using them to select paths, allocation sizes, or commands. Do not log secret variables during diagnostics. The example passes only one harmless fixture variable and checks it in a child mode; it does not copy the entire parent environment into test output.

The environment is a collection of strings conventionally formatted as name-value pairs. It influences behavior through locale, search paths, configuration locations, loader settings, and application-specific controls. Treat it as an input surface. Inheriting the parent's entire environment is convenient, but it can unintentionally couple a background job to an interactive session or expose information the child does not need.

An explicit environment makes a fixture reproducible. The example passes `HARBOR_MODE=fixture` and has the child verify that value. It does not assume that the current shell already set the variable. A real child might need additional well-defined entries; a minimal fixture environment is not a universal policy that every external tool can run with no inherited configuration.

Environment and argument storage must remain valid for the call according to the interface contract. When adapting C++ strings to the C API, construct the owning strings first, then form the pointer vectors after any operations that could reallocate them. Do not retain `c_str()` pointers while continuing to grow or replace the underlying strings. The null terminator of each string and the null sentinel of the pointer array are distinct requirements.

Avoid modifying a shared process environment as a way to configure one child while other threads may read or launch against it. Supply an explicit environment vector to the launch interface where available. This keeps the child's configuration local to the operation and avoids turning a per-task requirement into process-global mutation. Similarly, a child working directory should be specified through a controlled mechanism rather than racing a global `chdir` in a multithreaded parent.

Arguments and environment variables are not automatically appropriate secret stores. They can be visible through process inspection, debugging, logs, or inherited state depending on permissions and platform. Select a reviewed secret-delivery mechanism, redact diagnostics, and pass the least information needed. The teaching environment contains no credentials; its purpose is to make configuration provenance observable, not to model secure credential distribution.

12.2.2 Handling Exit Status

Waiting for a child reports a status value whose fields must be interpreted with platform macros. Check whether the child exited normally before extracting its exit code; signal termination is a different outcome. A child that has finished but has not been reaped can remain as a zombie entry until the parent collects its status.

Process-launch errors and child-program errors are separate. `posix_spawn` reports its own error number directly under its interface, while a successfully launched child can later return a nonzero application status. Record both layers. A timeout or cancellation policy is also needed for general long-running children; this fixture's child performs only a bounded environment check and exits immediately.

The parent's wait operation collects information about a child's state change. For the simple completed-child case, `waitpid` can wait for a specific PID. An interrupted wait may need to be retried, preserving the same child identity. A successful wait returns encoded status information; it is not the child's plain exit code. Use the appropriate predicates before selecting a decoding macro. ([WAIT](#))

First ask whether the child exited normally. Only then interpret its normal exit status. A child terminated by a signal has a different outcome, and attempting to describe it as an ordinary application error number loses information. Logs should distinguish launch failure, normal nonzero exit, signal termination, wait failure, and timeout or cancellation initiated by the supervisor.

The example deliberately has its child exit with seven. The parent considers that the expected fixture outcome, not a general success convention for all programs. Most command-line programs conventionally use zero for success, but each tool's documented interface defines the meaning of other values. Keep the expected status in the test contract rather than assuming every nonzero status represents the same kind of defect.

A child that has terminated still needs to be reaped according to the operating system's process model. Abandoning repeated child results can accumulate zombie entries and lose operational evidence. Conversely, waiting forever for a child whose input can stall is not an acceptable complete supervisor design. A deadline requires a policy for observing progress, requesting termination, escalating when necessary, and collecting the final status.

Termination is not rollback. The child may already have written files, sent requests, or acquired external resources. A supervisor should record what was attempted and what remains uncertain after forced termination. A robust job protocol can use transaction identifiers, staged outputs, or explicit completion markers to make recovery possible. Do not describe `kill` followed by `waitpid` as proof that no partial work happened.

Passing an Environment Variable and Decoding Child Exit Status

Purpose

Process control includes the data inherited at process creation and the encoded status returned when a child terminates.

Code or command

```
#include <cstdlib>
#include <sys/wait.h>
#include <unistd.h>
#include <iostream>

int main() {
    ::setenv("DEMO_MODE", "safe", 1);
    pid_t pid = ::fork();
    if (pid == 0) {
        const char* value = std::getenv("DEMO_MODE");
        _exit(value && std::string(value)=="safe" ? 23 : 24);
    }
    int status{}; ::waitpid(pid, &status, 0);
    if (WIFEXITED(status)) std::cout << WEXITSTATUS(status) <<
    "\n";
}
```

What the important parts do

`setenv` modifies the parent environment before `fork`, so the child inherits that value. `_exit(23)` supplies a small application exit code. `waitpid` retrieves an encoded status that must be interpreted with `WIFEXITED` and `WEXITSTATUS`.

Expected behavior or output

The program prints `23` when the environment value is inherited as expected.

Concrete experiment

Move `setenv` after `fork` but only in the parent branch. Predict the child result before running and explain why post-fork parent changes do not retroactively modify the child's environment.

Worked example — Launch a controlled child image with a minimal environment

Trace the implementation

The fixture runner launches the executable through an absolute path, so `argv[0]` is a usable path for the child image. A manually launched relative path can also work

when it still resolves from the current directory, but `argv[0]` is not a universal trusted executable locator in arbitrary applications. A production launcher should receive or resolve an explicit approved path.

The argument array and environment array remain alive during `posix_spawn`. No shell parses their contents. The child checks the supplied environment and returns seven as an intentional application result; the parent verifies normal termination before interpreting that number.

`waitpid` retries only an interrupted wait and reaps the specific child. The code does not confuse a launch failure with a child exit status. Generalizing this example to an arbitrary program requires a deadline, output handling, resource policy, and cancellation/reaping strategy, none of which are implied by this short successful fixture.

Follow the launch and the result as separate phases

The program has a parent mode and a `--child` mode. Parent mode builds a small argument vector and explicit environment, invokes the same executable, and receives a PID after successful creation. Child mode checks its expected environment and exits with the declared fixture value. The parent then waits for that exact child and validates the decoded termination category and code.

The executable path comes from the fixture invocation. Running the example through its supplied runner gives it an executable path that is meaningful from the temporary working directory. In a different launcher, do not assume that an arbitrary `argv[0]` string is an absolute or independently resolvable path. Production supervisors should establish executable identity explicitly.

```
parent: prepare path/argv/env -> spawn -> PID -> wait -> decode -> verify
child:                                start -> inspect env -> exit(7)
```

`posix_spawn` reports its own error number as a return value. A wrapper must interpret that value rather than assume every failure follows a `-1` plus `errno` convention. Interfaces with similar purposes can have different error channels; inspect the actual contract when wrapping them.

The fixture demonstrates launch configuration and result collection. It does not implement a timeout, kill escalation, resource isolation, process-group supervision, or secure executable discovery. Those requirements should be separate design decisions with independent tests, not silently attributed to a short successful launch.

Complete worked source

The current files for this example are under `source/book04/ch12/`. Listings are inserted from those files when this book is assembled.

main.cpp

```

#include "check.hpp"
#include <cerrno>
#include <cstdlib>
#include <cstring>
#include <spawn.h>
#include <string_view>
#include <system_error>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char** argv) {
    if (argc == 2 && std::string_view{argv[1]} == "--child") {
        const char* mode = std::getenv("HARBOR_MODE");
        return mode && std::strcmp(mode, "fixture") == 0 ? 7 : 8;
    }
    CHECK(argc == 1);
    char child_flag[] = "--child";
    char environment_value[] = "HARBOR_MODE=fixture";
    char* child_arguments[] {argv[0], child_flag, nullptr};
    char* child_environment[] {environment_value, nullptr};
    pid_t child{};
    const int error = ::posix_spawn(&child, argv[0], nullptr, nullptr,
    child_arguments, child_environment);
    if (error != 0) throw std::system_error(error, std::generic_category(),
    "posix_spawn");
    int status{};
    pid_t waited;
    do { waited = ::waitpid(child, &status, 0); } while (waited < 0 && errno ==
    EINTR);
    if (waited < 0) throw std::system_error(errno, std::generic_category(),
    "waitpid");
    CHECK(waited == child && WIFEXITED(status));
    CHECK(WEXITSTATUS(status) == 7);
    std::cout << "child_exit=" << WEXITSTATUS(status) << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 12
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses POSIX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — The child that survived the error path

Suppose a parent launches a child, then fails to create its logging file. It throws before calling wait. The child can continue working, while the operator sees only the parent's logging error. Repair the lifecycle ownership so every successful launch enters a supervisory scope immediately. That scope must decide whether to wait, request termination, or transfer responsibility to another owner.

A second failure comes from decoding status without checking the termination category. A signal-terminated child is logged as an ordinary exit number and sent

down the wrong retry policy. Preserve the category first, then attach the corresponding detail. An exit code is meaningful only within the normal-exit branch.

Another defect appears when a developer uses the parent's global environment to configure one child. Two concurrent launches overwrite the same variable and receive each other's settings. An explicit environment vector removes that coupling without requiring a global lock around unrelated work.

Finally, a child uses C++ streams and heap allocation after fork in a multithreaded parent and hangs before exec. The code can look perfectly valid when tested in a single-threaded command-line program. The missing precondition is the post-fork execution restriction. Use a launch design that avoids unsafe child-side work, or reduce that branch to the permitted operations required to enter the new image.

Practice and implementation lab

Add a second child mode that returns a different controlled error code and test that the parent reports it distinctly. Add explicit spawn file actions to redirect child output into a private pipe.

Design a timeout policy for a child that does not exit, including how it is signaled and how its final status is reaped.

Solution and review notes

Keep launch return codes separate from the status collected after successful creation. File actions should close unused pipe ends in both processes so EOF can be observed correctly. The next chapter examines those ownership relationships.

A timeout policy should use a monotonic deadline for supervision, issue an approved termination sequence, and still wait for the child to avoid leaving a zombie. Do not treat sending a signal as proof the process has exited or that its external work has rolled back.

Guided lab — Define a complete launch outcome table

Extend the self-launch fixture with three child modes: normal success, a documented nonzero status, and rejection of an incorrect environment value. For every mode, record the parent-side launch result and the collected child result separately. The test should fail if it declares completion before waiting.

Add an invalid executable-path case. Interpret the launch interface's error channel without reading stale `errno`. Do not construct the failure by deleting or replacing an unrelated system executable; use a nonexistent path inside the fixture directory or an explicitly invalid test input.

Write a design for a timed child supervisor without immediately implementing signal escalation. Specify the deadline origin, how the parent waits without losing the ability to enforce it, what constitutes graceful termination, what happens after

a grace period, and who performs the final wait. State what partial external effects remain possible.

For inheritance review, list every descriptor the parent owns at launch. Classify each as deliberately passed, close-on-exec, or accidentally inherited. A program that cannot explain its child descriptor set is not ready to launch untrusted or privileged work. Keep the exercise within owned test resources.

Review reasoning. Does exec create a second process? Not by itself. Does successful spawn prove the child's task succeeded? No. Is a wait status directly the exit code? No. Can a parent exception automatically terminate its child? Not without an explicit owner or supervisor policy. Why avoid global environment changes? They turn local launch configuration into shared mutable process state.

Completion criterion. Deliver a launch configuration object, a result type that preserves outcome categories, and tests for expected and failed launches. The parent must have an explicit owner for each child from creation through reaping, including the paths where the parent's own work fails.

Primary reference notes

Technical references: [SPAWN](#) [WAIT](#) [POSIXCLOSE](#)

Chapter 13 — Communicating Between Processes

Processes can exchange bytes through pipes or coordinate through shared mappings, but neither mechanism removes the need for ownership and synchronization. This Linux fixture combines a small pipe message with a process-shared semaphore protecting a shared value. The child uses a tightly bounded path and exits with `_exit`; the parent closes unused endpoints and reaps the child before destroying shared synchronization state.

A producer exits, but its consumer never sees end-of-file. The parent still owns a copied write descriptor, so the pipe remains logically open for writing. Every process performed its local work correctly; the system failed because nobody drew the complete ownership graph. Interprocess communication is as much about which references remain alive as it is about moving bytes.

This chapter develops two communication models. A pipe transfers a sequence of bytes through a kernel-managed channel. Shared memory lets processes access the same underlying storage, but requires a separately supported synchronization mechanism. The worked example uses both deliberately: a pipe carries a small completion message, while a process-shared semaphore coordinates access to a value in shared storage.

13.1 Exchanging Data Through Pipes

13.1.1 Creating Anonymous Pipes

An anonymous pipe provides a read endpoint and a write endpoint. After process creation, each process may inherit both descriptors unless they are closed or configured otherwise. Every unnecessary writer must be closed for readers to observe end-of-file when real writers finish. A forgotten inherited write descriptor can keep a reader blocked even after the intended producer exits.

Pipes carry bytes, not automatically complete application messages. Atomic-write limits for small writes under specified conditions do not mean every read returns one corresponding write. Define framing and handle partial reads. This example transfers one fixed four-byte token, so the receiver requests exactly that known length and rejects truncation.

An anonymous pipe has a read endpoint and a write endpoint represented by descriptors. It is commonly created before a child process is launched so the endpoints can be inherited or assigned through launch file actions. Each process should then retain only the endpoints required by its role. Closing unused copies is part of the protocol, not merely cosmetic cleanup. ([PIPE](#))

Pipe EOF depends on the absence of remaining writer references and the consumption of buffered data. One forgotten writer in a parent, child, or executed helper can prevent EOF indefinitely. The reader cannot infer that the intended producer has finished merely because no new bytes arrived recently. Draw every inherited copy and show the point where it is closed.

A pipe is a byte stream, not an arbitrary message transport. Small writes within the platform's documented atomic-write bound can have useful interleaving properties among multiple writers, but that does not mean a read returns exactly one application record. Reads can split or combine submitted byte sequences. If records matter, define framing and accumulation independently of convenient behavior observed with tiny fixtures.

The pipe buffer has finite capacity. A blocking writer can wait when the consumer does not drain it, and a blocking reader can wait when no data is available while a writer still exists. This creates a familiar deadlock when a parent waits for a child to exit before reading all the child's output, while the child waits for buffer space before it can exit. Drain and supervise according to the actual dependency graph.

Writing after readers disappear can raise SIGPIPE and produce an error according to the process's signal policy. Do not casually change a process-wide signal disposition inside a reusable library. A pipeline application should decide and document how broken channels are reported. Socket-specific flags used later are not automatically applicable to pipe writes; the interfaces have different control options.

13.1.2 Building Producer-Consumer Process Pipelines

A producer writes data that a consumer reads, and the pipe's bounded kernel buffer naturally creates backpressure when the consumer falls behind. A process pipeline must decide how failures propagate: an early consumer exit can cause a writer error or signal, while a producer failure can produce EOF before a complete record.

Close unused endpoints immediately and define a termination protocol. Do not assume that waiting for a child while leaving its output unread is safe for arbitrarily large output; the child can block on a full pipe while the parent waits for it to exit. Bound output or supervise reading and process completion together.

A producer-consumer process pipeline separates stages that can fail independently. The producer may parse input, the consumer may transform it, and a final stage may write output. Each link needs a byte or record contract, a capacity behavior, and a termination convention. Without those definitions, a pipeline can look correct while silently dropping a final partial record or waiting for data that will never arrive.

The simplest protocol is often a bounded fixed-size control message whose complete length is known. The teaching example sends four bytes spelling done. The receiver uses exact-read logic, so a two-byte prefix followed by EOF is not accepted as completion. A real stream of variable messages needs lengths,

delimiters with escaping, or another explicit framing design. Choose the protocol before selecting helper function names.

Backpressure occurs when a downstream stage cannot accept more data. A pipe's finite buffer can naturally slow an upstream producer, but the surrounding application must still decide what happens if the downstream stage stops permanently. Waiting without a deadline can be appropriate for an interactive shell pipeline and inappropriate for a request with a service deadline. The context, not the pipe itself, determines the acceptable policy.

Failure reporting should preserve the distinction between transport completion and task success. EOF can mean the producer completed, crashed, or closed its output after an error. A separate exit status or structured terminal record may be needed. Similarly, receiving done proves that those bytes arrived; it does not automatically prove every unrelated shared-memory field has been synchronized correctly.

Avoid assuming that process separation alone creates fault containment for all resources. Processes can still share files, inherited sockets, shared memory, credentials, and external services. A failed stage may leave partial output or hold a resource needed by recovery. An operational pipeline requires ownership and cleanup for those dependencies in addition to ordinary waits and pipe closes.

Sending a Framed Message Through an Anonymous Pipe

Purpose

Anonymous pipes provide one-way byte streams between related processes. Closing unused ends is part of the protocol because it controls EOF and resource lifetime.

Code or command

```
#include <sys/wait.h>
#include <unistd.h>
#include <cstdint>
#include <iostream>

int main() {
    int fds[2]; if (::pipe(fds) != 0) return 1;
    pid_t pid = ::fork();
    if (pid == 0) {
        ::close(fds[0]); std::uint32_t value=42; ::write(fds[1],
        &value, sizeof value); ::close(fds[1]); _exit(0);
    }
    ::close(fds[1]); std::uint32_t value{}; ssize_t
    n=::read(fds[0], &value, sizeof value); ::close(fds[0]);
    ::waitpid(pid,nullptr,0); std::cout << n << ' ' << value <<
    "\n";
}
```

What the important parts do

`pipe` returns separate read and write descriptors. The child closes its read end and writes four bytes. The parent closes its write end, reads the same byte count, and waits for child completion.

Expected behavior or output

The program should print a byte count equal to `sizeof(uint32\_t)` followed by `42`.

Concrete experiment

Send two integers instead of one and intentionally perform a one-byte read first. Observe that a pipe preserves byte order but not application message boundaries.

13.2 Sharing Data Between Processes

13.2.1 Using Shared Memory

A shared mapping makes a region accessible to multiple processes under the operating system's mapping semantics. Ordinary process heap pointers are not a

portable shared-memory data format, because mappings can appear at different addresses in independently started processes. Use offsets or explicitly designed shared structures when addresses are not inherited at the same location.

The fixture maps one small structure before fork, so both processes inherit the mapping relationship. It stores only a semaphore and an integer, not standard containers with private heap allocations. A production shared-memory protocol needs versioning, initialization ownership, crash recovery, permissions, and a policy for abandoned participants.

Shared memory provides storage that multiple processes can map into their address spaces. The mappings need not have the same virtual address in each process. For independent mappings or persistent shared formats, store offsets or carefully defined identifiers rather than process-local pointers. A pointer meaningful in one address space is not a portable reference in another. The inherited anonymous mapping in the example is a narrower teaching setup.

(MMAP)

Creating storage does not automatically create every desired C++ object in it. The example establishes the lifetime of its shared record explicitly before use and performs matching cleanup after the child is finished. Layout, alignment, and initialization matter even when the storage came from a system call rather than new. Do not reinterpret an arbitrary mapped byte sequence as an object with a nontrivial runtime representation.

A shared-memory record should have a versioned layout and a clearly defined ownership policy when used beyond a small fixture. Decide which process initializes it, when other participants may attach, how they detect partial initialization, and which participant destroys synchronization objects. Unmapping one process's view does not necessarily destroy the underlying shared object or prove that every other participant has stopped using it.

Ordinary C++ containers generally contain process-local allocation state and pointers. Placing a `std::vector` object in shared memory does not automatically put its elements there or make its allocator usable from another process. A robust shared-memory container requires a specialized representation and allocation scheme whose cross-process contract is explicit. For small systems, a bounded byte-oriented record can be much easier to audit.

Crash recovery adds another layer. A process can die halfway through a multi-field update or while holding a process-shared lock. A protocol must decide whether partially updated state is detectable, repairable, or discarded. Shared memory improves the path by which bytes are accessed; it does not supply transactionality, durable storage, or automatic owner-death recovery.

13.2.2 Synchronizing Shared Access

Shared bytes do not create a happens-before relationship by themselves. Use synchronization explicitly supported across processes, such as a properly initialized process-shared POSIX semaphore or mutex. A normal `std::mutex` placed

in a mapping is not automatically a portable interprocess synchronization primitive.

The child writes the value and posts the semaphore; the parent consumes that synchronization event before reading the value. Destruction happens only after the child is reaped. Crashes can still interrupt a protocol, so production designs need timeouts and recovery rules. A synchronization object is part of a protocol, not a guarantee that every participant remains alive.

Sharing storage and synchronizing access are separate requirements. Without an appropriate interprocess synchronization mechanism, one participant can observe incomplete or conflicting updates. The example uses a semaphore initialized for process sharing in the mapped region. The writer stores the value before posting; the reader successfully acquires the corresponding synchronization event before relying on the value. ([SEMAPHORE](#))

Do not assume that every ordinary `std::mutex` or `std::atomic` object becomes a supported interprocess primitive merely because its bytes are placed in a shared mapping. The C++ threading model and implementation contracts must be distinguished from a platform's process-shared facilities. Use documented interprocess primitives and their required initialization attributes. The portable language abstraction alone does not promise arbitrary cross-process placement behavior.

Initialization and destruction have ordering requirements. A semaphore must be successfully initialized before the child can use it. Destruction must occur only after no participant can still wait on or post it. If initialization fails, cleanup must not call destruction as though initialization succeeded. Track completed acquisition stages explicitly, just as an RAII owner tracks whether it actually owns a descriptor.

A counting semaphore retains a count; it is not identical to a condition-variable notification. Nevertheless, a semaphore count still needs an application meaning. One post might mean one available record, one completed phase, or one returned capacity unit. Posting twice by mistake can let a consumer proceed without corresponding data unless the surrounding state machine prevents that inconsistency.

The fixture's process finishes quickly under controlled inputs, but its demonstration is not a general timed synchronization service. Waiting for a child or semaphore indefinitely requires an operational decision. A production protocol needs cancellation, owner failure detection, and recovery rules appropriate to the shared state. These are not established simply by a successful run on one machine.

Sharing One Counter Across Processes with mmap

Purpose

Ordinary process memory becomes copy-on-write after `fork`, so explicit shared memory is needed when both processes must observe updates to the same storage.

Code or command

```
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>
#include <iostream>

int main() {
    auto* value =
static_cast<int*>(::mmap(nullptr, sizeof(int), PROT_READ |
PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0));
    if (value == MAP_FAILED) return 1; *value = 10;
    pid_t pid = ::fork();
    if (pid == 0) { *value = 25; _exit(0); }
    ::waitpid(pid, nullptr, 0);
    std::cout << *value << "\n";
    ::munmap(value, sizeof(int));
}
```

What the important parts do

`MAP\_SHARED | MAP\_ANONYMOUS` creates a mapping inherited by the child whose writes are visible to the parent. `waitpid` supplies synchronization here: the parent does not read until the child has finished writing.

Expected behavior or output

The parent prints `25`.

Concrete experiment

Remove the `waitpid` before the read and run repeatedly. Explain why shared visibility does not by itself provide ordering or synchronization.

Worked example — Coordinate one shared value and close every unused pipe endpoint

Trace the implementation

The mapping is initialized before fork, and the semaphore is explicitly marked process-shared. The child performs no iostream output, allocation, or C++

exception work after fork; it updates the shared scalar, posts the semaphore, writes a fixed token, and calls `_exit`.

The parent closes its write endpoint immediately. It reads the small token before waiting, so this fixture cannot deadlock by waiting on a producer whose large output fills the pipe. A readiness timeout bounds the initial pipe wait, and the child path itself contains no intentional blocking on external input. This is not a general arbitrary-child supervisor; broader timeout and cancellation handling belongs in that separate design.

The parent reaps the child, consumes the semaphore event, copies the shared value, and then destroys the semaphore and mapping before asserting results. Cleanup is deliberately performed even when receipt failed. The test validates a single controlled handoff, not crash recovery, multiwriter arbitration, or a reusable shared-memory queue.

Draw the copies before running the process branches

Before fork, the parent owns two pipe descriptors and one shared mapping containing a semaphore and integer. After fork, both processes have descriptor references and access to the mapping. The child closes the pipe's read endpoint; the parent closes its write endpoint. Only then does each role have the minimal channel shape intended by the protocol.

The child writes the shared value, posts the semaphore, sends the four-byte completion token, and exits through `_exit`. The parent receives the token, collects the child, verifies the semaphore event, and reads the value. The example is deliberately launched from a single-threaded state; it is not a recipe for running arbitrary C++ logic after fork in a multithreaded service.

parent mapping -----	shared record -----	child mapping
	value + semaphore	
parent pipe reader <-----	"done" -----	child writer
unused parent writer: closed		unused child reader: closed

The pipe and semaphore have distinct jobs. The pipe checks the byte-transport path. The semaphore establishes the intended shared-state coordination. The child's collected status supplies lifecycle evidence. A reviewer should not collapse these into one vague "the processes communicated" claim.

The child uses `_exit` rather than unwinding the parent's copied C++ state or flushing inherited buffered streams as though normal parent cleanup were occurring again. That choice is deliberate in this low-level branch. It also means child-owned cleanup that must happen before exit needs to be performed explicitly or safely delegated to operating-system process teardown under the stated contract.

Complete worked source

The current files for this example are under `source/book04/ch13/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include "linux_support.hpp"
#include <array>
#include <cerrno>
#include <memory>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>

struct Shared { sem_t ready; int value; };
int main() {
    using namespace harbor_linux;
    int raw[2];
    if (::pipe2(raw, 0_CLOEXEC) < 0) fail("pipe2");
    Fd read_end{raw[0]}, write_end{raw[1]};
    void* memory = ::mmap(nullptr, sizeof(Shared), PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (memory == MAP_FAILED) fail("mmap");
    auto* shared = std::construct_at(static_cast<Shared*>(memory));
    if (::sem_init(&shared->ready, 1, 0) < 0) {
        const int error = errno; (void)::munmap(memory, sizeof(Shared)); errno =
error; fail("sem_init");
    }
    const pid_t child = ::fork();
    if (child < 0) {
        const int error = errno; (void)::sem_destroy(&shared->ready);
        (void)::munmap(memory, sizeof(Shared)); errno = error; fail("fork");
    }
    if (child == 0) {
        (void)::close(read_end.get());
        shared->value = 42;
        if (::sem_post(&shared->ready) < 0) ::_exit(3);
        const char message[4]{'d', 'o', 'n', 'e'};
        std::size_t offset = 0;
        while (offset < sizeof(message)) {
            const auto count = ::write(write_end.get(), message + offset,
sizeof(message) - offset);
            if (count < 0 && errno == EINTR) continue;
            if (count <= 0) ::_exit(4);
            offset += static_cast<std::size_t>(count);
        }
        (void)::close(write_end.get());
        ::_exit(0); // Do not run duplicated parent stream/destructor state.
    }
    write_end.reset();
    std::array<std::byte, 4> message{};
    bool received = false;
    try { wait_readable(read_end.get()); received = read_exact(read_end.get(),
message); }
    catch (...) { received = false; }
    int status{};
    pid_t waited;
    do { waited = ::waitpid(child, &status, 0); } while (waited < 0 && errno ==
EINTR);
    const bool signaled = ::sem_trywait(&shared->ready) == 0;
    const int value = signaled ? shared->value : 0;
    const int destroyed = ::sem_destroy(&shared->ready);
    std::destroy_at(shared);
    const int unmapped = ::munmap(memory, sizeof(Shared));
    CHECK(waited == child && WIFEXITED(status) && WEXITSTATUS(status) == 0);
    CHECK(received && signaled && value == 42 && destroyed == 0 && unmapped ==
0);
}
```

```
CHECK(std::to_integer<char>(message[0]) == 'd');  
std::cout << "PASS\n";  
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 13
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — Shared bytes without shared meaning

A developer replaces the semaphore with a plain Boolean flag and adds a sleep before reading. The test passes until scheduling changes. The sleep creates no supported synchronization relationship and says nothing about which update is visible. Restore a documented synchronization primitive and a predicate or count meaning that matches the data protocol.

Another program keeps the parent's write endpoint open while waiting for the consumer to report EOF. The consumer waits because a writer still exists; the parent waits because the consumer has not finished. The repair is to close the unused reference at the branch boundary, not to increase a timeout indefinitely.

A shared record stores a pointer to a heap string allocated by one process. An independently attached process interprets the same numeric address and fails. Replace process-local pointers with a representation designed for the mapping, such as validated offsets into a bounded shared byte region, or transmit an owned copy through a channel.

A final error destroys the semaphore immediately after sending a cancellation signal to the child. Sending a signal does not prove the child has stopped accessing the shared record. Establish participant termination and collect the result before destroying shared synchronization state. Cleanup order is part of the concurrency protocol.

Practice and implementation lab

Verify all four token bytes rather than only the first. Add a child mode that closes the pipe before writing a complete token and confirm the parent reports failure while still reaping the child and releasing shared resources.

Design a reusable shared-memory record with sequence numbers and a bounded producer-consumer protocol. Specify crash and timeout outcomes before adding more writers.

Solution and review notes

A truncated token must fail the record contract, not be accepted because one leading byte matches. Cleanup should occur before reporting the failed check. A structured result can preserve the original I/O error rather than reducing every failure to a Boolean as this compact fixture does.

Sequence numbers distinguish successive records but do not provide synchronization alone. A complete protocol needs ownership of each slot, visibility rules, full/empty states, and recovery when a participant disappears. Do not infer those guarantees from one successful shared integer exchange.

Guided lab — Test ownership, progress, and terminal states

Create a two-process fixture with a fixed-size record and explicit close operations for every unused pipe endpoint. Verify the ordinary transfer and the child's normal exit status. Keep the child computation bounded and the outer fixture timeout enabled so an accidental ownership cycle produces a failed test rather than an indefinitely blocked build.

Add a deliberate early-child-exit mode before the complete record is sent. The parent must classify a partial record as truncation and still reap the child. Do not accept an all-zero tail produced by a default-initialized buffer as valid communication.

As a reasoning exercise, leave one extra write-end reference in the diagram without executing the hang. Predict why EOF cannot occur. Then identify which owner must release that reference and at what stage. Explaining the blocked dependency is more valuable than memorizing that “pipes sometimes hang.”

Design a shared-memory record without raw pointers. Include a version field, bounded payload length, state marker, and a documented synchronization operation. Explain initialization ownership, attach validation, and cleanup after participant failure. This design exercise does not claim to implement a persistent transactional shared-memory database.

Review reasoning. Does process exit by the intended writer guarantee pipe EOF if another writer reference remains? No. Does a shared mapping synchronize ordinary accesses automatically? No. Can an arbitrary standard container be placed in shared memory and used by another process? Not without a supporting representation and allocator contract. Why collect the child before destroying shared state? To establish that the participant can no longer access it.

Completion criterion. Deliver an endpoint ownership diagram, a record protocol, and tests for complete and truncated transfers. Every wait must have a stated condition that can actually become true, and every shared object's destruction must follow the end of all participant accesses.

Primary reference notes

Technical references: [PIPE](#) [MMAP](#) [SEMAPHORE](#) [WAIT](#)

Chapter 14 — Understanding Computer Networking

A network endpoint combines addressing, transport, and application protocol choices. A listening socket is not a complete service contract, and name resolution is not proof of reachability. This chapter builds a numeric loopback endpoint without contacting an external DNS server, inspects its normalized host and service strings, and opens a local listener on an operating-system-selected port.

A client reports that “the network is down.” The server is listening, but only on the loopback address of another machine. Name resolution succeeds, the route exists, and the application is healthy; the configured endpoint does not provide the path the client expected. Effective network diagnosis follows the request across layers rather than treating every failed connection as the same problem.

The chapter builds a precise vocabulary for addresses, transport endpoints, resolution, listeners, and accepted connections. Its executable uses numeric loopback addresses so the result does not depend on public DNS, external servers, firewall changes, or privileged ports. That deliberately narrow environment lets you verify the socket interface before adding infrastructure dependencies.

14.1 Understanding Network Layering

14.1.1 Understanding IP Addressing

IP addresses identify network-layer endpoints within an addressing and routing context. IPv4 and IPv6 use different address families and representations. A hostname can resolve to several addresses, and a usable client may need to try more than one result under a defined connection policy.

Loopback addresses refer to the local host’s network stack and are appropriate for isolated development fixtures. Binding to loopback differs from binding to all interfaces, which can expose a service beyond the machine. Containers and network namespaces add another scope: loopback inside one namespace is not automatically the host or another container.

An IP address identifies an interface or network endpoint within a particular addressing and routing context; it is not a permanent application identity. Addresses can change, one host can have several, and the same address value can be meaningful differently across isolated networks. A hostname adds a naming layer but does not remove the need to determine the actual address family and route used by a connection.

IPv4 and IPv6 are distinct address families with different representations and configuration concerns. Write endpoint code against the family reported by the

resolution result rather than casting every address to `sockaddr_in`. A generic address buffer such as `sockaddr_storage` can hold supported socket addresses, but its contents must still be interpreted using the actual family and returned length.

Loopback is intentionally local to the host or relevant network namespace. Binding to `127.0.0.1` makes the IPv4 examples reachable from that local context, not from another physical machine. This is a safety property for a teaching service without authentication or encryption. Replacing it with a wildcard address is a change in exposure, not a harmless portability improvement.

Routes, source-address selection, and network policy determine whether a packet can travel from one endpoint to another. Application code normally asks the operating system to perform this work rather than manually implementing routing. When a connection fails, preserve the destination, family, operation, and error category so an operator can distinguish resolution failure, unreachable paths, refused connections, and timeouts.

An address does not establish who controls the peer. Numeric equality, a successful handshake, or a reverse lookup is not authentication. A production service needs an application-appropriate identity and security design. The book's loopback fixtures deliberately avoid claiming that a bare socket connection supplies those properties; they teach transport mechanisms under an isolated, owned test boundary.

14.1.2 Understanding Ports and Transport Protocols

Ports distinguish transport endpoints for protocols such as TCP and UDP. The same numeric port under different transport protocols is not the same socket endpoint. TCP provides an ordered byte stream with connection semantics; UDP provides datagrams without the same delivery or ordering guarantee.

Network byte order is used for selected protocol fields and socket address values. Functions such as `htons` and `ntohs` convert suitable integer fields between host and network representation. They do not serialize an entire C++ object or validate an address family. Keep address construction and application message encoding as separate operations.

A transport port helps identify the application endpoint within an address and transport protocol. TCP port 8080 and UDP port 8080 are not the same transport endpoint. A service must agree with its clients about address family, address, transport, port, and application protocol. Matching only the number shown in a configuration file is insufficient.

TCP provides a reliable ordered byte-stream abstraction while a connection is functioning, but it does not preserve application write boundaries or guarantee that a remote application committed a request. UDP preserves datagram boundaries but does not provide delivery, ordering, or duplicate suppression as an application guarantee. These differences affect parser design, retry behavior, and the evidence needed to acknowledge work.

A listening TCP socket is not a connected client socket. The listener accepts connection attempts; each successful accept creates a new socket for a particular connection. The server retains the listener if it intends to accept more clients, while ownership of each accepted descriptor can move to a worker. Closing one accepted connection need not stop the listener, and closing the listener does not automatically define cleanup for every accepted socket.

Binding port zero asks the operating system to select an available local port. After binding, query the actual assigned endpoint rather than assuming zero is the port clients should use. This is useful in tests because it avoids choosing a fixed port that may already be occupied. It does not eliminate all resource limits or make concurrent tests immune to every environmental failure.

Do not infer application readiness solely from an open port. A server may listen before loading its configuration, or may accept while its worker queue is unable to make progress. Readiness requires a service-level contract. The simple example checks socket creation and binding; later chapters add framed requests and response validation to test a deeper path through the application.

Separating IP Address, Transport Protocol, and Port in an Endpoint

Purpose

The section distinguishes network-layer addressing from transport-layer ports. A `sockaddr_in` makes those fields visible without yet creating a socket.

Code or command

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <iostream>

int main() {
    sockaddr_in endpoint{};
    endpoint.sin_family = AF_INET;
    endpoint.sin_port = htons(8080);
    ::inet_pton(AF_INET, "127.0.0.1", &endpoint.sin_addr);
    char text[INET_ADDRSTRLEN]{};
    ::inet_ntop(AF_INET, &endpoint.sin_addr, text, sizeof text);
    std::cout << text << ':' << ntohs(endpoint.sin_port) << "
transport=TCP-or-UDP\n";
}
```

What the important parts do

`AF_INET` selects IPv4 addressing, `inet_pton` converts textual IP notation into binary network form, and `htons` converts the host integer 8080 to network byte order. The port alone does not determine whether TCP or UDP is used; that comes from socket type.

Expected behavior or output

The program prints `127.0.0.1:8080 transport=TCP-or-UDP`.

Concrete experiment

Change the port to 443 and then create one `SOCK_STREAM` and one `SOCK_DGRAM` socket. Explain why the same IP:port numeric pair can exist in separate transport namespaces.

14.2 Understanding Socket Communication

14.2.1 Creating Network Endpoints

Creating a socket selects an address family, socket type, and protocol relationship. A server typically binds a local address, listens for TCP connections, and accepts connected sockets; a client connects to a selected endpoint. UDP uses datagram operations and may or may not use a connected-socket convenience interface.

Every step can fail independently. A successful socket creation does not mean bind, listen, connect, or a later application request will succeed. Use explicit timeouts and resource ownership, and preserve the error at the failing operation. The companion's Linux helpers use close-on-exec descriptors and loopback-only endpoints for safe local testing.

Creating a socket selects a domain, type, and protocol combination. It returns an owned descriptor or a failure. The program must then perform the operations appropriate to its role: bind and listen for a TCP server; connect for a TCP client, or establish a local endpoint and exchange datagrams for a UDP example. The exact state machine depends on the transport. ([SOCKETS](#))

Use ownership wrappers from the moment acquisition succeeds. A bind or listen failure after socket creation still needs to close the socket. If the returned descriptor is immediately installed in a movable owner, exceptions from subsequent setup do not leak it. A wrapper should expose borrowing for system calls without making copying the native number look like copying ownership.

Byte order matters in socket structures. Port fields and certain address fields use network byte order. Use the documented conversion functions and resolver interfaces rather than assigning host integers directly into fields whose representation differs. A test that happens to use zero can conceal a missing conversion because zero has the same byte representation in either order.

A bound socket can report its effective local address through `getsockname`. A connected socket can report its peer through the corresponding peer query. Preserve the returned length and family. The operating system can fill values that were implicit at configuration time, such as a selected ephemeral port or source address. Diagnostics should report the effective state, not merely repeat the requested input.

Socket options have scope and semantics, not magical names. A timeout option may affect particular blocking calls without imposing an absolute deadline on an entire application request. An address-reuse option can change binding behavior without making two incompatible listeners a coherent service. Add only the options required by the demonstrated contract, and explain the consequences at the operation where they apply.

14.2.2 Resolving Names and Addresses

`getaddrinfo` converts a host and service description into candidate address structures under supplied hints. It reports errors through its own error-code interface rather than always through `errno`. The returned linked result list must be released with `freeaddrinfo`, and its nodes remain valid only until that release.

Numeric-only flags can prevent DNS resolution when the input is already a literal address and port. That is useful for a deterministic fixture. A real hostname-based client needs a policy for result ordering, address-family fallback, connection deadlines, and resolver failure. A successful lookup establishes candidate addresses, not that a server is listening or authorized.

Name and service resolution translates human-oriented inputs into candidate socket addresses. `getaddrinfo` returns a linked collection of results and an error channel distinct from ordinary `errno`-based system calls. Each result includes the family, socket type, protocol, address, and length needed for a compatible socket attempt. Free the returned collection using its matching release function.

([GETADDRINFO](#))

The example sets `numeric-host` and `numeric-service` flags. That makes `127.0.0.1` and `8080` explicit numeric inputs and avoids DNS or service-name lookup. Successful execution therefore establishes the numeric conversion path, not the health of a DNS resolver. This distinction belongs in the test report because a local deterministic fixture should not be presented as a network-infrastructure validation.

For a general client, resolution can produce multiple candidates. A client may need to try compatible addresses and preserve enough information to report why attempts failed. A single failing first address does not prove that every address for the name is unusable. Conversely, unbounded sequential retries can exceed the user's total deadline even when every individual attempt has a timeout.

A returned address list is owned data whose lifetime must enclose any borrowed pointers into it. Do not free the list while another component retains one of its `ai_addr` pointers. Copy the required address bytes into an owned endpoint object with a validated length, or keep the resolver result owner alive through connection setup. The same borrow principles used for vectors apply to native address structures.

Reverse or numeric formatting through `getnameinfo` is a separate operation. Numeric formatting avoids adding a reverse-DNS dependency to error reporting. A diagnostic path should not unexpectedly block on a name lookup while attempting to report an existing connection failure. Prefer bounded, explicit behavior for logs, and keep any human-friendly lookup outside critical request completion when its latency is not controlled.

Resolving localhost and Creating a Matching Socket Endpoint

Purpose

Socket communication begins with an endpoint description. `getaddrinfo` performs name/service resolution and returns parameters that can be passed directly to `socket`.

Code or command

```
#include <netdb.h>
#include <sys/socket.h>
#include <unistd.h>
#include <iostream>

int main() {
    addrinfo hints{}; hints.ai_family=AF_UNSPEC;
    hints.ai_socktype=SOCK_STREAM;
    addrinfo* result=NULLPTR;
    int rc=::getaddrinfo("localhost", "80", &hints, &result);
    if(rc!=0) return 1;
    int fd=::socket(result->ai_family, result->ai_socktype, result-
>ai_protocol);
    std::cout << "family=" << result->ai_family << " socket_ok=" <<
(fd>=0) << "\n";
    if(fd>=0) ::close(fd); ::freeaddrinfo(result);
}
```

What the important parts do

`AF_UNSPEC` permits IPv4 or IPv6, while `SOCK_STREAM` requests a TCP-style endpoint. The resolver supplies family, type, protocol, and address information; `socket` creates only the local endpoint object and does not connect it.

Expected behavior or output

The program prints the selected address family and `socket_ok=1` when endpoint creation succeeds.

Concrete experiment

Change `localhost` to the numeric service `443` and add `AI_NUMERICHOST` while keeping a hostname. Observe the resolver failure and explain what that flag requires.

Worked example — Resolve a numeric endpoint and inspect a loopback listener

Trace the implementation

The resolver receives numeric-only flags, so no external hostname lookup is needed. Its result list is immediately owned by a `unique_ptr` with the matching deleter. `getnameinfo` converts the endpoint back to numeric strings, checking representation without relying on reverse DNS.

The listener helper binds port zero on IPv4 loopback, asks the operating system for the assigned port, and starts listening. Tests therefore avoid collisions caused by every chapter hardcoding the same port. The printed port is observational and is not asserted against a fixed number.

No client connects in this chapter. A successful listener setup proves only that this local endpoint was created; it does not verify an application protocol, TLS, authentication, or external reachability. The next chapter adds actual TCP and UDP exchanges while retaining the same bounded local scope.

Verify the endpoint you actually created

The worked program resolves a numeric IPv4 address and service, formats the result back into numeric text, and verifies the expected values. It then creates a loopback TCP listener with an operating-system-selected port and checks that its socket type is the one requested. Each check corresponds to a specific layer of the construction process.

The resolver owner uses `freeaddrinfo`, not `delete`, because the allocation belongs to the native API. The socket owner uses the Linux descriptor cleanup policy. These two resources happen to appear in one small program, but their acquisition and release interfaces are unrelated. RAII unifies the lifetime structure without pretending the native resources are interchangeable.

```
numeric host + numeric service
-> address candidate with family/type/protocol
-> compatible socket
-> bind loopback:0
-> query effective selected port
-> listen for local clients
```

A successful listener setup does not include a request exchange. The next chapter adds `accept`, `connect`, framing, and response checks. Keeping the stages separate makes failures more informative: a bind error should not be described as a parser failure, and a parser rejection should not be described as a routing outage.

When running manually, record whether the program is in a container or other network namespace. Loopback belongs to that context. A client outside the namespace cannot necessarily use the same loopback endpoint even when both processes run on the same physical computer.

Complete worked source

The current files for this example are under `source/book04/ch14/`. Listings are inserted from those files when this book is assembled.

`main.cpp`

```
#include "check.hpp"
#include "linux_support.hpp"
#include <array>
#include <cstring>
#include <memory>
#include <netdb.h>
#include <string>

int main() {
    addrinfo hints{};
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_NUMERICHOST | AI_NUMERICSERV;
    addrinfo* raw = nullptr;
    const int error = ::getaddrinfo("127.0.0.1", "8080", &hints, &raw);
    if (error != 0) throw std::runtime_error(::gai_strerror(error));
    std::unique_ptr<addrinfo, decltype(&::freeaddrinfo)>
addresses{raw, ::freeaddrinfo};
    CHECK(addresses && addresses->ai_family == AF_INET);
    std::array<char, NI_MAXHOST> host{};
    std::array<char, NI_MAXSERV> service{};
    const int named = ::getnameinfo(addresses->ai_addr, addresses->ai_addrlen,
        host.data(), host.size(), service.data(), service.size(), NI_NUMERICHOST
| NI_NUMERICSERV);
    CHECK(named == 0);
    CHECK(std::string{host.data()} == "127.0.0.1" && std::string{service.data()}
== "8080");
    auto listener = harbor_linux::listen_loopback();
    CHECK(listener.port != 0);
    int type{}; socklen_t size = sizeof(type);
    CHECK(::getsockopt(listener.socket.get(), SOL_SOCKET, SO_TYPE, &type, &size)
== 0);
    CHECK(type == SOCK_STREAM);
    std::cout << "loopback_port=" << listener.port << "\nPASS\n";
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 14
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — The correct name, the wrong path

A service binds to loopback and a remote client times out. The first repair is not to disable the firewall. Establish the server's effective local address, the client's

destination, and whether those endpoints are intended to communicate across hosts. A deployment change to a non-loopback address requires a security and exposure review that the teaching fixture deliberately avoids.

Another client treats a `getaddrinfo` return code as an `errno` value and prints an unrelated error. Preserve the API-specific result and use the matching formatting function. An error-reporting helper should not assume one universal convention across all native interfaces.

A test hardcodes port 5000 and fails intermittently when another process owns it. Bind to port zero and communicate the selected port through an owned test channel. Do not “solve” the problem by terminating whatever happens to occupy the port; that process is outside the fixture’s authority.

Finally, a service performs reverse DNS inside every error log and appears to hang when the resolver is slow. Numeric diagnostic formatting removes that hidden dependency. Logging should help isolate the failing layer rather than introduce a new network operation whose failure obscures the original one.

Practice and implementation lab

Add an IPv6 numeric-resolution test for loopback and report unsupported address-family behavior explicitly. Do not silently replace an IPv6 requirement with IPv4 success.

Draw the path from name resolution through connection and application response, and identify one distinct failure that can occur at each stage.

Solution and review notes

Use the appropriate address family and numeric literal, and keep resolver success separate from the ability to create and bind an IPv6 socket in the current environment. A disabled or unavailable IPv6 stack is a platform result to report.

Resolver failure, route or connection failure, transport closure, and application rejection require different diagnostics. A single generic network error loses the evidence needed to choose the next test or recovery action.

Guided lab — Build an endpoint evidence sheet

Extend the numeric-resolution example to print the selected family, socket type, numeric address, and port without depending on external DNS. Confirm that the address object retains a correct length and remains owned for the duration of formatting and socket setup.

Add invalid numeric host and invalid numeric service cases using the numeric-only flags. The test should receive a resolver failure, not silently fall back to an external lookup. Check the error channel without asserting one platform-specific English error sentence as a portable contract.

Create two listeners bound to loopback port zero and verify that each receives a usable selected port while both are alive. Establish cleanup through the owners. Do not assert that the chosen numbers follow a particular sequence; port allocation policy is not part of the application contract.

Write a layered diagnosis for a client that cannot reach a service: input endpoint, resolution result, transport attempt, accepted connection, application framing, and application response. For each layer, identify one observation that supports a conclusion and one tempting conclusion that observation does not justify.

Review reasoning. Does numeric resolution test DNS? No. Does a listening port prove application readiness? No. Is loopback on a different machine your local loopback? No. Does a resolver result remain valid after its owner is freed? No. Why use ephemeral ports in tests? To avoid unnecessary dependence on an arbitrarily chosen fixed port.

Completion criterion. Deliver an owned endpoint representation and a diagnostic record of effective socket state. The program should clearly distinguish what the caller requested, what the resolver returned, what the operating system created, and what application behavior has actually been verified.

Primary reference notes

Technical references: [SOCKETS](#) [GETADDRINFO](#) [POSIXIO](#)

Chapter 15 — Building TCP and UDP Applications

Transport APIs move bytes or datagrams, but the application must define a message contract. A TCP read is not one message, and a UDP send is not proof of delivery. This chapter sends a binary-safe framed TCP request through a loopback server and performs a separate local UDP exchange. It keeps framing, timeout, cleanup, and thread-failure handling visible instead of hiding them behind an unbounded demonstration loop.

A TCP client sends a request with one write and expects the server to receive it with one read. The assumption survives a local demonstration, then fails under real buffering and scheduling. TCP preserves an ordered stream of bytes, not the boundaries of the application's calls. The protocol must reconstruct its own messages.

The chapter compares a framed TCP exchange with a bounded UDP datagram. Both use loopback and owned descriptors. The examples establish byte handling and endpoint behavior; they do not provide authentication, encryption, internet exposure controls, or exactly-once business operations. Those properties require additional protocols and operational decisions rather than a different socket flag.

15.1 Building Reliable TCP Connections

15.1.1 Implementing TCP Servers

A TCP server accepts a connected socket and interprets an ordered stream of bytes. The application must delimit messages, for example with a fixed header containing a bounded body length. Validate that length before allocating the body. A clean EOF before a new header differs from an EOF partway through a header or body.

The listener and accepted connection are separate resources. Closing one does not automatically substitute for managing the other. Define idle and operation timeouts, maximum message size, and shutdown behavior. A loopback teaching server with no authentication is not suitable for public exposure merely because its reads are length-checked.

A basic TCP server creates a listener, binds it to the intended local address, listens, and accepts connections. Each accepted socket represents one connection and receives its own ownership and state. A listener can remain alive while individual connections close. A server architecture should therefore separate listener lifetime from connection lifetime and from the workers that process application requests. ([ACCEPT](#), [SOCKETS](#))

Define framing before implementing the parser. The book's elementary protocol uses a four-byte unsigned length in network byte order followed by exactly that many payload bytes. The accepted maximum is 4096 bytes. The receiver validates the declared length before allocating the payload, so an oversized header does not request an arbitrary amount of memory. A length limit is a resource policy, not proof that the payload is semantically valid.

The header itself can arrive in fragments. Accumulate exactly four bytes, decode the value, validate the limit, and then accumulate the body. Clean EOF before a new header can mean no request; EOF after a partial header or before the declared body completes is truncation. A zero-length body is a complete empty frame, not a transport EOF. These distinctions are exercised more extensively in the supplemental codec tests.

A server must decide how many requests one connection carries. The teaching exchange uses one request and one response. A persistent connection needs a loop with per-message state, a total connection policy, and a clear response to malformed frames. Do not continue parsing at an arbitrary byte after a framing error unless the protocol explicitly defines resynchronization.

An accepted connection is not a trusted input source. Validate lengths, syntax, numeric bounds, and resource use. If public exposure is required, add an appropriate security design and review rather than simply binding to every interface. The chapter's loopback-only restriction is deliberate because the protocol supplies no authentication or confidentiality.

15.1.2 Implementing TCP Clients

A client chooses an endpoint, connects, sends its request framing and body, and reads the response under the same protocol. Both send and receive may make partial progress. A send success reports acceptance by the local interface, not that the peer has processed the request. Application acknowledgments or response semantics establish that later fact.

Retry policy must consider whether an operation may already have been applied. Reconnecting and blindly resending a non-idempotent request can duplicate work. This example's transformation is a pure byte reversal, so repeated requests have no external side effects, but a production update protocol would need request identity and duplicate-handling rules.

A client resolves or constructs the destination, creates a compatible socket, connects, sends a framed request, and reads the framed response. Each stage has a different failure meaning. A connection failure says the stream was not established through that attempt; a response timeout after sending leaves uncertainty about whether the server already processed the request. Retrying both cases identically can duplicate external effects.

Writes need a partial-progress loop. After a successful send of a prefix, advance the pointer and reduce the remaining count. On Linux, the helper uses `MSG_NOSIGNAL` for socket sends so a broken peer can be reported as an error without delivering `SIGPIPE` for that call. That option does not make the transfer

transactional, and it does not apply to every non-socket write interface.

[\(SOCKETSEND\)](#)

Reads require the same progress discipline. A response header can be split, the body can be empty, and the peer can close at any point. The response parser must enforce its own size bound rather than trusting a server merely because the connection was outbound. A compromised or defective peer is still a source of arbitrary bytes.

A total deadline should cover the intended operation, not be recreated every time progress occurs. The elementary socket helper applies bounded waits and per-call socket timeouts, which are useful safety measures for controlled fixtures but do not prove an absolute end-to-end request deadline. A slow peer can keep making small progress. The extended HarborRelay implementation carries a single steady-clock deadline through each framed operation.

A successful response should be validated at the application layer. The worked reverse service compares the exact returned byte string, including an embedded zero byte. It does not use C-string termination to decide payload length. For a request that changes durable state, a response would also need a documented acknowledgement meaning. “The bytes arrived” and “the transaction committed once” are different claims.

Loopback TCP Server and Client with Explicit Framing

Purpose

A reliable TCP example needs both connection establishment and an application framing decision. This loopback test uses a fixed four-byte message so the expected receive length is explicit.

Code or command

```
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>
#include <cstring>
#include <iostream>
#include <thread>

int main(){
    int server=::socket(AF_INET,SOCK_STREAM,0); sockaddr_in a{};
    a.sin_family=AF_INET; a.sin_addr.s_addr=htonl(INADDR_LOOPBACK);
    a.sin_port=0;
    if(::bind(server,reinterpret_cast<sockaddr*>(&a),sizeof a)!=0||::listen(server,1)!=0) return 1;
    socklen_t len=sizeof
a; ::getsockname(server,reinterpret_cast<sockaddr*>(&a),&len);
    std::thread client([&]{int
c=::socket(AF_INET,SOCK_STREAM,0);::connect(c,reinterpret_cast<sock
addr*>(&a),sizeof a);const char
msg[]="ping";::send(c,msg,4,0);::close(c);});
    int peer=::accept(server,nullptr,nullptr); char buf[4]{}; ssize_t
n=::recv(peer,buf,sizeof buf,MSG_WAITALL);
    std::cout.write(buf,n);
    std::cout<<"\n"; ::close(peer);::close(server);client.join();
}
```

What the important parts do

`bind` with port zero asks the kernel to choose an unused local port. `listen` and `accept` implement the server side, while the client calls `connect`. `MSG\_WAITALL` requests the full four-byte frame for this controlled demonstration; production protocols still need robust partial-I/O handling.

Expected behavior or output

The program prints `ping` without contacting any external host.

Concrete experiment

Replace the fixed message with a length-prefixed payload and implement loops around `send` and `recv` so partial transfers are handled explicitly.

15.2 Building Datagram-Based UDP Systems

15.2.1 Sending UDP Datagrams

UDP preserves datagram boundaries rather than presenting one continuous byte stream. A receive buffer must be large enough for the accepted datagram size or the application must detect truncation. A zero-length datagram is valid and must not be confused with TCP's EOF convention.

Datagrams can be lost, duplicated, or reordered, and a successful local send does not remove those possibilities. Keep messages within a justified size to avoid unnecessary fragmentation and define how the application recognizes peers and validates content. A loopback exchange is useful for testing API use but does not reproduce a lossy production network.

UDP sends discrete datagrams. Each receive obtains at most one datagram's content rather than continuing a byte stream across unrelated messages. This preserves message boundaries but introduces a separate buffer-sizing obligation: an incoming datagram can be larger than the supplied receive buffer. The application must detect truncation and reject incomplete data rather than parsing a silently shortened message. ([SOCKETREC.V](#))

The Linux example uses a truncation-aware receive path and checks the returned length against its bounded buffer. This behavior is platform-specific enough to require consulting the exact receive API and flags. Do not assume that every portable wrapper reports truncation through the same return value. A packet that does not fit is not a valid shorter packet.

A datagram can have zero payload bytes. A zero-length UDP receive is therefore not the same interpretation as zero from a nonzero stream read. The transport determines the meaning of the result. Shared helpers that flatten both interfaces into a single “zero means closed” rule are incorrect for this use.

The destination is specified for each unconnected send, and the receiver can obtain the sender address. A connected UDP socket can establish a default peer and filtering behavior without creating TCP's reliable stream semantics. The word connect in an API name does not imply identical transport guarantees across socket types.

Keep datagrams within the application and network design's practical bounds. Large messages may involve fragmentation and additional failure modes. The tiny loopback fixture avoids those issues to isolate serialization and receive behavior. It does not establish a safe production datagram size for every network, tunnel, or path MTU.

15.2.2 Handling Loss and Ordering at the Application Layer

An application can attach sequence numbers, acknowledgments, deadlines, and duplicate-detection state when its UDP use requires stronger semantics. Those mechanisms create another protocol with their own wraparound, replay, and

resource limits. A sequence number alone neither authenticates the sender nor guarantees eventual delivery.

Choose a policy appropriate to the data. A live status feed may discard late updates, while a reliable transfer needs retransmission and reassembly. Do not rebuild a complex transport protocol casually when an existing reliable protocol meets the requirement. Tests should deliberately simulate loss and reordering at a controlled boundary instead of waiting for random network failures.

UDP applications must decide what loss, duplication, and reordering mean. A telemetry display may accept occasional loss and keep the newest sample. A command protocol may require acknowledgement, bounded retries, duplicate detection, and a way to determine whether a previous command took effect. The transport does not choose those semantics for the application.

A sequence number helps identify relative order or duplicates only within a defined session and wrap policy. A number alone does not authenticate the sender or guarantee freshness across restart. A robust protocol can include a session identifier, bounded replay window, and explicit reset behavior. The chapter's sequence value is an illustrative field, not a complete reliable or secure datagram protocol.

Retries require congestion and resource consideration. An immediate unbounded resend loop can overload the very system whose responses are delayed. Define retry limits, backoff, timeout origin, and an eventual uncertain or failed outcome. Acknowledgement loss can cause a sender to retry work that the receiver already performed, so the receiver's operation semantics must tolerate duplicates or detect them.

Source-address checks are useful routing evidence but not a general authentication mechanism. The example verifies a loopback source family under a local test boundary. It does not claim that a packet source field proves the identity of an application or user. Security-sensitive use requires a reviewed protocol with appropriate integrity and identity properties.

Test message semantics independently of the real network. A deterministic model can feed packets in order, out of order, duplicated, missing, and after a simulated session change. This is more reliable than hoping a loopback test will naturally lose packets. Then use real socket fixtures to verify that the implementation integrates the model with actual datagram lengths and endpoint handling.

Detecting UDP Sequence Gaps at the Application Layer

Purpose

UDP preserves individual datagram boundaries but does not guarantee delivery or ordering. Sequence numbers are an application-level mechanism for detecting missing or reordered messages.

Code or command

```
#include <cstdint>
#include <iostream>
#include <vector>

struct Datagram { std::uint32_t sequence; int value; };

int main(){
    std::vector<Datagram> received{{10,1},{11,2},{13,4}};
    std::uint32_t expected=10;
    for(const auto& d:received){
        if(d.sequence!=expected) std::cout<<"gap expected="<<expected<<"
got="<<d.sequence<<"\n";
        expected=d.sequence+1;
    }
}
```

What the important parts do

Each datagram carries a monotonically increasing sequence. The receiver compares the observed value with the expected next value and reports a gap before advancing its expectation.

Expected behavior or output

The program reports `gap expected=12 got=13`.

Concrete experiment

Reorder the last two datagrams to 13 then 12. Extend the logic to distinguish a gap from a late/reordered packet rather than treating both conditions identically.

Worked example — Exchange a binary TCP frame and one bounded UDP datagram

Trace the implementation

The shared frame protocol uses a four-byte big-endian length and a maximum body of 4096 bytes. The request contains an embedded zero byte, so success demonstrates that framing uses explicit lengths rather than C-string termination.

The server catches exceptions inside its thread and transfers the `exception_ptr` to the joining caller; it does not let an exception escape the thread entry function.

The server result is read only after join, which synchronizes completion. Both sockets have bounded operation timeouts in the helper. The example performs one request and then exits; it is not an unbounded daemon or a claim of graceful handling for every network failure.

The UDP receive uses Linux `MSG_TRUNC` behavior to detect a datagram larger than the buffer and validates the returned byte count before constructing a string. The test verifies one local delivery, not UDP reliability. The textual sequence field is illustrative payload; no loss-recovery or authentication protocol is implemented here.

Preserve byte identity across the TCP exchange

The TCP part starts a loopback listener and a server thread. The client sends a bounded byte string that includes an embedded zero. The server receives one frame, reverses the bytes, and sends one framed result. The client compares the complete result using the string's explicit size. It never asks a zero byte to serve as a hidden message terminator.

The thread stores any exception for the supervising thread to examine after joining. A failure in a worker must not become an unobserved termination or be mistaken for a successful response. The socket owners ensure that early failures release completed acquisitions, while the outer test harness places a limit on a defective fixture's total runtime.

The UDP part uses a small datagram and a receive buffer with an explicit maximum. It verifies the payload and sender information. These are separate tests of two transport models, not evidence that UDP became reliable because the local exchange succeeded.

<pre>TCP: length header + payload -> byte stream -> accumulate -> complete frame UDP: one bounded datagram -> receive once -> verify length and source</pre>
--

For the TCP header, predict the exact four bytes for a payload of length five: three zero bytes followed by five. Then distinguish that wire representation from the host's native representation of an integer. The receiver decodes protocol bytes, not an in-memory struct dump.

Complete worked source

The current files for this example are under `source/book04/ch15/`. Listings are inserted from those files when this book is assembled.

main.cpp

<pre>#include "check.hpp" #include "linux_support.hpp" #include <algorithm> #include <array></pre>
--

```

#include <exception>
#include <string>
#include <thread>

int main() {
    using namespace harbor_linux;
    auto listener = listen_loopback();
    std::exception_ptr server_error;
    std::jthread server([&] {
        try {
            auto connection = accept_one(listener.socket.get());
            auto request = receive_frame(connection.get());
            if (!request) throw std::runtime_error("missing request");
            std::reverse(request->begin(), request->end());
            send_frame(connection.get(), *request);
        } catch (...) { server_error = std::current_exception(); }
    });
    auto client = connect_loopback(listener.port);
    const std::string request{"a\0bc", 4};
    send_frame(client.get(), request);
    const auto reply = receive_frame(client.get());
    server.join();
    if (server_error) std::rethrow_exception(server_error);
    CHECK(reply && *reply == std::string("cb\0a", 4));

    auto receiver = checked_fd(::socket(AF_INET, SOCK_DGRAM | SOCK_CLOEXEC, 0),
"UDP socket");
    auto sender = checked_fd(::socket(AF_INET, SOCK_DGRAM | SOCK_CLOEXEC, 0),
"UDP socket");
    socket_timeout(receiver.get()); socket_timeout(sender.get());
    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = 0;
    if (::bind(receiver.get(), reinterpret_cast<const sockaddr*>(&address),
sizeof(address)) < 0) fail("UDP bind");
    socklen_t address_size = sizeof(address);
    if (::getsockname(receiver.get(), reinterpret_cast<sockaddr*>(&address),
&address_size) < 0) fail("UDP name");
    const std::string datagram = "sequence=1";
    const auto sent = ::sendto(sender.get(), datagram.data(), datagram.size(),
0,
    reinterpret_cast<const sockaddr*>(&address), sizeof(address));
    CHECK(sent == static_cast<ssize_t>(datagram.size()));
    std::array<char, 64> buffer{};
    sockaddr_in peer{}; socklen_t peer_size = sizeof(peer);
    const auto received = ::recvfrom(receiver.get(), buffer.data(),
buffer.size(), MSG_TRUNC,
    reinterpret_cast<sockaddr*>(&peer), &peer_size);
    CHECK(received >= 0 && static_cast<std::size_t>(received) <= buffer.size());
    CHECK(std::string(buffer.data(), static_cast<std::size_t>(received)) ==
datagram);
    CHECK(peer.sin_family == AF_INET && peer.sin_addr.s_addr ==
htonl(INADDR_LOOPBACK));
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 15
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — A timeout that restarts forever

A service configures a five-second receive timeout and accepts a peer that sends one byte every four seconds. Every individual receive makes progress, yet the full request never finishes within five seconds. The implementation enforced a per-call inactivity policy, not the advertised total request deadline. Carry an absolute deadline through the entire header-and-body operation and recompute remaining time before each wait.

Another server assumes each receive returns one complete request. Two requests arrive together, or one request arrives in halves. The parser loses synchronization. Repair the explicit frame state machine; adding a larger buffer only changes how often the bug appears.

A UDP receiver parses a truncated packet as though it were the complete message. The missing final field becomes a default value and triggers the wrong action. Detect truncation before semantic parsing and reject the packet as a whole. For a framed TCP request, the analogous failure is EOF before the declared body length completes.

Finally, a client retries a state-changing request after losing its response. The server executes the change twice. Transport reliability does not solve application duplicate effects. Introduce an operation identifier and a documented idempotency or deduplication policy where the use case requires it; a socket reconnect alone cannot recover the missing acknowledgement meaning.

Practice and implementation lab

Add TCP tests for an empty frame, an oversized header, and a peer closing halfway through the body. Keep every server thread joined and every descriptor owned on failure paths.

Create a pure UDP-message state machine that receives controlled sequence numbers in duplicate and reordered order, and test a documented latest-update policy without relying on actual packet loss.

Solution and review notes

An empty frame is a valid zero-length body, while an oversized declared length should be rejected before allocation. Partial EOF is a framing error. Test these as separate cases so a clean connection close is not confused with malformed input.

A latest-update policy can reject sequence values older than the accepted state under a clearly bounded nonwrapping test domain. A real wrapping counter needs

a defined comparison window, and sender authentication remains separate. Pure state-machine tests make loss and ordering behavior deterministic.

Guided lab — Test fragmentation without depending on timing luck

Build a frame decoder that accepts arbitrary byte chunks. Feed a valid header and payload in every split position, including one byte at a time. The decoded payload must be identical to the original and the decoder must retain only bounded state. Include an empty frame, an embedded zero, the maximum payload, and an oversized declared length.

Test end-of-input before a header, inside the header, and inside the body. Record distinct outcomes. Do not use sleeps to make the network “probably fragment” a request; deterministic chunk input proves the parser behavior directly. A separate real-socket test can then verify integration with the receive loop.

For UDP, create a pure packet-policy exercise with sequence values and explicit session identity. Feed duplicates and reordered packets and explain whether each should be accepted, ignored, or reported. Keep security claims out of this model unless cryptographic identity is actually implemented.

Write a client retry decision table for connect failure, send failure after partial progress, response timeout, and an explicit application rejection. Mark which outcomes leave uncertainty about remote execution. This prevents an error wrapper from treating every exception as permission to repeat a non-idempotent operation.

Review reasoning. Does TCP preserve write boundaries? No. Is an empty UDP datagram an EOF signal? No. Does a per-call timeout always impose a total request deadline? No. Does a length limit authenticate the peer? No. Why test embedded zero bytes? To establish that payload length comes from the protocol rather than C-string conventions.

Completion criterion. Deliver a bounded frame state machine, a transport-versus-application error model, and a test matrix covering fragmentation and truncation. Every acknowledged result should state whether it proves byte delivery, semantic acceptance, or a stronger application-specific completion property.

Primary reference notes

Technical references: [SOCKETS](#) [POSIXIO](#) [THREADS](#)

Chapter 16 — Creating Concurrent Programs with Threads

Threads share a process address space but have independent execution progress and lifetime. That makes borrowing convenient and mistakes consequential: a worker can outlive the data it uses, or two workers can write the same object without synchronization. We will partition a bounded sum into disjoint ranges, give each worker a separate result slot, and join every worker before reading the combined result.

A sequential sum is correct, so a developer divides its loop among four threads. The new version sometimes returns a smaller value because the caller reads partial results before the workers finish. The arithmetic did not change; the program added a lifecycle and synchronization obligation that the original function never had.

Concurrency begins with ownership of work and its results. Determine which data each worker reads, which state it alone writes, how its lifetime is bounded, and what event allows the caller to consume its output. The chapter's partitioned sum is deliberately modest: it demonstrates disjoint work, bounded arithmetic, joined completion, and explicit result collection without claiming that more threads must make a small input faster.

16.1 Creating and Managing Threads

16.1.1 Starting Work with `std::thread`

Constructing `std::thread` starts an execution associated with the supplied callable when creation succeeds. The thread object manages the association, not the entire lifetime of captured data. A joinable `std::thread` must be joined or detached before destruction; destroying one while still joinable calls terminate. Detaching transfers no automatic ownership of referenced objects.

Exceptions cannot safely escape the thread entry function into the creator's call stack. Capture failures inside the worker and transfer them through an explicit result channel when needed. For a simple no-throw bounded arithmetic worker, keep the callable small and avoid hidden allocations or I/O. Thread creation itself can fail and must fit the surrounding resource-cleanup design.

A `std::thread` object represents a thread of execution after successful construction. The callable can begin running before the constructor's caller reaches its next statement. Therefore, every object captured by reference must already be initialized and remain alive for the worker's entire access period. Do

not publish a pointer to partially constructed application state simply because thread construction appears near the end of a function.

A joinable `std::thread` must be joined or detached before its destructor runs; otherwise destruction terminates the program. Joining waits for the represented thread to complete and establishes the required synchronization with that completion. Detaching removes the local ownership handle but does not make captured references live longer or arrange result collection. It is rarely an appropriate shortcut for a bounded function's worker tasks. ([THREADS](#))

Thread construction can fail. If a loop starts several workers and a later construction throws, already-started workers still exist. The surrounding scopes must clean them up while preserving every object they use. A design that is correct only after all workers have started is incomplete. Test or reason explicitly about failure at each launch position.

Exceptions do not automatically travel from a worker to the creating thread. An exception escaping a thread's initial callable results in termination. Catch within the worker and transfer an exception pointer, error value, or future result through a synchronized mechanism. The supervising thread can then join and report the failure through its ordinary error path.

A thread object and an operating-system thread identifier serve different purposes. The object expresses ownership and lifecycle operations; an identifier supports comparison and diagnostics. Do not use an observed identifier as permission to access another thread's stack or shared mutable data. Correctness comes from the program's synchronization and lifetime contracts, not from recognizing which worker probably ran first.

16.1.2 Managing Lifetime with `std::jthread`

C++20 `jthread` improves scoped lifetime management by requesting stop and joining on destruction when joinable. A worker can receive a `stop_token` and cooperate with cancellation, but requesting stop does not forcibly terminate arbitrary code. Blocking operations need their own wakeup or timeout integration.

Automatic joining prevents one common leak of execution lifetime, but it can still block indefinitely if the worker never finishes. Keep referenced data alive until the join completes and ensure shutdown can make progress. Destruction order matters when a `jthread` borrows members of an enclosing object; those members must not be destroyed before the worker stops using them.

`std::jthread` adds automatic joining and a cooperative stop mechanism to the basic thread model. When its destructor owns a joinable thread, it requests stop and joins. This simplifies ordinary scope cleanup, but it does not forcibly interrupt arbitrary work. The callable must observe the stop request or reach completion through another mechanism. A blocking native read or an unrelated condition-variable wait does not necessarily wake merely because the stop token changed. ([JTHREAD](#), [STOPTOKEN](#))

Cooperative cancellation is a protocol. Define the state changes that a stop request permits, which waits must be awakened, and whether accepted work is drained or discarded. A worker that waits on a queue may need queue closure as well as a stop request. A worker waiting on a socket may need bounded polling or an explicitly supported wakeup mechanism. Automatic join makes missing cancellation paths visible as hangs rather than repairing them.

Member and local declaration order matters. Destruction occurs in reverse order, so data used by workers must outlive the worker owners. In the partition example, partial-result storage exists before the vector of `jthreads` and remains alive until those `jthreads` have been cleared and joined. Reversing the order can leave workers accessing destroyed storage during exceptional cleanup.

The stop token should not become an excuse to skip invariants. A worker that notices cancellation halfway through updating shared state must leave that state in a documented condition. A bounded independent computation can often stop between complete units of work. A transaction-like operation may need to complete or roll back its local mutation before reporting cancellation.

For a function that returns a completed result, explicit joining before reading output is still useful even when `jthread` would join later during destruction. The return-value calculation can happen before local destructors execute. Relying on destructor timing while computing the result risks reading incomplete state. Join or otherwise synchronize at the point where the result becomes eligible for consumption.

Managing Thread Lifetime with `std::jthread` and Stop Tokens

Purpose

`std::thread` requires explicit joining or detaching, whereas `std::jthread` couples thread lifetime with automatic stop request and join behavior.

Code or command

```
#include <chrono>
#include <iostream>
#include <thread>

int main(){
    int iterations=0;
    {
        std::jthread worker([&](std::stop_token stop){
            while(!stop.stop_requested()){ ++iterations;
            std::this_thread::sleep_for(std::chrono::milliseconds(1)); }
        });
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
    }
    std::cout << (iterations>0) << "\n";
}
```

What the important parts do

The worker receives a `stop_token` and checks it cooperatively. Exiting the scope destroys the `jthread`; its destructor requests stop and joins, so `iterations` is no longer being modified when it is printed.

Expected behavior or output

The program prints `1` after at least one worker iteration.

Concrete experiment

Rewrite the program with `std::thread`. Intentionally omit `join` once in a disposable run and observe why `std::thread` lifetime management requires more care.

16.2 Dividing Work Across Threads

16.2.1 Partitioning Independent Work

A data-parallel sum can assign each worker a disjoint input interval and one private output slot. The input is read-only, and no two workers write the same scalar result object. After all workers join, the caller combines the partial sums. This structure avoids a lock on every input element.

Partition boundaries must cover the input exactly once, including uneven division and fewer elements than requested workers. Empty input should have a defined result without creating unnecessary threads. Avoid `vector<bool>` for independent Boolean result slots because its packed representation can make distinct logical elements share a storage location relevant to concurrent modification.

Independent partitioning assigns each input element to exactly one worker and gives each worker a private result slot. The caller then combines those results after all workers finish. This avoids contention on a single shared sum and makes the ownership argument straightforward: input is read-only, each partial is single-writer, and final reduction occurs after joined completion.

The quotient-and-remainder partition distributes n elements among k workers without assuming that n is divisible by k . Each worker receives either n / k elements or one additional element for the first $n \% k$ workers. Track the next beginning index and assert that the final endpoint equals n . Empty input and more requested workers than elements need explicit policies.

Arithmetic bounds are part of parallel correctness. Changing the order of integer additions is safe only when all evaluated operations stay within their types' valid range. The example bounds input size and element magnitude so the partial and final long long sums remain representable. For floating-point data, different reduction orders can change rounding, so equality with a sequential result may require a defined tolerance or reproducibility policy rather than exact bitwise identity.

Capture the assigned range by value or store it in a stable task object. Accidentally capturing the loop's mutable index by reference can make every worker use a later value or access an object after its scope ends. Capturing a span by value copies the view, not the underlying data; the source container must remain alive and unmodified in ways that invalidate the span.

Partitioning proves only the independence you actually enforce. A supposedly read-only callback that updates a hidden global cache reintroduces shared state. An element operation that accesses an external device may have ordering requirements even when the array indices are disjoint. Define the work unit's effects, not just its input range, before declaring it embarrassingly parallel.

16.2.2 Understanding Scheduling Overhead

Creating threads, scheduling them, and combining results add work. For tiny inputs, these costs can exceed any benefit from parallel arithmetic. `hardware_concurrency` is a hint that can return zero; it is not a guarantee that many threads will run simultaneously or that the process has that many available cores under container or scheduler limits.

Choose worker count from measured workload and deployment constraints. Oversubscription, memory bandwidth, and cache effects can limit scaling. This chapter tests correctness across several worker counts but does not claim a speedup. Reusing a worker pool can reduce repeated creation cost when the application has many tasks.

Starting, scheduling, synchronizing, and joining threads costs work. A tiny sum can take longer in a parallel implementation than in a sequential loop because the useful computation is smaller than the coordination overhead. This is not a failure of correctness. Performance evaluation must compare complete operation costs under representative input sizes and runtime conditions.

Requested worker count is not a guarantee of simultaneous execution. The operating system schedules runnable threads according to available resources and policy. A container can have CPU limits different from the host's physical core count. `hardware_concurrency` is a hint and can return zero; it is not a complete capacity plan or a mandate to start that many workers for every request.

Oversubscription can increase context switching and contention, especially when each incoming request creates its own set of threads. A reusable bounded worker pool can amortize creation costs and control concurrent work, but introduces queue capacity, shutdown, and task-result ownership requirements. Chapter 21 develops that architecture rather than treating a pool as a free replacement for every loop.

Measure enough repetitions to characterize variability, and keep validation outside the timed region when it does not belong to the production operation. Still verify every competing implementation produces the required result. Avoid asserting that the parallel version must be faster in a unit test: shared CI environments and small workloads make such a threshold unreliable and conceptually different from functional correctness.

Cache behavior can also change with partition size and result layout. Separate workers writing adjacent result slots may share cache lines even though the C++ accesses are race-free. That possible false-sharing cost does not invalidate the ownership design; it identifies a later optimization question. First establish correctness, then use measurements to determine whether layout or grain size is worth changing.

Partitioning a Vector into Independent Thread Ranges

Purpose

Independent partitions avoid synchronization inside the hot loop because each worker reads a disjoint range and writes a distinct result variable.

Code or command

```
#include <iostream>
#include <numeric>
#include <thread>
#include <vector>

int main(){
    std::vector<int> v(1000,1); long long left=0,right=0;
    std::jthread a([&]{left=std::accumulate(v.begin(),v.begin()+500,0LL);});
    std::jthread b([&]{right=std::accumulate(v.begin()+500,v.end(),0LL);});
    a.join(); b.join(); std::cout<<left+right<<"\n";
}
```

What the important parts do

The vector is split at element 500. Each `jthread` performs its own reduction, then the main thread joins both before combining results. The join establishes completion before the final read.

Expected behavior or output

The program prints `1000`.

Concrete experiment

Benchmark sizes 10, 1,000, and 10,000,000 against a single-threaded `accumulate`. Identify where thread startup and scheduling overhead exceed the useful parallel work.

Worked example — Partition a bounded sum and join before consuming partial results

Trace the implementation

The partition formula divides the input into a base amount per worker plus one extra element for the first remainder workers. It avoids gaps and overlaps without multiplying total input size by a worker index. The uneven length of 1003 specifically exercises remainder handling.

Each worker writes one long long element in partial. Those are distinct objects even if the hardware places several on one cache line; false sharing may affect performance but does not create the same correctness issue as two workers writing one element. The input remains alive and unchanged until all jthreads have been destroyed and joined.

The input-size bound combined with the platform's int range should be considered when generalizing the sum. On the tested common 32-bit-int platforms, one million int values fit in long long accumulation; a platform with a wider int would need a stronger explicit value bound. The supplied fixture values are small and safe independently of that broader interface consideration.

Prove the partition before measuring it

The worked function validates worker count, input size, and element magnitude. It allocates partial results, creates disjoint ranges, starts jthreads, and explicitly clears the thread vector to join before reducing the partials. Tests compare several worker counts and small boundary inputs against the same expected mathematical result.

Use a seven-element input and three workers to trace the range calculation. The quotient is two and the remainder is one, so the ranges contain three, two, and two elements. The half-open intervals touch at their boundaries without overlap. No worker accesses the one-past endpoint, and the final endpoint is exactly the input size.

```
input [0,7): [0,3) -> partial[0]
              [3,5) -> partial[1]
              [5,7) -> partial[2]
join all workers -> reduce partials -> return owned scalar
```

The vector holding the input remains unchanged while workers borrow it. Each partial slot has only one writer. The caller's reads occur after join, so it does not need an atomic for every partial result. Adding atomics everywhere would not improve the lifetime argument and could obscure the simpler synchronization already provided by completion.

The demonstration's accepted numeric bounds make its chosen accumulator sufficient. A general-purpose numeric reduction API would need a more flexible overflow and precision contract. Do not remove the validation while retaining the same correctness claim.

Complete worked source

The current files for this example are under `source/book04/ch16/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <algorithm>
#include <cstdint>
```

```

#include <numeric>
#include <span>
#include <stdexcept>
#include <thread>
#include <vector>

long long parallel_sum(std::span<const int> values, std::size_t requested) {
    if (requested == 0 || requested > 64 || values.size() > 1000000)
        throw std::invalid_argument("work bounds");
    for (int value : values)
        if (value < -1000000 || value > 1000000)
            throw std::invalid_argument("value bounds");
    if (values.empty()) return 0;
    const auto workers = std::min(requested, values.size());
    std::vector<long long> partial(workers, 0);
    std::vector<std::jthread> threads;
    threads.reserve(workers);
    for (std::size_t id = 0; id < workers; ++id) {
        const auto base = values.size() / workers;
        const auto extra = values.size() % workers;
        const auto begin = id * base + std::min(id, extra);
        const auto end = begin + base + (id < extra ? 1 : 0);
        threads.emplace_back([&, id, begin, end] {
            long long sum = 0;
            for (auto i = begin; i < end; ++i) sum += values[i];
            partial[id] = sum;
        });
    }
    threads.clear(); // jthread destruction joins before partial is read.
    return std::accumulate(partial.begin(), partial.end(), 0LL);
}

int main() {
    int observed = 0;
    std::thread simple([&] { observed = 7; });
    simple.join();
    CHECK(observed == 7);
    std::vector<int> values(1003);
    std::iota(values.begin(), values.end(), -501);
    const auto expected = std::accumulate(values.begin(), values.end(), 0LL);
    for (std::size_t workers : {1U, 2U, 3U, 8U, 64U})
        CHECK(parallel_sum(values, workers) == expected);
    CHECK(parallel_sum({}, 4) == 0);
    const std::vector<int> tiny{5, 7};
    CHECK(parallel_sum(tiny, 8) == 12);
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 16
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — Automatic joining at the wrong point

A function launches jthreads and immediately returns the sum of its partial-result vector. Local destructors join later, but the return expression already read the vector while workers were updating it. The repair is explicit completion before

result evaluation. Automatic cleanup is not a substitute for a synchronization point in the algorithm.

Another function declares its task data after its thread owner. On an exception, reverse destruction removes the task data before the worker is joined. Rearrange ownership so worker-accessed state outlives worker cleanup, or encapsulate both in an object whose destructor explicitly stops and joins before destroying the state.

A stop request is issued to a worker blocked forever on a queue whose predicate never changes. The stop token is not connected to the wait. Close the queue or use an appropriate stop-aware wait, and notify the blocked participants. The shutdown contract must explain how every wait becomes eligible to finish.

A final defect captures the launch loop index by reference. Every worker observes whatever value the loop later reaches. Capture the assigned range values, not a mutable scheduling variable. A test with many worker counts and uneven sizes is more likely to expose the error than one evenly divisible input.

Practice and implementation lab

Test invalid worker counts and values just outside the explicit per-value bounds. Explain why at most one million accepted values, each no greater than one million in magnitude, fit in the long long accumulator.

Measure serial and parallel versions across input sizes, including thread creation time. Then compare repeated tasks using a persistent worker pool.

Solution and review notes

Validate each input value against a documented magnitude and prove maximum count times magnitude fits the accumulator. Validation itself costs work and belongs either in the measured operation or an explicitly separate ingestion phase.

A persistent pool changes the measured operation by amortizing creation. Report that distinction and verify every result. More workers can lose to fewer workers when scheduling or memory traffic dominates, so no fixed speedup should be asserted in correctness tests.

Guided lab — Separate partition proof from scheduling observations

Extract the range-partition calculation into a pure function. Test zero elements, one element, uneven division, and more requested workers than elements. Verify complete coverage, non-overlap, and bounds for every returned range. These deterministic properties do not depend on thread scheduling.

Run the parallel computation over bounded positive, negative, and mixed inputs. Compare with an independent sequential calculation using the same accepted arithmetic domain. Include rejected worker counts and rejected input bounds. Keep check functions active in Release builds rather than depending only on `assert`.

Add a controlled launch-failure seam to a small worker-owner exercise. Fail before the first launch and after one successful launch. Verify that already-created workers are made able to finish and are joined while their referenced state remains alive. Avoid exhausting system threads as a way to provoke the test; deterministic injection is safer and reproducible.

For performance practice, measure a range of input sizes in a separate Release benchmark. Record compiler options, worker count, hardware context, and the complete measured operation. Report crossover behavior as an observation under those conditions, not as a universal promise that parallel execution is faster.

Review reasoning. Does copying a span copy its elements? No. Does jthread stop forcibly cancel a blocking system call? No. Can partial results be ordinary variables when each has one writer and the caller reads after join? Yes. Does race freedom guarantee no false sharing? No. Why test uneven partitions? They expose boundary assumptions hidden by divisible input sizes.

Completion criterion. Deliver a partition invariant, a lifetime diagram, and tests for partial startup failure. The result should become readable at a visible completion boundary, and shutdown should not depend on a worker noticing a condition that can never change.

Primary reference notes

Technical references: [THREADS](#) [MEMORYMODEL](#) [NUMERIC](#)

Chapter 17 — Protecting Shared Data with Mutexes and Locks

When operations share mutable state, correctness depends on protecting the entire invariant rather than only individual assignments. A transfer between two accounts must preserve the total while preventing negative balances. This chapter uses RAII locks and multi-mutex acquisition to make that transaction visible, then separates deadlock avoidance for one operation from the wider lock-order discipline of an application.

Two accounts each have a thread-safe balance accessor, yet a transfer loses money. The implementation locks one account, subtracts, unlocks it, then locks the other and adds. An observer can see an intermediate state, and a later failure can leave only half the operation applied. Protecting individual variables is not the same as protecting the invariant of a multi-object operation.

The chapter uses a bounded transfer model to connect locks to invariants. It does not model a banking system's durability, authorization, or distributed transactions. Its purpose is narrower and testable: make a two-account update indivisible with respect to the program's cooperating threads, reject invalid operations without changing balances, and acquire the required locks without creating an avoidable circular wait.

17.1 Serializing Shared Access

17.1.1 Protecting Critical Sections

A mutex allows one thread at a time to hold a protected critical section. Every access participating in the same shared invariant must follow the chosen synchronization policy. Protecting writes but reading the same ordinary state without synchronization can still create a data race. A mutex member does not enforce correct use by itself.

Keep the protected operation coherent. Checking a balance under one lock and later subtracting under another unlocked interval allows another thread to invalidate the check. The check and mutation belong to one critical section. Avoid calling unknown callbacks or slow external I/O while holding a lock unless the blocking and reentrancy consequences are deliberate.

A critical section is the region in which shared state is accessed under a synchronization discipline. The mutex is meaningful only if every conflicting access follows that discipline. Locking the writer while allowing unsynchronized readers does not make an ordinary non-atomic variable safe. Document which

mutex protects which fields and ensure that helper functions preserve the rule. ([THREADS](#), [MEMORYMODEL](#))

Start with the invariant rather than the member layout. For a transfer, each balance must remain within its permitted range and a successful operation preserves the combined total. Checking and changing those values must occur while the relevant locks are held. A funds check performed before acquiring the lock can become stale before the subtraction, even when the subtraction itself is inside a critical section.

A mutex establishes both exclusion and synchronization. Unlocking and a subsequent successful lock acquisition on that mutex provide the ordering needed for protected state to become visible according to the language model. This is not merely a scheduling convention that tells threads to be polite. Bypassing the mutex because a read “usually happens later” breaks the formal basis for the access.

The critical section should encompass the smallest coherent state transition, not necessarily the smallest number of source lines. Splitting one invariant across several separately locked fragments can be incorrect. Conversely, holding a global lock through unrelated I/O can unnecessarily block other work. Identify the commit point where validated state changes become visible together, and keep avoidable external work outside that protected transition.

Returning a reference to protected data can leak the lock discipline beyond the method boundary. A getter that locks, returns a reference, and unlocks before the caller uses it does not protect the caller’s later access. Return an owned snapshot when appropriate, or expose a carefully designed operation that retains the lock through the access. Make the ownership and synchronization scope visible in the interface.

17.1.2 Using RAII Lock Types

`lock_guard` acquires a mutex and releases it at scope exit. `unique_lock` supports additional ownership operations such as deferred locking and temporary unlock, which condition variables need. `scoped_lock` can acquire several mutexes with a deadlock-avoidance strategy for that acquisition. These types connect lock release to ordinary and exceptional control flow.

Select the least flexible lock type that expresses the operation. Manual lock and unlock pairs are easy to separate accidentally with an early return or exception. RAII prevents that cleanup mistake but does not prove that the chosen critical section is large enough, that all accesses use the mutex, or that the program’s complete lock graph is acyclic.

RAII lock types associate lock ownership with object lifetime. `lock_guard` handles a simple scope. `unique_lock` supports deferred acquisition, unlocking, relocking, and integration with condition-variable waits. `scoped_lock` can acquire multiple mutexes using a deadlock-avoidance algorithm. Select the type according to the required operation rather than using the most flexible wrapper everywhere.

A lock object's destructor releases only locks it owns. This makes exception paths less fragile than manual lock/unlock pairs spread across branches. It does not make the protected operation automatically exception-safe. If a mutation changes one field and then throws before restoring the invariant, releasing the mutex exposes the damaged state correctly synchronized but logically invalid. Plan mutation order and commit behavior as well as cleanup.

The transfer example validates amounts and destination capacity before performing its two nonthrowing arithmetic updates. Under the declared numeric bounds, the updates cannot overflow and do not allocate. Once validation succeeds while both locks are held, the commit is small and nonthrowing. This combination of locking and arithmetic reasoning is stronger than merely surrounding unsafe additions with a mutex.

Do not hold a lock across an arbitrary callback unless the contract explicitly permits reentrancy and blocking behavior. The callback can acquire other locks, call back into the same object, throw, or perform slow I/O. A safer architecture often gathers an owned event under the lock and invokes external behavior afterward, while acknowledging any ordering implications of that separation.

Moving a `unique_lock` transfers lock ownership, not ownership of the mutex object's lifetime. The mutex must outlive every lock wrapper referring to it. Likewise, a locked resource that crosses an asynchronous boundary creates a long and potentially unpredictable critical section. Favor bounded synchronous state transitions when possible, and make any longer lock lifetime an explicit reviewed design choice.

Protecting a Shared Counter with `std::scoped_lock`

Purpose

A critical section is the smallest region that must be serialized to preserve a shared invariant. RAII lock objects ensure the mutex is released on every scope exit.

Code or command

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int main(){
    std::mutex m; int counter=0; std::vector<std::jthread> threads;
    for(int t=0;t<4;++t) threads.emplace_back([&]{ for(int
i=0;i<10000;++i){ std::scoped_lock lock(m); ++counter; } });
    threads.clear();
    std::cout<<counter<<"\n";
}
```

What the important parts do

Each increment is a read-modify-write on shared state, so the mutex protects it. `std::scoped_lock` acquires in its constructor and releases in its destructor. Clearing the vector destroys and joins the `jthread` objects before the result is read.

Expected behavior or output

The program prints `40000`.

Concrete experiment

Move the lock outside the inner loop so each thread holds the mutex for all 10,000 increments. Compare correctness and runtime while explaining the change in contention granularity.

17.2 Preventing Deadlocks

17.2.1 Understanding Circular Wait

Deadlock can occur when threads each hold resources needed by the other and no one can progress. Two transfers that lock source then destination in opposite directions can create a classic circular wait. A correct individual balance update does not prevent that execution-level failure.

Deadlock is different from starvation and from a data race. A race detector does not necessarily prove the absence of deadlock, and a timeout can hide rather than repair an incorrect lock protocol. Identify every resource held while waiting for another, including locks acquired indirectly through helper calls or callbacks.

Deadlock occurs when participants cannot make progress because each waits for a resource or event that depends on another blocked participant. Circular lock acquisition is a common cause: one thread owns A and waits for B, while another owns B and waits for A. Each lock operation can be individually correct; the combined acquisition graph is not.

A two-object operation must handle identity as well as order. Transferring an account to itself would present the same mutex twice to a multi-lock operation. The example rejects that case before acquisition. More generally, normalize duplicate resources before applying an algorithm designed for distinct lockable objects. A data structure that aliases two logical names to one underlying owner can hide this case.

Deadlock is not limited to mutexes. A thread can hold a lock while waiting for a future whose producer needs that lock. A parent can wait for a child while the child waits for pipe capacity the parent is not draining. A worker pool can block every worker waiting for tasks queued to the same saturated pool. Draw wait dependencies across APIs, not only explicit lock calls.

A timeout may turn an indefinite wait into a reported failure, but it does not by itself repair the resource-ordering design or restore partially mutated state. If timed acquisition is part of the policy, define what the operation owns when it times out and how it releases or rolls back that state. Repeated immediate retries can create livelock even when no thread remains permanently blocked.

Stress tests can expose a deadlock but cannot prove its absence for all schedules. Use structural arguments: global lock order, atomic multi-lock acquisition where appropriate, no blocking callbacks under protected invariants, and explicit shutdown dependencies. Then use bounded tests and diagnostic logging to detect implementation violations of those rules.

17.2.2 Establishing Lock Ordering

A global lock order can prevent cycles when all code follows it. Alternatively, `scoped_lock` can acquire a known set of mutexes together using an appropriate deadlock-avoidance algorithm. Passing the same nonrecursive mutex twice is not a valid way to represent a self-transfer; handle that case before acquiring the pair.

An application can still deadlock if code holds some other lock while entering the transfer or if a callback acquires locks in another order. Document lock ownership at interface boundaries and avoid hidden nested acquisition. The example has only the two account locks and no callbacks or external I/O inside the critical section.

A lock-ordering policy assigns an ordering to resources and requires every operation that needs several to acquire them in that order. The order can be based

on stable logical IDs or a designed hierarchy. It must remain consistent across all code paths, including callbacks, cleanup, and error reporting. A rule documented in one function is insufficient if a lower-level helper acquires the same locks in reverse.

For a fixed pair of mutexes, `std::scoped_lock` can express coordinated acquisition without manually selecting an order. It uses the standard library's deadlock-avoidance machinery for the supplied lockables. This solves the acquisition pattern at that call site, not every possible cycle involving other resources or code invoked while the locks are held.

Avoid comparing unrelated raw object pointers as a casual portable ordering scheme without understanding the language's comparison rules and the required strict total ordering facility. Stable application identifiers are often easier to explain. Whatever ordering is chosen, it must be independent of mutable state that another thread can change while the operation is deciding which lock comes first.

Lock hierarchy should align with ownership boundaries. A component that can call arbitrary external code while holding its internal mutex cannot easily guarantee how that callback participates in the larger order. Minimize such dependencies and separate internal state capture from external notification when the semantics permit it. This is an API design issue, not just a local optimization.

A consistent snapshot spanning multiple objects may require acquiring all their locks together. Calling two individually safe getters produces two individually valid observations at potentially different times. It does not necessarily describe one simultaneous system state. The tests in this chapter inspect final balances after joining all transfer workers, which is a simpler quiescent observation boundary than a live multi-object snapshot.

Acquiring Two Mutexes Without Circular Wait

Purpose

Deadlock requires a cycle of incompatible waits. `std::scoped_lock` can acquire multiple mutexes using deadlock-avoidance logic instead of letting different code paths pick conflicting orders.

Code or command

```
#include <iostream>
#include <mutex>
#include <thread>

int main(){
    std::mutex a,b; int x=0;
    auto work=[&]{ for(int i=0;i<1000;++i){ std::scoped_lock
lock(a,b); ++x; } };
    std::jthread t1(work), t2(work); t1.join(); t2.join();
    std::cout<<x<<"\n";
}
```

What the important parts do

Both workers request the same pair in one operation. The library coordinates acquisition so one thread does not hold `a` while waiting forever for `b` as another thread does the reverse.

Expected behavior or output

The program terminates and prints `2000`.

Concrete experiment

For study only, replace `scoped_lock(a,b)` with two separate `lock_guard` objects and reverse their order in one worker. Explain the circular-wait condition even if a particular run does not deadlock.

Worked example — Preserve a two-object invariant under opposing transfers

Trace the implementation

Both account mutexes are held during validation and the paired update. The destination bound check uses one million minus amount, which is safe after validating amount, and prevents exceeding the account's invariant. Rejected transfers leave both balances unchanged.

Each worker writes only its own bool in a plain array, not `vector<bool>`. The parent reads those flags after joining. Opposing transfer directions exercise the multi-lock

acquisition path without relying on a lucky fixed scheduling order. The final balances and total verify the transaction's intended effect.

The two separate `balance()` calls would not provide one atomic snapshot while transfers were still active. They are safe individually but could observe different moments. Here all workers have joined before the combined check. A live combined snapshot would need a function that locks both accounts together.

Locate the transfer's commit point

The example rejects invalid amounts and self-transfer before taking the pair of account locks. Under both locks, it checks source funds and destination capacity. Only after both conditions hold does it subtract from one balance and add to the other. The updates are bounded integer operations with no intervening allocation or callback.

Two workers repeatedly transfer in opposite directions. The helper coordinates lock acquisition despite the reversed argument order, so neither worker relies on winning one mutex before the other. Results are collected only after joining. The final checks verify accepted operations and the expected balances under the fixture's bounded sequence.

```
validate identities/amount
-> acquire both mutexes
-> validate funds and destination range
-> debit + credit (nonthrowing commit)
-> release both mutexes
```

There is no durable storage and no network client in this model. A process crash can end the program regardless of the lock discipline, and a real financial service would require substantially more semantics. The lesson is how a local invariant determines lock scope and update order.

Read the negative paths as carefully as the success path. A rejected transfer should preserve both balances. A self-transfer is a domain decision made before passing duplicate mutexes to the acquisition operation. Validation does not merely improve error messages; it removes invalid states from the mutation path.

Complete worked source

The current files for this example are under `source/book04/ch17/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <array>
#include <mutex>
#include <stdexcept>
#include <thread>

class Account {
    mutable std::mutex mutex_;
    int balance_;
```

```

public:
    explicit Account(int balance) : balance_(balance) {
        if (balance < 0 || balance > 1000000) throw
std::invalid_argument("balance");
    }
    int balance() const { std::lock_guard lock{mutex_}; return balance_; }
    friend bool transfer(Account& from, Account& to, int amount) {
        if (&from == &to || amount <= 0 || amount > 1000000) return false;
        std::scoped_lock lock{from.mutex_, to.mutex_};
        if (from.balance_ < amount || to.balance_ > 1000000 - amount) return
false;
        from.balance_ -= amount;
        to.balance_ += amount;
        return true;
    }
};

int main() {
    Account a{10000}, b{10000};
    std::array<bool, 2> accepted{true, true};
    std::jthread first([&] {
        for (int i = 0; i < 1000; ++i) if (!transfer(a, b, 1)) accepted[0] =
false;
    });
    std::jthread second([&] {
        for (int i = 0; i < 1000; ++i) if (!transfer(b, a, 1)) accepted[1] =
false;
    });
    first.join(); second.join();
    CHECK(accepted[0] && accepted[1]);
    CHECK(a.balance() == 10000 && b.balance() == 10000);
    CHECK(!transfer(a, a, 1));
    CHECK(!transfer(a, b, 20000));
    CHECK(a.balance() + b.balance() == 20000);
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 17
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — Thread-safe parts, unsafe composition

A caller checks a source balance through a locked getter, unlocks, then invokes a separately locked subtraction. Another thread changes the balance between those operations. The fix is an atomic application operation that checks and mutates under the same required lock set. More getters do not close the check-then-act window.

Another operation performs a debit, constructs a log string that can allocate, then credits the destination. An allocation failure leaves the invariant broken even though RAII releases every mutex. Prepare potentially throwing data before commit or redesign the transition so failure cannot expose half an update. Locks and exception guarantees must be reasoned about together.

A debug logger called while holding an account mutex obtains a global log mutex, while another code path holds the log mutex and queries the account. The deadlock crosses a subsystem boundary. Moving logging outside the critical section with an owned event can remove the cycle, but review what event ordering the application requires.

Finally, a test sums balances using two separate live getters and occasionally observes an unexpected total. The transfer itself may be correct; the observation was not a simultaneous snapshot. Either acquire the needed locks together or inspect after all workers have joined. Evidence must match the invariant's observation conditions.

Practice and implementation lab

Add a combined snapshot function that returns both balances while holding both mutexes. Test total conservation while transfers run without composing two separately timed reads.

Add a documented failure channel for an unexpected worker exception rather than allowing it to escape the thread entry function.

Solution and review notes

The snapshot must acquire the same pair safely and copy both values before releasing the locks. Self-pairs need a separate case just as transfers do. A coherent snapshot is a stronger operation than two individually synchronized reads.

Catch exceptions in each worker, store an `exception_ptr` in a separately owned result slot, join, and rethrow at the calling boundary. This preserves cleanup and makes failure visible without pretending that the creator can catch an exception directly from another thread.

Guided lab — Test rejection without partial mutation

Add tests for zero and negative amounts, insufficient funds, destination overflow, and self-transfer. Record both balances before each rejected operation and verify that neither changes. Use the same public contract as ordinary callers rather than modifying private state through representation tricks.

Create a bounded generated sequence of accepted and rejected transfers and compare the final result with a sequential reference model after all worker activity has ended. Keep the model's arithmetic domain small enough that its own operations are defined. Distinguish individual operation checks from the number of independently registered test cases.

Draw a wait-for graph for a hypothetical logger callback under the account lock. Identify the reverse path that can close a cycle. Propose a design that captures an owned event under protection and publishes it afterward, and state whether event ordering changes.

Write a lock ownership comment for every shared field in a small component. Then review every returned reference, callback, and asynchronous task for ways that access might escape the protected scope. This review often finds defects that adding another stress-test loop does not reveal.

Review reasoning. Does protecting each field separately always protect a multi-field invariant? No. Does RAII locking automatically provide the strong exception guarantee? No. Can two individually safe getters form an inconsistent live snapshot? Yes. Does scoped multi-lock acquisition solve cycles involving an external future or callback? Not by itself. Why reject self-transfer before locking? To define the domain and avoid duplicate-lock acquisition.

Completion criterion. Deliver an invariant statement, a lock map, a visible nonthrowing commit point, and a rejection test matrix. The explanation should show both why conflicting accesses are synchronized and why every exposed state remains logically valid.

Primary reference notes

Technical references: [THREADS MEMORYMODEL GUIDELINES](#)

Chapter 18 — Coordinating Threads with Condition Variables

A condition variable lets a thread wait for shared state to become useful without repeatedly polling it. The state remains the source of truth; the notification is only a prompt to check again. This chapter builds a bounded queue with explicit close semantics, then verifies that accepted values are drained before a consumer observes end-of-stream and that blocked producers can stop when the queue closes.

A worker receives a notification, wakes, and finds no task. Another worker consumed the available item first. The program crashes because it treated the notification as a reservation. A condition variable does not own the shared state or assign work to a particular thread; it lets threads wait while a predicate about protected state is false.

This chapter makes a bounded queue's state machine explicit. The queue can be open, full, empty, or closed with remaining items to drain. Producers and consumers wait on predicates protected by one mutex. Closure changes state and wakes both groups. The design teaches backpressure and shutdown together because a queue that works only while every participant stays healthy is not a complete concurrent component.

18.1 Waiting for Shared State Changes

18.1.1 Waiting with Predicates

A condition-variable wait uses a `unique_lock` so it can atomically release the mutex while waiting and reacquire it before returning to inspect state. The predicate form expresses the condition under the same mutex, such as queue not empty or queue closed. This prevents a notification from being treated as a durable message independent of state.

Modify the state under its mutex, then notify appropriate waiters. Whether notification happens just before or after releasing the lock can affect scheduling, but the predicate and mutex establish correctness. Waiting on one mutex while the producer changes the relevant state under another unrelated mutex does not create the required protocol.

A condition variable is used with a mutex and a predicate over shared state. The waiting thread holds the mutex, checks whether it may proceed, and otherwise waits through an operation that releases the mutex and later reacquires it before returning. The predicate overload repeats the check as needed. The state, not the notification count, determines whether work is available. ([CONDVAR](#))

For the consumer, the predicate is typically “closed or not empty.” After waking with the mutex held, it can distinguish a remaining item from a closed-and-empty terminal state. For the producer, the predicate is “closed or there is space.” A closed queue rejects new work even if capacity is available. These predicates encode different role permissions over the same state machine.

Every field used by the predicate must follow the same synchronization discipline. Setting a plain `closed` flag outside the mutex while other threads read it inside the mutex is not sufficient. Either protect it consistently with that mutex or design a separately proved atomic protocol. Mixing approaches casually can create data races or lost state transitions.

Wait operations release the mutex while blocked so another participant can change the state. Holding an unrelated required lock across the wait can still create a deadlock if the producer needs that other lock to make the predicate true. Review the complete dependency graph, especially when queue operations are called from code already holding component locks.

The queue’s return type should express terminal state without inventing a fake data item. An optional result can distinguish a real value from closed-and-empty. An empty string, zero integer, or null-looking payload may be a legitimate item and should not be overloaded as shutdown unless the domain explicitly reserves it. Ownership transfers through the returned value when an item is successfully removed.

18.1.2 Handling Spurious Wakeups

A wait can return even when the desired state has not become true, and another awakened thread can consume the state first. Therefore recheck the predicate in a loop or use the predicate overload that performs that loop. An `if` statement followed by a bare wait is not a complete substitute.

Notifications can also occur before a waiter begins waiting. A correct state predicate handles this because the waiter sees the already changed state and need not sleep. A design that relies on counting notifications instead of storing queue or readiness state can lose work. Use a semaphore when counted permits are the intended abstraction, not an undocumented imitation with condition variables.

A condition-variable wait can wake without the predicate becoming true. Even when a notification corresponds to a real state change, another thread may consume that opportunity before this waiter reacquires the mutex. Both cases require rechecking the predicate. An `if` around a raw wait is not an adequate general replacement for a predicate loop.

Notifications are not stored as ordinary messages for future waiters. If no thread is waiting, a `notify` does not create a token that a later waiter consumes. This is safe when the actual state change remains represented in the protected queue: a later consumer sees the nonempty predicate and never needs the old notification. Designs that rely on notification alone without durable state are vulnerable to missed progress.

Notify after modifying state under the mutex discipline. It is often useful to unlock before notifying to avoid waking a thread only to have it immediately block on the same mutex, but correctness depends on the protected state transition and predicate protocol, not on a universal rule that notify must always occur inside or outside the lock. Preserve the state object's lifetime through the notification.

Use `notify_one` when one newly available opportunity should wake one suitable waiter, and `notify_all` for a global transition such as closure that may make every waiter eligible to stop. Separate producer and consumer condition variables can make the intended wake group explicit. A wrong notification target can produce a liveness defect even when all accesses are race-free.

Tests should verify behavior through state transitions rather than depend on an exact scheduler order. Arrange known preconditions, release a gate, and verify eventual bounded completion under the outer harness. A test that sleeps for a millisecond and assumes a worker is now waiting can be flaky and fail to test the intended path. Where a deterministic seam is possible, expose the state transition instead of guessing timing.

Waiting on a Predicate with `std::condition_variable`

Purpose

Condition variables separate waiting from polling. The predicate is the actual state condition; the notification is only a hint that the waiter should re-check it.

Code or command

```
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <thread>

int main(){
    std::mutex m; std::condition_variable cv; bool ready=false;
    std::jthread worker([&]{ std::unique_lock lock(m); cv.wait(lock,
    [&]{return ready;}); std::cout<<"ready\n"; });
    { std::lock_guard lock(m); ready=true; }
    cv.notify_one();
}
```

What the important parts do

`unique_lock` is required because `wait` atomically releases the mutex while sleeping and reacquires it before testing the predicate. The predicate loop built into this overload makes spurious wakeups harmless.

Expected behavior or output

The worker prints `ready` and exits.

Concrete experiment

Call `notify_one` before setting `ready`. Confirm that the predicate prevents incorrect progress, then restore the correct update-before-notify sequence.

18.2 Implementing Producer-Consumer Systems

18.2.1 Building Thread-Safe Queues

A blocking queue coordinates push and pop under one state invariant. The example stores a bounded deque, a closed flag, and two condition variables for not-empty and not-full conditions. Pop waits until an item exists or closure is visible; if closed and empty, it returns absence. Existing accepted items are drained after close.

Push rejects new values after closure. Moving a value out of the front should have a suitable exception contract so a failed move does not leave a partially consumed

logical item. This generic teaching queue requires nothrow move construction. Container allocation during push can still fail, and callers must arrange closure so other threads are not stranded when a producer fails.

A thread-safe queue must define more than mutual exclusion around a container. Its contract includes capacity, insertion failure, removal failure, closure, draining, exception behavior, and ownership transfer. The teaching queue bounds the number of queued elements, rejects pushes after closure, and lets consumers drain already-accepted items before returning a terminal empty result.

The push parameter is taken by value. That means the caller's argument may already have been copied or moved before the function discovers closure. A false return means the queue did not accept the item; it does not promise that a moved-from caller argument has been restored. This matters for unique resource owners such as accepted sockets. Choose a by-reference or returned-rejected-item design when preserving caller ownership after rejection is required.

The implementation requires nothrow move construction for its item type so removing an item and transferring it to a caller can maintain a simple state transition. The underlying deque can still allocate during insertion. A queue capacity measured in elements does not imply allocation-free operation or a strict byte bound. For a hard fixed-storage requirement, choose a bounded ring representation and define its object-lifetime behavior explicitly.

Closure is idempotent: repeated calls leave the same closed state and wake blocked participants. It is not the same as cancellation of already-running work. Once an item has left the queue, the consumer owns it and must follow its own completion or cancellation policy. The server architecture distinguishes queued connections from active connections for this reason.

Destruction requires participant quiescence. A queue object cannot safely disappear while producers or consumers still execute its methods or wait on its condition variables. The owner must close, make workers able to finish, join them, and only then destroy the queue. Declaring a queue near a worker vector is not enough; inspect reverse destruction order and partial-startup failure paths.

18.2.2 Applying Backpressure

Backpressure slows or rejects producers when downstream capacity is exhausted. A bounded queue blocks push while full, preventing unbounded memory growth from an indefinitely faster producer. The bound does not make the entire application bounded if items themselves own unbounded data or if other queues continue accumulating work.

Define shutdown and overload separately. Closing wakes both producer and consumer waiters, while temporary fullness waits for space. A production API may also need `try_push`, deadlines, cancellation, or drop policies. Those behaviors should be explicit rather than hidden as arbitrary sleeps or silent overwrites.

Backpressure connects admission to available processing capacity. A blocking bounded queue slows producers when the queue is full. A nonblocking rejection

policy returns immediately instead. A timed admission policy waits only within a declared deadline. Each can be correct for a particular system, but callers must know whether submission can block, fail, or consume their payload before acceptance.

Queue capacity is not the complete work-in-progress bound. With capacity q and w workers, up to q items can be queued while up to w more are actively processed. The accepting thread may temporarily own another item, and upstream buffers may retain additional input. A meaningful resource budget counts all these stages rather than quoting only the queue's configured size.

Blocking a producer while it holds a resource needed by consumers can deadlock the system. For example, a producer that retains a global processing lock while waiting for queue space may prevent workers from completing their current items. Define lock and ownership boundaries before calling a potentially blocking submit operation.

When a worker fails, the system must decide whether to continue with reduced capacity, reject new work, close the queue, or abort the operation. If every consumer exits while a producer waits on a full open queue, no future dequeue can make progress. A failure path that closes and notifies the queue lets the producer observe terminal state instead of blocking forever.

A bounded queue controls retained work but does not by itself control per-item latency. A single item can stall in a blocking system call while the queue remains perfectly bounded. Pair admission limits with request deadlines and cancellation paths when the service requires bounded response behavior. Even then, ordinary operating-system scheduling prevents these teaching bounds from constituting a hard real-time guarantee.

Bounded Producer-Consumer Queue with Backpressure

Purpose

A bounded queue must coordinate both empty and full states. The full predicate is the backpressure mechanism that stops producers from consuming unbounded memory.

Code or command

```
#include <condition_variable>
#include <deque>
#include <iostream>
#include <mutex>
#include <thread>

class Queue{
    std::mutex m; std::condition_variable not_empty,not_full;
    std::deque<int> q; const std::size_t cap=2;
public:
    void push(int v){std::unique_lock l(m);not_full.wait(l,[&]{return
q.size()<cap;});q.push_back(v);not_empty.notify_one();}
    int pop(){std::unique_lock l(m);not_empty.wait(l,[&]{return !
q.empty();});int
v=q.front();q.pop_front();not_full.notify_one();return v;}
};

int main(){Queue q; std::jthread p([&
{q.push(1);q.push(2);q.push(3);}); std::cout<<q.pop()+q.pop()
+q.pop()<<"\n";}
```

What the important parts do

`not\_empty` protects consumers from reading an empty deque, while `not\_full` blocks producers at capacity. Each state transition notifies the side that may now make progress.

Expected behavior or output

The program prints `6` and completes without allowing more than two queued elements.

Concrete experiment

Set capacity to 1 and add a deliberate 100 ms delay between consumer pops. Measure how producer completion is delayed by backpressure.

Worked example — Drain a bounded queue and wake every waiter during closure

Trace the implementation

The queue's two predicates include closed so shutdown can wake both kinds of waiter. A closed queue still yields accepted items until empty, then pop returns nullopt. That is an explicit drain policy rather than cancellation that discards everything immediately.

The producer closes the queue on both success and a caught failure. The consumer also closes on failure so a blocked producer can wake and stop. Each exception_ptr is written by one worker and read after join. The received vector is owned by the consumer during execution and read by main only after completion.

Thread construction itself is another failure boundary. The closure guard is declared after the worker container and before any workers start. If a later thread creation fails, the guard closes the queue before the already-started workers are joined during unwinding. The guard does not cancel arbitrary user code; its guarantee relies on this queue's closed predicate being part of every wait.

Trace startup, production, draining, and failure

The worked program creates the queue before the worker owners. It also creates a cleanup guard after the worker vector so exceptional unwinding closes the queue before the vector's jthreads join. That order matters if a producer or consumer has started but a later thread construction fails. A blocked participant must be made able to exit before automatic joining waits for it.

The producer submits a bounded sequence, then closes the queue. The consumer removes values until the queue is both closed and empty. Each worker catches its own exception, records it in a dedicated slot, and ensures the closure needed by other participants. The supervisor joins before reading those slots or the accumulated result.

```
open+space -> push -> possibly full -> producer waits
open+items -> pop  -> possibly empty -> consumer waits
close      -> reject new pushes; wake everyone
closed+items -> drain
closed+empty -> consumers finish -> join -> destroy queue
```

Observe that a close does not erase accepted items in this design. A cancellation queue could choose differently, but its discarded-item cleanup and reporting would need their own contract. The current example prioritizes a simple drain-on-close model and exposes that choice in the tests.

The queue's checks remain active independently of release-mode assert removal. Concurrency demonstrations need executable acceptance criteria, not printed output that merely looks plausible on one schedule.

Complete worked source

The current files for this example are under `source/book04/ch18/`. Listings are inserted from those files when this book is assembled.

`bounded_queue.hpp`

```
#ifndef HARBOR_SERIES_BOUNDED_QUEUE_HPP
#define HARBOR_SERIES_BOUNDED_QUEUE_HPP
#include <condition_variable>
#include <cstdint>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <type_traits>
#include <utility>

template<class T>
class BoundedQueue {
    static_assert(std::is_nothrow_move_constructible_v<T>);
    std::mutex mutex_;
    std::condition_variable not_empty_, not_full_;
    std::deque<T> data_;
    std::size_t capacity_;
    bool closed_{};
public:
    explicit BoundedQueue(std::size_t capacity) : capacity_(capacity) {
        if (capacity == 0) throw std::invalid_argument("queue capacity");
    }
    BoundedQueue(const BoundedQueue&) = delete;
    BoundedQueue& operator=(const BoundedQueue&) = delete;
    bool push(T value) {
        std::unique_lock lock{mutex_};
        not_full_.wait(lock, [&] { return closed_ || data_.size() <
capacity_ ; });
        if (closed_) return false;
        data_.push_back(std::move(value));
        lock.unlock();
        not_empty_.notify_one();
        return true;
    }
    std::optional<T> pop() {
        std::unique_lock lock{mutex_};
        not_empty_.wait(lock, [&] { return closed_ || !data_.empty(); });
        if (data_.empty()) return std::nullopt;
        T value = std::move(data_.front());
        data_.pop_front();
        lock.unlock();
        not_full_.notify_one();
        return value;
    }
    void close() {
        { std::lock_guard lock{mutex_}; closed_ = true; }
        not_empty_.notify_all();
        not_full_.notify_all();
    }
};
#endif
```

`main.cpp`

```

#include "check.hpp"
#include "bounded_queue.hpp"
#include <exception>
#include <numeric>
#include <thread>
#include <vector>

int main() {
    BoundedQueue<int> queue{3};
    std::vector<int> received;
    std::exception_ptr producer_error, consumer_error;
    std::vector<std::jthread> workers;
    struct CloseFirst { BoundedQueue<int>& queue; ~CloseFirst()
{ queue.close(); } };
    CloseFirst startup_guard{queue}; // Destroyed before workers on failure.
    workers.emplace_back([&] {
        try { while (auto item = queue.pop()) received.push_back(*item); }
        catch (...) { consumer_error = std::current_exception(); }
    queue.close(); }
    });
    workers.emplace_back([&] {
        try {
            for (int value = 0; value < 100; ++value)
                if (!queue.push(value)) throw std::runtime_error("unexpected
closure");
        } catch (...) { producer_error = std::current_exception(); }
        queue.close();
    });
    workers.clear();
    if (producer_error) std::rethrow_exception(producer_error);
    if (consumer_error) std::rethrow_exception(consumer_error);
    std::vector<int> expected(100); std::iota(expected.begin(), expected.end(),
0);
    CHECK(received == expected);
    CHECK(!queue.push(101) && !queue.pop());
    queue.close(); // Repeated closure is harmless.
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 18
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — Race-free but permanently blocked

All queue fields are protected by the mutex, yet shutdown hangs. The owner requests stop on a jthread without closing the queue, so the worker's condition-variable predicate remains false. There is no data race; there is a missing liveness transition. Add closure or a stop-aware predicate path and notify the relevant waiters.

Another queue uses `if (empty) wait(lock)` and then unconditionally removes the front item. A spurious wake or competing consumer leaves the queue empty

when the thread resumes. Use a predicate loop and interpret closed-and-empty explicitly. The notification was never a reservation for a particular item.

A server describes a capacity-four queue as “at most four open connections,” while two workers and the acceptor hold three more. Correct the resource accounting to include active and transient owners. The capacity setting still helps, but the documentation overstated what it bounded.

A final failure arises during startup: one consumer launches successfully, then producer construction throws. The `jthread` destructor tries to join the consumer while the queue remains open and empty. A close-before-join guard fixes the lifetime order and makes the blocked wait eligible to finish. Test this path through a controlled launch seam rather than relying on rare system exhaustion.

Practice and implementation lab

Explain why the startup guard must be declared after the worker container. Test closure while a producer waits on a full queue and while a consumer waits on an empty queue.

Add a nonblocking `try_push` with a result distinguishing full from closed, rather than overloading one Boolean with both meanings.

Solution and review notes

Declare the guard after the worker container so it is destroyed first during unwinding, waking waiters before `jthread` destructors join them. A deterministic test can use promises or latches to establish that the desired state has been reached instead of guessing with sleep durations.

A small enum can report accepted, full, or closed. The operation should examine and mutate state under the same mutex. A full result leaves the queue unchanged, while closure is a persistent state transition under the chosen policy.

Guided lab — Verify the queue as a state machine

Test ordinary push and pop, capacity-one operation, closure before any item, closure with pending items, repeated closure, and rejected push after closure. Confirm that accepted items are delivered exactly once in the single-consumer model and that closed-and-empty is distinct from a legitimate empty payload.

Add multiple producers with uniquely identified items and multiple consumers. After joining, compare the collected IDs with the accepted set. Do not require a total order among concurrent producers unless the API promises one. The useful invariant is no loss or duplication of accepted items under the stated ownership model.

Introduce a worker failure that closes the queue while another participant could be blocked. Verify bounded completion through the test harness and that all owned payload resources are released. Include partial thread startup failure in a separate deterministic exercise.

Write a capacity budget for a server with queue capacity four, two workers, one acceptor, and maximum frame size 4096. Distinguish descriptors, queued objects, active payload buffers, and temporary encoded responses. State which allocations occur before admission and therefore are not covered by the queue's element bound.

Review reasoning. Does a notification reserve an item? No. Does a bounded queue guarantee allocation-free insertion? No. Does closure cancel work already removed by a consumer? Not in this design. Does race freedom imply shutdown progress? No. Why close before joining workers during failure? So blocked predicates can become terminal while their state remains alive.

Completion criterion. Deliver a state-transition table and tests for startup, normal draining, and failure closure. Every blocking path must identify the state change and notification that allows it to finish, including when the other participant fails.

Primary reference notes

Technical references: [THREADS](#) [CONDVAR](#) [MEMORYMODEL](#)

Chapter 19 — Building Task-Based Asynchronous Systems

Sometimes the useful abstraction is a result that will become available, not a thread that must be managed directly. Futures and promises provide a one-result communication channel, while `async` can arrange execution under a selected policy. This chapter transfers both a value and an exception, distinguishes deferred work from asynchronous work, and explains why a future's lifetime can still affect blocking behavior.

An asynchronous function returns a future immediately, but the caller's scope still blocks when an unused future is destroyed. Another task returns a future that never becomes ready because its producer was abandoned. The word asynchronous describes a separation in time; it does not specify launch policy, ownership, cancellation, or completion behavior by itself.

This chapter treats futures and promises as a result channel. A producer makes one shared state ready with a value or exception. A consumer observes readiness and retrieves the result according to the future's ownership rules. Task execution remains a separate design question: who runs the callable, what resources it owns, how it is stopped, and what happens when the caller no longer wants the answer.

19.1 Communicating Results with Futures and Promises

19.1.1 Producing Deferred Results

A promise owns the producer side of a shared result state, and its future provides a consumer interface. The producer can satisfy the state with a value or an exception. It should do so exactly once under the interface rules. Destroying an unsatisfied promise makes the corresponding future observe a broken-promise error rather than waiting forever for a producer that no longer exists.

This is a one-shot result channel, not a general queue. Repeated messages need another abstraction. Move promises into the worker that owns responsibility for completing them, and catch failures inside the worker so `set_exception` can transfer them. Keep result-state ownership separate from the lifetime of any borrowed input the task uses.

A promise and its future refer to a shared result state. The promise is the producer-facing endpoint; the future is the consumer-facing endpoint. A producer can store a value or an exception, making the state ready. The result channel does not itself start a thread. A worker, event loop, or synchronous operation can fulfill the promise according to the surrounding execution model. ([FUTURE](#))

The ordinary promise/future pair is single-result. Retrieve the future once, then fulfill the promise once. Attempts to repeat operations outside their contract report errors rather than becoming a multi-message queue. For a stream of results, use a channel or queue whose state model explicitly supports multiple items and closure.

Moving the promise into the worker transfers the producer endpoint's ownership. Moving the future transfers the consumer endpoint. The shared state allows those endpoints to live in different scopes, but any separately captured references still require their own lifetime argument. A future holding a result does not keep every arbitrary object used by the producer alive.

A worker should catch failures and make them available through the result channel. The example calls `checked arithmetic` inside a `try` block and stores `current_exception` on failure. This prevents an exception from escaping the thread entry function and lets the caller handle the failure at `get`. The producer still needs a clear policy if it cannot fulfill the state as intended; do not silently swallow all exceptions and leave the consumer waiting.

Abandoning a promise before supplying a result makes the associated state report a broken-promise condition rather than a normal value. This is useful evidence that the result producer disappeared, not a substitute for supervising its task. The caller may still need to join the worker and clean up external resources. Result-state readiness and task-resource completion are related but not interchangeable responsibilities.

19.1.2 Retrieving Asynchronous Results

`future::get` waits for readiness when necessary, returns the stored value, or rethrows the stored exception. For an ordinary future, `get` consumes its result association; repeated retrieval is not the same as repeatedly reading a variable. `shared_future` supports a different multiple-observer pattern with its own value-access considerations.

Waiting for a result can synchronize the producer's preceding work with the consumer under the future's rules, but it does not make arbitrary unrelated mutable objects safe to access concurrently. Prefer returning an owning result instead of publishing side effects through undocumented globals. A future can make the result boundary explicit without erasing every other lifetime dependency.

Calling `get` on an ordinary future waits for readiness if necessary, then returns the stored value or rethrows the stored exception. The operation consumes the ordinary future's result association. Check the intended ownership model before calling it from multiple places. A `shared_future` provides a different interface for shared observation, with its own value-access and lifetime semantics.

Waiting for readiness can be useful when the caller has other work to perform, but a wait timeout does not cancel the producer. The task can continue consuming CPU, holding descriptors, or writing external state after the consumer decides it no

longer wishes to wait. Cancellation needs a separate cooperative mechanism and an owner responsible for eventual completion and cleanup.

A timed wait should be interpreted as an observation at that point in time. Ready means the state has a stored outcome. Timeout means it was not ready within the requested wait. Deferred indicates a task whose execution has not been launched under the relevant async policy. Treating all non-ready outcomes as the same event can cause either accidental blocking or abandoned work.

The synchronization associated with readiness makes the stored result available under the future contract. It does not permit arbitrary concurrent mutation of unrelated shared objects captured by the producer. Prefer returning an owned result through the future instead of publishing partially initialized state elsewhere and using the future as an informal completion flag.

Error handling should retain the operation context. An exception rethrown by `get` may describe invalid input, allocation failure, or native I/O failure from the producer. Attach a task identifier and bounded diagnostic context without including secrets or unbounded payloads. The point of the channel is to preserve the failure, not to replace every cause with a generic “async failed” message.

Delivering One Deferred Result with `std::promise` and `std::future`

Purpose

A promise owns the producer side of a one-shot shared state, while the future owns the consumer side. The pair communicates both completion and a value.

Code or command

```
#include <future>
#include <iostream>
#include <thread>

int main(){
    std::promise<int> promise; std::future<int>
    result=promise.get_future();
    std::jthread worker([p=std::move(promise)]() mutable
    { p.set_value(6*7); });
    std::cout<<result.get()<<"\n";
}
```

What the important parts do

`get_future` obtains the consumer handle before the promise is moved into the worker. `set_value` makes the result ready; `future::get` waits if necessary, returns 42, and consumes the future's value.

Expected behavior or output

The program prints `42`.

Concrete experiment

Replace `set_value` with `set_exception(std::make_exception_ptr(...))` and catch the exception around `future::get` to observe error propagation through the same channel.

19.2 Launching Asynchronous Tasks

19.2.1 Using `std::async`

`std::async` returns a future associated with a callable's result. With `launch::async`, it arranges asynchronous execution under the facility's rules. With `launch::deferred`, invocation is postponed until a suitable waiting or retrieval operation requests it. The default policy can permit either behavior; so code that requires actual concurrency should not silently rely on the default choosing a new execution path.

Arguments and captures still follow ordinary ownership rules. Passing references requires the referenced objects to outlive their use. Exceptions from the callable

are stored for retrieval through the future rather than escaping directly into the creator's call stack. Resource exhaustion can also make launching fail at the call boundary.

`std::async` combines a callable with a standard-library-managed result state and an execution policy. With an explicit asynchronous policy, it runs the task in the specified asynchronous manner and makes the outcome available through a future. With a deferred policy, execution is postponed until the appropriate wait or retrieval triggers it. An omitted policy permits implementation choices that callers must not silently assume away.

The worked example supplies policies explicitly. This makes the lesson observable: one operation is asynchronous, and another is intentionally deferred. A zero-duration wait on the deferred future identifies the deferred state, and `get` then executes and retrieves its result. The test is about documented policy, not about which thread happened to win a race on a particular machine.

Arguments and callable state are stored according to the invocation interface's rules. Passing a value can create an independent copy or move; using a reference wrapper or capturing by reference retains a borrow. An asynchronous launch therefore still needs the same input lifetime and mutation analysis as an explicit thread. Convenience in task creation does not remove ownership obligations.

Launching one asynchronous operation per tiny item can create excessive concurrency or overhead. `std::async` is not a universal bounded executor configuration API. For a service with a strict admission budget, a separately owned worker pool and task queue may make capacity and shutdown clearer. The result channel can still be a future, but the execution policy belongs to the pool.

Do not discard the returned future casually. The lifetime of a future associated with an asynchronously launched operation has special completion behavior; destruction of the last relevant reference can wait for completion under the standard's conditions. Code that constructs and destroys a temporary future each loop iteration may serialize work unexpectedly. Retain futures deliberately and collect them at a visible completion boundary. ([FUTURE](#))

19.2.2 Understanding Task Execution Policies

A deferred future can report `future_status::deferred` to `wait_for`, while an asynchronously executing task may report `timeout` or `ready`. Waiting status is not a performance measurement or a guarantee that the task has made a particular amount of progress. Avoid polling a future in a tight loop when a blocking result wait or event-driven design fits the requirement.

The last future associated with an `async`-launched task can block during destruction under the specified conditions. Discarding a temporary future can therefore serialize work unexpectedly. Retain futures deliberately and define where results are joined. Futures do not provide universal cancellation; long-running tasks need a separate cooperative stop design.

Execution policy changes when and where the callable can run, and therefore changes the lifetime and blocking analysis. A deferred task may execute in the thread that first performs the triggering wait or retrieval. That thread might hold locks or have latency responsibilities that the original author did not consider. Avoid assuming that a callable will always execute on a background worker unless the policy establishes it.

An explicit asynchronous task can fail to launch because required resources are unavailable. That failure occurs at submission rather than through a later successful task result. If several tasks have already launched, the supervisor must still collect or clean up their owners while handling the new launch failure. Batch task construction needs the same partial-startup reasoning as a vector of threads.

A future does not make dependency cycles impossible. If task A waits for task B while B waits for a resource held by A, the shared result states simply encode the deadlock. A bounded worker pool can deadlock when all its workers block waiting for tasks queued behind them. Design dependencies so ready work can actually execute, or use continuation and scheduling mechanisms appropriate to the system.

Cancellation and timeout policy should say whether a result can be ignored, whether work should stop cooperatively, and whether external effects remain possible. An application can choose to abandon a response while still supervising the task to completion. It can also pass a stop token checked at safe boundaries. Neither choice implies rollback of bytes already written or remote operations already requested.

Measure tasks at the operation boundary relevant to users. Submission latency, queue delay, execution time, result retrieval, and cleanup are different quantities. A fast-returning submission call can hide a long queue, and a future destructor can move waiting into an unexpected scope. Make those stages visible before claiming that an asynchronous design improves end-to-end latency.

Comparing Deferred and Asynchronous `std::async` Policies

Purpose

`std::async` is not one fixed scheduling mechanism. Explicit launch policies make the difference between lazy execution in the waiting thread and a separately scheduled asynchronous task observable.

Code or command

```
#include <future>
#include <iostream>
#include <thread>

std::thread::id work(){ return std::this_thread::get_id(); }
int main(){
    auto caller=std::this_thread::get_id();
    auto deferred=std::async(std::launch::deferred,work);
    auto async=std::async(std::launch::async,work);
    std::cout<<(deferred.get()==caller)<<' ' <<(async.get()!
=caller)<<"\n";
}
```

What the important parts do

`launch::deferred` postpones `work` until `get`, so it runs on the caller thread. `launch::async` requires asynchronous execution and therefore uses a different execution thread for this task.

Expected behavior or output

The program should print `1 1` on a conforming implementation.

Concrete experiment

Use the default policy `std::async(work)` and record which thread executes it on your standard library. Explain why code must not assume one policy when none is specified.

Worked example — Transfer task failures and distinguish deferred execution

Trace the implementation

The promise is moved into the worker, establishing one producer responsible for the shared state. The input is a small value rather than a borrowed reference. The worker catches any task failure and stores it, and main retrieves the value before explicitly joining the jthread's execution lifetime.

The failed async task intentionally receives an out-of-domain value. Its exception emerges from `get` in the calling thread, where the test checks the expected category. The deferred task proves that its policy is observable before `get` triggers execution; the test does not assume the default async policy.

The abandoned promise case distinguishes a missing producer from a task that legitimately returns zero or no value. That error is delivered through the future state. None of these mechanisms provides a stream of results or a cancellation command, so a larger task system should not overload one-shot futures with those additional responsibilities.

Trace three result channels, not three kinds of magic

The first demonstration constructs a promise/future pair and moves the promise into a `jthread`. The worker performs checked squaring and sets either the value or an exception. The caller retrieves the result and joins the worker while the surrounding state remains alive.

The second demonstration launches checked work with an explicit asynchronous policy and verifies that invalid input is rethrown through `get`. The third uses an explicit deferred policy, observes that status without running the callable, then retrieves the expected result. Together they separate result storage, execution policy, and error propagation.

```
promise producer -> value or exception -> shared result state -> future.get()
async policy     -> scheduled work    -> shared result state -> future.get()
deferred policy  -> stored callable   -> triggering get runs callable -> result
```

The arithmetic bounds prevent signed overflow before the operation is evaluated. An asynchronous context does not make undefined arithmetic safer; it merely changes where the defect occurs. Keep input validation within the callable's declared contract and test both accepted and rejected values.

The examples complete bounded computations and do not implement task cancellation. A timeout-based service should not copy this small result-channel demonstration and assume that dropping a future stops an I/O-bound worker. The extended server uses explicit stop state, bounded waits, and joined ownership instead.

Complete worked source

The current files for this example are under `source/book04/ch19/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <chrono>
#include <exception>
#include <future>
#include <stdexcept>
#include <thread>
```

```

int checked_square(int value) {
    if (value < -1000 || value > 1000) throw std::out_of_range("square domain");
    return value * value;
}
int main() {
    std::promise<int> producer;
    auto result = producer.get_future();
    std::jthread worker([promise = std::move(producer)]() mutable {
        try { promise.set_value(checked_square(12)); }
        catch (...) { promise.set_exception(std::current_exception()); }
    });
    CHECK(result.get() == 144);
    worker.join();
    auto failed = std::async(std::launch::async, checked_square, 2000);
    bool rejected = false;
    try { (void)failed.get(); } catch (const std::out_of_range&) { rejected =
true; }
    CHECK(rejected);
    auto deferred = std::async(std::launch::deferred, checked_square, 7);
    CHECK(deferred.wait_for(std::chrono::seconds{0}) ==
std::future_status::deferred);
    CHECK(deferred.get() == 49);
    std::future<int> abandoned;
    { std::promise<int> incomplete; abandoned = incomplete.get_future(); }
    bool broken = false;
    try { (void)abandoned.get(); }
    catch (const std::future_error& error) {
        broken = error.code() ==
std::make_error_code(std::future_errc::broken_promise);
    }
    CHECK(broken);
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 19
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — Timed out, still running

A caller uses `wait_for`, reports timeout, and destroys the surrounding input object while the producer still holds a reference to it. The task continues and accesses expired storage. Repair ownership first: either move owned task data into the producer or keep the borrowed data alive through supervised completion. Then add a cooperative stop protocol where the use case requires it.

Another loop calls `std::async` and immediately discards the returned future. Temporary future destruction can wait, making the loop behave serially despite the asynchronous launch policy. Retain the futures and collect them deliberately, while still respecting a bounded concurrency policy rather than starting unlimited tasks.

A worker catches an exception but neither sets a promise result nor completes the promised protocol. The consumer cannot distinguish useful waiting from abandoned work until the producer endpoint is destroyed or another policy intervenes. Store the exception through the channel and ensure endpoint lifetime corresponds to task ownership.

A final bug requests a deferred task while holding a mutex, then calls `get`. The callable runs in that context and attempts to acquire the same mutex. Explicit policy does not automatically make a design safe; it gives you enough information to reason about the execution location. Review locks and reentrancy at the retrieval point.

Practice and implementation lab

Launch several independent tasks, retain every future, and collect all failures without abandoning the remaining tasks. Define whether the application reports the first failure or an aggregate.

Add a cooperative stop_token to a long-running task wrapper and specify what result represents cancellation.

Solution and review notes

Store futures in an owning collection and retrieve each result inside its own error-handling boundary. A failure in one `get` should not skip cleanup or observation of the remaining tasks. Report the policy explicitly rather than letting destruction order decide which work is waited on.

Cancellation needs a state the worker checks and a documented result or exception category. A stop request is not proof the task stopped immediately, and external side effects already performed may require their own compensation or idempotency policy.

Guided lab — Separate readiness, cancellation, and cleanup

Create a task returning an owned result and test success, checked-input failure, and producer abandonment. For the failure case, verify the exception category or structured error rather than relying only on a printed message. For abandonment, distinguish the broken result channel from a successfully computed default value.

Add a deferred task with an observable counter. Verify that a non-triggering status check reports deferred without incrementing the counter, then that retrieval runs the task once and produces the intended value. Keep the counter access synchronized or confined to the executing test thread.

Design a cancellable bounded computation with an explicit stop token and documented checkpoints. Test cancellation before work and after a controlled gate, then join the worker before destroying its inputs. Do not use elapsed-time sleeps as proof that a particular iteration has been reached.

Write an ownership table for a batch of task submissions. Include the path where the third submission fails after two successful launches. State which objects retain the first two results, who requests any cancellation, and where final cleanup occurs. A task batch should remain coherent even when it never reaches its intended full size.

Review reasoning. Does a promise start a thread? No. Does wait timeout cancel the producer? No. Does deferred execution necessarily occur on a background thread? No. Can discarding an async future introduce waiting? Under the specified future-lifetime conditions, yes. Why return owned results? To make the result lifetime independent of temporary producer-local storage.

Completion criterion. Deliver a task/result ownership diagram and tests for each outcome category. The design must name the event that makes a result ready and separately name the event that proves the producer's resources can be destroyed.

Primary reference notes

Technical references: [THREADS](#) [FUTURE](#) [MEMORYMODEL](#)

Chapter 20 — Understanding Atomics and the C++ Memory Model

Atomics prevent certain conflicting accesses from becoming data races and provide ordering tools for communication. They do not make an entire algorithm correct merely by replacing `int` with `atomic<int>`. This chapter separates a relaxed statistics counter from release-acquire publication of ordinary data, then explains why ordering, lifetime, and logical ownership must be reasoned about together.

A counter becomes atomic and the program stops losing increments. The developer then replaces every mutex with atomic flags and assumes the larger data structure is safe. Its nodes can still be destroyed while another thread reads them, and two fields can still describe incompatible states. Atomic access to one object is a local property, not a complete concurrent algorithm.

This chapter separates atomicity, ordering, notification, and lifetime. A relaxed counter needs indivisible updates but no publication of associated payload. A one-shot handoff needs a release/acquire relationship connecting the writer's ordinary payload update to the reader's later access. Neither example solves reusable queues, lock-free reclamation, or every possible startup and shutdown failure.

20.1 Performing Atomic Operations

20.1.1 Performing Atomic Loads and Stores

An atomic object supports indivisible operations under the type's interface. Loads and stores use explicit or default memory orders, and implementations may use different mechanisms depending on type and platform. Atomic does not universally mean lock-free; query the appropriate property when a design actually requires lock freedom.

Do not mix atomic and non-atomic accesses to the same object through casts or aliases. Keep the object's lifetime valid for every participating thread. An atomic pointer protects the pointer value's access, not automatically the lifetime of the pointed-to object. Safe reclamation is a separate and often difficult part of lock-free data structures.

An atomic object supports indivisible operations according to its type and interface. Concurrent accesses through that atomic interface avoid an ordinary data race on that object. This does not authorize unsynchronized accesses to adjacent non-atomic fields, nor does it make a sequence of several atomic operations one indivisible transaction. Define the logical operation before deciding which primitive is sufficient. ([ATOMICS](#), [MEMORYMODEL](#))

Loads and stores access one atomic value. A load can request an ordering appropriate to the reader; a store can request an ordering appropriate to the writer. Not every ordering is valid for every operation. Use the documented combinations rather than passing a memory-order enum selected by a generic “stronger is always safer” rule without understanding the operation.

Atomic types are not universally lock-free. The implementation can use internal locking for some sizes and platforms. `is_lock_free` reports a property of the relevant implementation or object; `is_always_lock_free` expresses a compile-time guarantee for the specialization when true. A false result does not make the operation non-atomic. A true result does not establish that the entire application is lock-free or bounded in latency.

Initialization and destruction still require ownership. An atomic object cannot safely be destroyed while another thread may access it, and publishing a pointer to an incompletely initialized owner is not repaired by making one member atomic. Place shared state in a lifetime that encloses every accessing thread, then use synchronization to govern the accesses during that lifetime.

Avoid `volatile` as a substitute for thread synchronization. Its language role concerns particular observable accesses and implementation interfaces, not the release/acquire relationships required for ordinary inter-thread communication. Device programming may use `volatile` under a platform-specific contract, but that is a different concern from making a shared queue or flag race-free.

20.1.2 Performing Read-Modify-Write Operations

Operations such as `fetch_add` atomically read a value, compute an update, and return the prior value under one modification. A separate load followed by store is not equivalent when several threads update concurrently; intervening updates can be lost. Compare-exchange supports conditional updates but requires a carefully designed retry and failure-order policy.

A counter can use relaxed ordering when only the counter’s own value matters and another synchronization event establishes when final observation occurs. That does not publish unrelated data. Use the weakest order justified by a written argument, not by copying a pattern whose surrounding lifetime and synchronization assumptions differ.

Read-modify-write operations combine reading the previous value and storing a new value as one atomic operation. `fetch_add` returns the old value while applying the increment; compare-exchange conditionally replaces a value according to whether it matches the expected state. These primitives can express counters, ticket assignment, and more complex algorithms, but each algorithm still needs an invariant and progress argument.

A relaxed fetch-add is appropriate for the chapter’s independent event counter. Each increment must be counted without loss, but no associated non-atomic payload is being published through that increment. After joining both workers, the supervisor can read the final count and compare it with the known number of

operations. The join supplies the completion boundary; relaxed does not mean “the final result may randomly be wrong.”

A check followed by a separate increment is not equivalent to a conditional atomic update. Two threads can both observe capacity before either increments. Use an appropriate compare-exchange loop or a lock protecting the complete reservation transition when the invariant requires “increment only if below a limit.” The simpler operation may be safer and easier to verify than a custom lock-free loop.

Compare-exchange updates its expected argument when comparison fails, and weak compare-exchange can fail spuriously. A retry loop must incorporate the new observed state and preserve the algorithm’s validity across repeated attempts. Side effects performed inside the retry loop can be repeated; do not send messages or modify unrelated state as though the body will execute exactly once.

Counter wrap, ABA, and reclamation are separate obligations. A value can change from A to B and back to A while another thread is paused; equality of the final bits does not prove nothing happened. A pointer can be removed and its storage reused while another thread retains a stale observation. Generation tags help some identity problems but do not by themselves keep objects alive. These advanced issues are why the chapter stops short of presenting a casual lock-free linked structure.

Using Atomic Fetch-Add as One Indivisible Read-Modify-Write

Purpose

Atomic read-modify-write operations prevent lost updates to a shared scalar without requiring a separate mutex.

Code or command

```
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>

int main(){
    std::atomic<int> counter{0}; std::vector<std::jthread> threads;
    for(int t=0;t<4;++t) threads.emplace_back([&]{for(int
i=0;i<10000;++i) counter.fetch_add(1,std::memory_order_relaxed);});
    threads.clear(); std::cout<<counter.load()<<"\n";
}
```

What the important parts do

`fetch\_add` reads the old value and stores the incremented value as one atomic operation. Relaxed ordering is sufficient here because only the counter's numeric integrity matters; no other data is being published through it.

Expected behavior or output

After all `jthread` objects are destroyed and joined, the program prints `40000`.

Concrete experiment

Replace the atomic with a plain `int` only in a ThreadSanitizer exercise. Observe the data-race diagnostic rather than treating any accidental numeric result as valid evidence.

20.2 Understanding Memory Ordering

20.2.1 Understanding Acquire and Release Semantics

A release operation can publish preceding writes when an acquire operation observes the appropriate released value under the memory model's rules. In a one-shot ready flag, the producer writes ordinary payload data and then stores true with release; a consumer that observes true with acquire can read the published payload after that synchronization.

The flag must not be reused carelessly for another payload while a consumer may still read the previous one. A single Boolean protocol that works once does not automatically become a reusable queue. Identify which write an acquire observes, when objects remain alive, and when the producer may modify them again. Memory-order syntax is only one part of that protocol.

Release/acquire ordering can publish ordinary data when the reader's acquire operation observes the relevant release operation, directly or through a supported synchronization relationship. In the one-shot example, the producer writes the payload, then stores true to the ready flag with release semantics. The consumer waits and observes ready with acquire semantics before reading the payload. The ordering connects those ordinary accesses. ([MEMORYMODEL](#))

The relationship is not “any acquire anywhere synchronizes with any release.” It depends on which value or modification is observed and the standard's rules for the operations involved. An unrelated atomic flag does not automatically order every shared variable in the process. Draw the actual writer, publication operation, observation, and reader sequence for the data being communicated.

The one-shot restriction is important. After publication, the producer does not overwrite the payload while the consumer reads it. Resetting the Boolean to false and reusing the same storage would require a handshake proving the previous reader has finished before the next write. Without that reverse coordination, the next payload update can race with the old reader even though each ready flag access remains atomic.

Atomic wait and notify provide a waiting mechanism around a value, but notification is not the memory publication itself. The store changes the predicate, and the acquire observation establishes the relevant ordering. Notify helps wake a waiter. A changed value can be observed even if notification occurred before waiting; conversely, a notification without the required value transition does not make the payload ready.

Startup and shutdown must preserve the ability to change the awaited state. A consumer launched before a producer can block forever if producer construction throws and no cleanup path publishes a terminal state. The revised chapter source starts the finite producer first, then creates the consumer. If consumer creation fails, the producer can still finish and be joined without waiting for an event only the missing consumer could provide.

20.2.2 Understanding Sequential and Relaxed Ordering

Sequentially consistent operations provide an additional global ordering framework for those operations, while relaxed operations provide atomicity and modification-order guarantees without the same cross-object ordering. The default atomic operations commonly use sequential consistency unless a different order is supplied.

Stronger ordering can simplify reasoning, but it does not turn several operations into one transaction or solve lifetime hazards. Relaxed ordering can be appropriate for independent statistics, but using it for a publication flag without

another synchronization mechanism can leave ordinary payload accesses unsafely ordered. Test executions can expose failures but cannot exhaustively prove a memory-model argument.

Sequential consistency provides a stronger ordering model for the participating atomic operations, including a single total order consistent with the required constraints. It can simplify reasoning, and the default ordering is often a sensible starting point. It does not turn several atomic fields into one transaction or solve object lifetime. A reader can still observe a combination that the application never intended if its multi-field protocol is incomplete.

Relaxed ordering retains atomicity and the atomic object's modification-order properties while omitting the additional inter-object ordering requested by stronger operations. It is useful for statistics and counters when no associated data publication relies on the operation. Selecting relaxed merely because it benchmarks faster is not a correctness argument. First state which ordering relationships the algorithm does and does not need.

Acquire and release offer a targeted publication protocol. They can be sufficient when the dataflow is clear, as in the one-shot example. A weaker ordering should be justified by a happens-before argument tied to actual observations, not by a processor-specific intuition such as “x86 usually keeps this order.” C++ correctness must survive the compiler's permitted transformations as well as the hardware's execution behavior.

Testing atomic algorithms is necessary but not sufficient. A million successful executions on one architecture do not enumerate every language-permitted interleaving or compiler transformation. Combine small-state reasoning, ownership proofs, targeted tests, and tools such as ThreadSanitizer where supported. A race detector can identify certain executed conflicting accesses; it does not certify lock freedom, linearizability, or deadline compliance.

Document why each non-default memory order exists. A useful comment identifies the payload being published or explains that the counter carries no associated data and is observed after joined completion. A comment that simply restates “use acquire” adds little. Future maintainers need to know which invariant would break if the order changed or if the data were reused before the reader finished.

Publishing Data with Release and Acquiring It in Another Thread

Purpose

Acquire/release ordering is useful when one atomic flag publishes preceding non-atomic writes to another thread.

Code or command

```
#include <atomic>
#include <iostream>
#include <thread>

int main(){
    int payload=0; std::atomic<bool> ready{false};
    std::jthread producer([&]{payload=99;
    ready.store(true,std::memory_order_release);});
    std::jthread consumer([&]{while(!
    ready.load(std::memory_order_acquire)){
    std::cout<<payload<<"\n";});
}
```

What the important parts do

The producer writes `payload` before the release store. The consumer spins until an acquire load observes that release; the synchronizes-with relationship makes the preceding payload write visible before the consumer reads it.

Expected behavior or output

The consumer prints `99`.

Concrete experiment

Change both flag operations to `memory\_order\_relaxed` and explain why the program loses the publication guarantee even if it still appears to print 99 on your machine.

Worked example — Distinguish a relaxed counter from release-acquire publication

Trace the implementation

The counter's relaxed fetch_add operations cannot lose increments because each update is an atomic read-modify-write. Joining both workers establishes completion before the final observation. The counter is not used to announce that some separate payload is ready, so it needs no publication role.

The second example has a different contract. payload is written once, before the release store. The consumer's acquire wait observes the flag transition before reading payload. The value remains unchanged afterward, and main reads observed only after the consumer joins. These lifetime and ordering facts make the ordinary integers safe in this one-shot exchange.

notify_one wakes a waiter but is not a substitute for the atomic state change and memory order. The printed lock-free property is observed rather than asserted across all implementations. This program is not a reusable lock-free queue, a memory-reclamation scheme, or a proof of hard real-time progress.

Two examples with two different synchronization needs

The event counter has two workers, each performing ten thousand relaxed increments. The main thread joins both, then checks the final twenty thousand. There is no non-atomic payload attached to each increment. The relevant property is that updates to the counter are not lost, followed by a completion boundary before the final observation.

The publication example has one payload write and one payload read. The producer writes forty-two and publishes ready with release semantics. The consumer waits for ready with acquire semantics, then copies the payload into its own result. The supervisor joins before checking that result.

```
producer: payload = 42 -> ready.store(true, release) -> notify
           |
consumer: wait/observe ready with acquire -> read payload
           |
supervisor: join both -> inspect owned observed result
```

The revised startup order makes the finite producer independent of consumer construction. This addresses a lifecycle failure without changing the memory-order lesson. It does not imply that “always start producers first” is a universal rule: a producer blocked on a bounded queue may itself need a close-before-join guard. The dependency graph determines the safe order.

The reported lock-free property is an observation, not an asserted platform requirement. The correctness checks should pass on a supported implementation whether the atomic type uses hardware instructions or an internal lock, provided the documented atomic contract is met.

Complete worked source

The current files for this example are under `source/book04/ch20/`. Listings are inserted from those files when this book is assembled.

main.cpp

```
#include "check.hpp"
#include <atomic>
#include <thread>

int main() {
```

```

    std::atomic<unsigned> count{0};
    std::jthread a([&] { for (unsigned i = 0; i < 10000; ++i) count.fetch_add(1,
std::memory_order_relaxed); });
    std::jthread b([&] { for (unsigned i = 0; i < 10000; ++i) count.fetch_add(1,
std::memory_order_relaxed); });
    a.join(); b.join();
    CHECK(count.load(std::memory_order_relaxed) == 20000);

    int payload = 0;
    int observed = 0;
    std::atomic<bool> ready{false};
    // Start the finite producer first. If consumer construction fails,
    // producer cleanup can still finish without waiting for a missing peer.
    std::jthread producer([&] {
        payload = 42;
        ready.store(true, std::memory_order_release);
        ready.notify_one();
    });
    std::jthread consumer([&] {
        ready.wait(false, std::memory_order_acquire);
        observed = payload;
    });
    producer.join(); consumer.join();
    CHECK(observed == 42);
    std::cout << "unsigned_atomic_lock_free=" << count.is_lock_free() <<
"\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 20
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Failure clinic — An atomic flag around a data race

A writer stores a new payload repeatedly and toggles a ready flag. The reader sees ready, starts reading, and the writer begins the next update. The flag is atomic, but the payload accesses overlap without the required ownership handoff. Introduce an acknowledgement, double-buffer ownership protocol, queue, or lock that prevents reuse while the reader still owns access.

Another program replaces an atomic increment with a load followed by a store of one greater value. Two threads can read the same old count and store the same new count, losing an event. Use an atomic read-modify-write for that operation, or protect the combined transition with a mutex.

A wait is notified without changing its watched value, and the developer expects it to return as though a message arrived. Atomic wait is value-oriented; the state transition must represent the event. For repeated events, a monotonic sequence with a wrap policy or a proper queue may be more appropriate than a reusable Boolean.

Finally, a program passes every stress test and is declared lock-free. The tests show observed behavior on the tested schedules, not a formal progress property. Distinguish atomic implementation lock freedom, algorithmic progress guarantees, and measured latency. They are related engineering concerns but not interchangeable labels.

Practice and implementation lab

Write the happens-before chain for the payload from producer assignment through consumer read and final main-thread observation. Identify which edges come from release/acquire and which come from join.

Design a reusable single-producer single-consumer slot with explicit empty and full ownership states. Do not merely reset ready to false without proving when both parties may access the payload.

Solution and review notes

The producer's ordinary write is sequenced before release publication; the consumer's observing acquire synchronizes with that publication; its payload read follows. Consumer completion and join then order main's observation. Each edge serves a distinct part of the argument.

A reusable slot requires a return-of-ownership transition so the producer cannot overwrite data still being read. Sequence counters or separate states can express that protocol, but object lifetime and shutdown still need analysis. Prefer a mutex-based queue until a measured requirement justifies the more subtle design.

Guided lab — Write the happens-before argument

For the counter, list every shared object and explain why only the counter requires atomic updates. Identify the join operations that precede the supervisor's final observations. Add boundary tests for zero increments and several bounded worker counts without weakening the lifecycle checks.

For publication, draw the producer's payload write, release store, consumer acquire observation, and payload read. Explain which operation observes which state. Then describe why a second producer write would be unsafe without a reverse handoff. Do not execute an intentionally racy version as part of the ordinary passing suite.

Create a small startup-failure model in which the consumer construction is rejected after the finite producer starts. Verify that the producer can complete and be joined without waiting for missing work. A separate queue exercise should show why producer-first alone is insufficient when the producer can block on capacity.

Review a proposed atomic reservation counter. Determine whether a separate load and increment can exceed capacity, then specify the invariant a compare-exchange

loop or mutex must maintain. Include the behavior at capacity and at the numeric limit, not only the successful reservation path.

Review reasoning. Does atomicity of one field make a whole struct transactionally consistent? No. Does relaxed mean updates may be lost? No, not for the atomic read-modify-write used here. Does notify publish ordinary payload data by itself? No. Does release/acquire allow the producer to overwrite a payload still being read? No. Does a successful stress test prove every allowed schedule correct? No.

Completion criterion. Deliver an object-lifetime argument and a happens-before diagram for each shared-data path. Every memory-order choice should have a stated purpose, and every waiter should retain a reachable completion or cancellation transition even during partial startup failure.

Primary reference notes

Technical references: [ATOMICS](#) [MEMORYMODEL](#) [THREADS](#)

Chapter 21 — Building a Concurrent Networked System

HarborRelay accepts a bounded request over a loopback TCP connection, validates it, and returns a bounded response. A small worker pool separates accepting connections from processing them. This capstone connects ownership, framing, partial I/O, thread lifetime, queue closure, and error reporting without pretending that a teaching server is an internet-ready service.

The application deliberately serves a specified number of connections and then drains its queue. That finite lifetime makes integration tests reproducible and exposes a shutdown policy that is often hidden in an endless accept loop. Authentication, TLS, authorization, admission control, and operational deployment remain separate production requirements.

A server can accept connections, create threads, and return correct answers while still lacking a safe shutdown path. Idle clients can occupy every worker, an unbounded queue can retain unlimited descriptors, and a failed worker can leave the acceptor waiting for capacity that never returns. A concurrent networked system needs one architecture connecting admission, ownership, protocol state, application logic, cancellation, and evidence.

The chapter first assembles the compact HarborRelay example from the original curriculum. It is a deliberately bounded loopback demonstration with a fixed connection count. The extended application in the appendix then adds explicit nonblocking progress, absolute operation deadlines, bounded nonblocking admission, independent stop state, and a broader failure-test matrix. The two implementations have different contracts; the larger one is not silently substituted for every claim made about the smaller teaching example.

21.1 Designing the Server Architecture

21.1.1 Separating Networking from Application Logic

The request processor accepts a string view and returns a response string. It knows nothing about file descriptors, threads, ports, or clocks. Its grammar is SUM followed by one to sixty-four decimal integers, each between minus one million and one million. This bounded domain also bounds the accumulator well within long long.

The transport adapter receives a length-prefixed frame, invokes the processor, and sends the result. A syntax error is an ordinary application response, while an incomplete frame is a transport failure. Keeping these categories separate makes it

possible to test malformed requests without opening sockets and to test the network contract without redesigning the arithmetic.

Separate the pure request processor from the transport adapter. The processor receives an owned or borrowed bounded request value and returns an owned response. It does not accept sockets, poll, launch threads, or modify process-wide configuration. This permits deterministic tests of syntax and arithmetic without requiring a live network. The transport layer remains responsible for framing, partial transfers, EOF classification, and connection ownership.

The example grammar is intentionally small: a command SUM followed by one to sixty-four bounded integers. Each token must parse completely; trailing junk is not accepted as a valid prefix. Numeric bounds make the accumulator's arithmetic representable. Unknown commands, missing values, too many values, and invalid numeric tokens produce explicit application error responses rather than transport exceptions.

A framing error and an application rejection belong to different layers. A correctly framed payload containing an unknown command is still a complete transport message. An oversized declared length is rejected before allocating or parsing that payload. A peer that closes halfway through a header has not submitted a complete request at all. Preserve these distinctions in metrics and tests.

Keep platform-specific code behind a narrow boundary. Linux descriptor ownership, nonblocking flags, poll events, and MSG_NOSIGNAL belong in the native adapter. The pure grammar and bounded arithmetic can remain portable C++. This separation does not guarantee that the entire server compiles on Windows; it identifies which components would need another implementation and which tests can be reused.

Dependencies should point toward policy rather than force policy to know every mechanism. A worker can call a request processor after receiving a frame. The request processor should not secretly reach back into global server state, allocate another listener, or launch unbounded tasks. Explicit interfaces make failure injection and independent correctness reasoning possible without replacing the real transport tests.

21.1.2 Managing Client Connections

Each accepted descriptor has one owner. The acceptor moves it into the queue, a worker moves it out, and the worker's local RAII wrapper closes it when processing ends. Queue insertion failure or exceptional unwinding also destroys the owned wrapper; no second component needs to guess who should close the connection.

The shared Linux support implements four-byte big-endian length framing with a 4,096-byte payload ceiling. It handles partial transfers and suppresses SIGPIPE on sends. The fixture binds only to loopback and uses bounded waits. Those constraints reduce laboratory hazards but are not an authentication boundary against other local processes.

A connection should have one clear owner at every stage: accepted by the listener, temporarily held by the acceptor, transferred into the queue if admitted, moved to a worker, and finally closed by that worker's scope. A rejected queue insertion must release or return the connection according to the API contract. Copying descriptor integers across these stages obscures who is responsible for closing them.

Bound admission before retaining unbounded work. A queue capacity limits queued items, while active workers and the acceptor can own additional descriptors. The extended server uses a nonblocking try-push policy: when the queue is full, the newly accepted connection is rejected and closed instead of blocking the acceptor indefinitely. This policy favors bounded retention and responsive stop checks over guaranteed admission of every arriving client.

Each accepted request also has a byte and time budget. The protocol caps payload size before allocation. An absolute deadline limits the allowed time for a complete framed read and response write in the extended implementation. The deadline is carried through partial progress, so a peer cannot indefinitely restart it by supplying one more byte. Ordinary OS scheduling still prevents this from being a hard real-time guarantee.

Connection cancellation should not depend on unsafely closing a descriptor from another thread while a worker is using it. The extended adapter uses nonblocking operations and bounded poll slices that recheck stop state. The owner closes the connection after leaving the operation. This preserves ownership and avoids assuming that a cross-thread close reliably interrupts every blocked call on every platform.

A listener's stop path must be independent of receiving a final client. A fixed connection count is useful for a deterministic elementary fixture but unsuitable as the only shutdown mechanism for a long-running service. The extended server checks an explicit stop state while waiting for new connections and while workers process active ones, then closes the queue and joins before destroying shared state.

Separating a Line Protocol Parser from the Socket Layer

Purpose

A scalable server is easier to test when networking transports bytes and application code interprets requests through a separate pure function.

Code or command

```
#include <iostream>
#include <optional>
#include <string_view>

std::optional<int> parse_request(std::string_view line){
    if(!line.starts_with("ADD ")) return std::nullopt;
    int value=0; for(char c:line.substr(4)){ if(c<='0' || c>'9') return
std::nullopt; value=value*10+(c-'0'); }
    return value;
}

int main(){ auto r=parse_request("ADD 42"); std::cout<<(r?
*r:-1)<<"\n"; }
```

What the important parts do

`parse_request` has no socket dependency; it receives a string view and returns either a validated value or `nullopt`. The networking layer can therefore manage connections independently of protocol correctness.

Expected behavior or output

The program prints `42` for the valid request.

Concrete experiment

Add tests for `ADD`, `ADD -1`, `ADD 999`, and `DELETE 1`. Decide which inputs belong to syntax validation versus higher-level application policy.

21.2 Scaling Concurrent Work

21.2.1 Implementing a Worker Pool

A fixed set of workers consumes accepted connections from a bounded queue. When the queue fills, the acceptor waits rather than allocating an unlimited number of pending tasks. The kernel's listen backlog is another queue with a separate capacity and failure behavior; bounding one queue does not eliminate all overload states.

Closing the work queue wakes idle workers and prevents additional insertions. Accepted items are drained before workers exit. A guard declared after the worker container closes the queue before `jthread` destruction can join workers during

exceptional startup. Per-connection failures are counted and isolated, while an acceptor failure is rethrown after orderly cleanup.

A worker pool separates task admission from execution. A fixed number of worker owners consume a bounded queue. This controls concurrency and avoids creating a new thread for every connection, but it adds queue and shutdown semantics that the one-thread-per-example model did not need. Define those semantics before choosing a container or thread-count default.

The compact chapter implementation uses two workers and a queue of four connections. Its accept loop handles a known number of clients, then closes and drains the queue. A close-before-join guard covers exceptional startup and accept paths. This demonstrates structured ownership, but per-call socket timeouts and a fixed accept count are not an independent total-deadline or service-stop design.

The extended implementation distinguishes normal completion, rejected admission, transport failure, and cancellation. Workers catch per-connection errors so one malformed or disconnected peer does not escape the thread entry function. A component-level failure still needs a supervisory policy; catching every exception is not permission to claim the server remains healthy after arbitrary resource exhaustion.

Queue closure and active cancellation can be combined deliberately. Closing prevents new admission and wakes idle workers. Workers can stop rather than drain every queued request when the documented shutdown policy chooses cancellation. Each queued owner must still be destroyed exactly once, and active owners remain alive until their operations observe stop or deadline and exit. The report should say whether pending work was completed or cancelled.

Startup must be exception-safe before the pool is fully populated. Create shared queue and stop state before workers, install the cleanup that can make blocked workers finish, and only then start additional threads. If worker creation fails partway through, close or cancel the queue, request stop where needed, and join already-started workers while all referenced state remains alive. This is a required architecture path, not a rare implementation detail to leave untested.

21.2.2 Measuring Throughput and Latency

Throughput counts completed work per unit of time; latency measures the delay experienced by a particular request. Report both with the workload definition, concurrency, message sizes, and error count. A server that rejects requests quickly can have excellent response timing without providing useful service.

The self-test records an end-to-end elapsed duration but makes no speed assertion. Its sequential clients establish functional behavior, not saturation throughput. A separate benchmark should vary concurrent clients, preserve correctness checks, record percentile distributions, and include queue waiting rather than timing only the arithmetic function.

Throughput measures completed work per unit time; latency measures how long an individual operation takes. A system can increase throughput by batching while

making individual latency worse. It can reduce average latency while leaving a long tail caused by queueing or slow peers. Report the metric and boundary being measured rather than calling every elapsed duration “performance.”

For the server, distinguish connection setup, queue delay, request read, processing, response write, and total client-observed duration. A worker-only timer excludes time waiting for admission. A client timer includes effects that may not belong to the request processor itself. Both can be useful, but they answer different questions and should not be compared as though their boundaries match.

Functional tests should assert protocol outcomes, ownership invariants, and bounded test completion, not demand a particular microsecond threshold on a shared test machine. A timeout fixture can use a generous outer bound to catch hangs while validating that the intended deadline error occurs. The exact scheduler delay around that deadline is an observation, not a hard real-time promise.

Capacity measurements need workload context: connection concurrency, payload distribution, computation per request, client behavior, CPU allocation, build type, and memory limits. A loopback benchmark with tiny sums primarily measures local scheduling and transport overhead. It does not predict WAN performance, TLS cost, persistent-storage latency, or throughput for a CPU-intensive application.

Observability should preserve enough categories to explain the system. Count admitted and rejected connections, completed responses, application rejections, transport failures, deadline expirations, and cancellations according to the actual implementation. Do not claim counters that the code does not expose. Protect payload privacy and bound log volume; error reporting can otherwise become another denial-of-service path in a service receiving arbitrary input.

Fixed Worker Pool with Queue Depth and Latency Measurements

Purpose

A worker pool bounds the number of execution threads while queue depth absorbs bursts. The section's throughput/latency concern is made observable by timing a known job count.

Code or command

```
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <deque>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int main(){
    using clock=std::chrono::steady_clock; std::mutex m;
    std::condition_variable cv; std::deque<int> jobs; bool done=false;
    std::atomic<int> processed{0};
    std::vector<std::jthread> workers; for(int w=0;w<2;++w)
workers.emplace_back([&]{for(;;){int job;{std::unique_lock
l(m);cv.wait(l,[&]{return done||!
jobs.empty();});if(jobs.empty()&&done)break;job=jobs.front();jobs.p
op_front();}(void)job;++processed;}}});
    auto start=clock::now(); {std::lock_guard l(m);for(int
i=0;i<10000;++i)jobs.push_back(i);done=true;} cv.notify_all();
workers.clear();
    auto
us=std::chrono::duration_cast<std::chrono::microseconds>(clock::now
()-start).count(); std::cout<<" processed<<" jobs in "<<us<<" us\n";
}
```

What the important parts do

Two workers repeatedly wait for queued jobs and exit only after `done` and an empty queue. The main thread enqueues 10,000 units, records elapsed time, then destroys the `jthread` workers to join them before reporting completion.

Expected behavior or output

The first printed field is `10000`; the elapsed microseconds vary by machine and are evidence only for that run.

Concrete experiment

Run with 1, 2, 4, and 8 workers while recording total time and maximum queue depth. Explain where more workers stop helping and whether queueing latency increases under overload.

Worked example — Build and exercise HarborRelay

Trace the implementation

The source/support directory contains the shared `bounded_queue.hpp` and `linux_support.hpp` used here. The queue is the implementation from Chapter 18; the transport helper is the bounded framing adapter used in Chapter 15. These files are supplied in full, not fetched from an external project. The CMake target links `relay.cpp` as a library and keeps the CLI in `main.cpp`.

Run the default program for its self-test. To inspect the client/server boundary interactively, start `b04_c21 --serve 1`, read the printed loopback port, and run `b04_c21 --request PORT "SUM 2 3"` in another terminal within the accept timeout. PORT must be replaced with the observed integer. The server exits after that one connection rather than remaining as an unmanaged background process.

The worker's descriptor stays alive through the entire receive/process/send sequence. completed counts a successfully transmitted response, including a valid error response to malformed application text; failed counts transport or processing exceptions. Those definitions prevent an invalid request from being confused with a broken connection. The self-test checks four independent connections and the complete response bytes.

A per-operation socket timeout is not a complete defense against clients that slowly deliver a long frame. A deployed service needs an absolute request deadline, authentication, connection limits, resource accounting, and a shutdown trigger independent of a known connection count. Treat those as acceptance criteria to implement and test, not as properties implied by the word concurrent.

Assemble the compact server without overstating it

The original worked program separates `process_request` from `serve_n`. The processor validates the bounded SUM grammar. The server accepts a fixed number of loopback connections, moves each accepted owner into a bounded queue, and lets two workers receive, process, and answer one framed request each. The supervisor closes the queue, joins workers, and checks the collected counters.

The default self-test sends a short series of valid and invalid application requests. It checks exact responses, including rejection categories, and reports an elapsed duration without asserting that the run must be faster than another implementation. A usage test checks the command-line boundary separately.

```
listener -> acceptor owner -> bounded queue -> worker owner
                                     -> read one frame
                                     -> pure SUM processor
                                     -> send one frame
                                     -> close connection
fixed accept count reached -> close queue -> drain -> join -> inspect counters
```

This is a complete executable for its stated finite exercise. It is not an internet-ready daemon. Its known-client-count termination and per-operation helper limits are intentionally visible. The appendix's extended HarborRelay code is a separate

application with independent stop state and absolute-deadline transport tests, so the reader can compare the additional design responsibilities explicitly.

Before running, identify the selected loopback port and the maximum number of retained connections implied by queue, workers, and acceptor. Do not expose the service on a wildcard address or remove payload limits to make a manual test more convenient.

Complete worked source

The current files for this example are under `source/book04/ch21/`. Listings are inserted from those files when this book is assembled.

relay.hpp

```
#ifndef HARBOR_RELAY_HPP
#define HARBOR_RELAY_HPP
#include <cstdint>
#include <string>
#include <string_view>
namespace harbor {
struct RelayStats { std::size_t completed{}, failed{}; };
std::string process_request(std::string_view request);
RelayStats serve_n(int listener, std::size_t connections);
}
#endif
```

relay.cpp

```
#include "relay.hpp"
#include "linux_support.hpp"
#include "bounded_queue.hpp"
#include <atomic>
#include <charconv>
#include <exception>
#include <sstream>
#include <thread>
#include <vector>

namespace harbor {
std::string process_request(std::string_view request) {
    if (request.size() > 4096) return "ERR size";
    std::istringstream input{std::string(request)};
    std::string token;
    if (!(input >> token) || token != "SUM") return "ERR command";
    long long sum = 0;
    std::size_t count = 0;
    while (input >> token) {
        if (++count > 64) return "ERR count";
        int value{};
        const auto [end, error] = std::from_chars(token.data(), token.data()
+token.size(), value);
        if (error != std::errc{} || end != token.data()+token.size() ||
            value < -1000000 || value > 1000000) return "ERR value";
        sum += value;
    }
    if (count == 0) return "ERR empty";
    return "OK " + std::to_string(sum);
}
RelayStats serve_n(int listener, std::size_t connections) {
    using harbor_linux::Fd;
```

```

    if (connections == 0 || connections > 1000)
        throw std::invalid_argument("connection count must be 1..1000");
    BoundedQueue<Fd> work{4};
    std::atomic<std::size_t> completed{}, failed{};
    std::vector<std::jthread> workers;
    struct CloseFirst {
        BoundedQueue<Fd>& queue;
        ~CloseFirst() { queue.close(); }
    } guard{work};
    for (int id = 0; id < 2; ++id) {
        workers.emplace_back([&] {
            while (auto connection = work.pop()) {
                try {
                    auto request = harbor_linux::receive_frame(connection->
>get());
                    if (!request) throw std::runtime_error("missing request");
                    const auto response = process_request(*request);
                    harbor_linux::send_frame(connection->get(), response);
                    completed.fetch_add(1, std::memory_order_relaxed);
                } catch (...) {
                    failed.fetch_add(1, std::memory_order_relaxed);
                }
            }
        });
    }
    std::exception_ptr accept_error;
    try {
        for (std::size_t i = 0; i < connections; ++i)
            if (!work.push(harbor_linux::accept_one(listener)))
                throw std::runtime_error("queue unexpectedly closed");
    } catch (...) { accept_error = std::current_exception(); }
    work.close();
    workers.clear();
    if (accept_error) std::rethrow_exception(accept_error);
    return {completed.load(), failed.load()};
}
}

```

main.cpp

```

#include "check.hpp"
#include "relay.hpp"
#include "linux_support.hpp"
#include <array>
#include <charconv>
#include <chrono>
#include <exception>
#include <string_view>
#include <thread>

unsigned number(std::string_view text, unsigned maximum) {
    unsigned result{};
    auto [end, error] = std::from_chars(text.data(), text.data()+text.size(),
result);
    if (error != std::errc{} || end != text.data()+text.size() || result == 0 ||
result > maximum)
        throw std::invalid_argument("numeric argument out of range");
    return result;
}

std::string request(unsigned port, std::string_view message) {
    auto connection =
harbor_linux::connect_loopback(static_cast<std::uint16_t>(port));
    harbor_linux::send_frame(connection.get(), message);
    auto response = harbor_linux::receive_frame(connection.get());
    if (!response) throw std::runtime_error("missing response");
}

```

```

        return *response;
    }
    int main(int argc, char** argv) {
        try {
            if (argc == 3 && std::string_view(argv[1]) == "--serve") {
                const auto count = number(argv[2], 1000);
                auto listener = harbor_linux::listen_loopback();
                std::cout << "127.0.0.1:" << listener.port << std::endl;
                auto stats = harbor::serve_n(listener.socket.get(), count);
                std::cout << "completed=" << stats.completed << " failed=" <<
stats.failed << '\n';
                return stats.failed == 0 ? 0 : 1;
            }
            if (argc == 4 && std::string_view(argv[1]) == "--request") {
                std::cout << request(number(argv[2], 65535), argv[3]) << '\n';
                return 0;
            }
            if (argc != 1) { std::cerr << "usage: relay [--serve COUNT | --request
PORT TEXT]\n"; return 2; }
            CHECK(harbor::process_request("SUM 2 -1 4") == "OK 5");
            CHECK(harbor::process_request("SUM") == "ERR empty");
            CHECK(harbor::process_request("SUM 4x") == "ERR value");
            auto listener = harbor_linux::listen_loopback();
            harbor::RelayStats stats;
            std::exception_ptr server_error;
            const auto start = std::chrono::steady_clock::now();
            std::jthread server([&] {
                try { stats = harbor::serve_n(listener.socket.get(), 4); }
                catch (...) { server_error = std::current_exception(); }
            });
            const std::array<std::string_view, 4> messages{"SUM 2 3", "BAD", "SUM -7
2", "SUM 1000001"};
            const std::array<std::string_view, 4> expected{"OK 5", "ERR command",
"OK -5", "ERR value"};
            for (std::size_t i = 0; i < messages.size(); ++i)
CHECK(request(listener.port, messages[i]) == expected[i]);
            server.join();
            if (server_error) std::rethrow_exception(server_error);
            CHECK(stats.completed == 4 && stats.failed == 0);
            const auto elapsed = std::chrono::duration<double,
std::milli>(std::chrono::steady_clock::now()-start).count();
            std::cout << "completed=" << stats.completed << " elapsed_ms=" <<
elapsed << "\nPASS\n";
            return 0;
        } catch (const std::exception& error) { std::cerr << error.what() << '\n';
return 1; }
    }

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 4 21
```

The runner configures this companion, builds the chapter target, and executes its fixture cases. See the verification report for the configurations actually run.

Platform boundary: the worked example uses LINUX facilities. Its native interfaces are not implemented for Windows.

Failure clinic — Healthy parser, unhealthy service

Every pure parser test passes, but an idle client occupies the only worker indefinitely. The parser has no input to inspect, so its correctness cannot solve the stalled transport. Add a deadline across the framed operation and ensure the wait path checks it while no bytes arrive.

Another server blocks its acceptor inside a full queue's push operation. The shutdown flag changes, but the acceptor cannot return to observe it because no consumer is freeing capacity. Choose a stop-aware blocking admission protocol or a nonblocking rejection policy. The extended application uses the latter to keep admission and shutdown bounded by explicit state rather than an unbounded capacity wait.

A worker catches an exception, increments a failure count, and exits permanently without notifying the supervisor. Repeated failures silently reduce the pool to zero workers while the listener continues accepting. A service-level failure policy must distinguish a per-connection rejection from a worker-loop failure. Keep the worker loop alive around expected connection errors or stop the service when its execution capacity is no longer trustworthy.

A final deployment report calls the service production-ready because all loopback tests passed. Those tests establish the implemented local contract under recorded conditions. Authentication, encryption, deployment supervision, resource limits, monitoring integration, threat review, and recovery remain separate work. The final acceptance checklist makes these limits explicit instead of converting a successful demo into an unsupported certification.

Practice and implementation lab

Add pure-function cases for more than sixty-four values, embedded nonnumeric tokens, and a maximum accepted sum. Then write a transport test that advertises a frame larger than 4,096 bytes and verify rejection before payload allocation.

Extend the server with an absolute request deadline and a stop mechanism. Preserve ownership on every exit path and ensure blocked accept, receive, and queue waits can all terminate. Measure end-to-end latency with multiple concurrent clients without removing the functional assertions.

Solution and review notes

Sixty-four values of one million produce 64,000,000, safely inside the stated accumulator contract. The sixty-fifth value must produce ERR count rather than silently truncating the request. An oversized advertised frame is rejected by `receive_frame` before constructing its payload string.

A stop token alone does not interrupt every operating-system call. A complete design can use nonblocking descriptors and a pollable wakeup descriptor, or another explicitly bounded I/O strategy, and propagate one deadline through all partial operations. The queue's close policy should be documented as drain or

cancel. Compare latency only after the stop and error paths pass reproducible tests.

Guided lab — Review the complete ownership and failure graph

Start with pure processor tests for valid sums, empty input, unknown command, malformed integer, too many values, and numeric bounds. Then run transport tests with fragmented headers, fragmented bodies, empty frames, oversized declarations, and truncated input. Keep the two test layers separate so a failure identifies the responsible contract.

Exercise the extended server's independent stop path with no clients, an idle accepted client, and completed clients. Verify that the listener and worker owners are joined before shared state is destroyed. Use controlled synchronization for startup; do not assume a server is ready merely because a thread has been created.

Test bounded admission deterministically at the queue level and exercise concurrent client integration separately. Real scheduling can make an exact rejection count nondeterministic under a burst, so assert accounting relationships and completed response correctness rather than one incidental ordering of connections.

Prepare a production-boundary review with requirements not implemented by the teaching service. Include peer identity, encrypted transport, process supervision, configuration validation, log retention, operational metrics, deployment rollback, and overload behavior. Mark each as implemented, tested, specified only, or out of scope. A truthful readiness record is more useful than an unqualified pass label.

Review reasoning. Does a correct parser stop an idle client? No. Does queue capacity equal all live connections? No. Does closing the listener automatically define active-connection cleanup? No. Does local throughput predict internet service performance? No. Why retain a pure processor? To test application semantics independently from transport scheduling and native resource behavior.

Completion criterion. Deliver a runnable loopback server, an ownership diagram from accept through close, a bounded failure matrix, and an honest readiness review. Every accepted resource should have one owner, every wait should have a completion or cancellation path, and every reported success should correspond to an observed contract rather than an assumed one.

Primary reference notes

Technical references: [SOCKETS](#) [THREADS](#) [CONDVAR](#) [MEMORYMODEL](#)

Appendix A — The Systems Programmer's Diagnostic Desk Reference

A useful diagnostic starts with the narrowest claim the evidence can support. “The descriptor is valid” is weaker than “the descriptor identifies the intended open resource.” “The thread returned” is weaker than “every accepted task completed.” “The payload parsed” is weaker than “the peer was authenticated.” This reference organizes recurring failures by the boundary that needs investigation, not by the last function name visible in a stack trace.

Preserve the first useful evidence

Record the operation, the relevant owned object, the exact return channel, the progress already made, and the intended next state. Capture `errno` immediately for interfaces that use it. Preserve direct error values for interfaces such as `posix_spawn` and `getaddrinfo` that do not follow the same convention. Do not overwrite the original cause with an unrelated cleanup error; report both when the completion policy requires it.

A diagnostic record should be bounded. Include numeric endpoints, resource roles, phase names, and counts rather than copying unlimited input or secret-bearing buffers. An attacker-controlled payload is not a safe log template. The teaching service uses category counters and does not log request contents. A production logging adapter would need its own retention, privacy, and failure behavior.

Symptom	Inspect first	Conclusion not yet justified
A newly opened descriptor is zero	Ownership and open return value	Zero is invalid. Negative values mark acquisition failure here.
Closing a descriptor reports an error	Linux close contract and saved error	Retrying that integer is safe.
A read returns fewer bytes	Returned count and accumulated state	The complete application record has arrived.
A pipe reader never reaches EOF	Every inherited write-end reference	The intended producer is still computing.
A socket read returns zero	Stream state and framing phase	A partially received request is valid.
A UDP receive returns zero	Datagram semantics and buffer status	The peer closed a byte stream.

Symptom	Inspect first	Conclusion not yet justified
Mapping succeeds but later access faults	File stability, extent, mapping lifetime	The original size check protects against concurrent truncation.
Layout changes between builds	Types, ABI, alignment, compiler options	The compiler violated the language.
A worker crashes the process	Exception escaping its initial callable	The creating thread can automatically catch that exception.
Shutdown blocks	Predicate transitions and join order	More notifications without state changes will repair it.
A counter loses updates	Atomic read-modify-write or lock scope	A separate atomic load and store form one increment.
Payload races despite an atomic flag	Publication and reuse handoffs	Atomicity of the flag protects all surrounding data.
A future times out	Producer ownership and cancellation policy	The task has stopped.
A server port is open	Request path through queue and workers	The application is ready to serve its contract.
A request fails after partial send	Accepted prefix and remote outcome uncertainty	Retrying cannot duplicate effects.
A benchmark changes under a sanitizer	Instrumentation and measurement boundary	The uninstrumented production performance changed by the same amount.

A resource incident, worked through

Suppose a service's descriptor count rises during rejected requests. Begin with a reproducible owned fixture, not by changing the process limit. Trace a connection from accept to queue admission. If admission rejects, determine whether the argument was consumed by value, returned to the caller, or leaked through an unowned integer. An ownership wrapper should close the rejected connection when its local value leaves scope.

Next examine the successful path. The worker receives one owner, processes one request, and destroys that owner after completion or failure. A reference to the native descriptor is only a borrow for a call. It must not be copied into a delayed task after the worker's owner is destroyed. Distinguish a leak from a stale-descriptor bug: the latter can close or use an unrelated resource after the number is reused.

Use a bounded repeated test and inspect resource state before and after quiescence. A changing count while work is in flight can be legitimate. Compare at

a point where all relevant workers have joined and all temporary owners have been destroyed. The same principle applies to allocations, mappings, and child processes: the observation boundary must match the lifecycle claim.

A stalled request, worked through

A worker waits for the rest of a frame. Identify whether it is reading the header or the body, how many bytes were accepted, the maximum allowed length, and the absolute deadline. If a per-call timeout is recreated on each read, a slowly progressing client can renew it indefinitely. The extended adapter carries the same deadline through the complete request and response operation.

A readiness event does not guarantee that the next nonblocking read completes the requested transfer. Retry temporary unavailability through the readiness loop while preserving progress and deadline. Interpret EOF according to the frame state. An error after a response prefix was sent leaves an externally visible partial effect; it is not a clean rollback to “nothing happened.”

When stop is requested, verify that every blocked operation can observe it. The extended server polls in short bounded slices and checks stop state rather than assuming a cross-thread close safely cancels another thread’s descriptor use. This is a cooperative design on an ordinary scheduler. A twenty-five-millisecond poll slice is not a promise of twenty-five-millisecond worst-case shutdown under all scheduling conditions.

A race investigation, worked through

Identify the conflicting accesses, the object whose lifetime contains them, and the synchronization relationship intended to order them. Do not stop at a line saying atomic or mutex. A mutex protects accesses only when all participating paths obey the same discipline. A release operation publishes data only through the corresponding observation relationship, and reuse requires its own handoff.

Run a race detector where the environment supports it, but preserve the source-level argument even when the tool reports no issue. Instrumentation observes executions; it does not enumerate all allowed schedules. Conversely, an unavailable race-detector runtime must be reported as unavailable, not converted into a clean test result. The package’s verification report distinguishes successful compiler/sanitizer runs from any unsupported diagnostic attempt.

The recovery question

For every incident, ask what changed outside the current process. Memory cleanup can release local owners; it cannot retract bytes already delivered, reverse a child program’s completed file mutation, or guarantee that a remote request was never executed. Recovery needs an application protocol when external effects are

involved. Local exception safety and distributed outcome certainty are different guarantees.

Close the investigation with a bounded regression case and a statement of the corrected invariant. “Added a sleep” is not a synchronization explanation. “Raised the buffer size” is not a framing explanation. “Retried every error” is not an ownership explanation. A repair is defensible when its mechanism addresses the specific failed boundary and its tests would distinguish that repair from the original misleading success.

Appendix B — Complete Systems Laboratories and Shared Support

These laboratories complement the compact chapter programs. Each is a real executable in the companion source tree, with checks that remain active in Release builds. The ordinary suite does not intentionally dereference invalid pointers, create data races, or operate on unowned devices. The separate optional sanitizer-negative example is not part of these passing cases.

Laboratory B.1 — Descriptor identity and open-description state

The descriptor lab creates and unlinks its own temporary file, marks the original descriptor close-on-exec, and duplicates it. The two descriptor numbers differ, their close-on-exec flags differ under the chosen dup operation, and their reads advance the same open-file offset. It then closes one owner and continues through the other. This distinguishes descriptor-local state from state shared through an open file description. ([OPEN](#), [POSIXCLOSE](#))

The one-byte reads and three-byte write are a controlled regular-file fixture whose exact counts are checked. They are not general transfer helpers. The lab fails rather than silently assuming progress if the environment reports a different result. General socket transfer loops appear in Appendix C. The temporary filename is removed only after the lab itself has successfully created it.

```
#include "native.hpp"
#include "check.hpp"
#include <array>
#include <algorithm>
#include <cerrno>
#include <cstdlib>
#include <fcntl.h>
#include <string>
#include <unistd.h>

namespace {
void transfer(harbor::Fd& target, harbor::Fd&& source) { target =
std::move(source); }
}
int main() {
    try {
        std::array<char, 26> name{};
        const std::string pattern = "/tmp/harbor-fd-XXXXXX";
        std::copy(pattern.begin(), pattern.end(), name.begin());
        const int raw = ::mkstemp(name.data());
        if (raw < 0) throw std::runtime_error("mkstemp");
        harbor::Fd first(raw);
        if (::unlink(name.data()) < 0) throw std::runtime_error("unlink owned
scratch file");
        CHECK(::fcntl(first.get(), F_SETFD, FD_CLOEXEC) == 0);
```

```

        CHECK(::write(first.get(), "abc", 3) == 3); // small regular-file
    fixture
        CHECK(::lseek(first.get(), 0, SEEK_SET) == 0);
        const int copied = ::dup(first.get());
        if (copied < 0) throw std::runtime_error("dup");
        harbor::Fd second(copied);
        CHECK(first.get() != second.get());
        CHECK(::fcntl(first.get(), F_GETFD) & FD_CLOEXEC != 0);
        CHECK(::fcntl(second.get(), F_GETFD) & FD_CLOEXEC == 0);
        char a = 0, b = 0, c = 0;
        CHECK(::read(first.get(), &a, 1) == 1);
        CHECK(::read(second.get(), &b, 1) == 1);
        CHECK(a == 'a' && b == 'b'); // duplicated descriptors share offset
        errno = E2BIG;
        first.reset();
        CHECK(errno == E2BIG);
        CHECK(first.get() == -1);
        harbor::Fd moved(std::move(second));
        CHECK(second.get() == -1);
        CHECK(::read(moved.get(), &c, 1) == 1 && c == 'c');
        CHECK(::read(moved.get(), &c, 1) == 0);
        transfer(moved, std::move(moved)); // aliased self-move through the
owner interface
        CHECK(moved.get() >= 0);
        std::cout << "PASS descriptor ownership, shared offset, and per-
descriptor flags\n";
        return 0;
    } catch (const std::exception& e) { std::cerr << e.what() << '\n'; return
1; }
}

```

Reason through the result. Closing the first descriptor does not close the duplicate. Moving the duplicate transfers its one ownership obligation. The test's self-move goes through the owner's public move-assignment contract. It does not re-adopt a borrowed integer through reset; that operation requires a genuinely new ownership value or the invalid sentinel.

Laboratory B.2 — Framing under every bounded three-chunk split

The frame lab constructs deterministic payloads of lengths zero through sixty-four and feeds their wire encodings at every pair of chunk boundaries. Some chunks are empty, some split the header, and others split the payload. Every history must yield the same exact byte string. A manually specified wire value with byte 255 provides an independent check rather than relying on round-trip consistency alone.

The shared FrameDecoder implementation is listed in Appendix C. Its returned consumed count stops at the end of one frame, allowing the caller to retain subsequent bytes for another decoder. Failed decoders cannot be reused as though a framing error had not happened. The separate registered codec cases test oversized headers, maximum-length payloads, and truncated input.

```

#include "frame.hpp"
#include "check.hpp"
#include <iostream>

```

```

#include <span>
#include <string>

int main() {
    try {
        std::size_t trials = 0;
        for (std::size_t size = 0; size <= 64; ++size) {
            std::string original(size, '\\0');
            for (std::size_t i = 0; i < size; ++i)
                original[i] = static_cast<char>((i * 37 + size) & 255);
            auto wire = harbor::encode_frame(original);
            // Exercise every pair of chunk boundaries for each bounded payload.
            for (std::size_t a = 0; a <= wire.size(); ++a) {
                for (std::size_t b = a; b <= wire.size(); ++b) {
                    harbor::FrameDecoder decoder;
                    CHECK(decoder.feed(std::span(wire).first(a)) == a);
                    CHECK(decoder.feed(std::span(wire).subspan(a, b - a)) == b -
a);
                    CHECK(decoder.feed(std::span(wire).subspan(b)) ==
wire.size() - b);
                    CHECK(decoder.finish()); CHECK(decoder.payload() ==
original);
                    ++trials;
                }
            }
            // A manually specified independent wire value, not round-trip alone.
            const std::byte bytes[]{std::byte{0}, std::byte{0}, std::byte{0},
std::byte{1}, std::byte{255}};
            harbor::FrameDecoder one;
            CHECK(one.feed(bytes) == 5);
            CHECK(one.payload().size() == 1);
            CHECK(static_cast<unsigned char>(one.payload())[0] == 255);
            std::cout << "PASS chunk_histories=" << trials << '\\n';
            return 0;
        } catch (const std::exception& e) { std::cerr << e.what() << '\\n'; return
1; }
    }
}

```

Reason through the result. This finite exhaustive split domain tests parser state transitions, not every possible network schedule. Actual socket integration is tested separately. A sequence of one-byte send calls does not force one-byte receive calls, so deterministic chunk tests remain valuable even when the real socket fixture also succeeds.

Laboratory B.3 — Accepted-item conservation and startup cleanup

Two producers submit disjoint ID ranges through a capacity-three blocking queue, while three consumers collect into separate vectors. After joining, the test combines and sorts the observed IDs. It expects exactly the accepted set with no duplicates or omissions. It does not require an arbitrary total order among concurrent producers.

The lab also injects failure after a consumer starts but before its partner is launched. A close-before-join guard changes the waiting predicate during unwinding. The already-started consumer exits while its queue remains alive. This

deterministic seam tests the lifecycle policy without exhausting actual operating-system thread resources.

```
#include "bounded_queue.hpp"
#include "check.hpp"
#include <algorithm>
#include <array>
#include <atomic>
#include <exception>
#include <iostream>
#include <thread>
#include <vector>

namespace {
struct CloseQueue {
    BoundedQueue<int>& queue;
    ~CloseQueue() { queue.close(); }
};
void partial_startup(std::atomic<bool>& finished) {
    BoundedQueue<int> queue(1);
    std::vector<std::jthread> workers;
    CloseQueue cleanup{queue}; // closes before workers join on exception
    workers.emplace_back([&] { while (queue.pop()) {} finished.store(true); });
    throw std::runtime_error("injected next-thread startup failure");
}
}

int main() {
    try {
        BoundedQueue<int> queue(3);
        std::array<std::vector<int>, 3> outputs;
        std::array<std::exception_ptr, 5> errors;
        std::vector<std::jthread> consumers, producers;
        CloseQueue cleanup{queue};
        for (std::size_t c = 0; c < outputs.size(); ++c)
            consumers.emplace_back([&, c] {
                try { while (auto item = queue.pop()) outputs[c].push_back(*item); }
                catch (...) { errors[c] = std::current_exception(); queue.close(); }
            });
        for (int p = 0; p < 2; ++p) producers.emplace_back([&, p] {
            try { for (int i = 0; i < 500; ++i)
                if (!queue.push(p * 500 + i)) throw
                    std::runtime_error("unexpected closure"); }
            catch (...) { errors[3 + p] = std::current_exception();
                queue.close(); }
            });
        producers.clear(); queue.close(); consumers.clear();
        for (auto error : errors) if (error) std::rethrow_exception(error);
        std::vector<int> collected;
        for (auto& values : outputs) collected.insert(collected.end(),
            values.begin(), values.end());
        std::sort(collected.begin(), collected.end());
        CHECK(collected.size() == 1000);
        for (int i = 0; i < 1000; ++i)
            CHECK(collected[static_cast<std::size_t>(i)] == i);
        CHECK(!queue.push(1001)); CHECK(!queue.pop());
        std::atomic<bool> finished{false};
        bool injected = false;
        try { partial_startup(finished); } catch (const std::runtime_error&)
        { injected = true; }
        CHECK(injected); CHECK(finished.load());
        std::cout << "PASS 1000 unique accepted items and partial-startup
            cleanup\n";
        return 0;
    } catch (const std::exception& e) { std::cerr << e.what() << '\n'; return
```

```
1; }  
}
```

Reason through the result. Each consumer owns its output vector, and the supervisor reads them only after joining. Shared queue state is protected internally. The guard's declaration order ensures it closes before worker-vector destruction joins. Moving the guard earlier would change exceptional cleanup order and could reintroduce the hang the test is designed to prevent.

Laboratory B.4 — A reusable single-producer/single-consumer handoff

The publication lab adds the reverse ownership handoff missing from a reusable Boolean-ready sketch. State zero grants the producer access to the payload; state one grants it to the consumer; state two closes the channel. Release/acquire operations transfer permission in both directions. Compare-exchange prevents either participant from overwriting the terminal closed state during a concurrent stop.

This is exactly one producer and one consumer with one integer slot. It is not a multi-producer queue, not a lock-free reclamation scheme, and not a guarantee that the standard library's atomic waiting implementation never uses operating-system blocking. A close can abandon an in-flight value; the lab's ordinary path instead transfers all five thousand values before closure.

```
#include "check.hpp"  
#include <atomic>  
#include <exception>  
#include <iostream>  
#include <optional>  
#include <thread>  
#include <vector>  
  
namespace {  
    // Exactly one producer and one consumer. State 0 grants the producer access;  
    // state 1 grants the consumer access; state 2 is terminal. close never reopens.  
    class Handoff {  
        std::atomic<unsigned> state_{0};  
        int payload_ = 0;  
    public:  
        bool publish(int value) {  
            unsigned state = state_.load(std::memory_order_acquire);  
            while (state == 1) {  
                state_.wait(1, std::memory_order_acquire);  
                state = state_.load(std::memory_order_acquire);  
            }  
            if (state == 2) return false;  
            payload_ = value; // consumer's prior release granted reuse  
            unsigned expected = 0;  
            if (!state_.compare_exchange_strong(expected, 1,  
std::memory_order_release,  
std::memory_order_relaxed)) return  
false;  
            state_.notify_one();  
            return true;  
        }  
        std::optional<int> consume() {
```

```

        unsigned state = state_.load(std::memory_order_acquire);
        while (state == 0) {
            state_.wait(0, std::memory_order_acquire);
            state = state_.load(std::memory_order_acquire);
        }
        if (state == 2) return std::nullopt;
        const int value = payload_;
        unsigned expected = 1;
        state_.compare_exchange_strong(expected, 0, std::memory_order_release,
                                       std::memory_order_relaxed);
        state_.notify_one(); // CAS cannot overwrite terminal state 2
        return value;
    }
    void close() noexcept {
        state_.store(2, std::memory_order_release);
        state_.notify_all();
    }
};
struct Close { Handoff& channel; ~Close() { channel.close(); } };
void injected_failure() {
    Handoff channel;
    std::vector<std::jthread> workers;
    Close cleanup{channel};
    workers.emplace_back([&] { for (int i = 0; i < 5000; ++i)
        if (!channel.publish(i)) break; });
    throw std::runtime_error("injected missing consumer");
}
}
int main() {
    try {
        Handoff channel;
        std::vector<int> received;
        received.reserve(5000); // bounded, nonthrowing push for these int
values
        std::vector<std::jthread> workers;
        Close cleanup{channel};
        workers.emplace_back([&] { for (int i = 0; i < 5000; ++i)
            if (!channel.publish(i)) break; });
        workers.emplace_back([&] { for (int i = 0; i < 5000; ++i) {
            auto value = channel.consume();
            if (!value) break;
            received.push_back(*value);
        }});
        workers.clear();
        CHECK(received.size() == 5000);
        for (int i = 0; i < 5000; ++i)
CHECK(received[static_cast<std::size_t>(i)] == i);
        channel.close(); CHECK(!channel.publish(9)); CHECK(!channel.consume());
        bool injected = false;
        try { injected_failure(); } catch (const std::runtime_error&) { injected
= true; }
        CHECK(injected);
        std::cout << "PASS 5000 two-way ownership handoffs and cancelled
startup\n";
        return 0;
    } catch (const std::exception& e) { std::cerr << e.what() << '\n'; return
1; }
}

```

Reason through the result. The consumer's release of state zero orders its completed read before the producer's next write. The producer's release of state one orders its write before the consumer's read. Terminal state prevents reopening during cancellation. The cleanup guard remains outside both workers and closes

before joining during injected startup failure. Payload lifetime still ends only after those joins.

Shared assertion helper

The checks throw an exception with source location when a condition fails. They are not compiled out by NDEBUG, which is why Release fixture execution remains meaningful. Worker functions must not let these exceptions escape a thread entry point; they either avoid checks inside the worker or catch and transfer failures to the supervisor.

```
#ifndef HARBOR_SERIES_CHECK_HPP
#define HARBOR_SERIES_CHECK_HPP
#include <iostream>
#include <source_location>
#include <stdexcept>
#include <string>

namespace harbor_test {
inline void check(bool passed, const char* expression,
                  std::source_location location =
std::source_location::current()) {
    if (!passed) {
        throw std::runtime_error(std::string(location.file_name()) + ":" +
                                std::to_string(location.line()) + ": check failed: " + expression);
    }
}
}
#define CHECK(...) ::harbor_test::check(static_cast<bool>((__VA_ARGS__)),
#_VA_ARGS__)
#endif
```

Shared blocking queue used by Chapters 18 and 21

This queue is element-count bounded and drains accepted items after closure. Its by-value push can consume a caller argument before returning false. The deque may allocate during insertion, and the item type must be nothrow move-constructible. Keep these properties distinct from the extended server's nonblocking admission queue in Appendix C.

```
#ifndef HARBOR_SERIES_BOUNDED_QUEUE_HPP
#define HARBOR_SERIES_BOUNDED_QUEUE_HPP
#include <condition_variable>
#include <cstdint>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <type_traits>
#include <utility>

template<class T>
class BoundedQueue {
    static_assert(std::is_nothrow_move_constructible_v<T>);
    std::mutex mutex_;
    std::condition_variable not_empty_, not_full_;
```

```

        std::deque<T> data_;
        std::size_t capacity_;
        bool closed_{};
public:
    explicit BoundedQueue(std::size_t capacity) : capacity_(capacity) {
        if (capacity == 0) throw std::invalid_argument("queue capacity");
    }
    BoundedQueue(const BoundedQueue&) = delete;
    BoundedQueue& operator=(const BoundedQueue&) = delete;
    bool push(T value) {
        std::unique_lock lock{mutex_};
        not_full_.wait(lock, [&] { return closed_ || data_.size() <
capacity_; });
        if (closed_) return false;
        data_.push_back(std::move(value));
        lock.unlock();
        not_empty_.notify_one();
        return true;
    }
    std::optional<T> pop() {
        std::unique_lock lock{mutex_};
        not_empty_.wait(lock, [&] { return closed_ || !data_.empty(); });
        if (data_.empty()) return std::nullopt;
        T value = std::move(data_.front());
        data_.pop_front();
        lock.unlock();
        not_full_.notify_one();
        return value;
    }
    void close() {
        { std::lock_guard lock{mutex_}; closed_ = true; }
        not_empty_.notify_all();
        not_full_.notify_all();
    }
};
#endif

```

Shared elementary Linux adapter

The compact chapter examples use the following adapter for descriptor ownership, exact byte transfers, bounded framing, and loopback endpoints. Its socket timeouts and readiness waits are useful for controlled finite examples. They do not form one absolute deadline across every header, body, response, and admission stage. The extended adapter in Appendix C supplies that stronger operation-level mechanism separately.

```

#ifndef HARBOR_SERIES_LINUX_SUPPORT_HPP
#define HARBOR_SERIES_LINUX_SUPPORT_HPP
// Linux-only teaching support. Sockets bind/connect only to IPv4 loopback.
#include <array>
#include <cerrno>
#include <chrono>
#include <cstdint>
#include <optional>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <system_error>
#include <utility>

```

```

#include <arpa/inet.h>
#include <fcntl.h>
#include <netinet/in.h>
#include <poll.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <unistd.h>

namespace harbor_linux {
[[noreturn]] inline void fail(const char* operation) {
    const int error = errno;
    throw std::system_error(error, std::generic_category(), operation);
}
class Fd {
    int value_{-1};
public:
    explicit Fd(int value = -1) noexcept : value_(value) {}
    Fd(const Fd&) = delete;
    Fd& operator=(const Fd&) = delete;
    Fd(Fd&& other) noexcept : value_(other.release()) {}
    Fd& operator=(Fd&& other) noexcept {
        if (this != &other) { reset(); value_ = other.release(); }
        return *this;
    }
    ~Fd() { reset(); }
    int get() const noexcept { return value_; }
    explicit operator bool() const noexcept { return value_ >= 0; }
    int release() noexcept { return std::exchange(value_, -1); }
    void reset() noexcept {
        const int saved = errno;
        if (value_ >= 0) (void)::close(release());
        errno = saved;
    }
};
inline Fd checked_fd(int value, const char* operation) {
    if (value < 0) fail(operation);
    return Fd{value};
}
inline void write_all(int fd, std::span<const std::byte> data) {
    while (!data.empty()) {
        const auto count = ::write(fd, data.data(), data.size());
        if (count < 0) { if (errno == EINTR) continue; fail("write"); }
        if (count == 0) throw std::runtime_error("write made no progress");
        data = data.subspan(static_cast<std::size_t>(count));
    }
}
// false: clean EOF before any requested byte; partial EOF is a framing error.
inline bool read_exact(int fd, std::span<std::byte> data) {
    const auto original = data.size();
    while (!data.empty()) {
        const auto count = ::read(fd, data.data(), data.size());
        if (count < 0) { if (errno == EINTR) continue; fail("read"); }
        if (count == 0) {
            if (data.size() == original) return false;
            throw std::runtime_error("truncated input");
        }
        data = data.subspan(static_cast<std::size_t>(count));
    }
    return true;
}
inline void socket_timeout(int fd) {
    const timeval timeout{5, 0};
    if (::setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) <
0) fail("receive timeout");
    if (::setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &timeout, sizeof(timeout)) <
0) fail("send timeout");
}

```

```

}
inline void wait_readable(int fd, std::chrono::milliseconds duration =
std::chrono::seconds{5}) {
    const auto deadline = std::chrono::steady_clock::now() + duration;
    for (;;) {
        const auto remaining =
std::chrono::duration_cast<std::chrono::milliseconds>(deadline -
std::chrono::steady_clock::now());
        if (remaining.count() <= 0) throw std::runtime_error("readiness
timeout");
        pollfd item{fd, POLLIN, 0};
        const int result = ::poll(&item, 1,
static_cast<int>(remaining.count()));
        if (result < 0) { if (errno == EINTR) continue; fail("poll"); }
        if (result == 0) throw std::runtime_error("readiness timeout");
        if (item.revents & POLLNVAL) throw std::runtime_error("invalid
descriptor in poll");
        if (item.revents & (POLLIN | POLLHUP | POLLERR)) return;
    }
}
inline void send_all(int fd, std::span<const std::byte> data) {
    while (!data.empty()) {
        const auto count = ::send(fd, data.data(), data.size(), MSG_NOSIGNAL);
        if (count < 0) { if (errno == EINTR) continue; fail("send"); }
        if (count == 0) throw std::runtime_error("send made no progress");
        data = data.subspan(static_cast<std::size_t>(count));
    }
}
inline void send_frame(int fd, std::string_view text) {
    if (text.size() > 4096) throw std::length_error("frame exceeds 4096 bytes");
    const auto length = static_cast<std::uint32_t>(text.size());
    const std::array<std::byte, 4> header{
        std::byte((length >> 24) & 255U), std::byte((length >> 16) & 255U),
        std::byte((length >> 8) & 255U), std::byte(length & 255U)};
    send_all(fd, header);
    send_all(fd, std::as_bytes(std::span{text.data(), text.size()}));
}
inline std::optional<std::string> receive_frame(int fd) {
    std::array<std::byte, 4> header{};
    if (!read_exact(fd, header)) return std::nullopt;
    std::uint32_t length = 0;
    for (const auto byte : header) length = (length << 8) |
std::to_integer<unsigned char>(byte);
    if (length > 4096) throw std::length_error("frame exceeds 4096 bytes");
    std::string result(length, '\0');
    if (!read_exact(fd, std::as_writable_bytes(std::span{result.data(),
result.size()})))
        throw std::runtime_error("missing frame body");
    return result;
}
struct Listener { Fd socket; std::uint16_t port; };
inline Listener listen_loopback() {
    auto socket = checked_fd(::socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0),
"socket");
    socket_timeout(socket.get());
    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = 0;
    if (::bind(socket.get(), reinterpret_cast<const sockaddr*>(&address),
sizeof(address)) < 0) fail("bind");
    if (::listen(socket.get(), 16) < 0) fail("listen");
    socklen_t size = sizeof(address);
    if (::getsockname(socket.get(), reinterpret_cast<sockaddr*>(&address),
&size) < 0) fail("getsockname");
    return {std::move(socket), ntohs(address.sin_port)};
}

```

```

}
inline Fd accept_one(int listener) {
    wait_readable(listener);
    const int raw = ::accept4(listener, nullptr, nullptr, SOCK_CLOEXEC);
    auto accepted = checked_fd(raw, "accept4");
    socket_timeout(accepted.get());
    return accepted;
}
inline Fd connect_loopback(std::uint16_t port) {
    auto socket = checked_fd(::socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0),
"socket");
    socket_timeout(socket.get());
    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons(port);
    if (::connect(socket.get(), reinterpret_cast<const sockaddr*>(&address),
sizeof(address)) < 0) fail("connect");
    return socket;
}
}
#endif

```

Running the laboratories

```

cmake -S source -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j 2
ctest --test-dir build --output-on-failure -R '^supplemental_'

```

The descriptor lab requires Linux. The frame, queue, and publication laboratories use C++20 facilities, but execution on other operating systems remains a separate validation task. Read the actual test report for the configurations run while preparing this edition.

Appendix C — HarborRelay: A Bounded, Cancellable Loopback Service

HarborRelay brings the volume’s mechanisms together in a separate multi-file application. It accepts one length-prefixed request per connection, computes a bounded integer sum, returns a framed result, and releases the connection. It has explicit admission capacity, a fixed worker count, a request deadline beginning at acceptance, and a stop path that does not depend on another client arriving.

The application is deliberately local and educational. It does not implement TLS, authentication, durable state changes, rate limiting by identity, service-manager integration, or a production threat model. Its fixed maximums bound particular resource dimensions; they do not make the process allocation-free or establish a hard-real-time response guarantee. Those limits are part of the specification, not fine print added after the demonstration.

C.1 Requirements and acceptance contract

The wire format is four unsigned length bytes in network order followed by at most 4096 payload bytes. An empty payload is a valid frame but an invalid SUM command. The application grammar recognizes the uppercase token SUM followed by one to sixty-four decimal integers, each between minus one million and one million. Parsing uses the classic locale for token separation and `from_chars` for whole-token integer conversion. A leading plus sign is rejected by the chosen integer parser. Embedded zero bytes do not terminate the frame and make an otherwise numeric token invalid.

Application errors are framed responses: ERR command, ERR empty, ERR count, ERR value, or ERR size at the pure processor boundary. Transport errors instead terminate the connection and update a category counter. A network request declaring more than 4096 bytes is rejected at the framing boundary before the processor is called, so it does not receive a promise of an ERR size response.

The server defaults to two workers, four queued connections, and a one-second request budget. The budget starts when accept returns, not when a worker finally dequeues the connection. Queue delay therefore consumes the same request budget as reading, processing, and writing. A queued request already past its deadline fails when a worker begins its operation rather than receiving a fresh allowance.

Admission is nonblocking. A full or closed inbox rejects the newly accepted owned socket, whose destruction closes it. This is a resource policy, not a negotiated overload response protocol. At quiescence after joined shutdown, accepted connections equal admitted plus rejected, and admitted connections equal

completed plus failed plus cancelled. Application-error responses are a subset of completed responses; timeouts are a subset of transport failures. Live snapshots use separate atomic loads and are not promised to form a simultaneous accounting snapshot.

C.2 Independent byte framing

The encoder and incremental decoder are independent of sockets and threads. They are used by deterministic split and boundary tests. Native receive uses the same declared length limit, while exact-transfer helpers deal with the operating system's progress and termination behavior.

```
#pragma once
#include <array>
#include <cstdint>
#include <climits>
#include <cstdint>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace harbor {
static_assert(CHAR_BIT == 8, "Wire protocol requires eight-bit bytes");
inline constexpr std::size_t max_payload = 4096;
inline std::size_t decode_length(std::span<const std::byte, 4> h) {
    std::uint32_t n = 0;
    for (auto b : h) n = (n << 8) | std::to_integer<unsigned>(b);
    if (n > max_payload) throw std::length_error("frame exceeds 4096 bytes");
    return n;
}
inline std::vector<std::byte> encode_frame(std::string_view payload) {
    if (payload.size() > max_payload) throw std::length_error("payload too large");
    const auto n = static_cast<std::uint32_t>(payload.size());
    std::vector<std::byte> bytes(4 + payload.size());
    for (unsigned i = 0; i != 4; ++i)
        bytes[i] = static_cast<std::byte>((n >> (24 - 8 * i)) & 255u);
    for (std::size_t i = 0; i != payload.size(); ++i)
        bytes[i + 4] = static_cast<std::byte>(static_cast<unsigned char>(payload[i]));
    return bytes;
}
// One frame per decoder. feed returns how many bytes it consumed, leaving
// any next frame to the caller. No unbounded input buffer is retained.
class FrameDecoder {
    std::array<std::byte, 4> header_;
    std::size_t header_used_ = 0;
    std::size_t needed_ = 0;
    bool complete_ = false;
    bool failed_ = false;
    std::string payload_;
public:
    std::size_t feed(std::span<const std::byte> input) {
        if (failed_) throw std::logic_error("decoder is failed");
        std::size_t consumed = 0;
        try {
            while (consumed < input.size() && !complete_) {
                if (header_used_ < 4) {
```

```

        header_[header_used_++] = input[consumed++];
        if (header_used_ == 4) {
            needed_ = decode_length(header_);
            payload_.reserve(needed_);
            complete_ = needed_ == 0;
        }
    } else {
        payload_.push_back(static_cast<char>(
            std::to_integer<unsigned char>(input[consumed++])));
        complete_ = payload_.size() == needed_;
    }
}
} catch (...) { failed_ = true; throw; }
return consumed;
}
bool complete() const noexcept { return complete_; }
bool finish() const {
    if (failed_) throw std::logic_error("decoder is failed");
    if (!complete_ && header_used_ != 0)
        throw std::runtime_error("truncated frame");
    return complete_; // false is clean EOF before a new header
}
const std::string& payload() const {
    if (!complete_ || failed_) throw std::logic_error("frame not complete");
    return payload_; // borrowed until this decoder is changed/destroyed
}
};
} // namespace harbor

```

The decoder retains only a four-byte header and a payload bounded by the accepted length. A completed decoder consumes no bytes from a subsequent frame. If an allocation or parsing error occurs during feed, the decoder enters a failed state rather than offering an ambiguous partial result. The payload accessor is borrowed; copy the returned string if it must outlive the decoder.

C.3 Bounded admission queue

The inbox uses a mutex and predicate wait for consumers, but producers use `try_push` and never wait for space. It drains accepted owners after closure. During server cancellation, workers still pop those owners, immediately observe stop in their operation, classify cancellation, and release the descriptors. This keeps accepted-item accounting explicit.

```

#pragma once
#include <condition_variable>
#include <cstdint>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <type_traits>
#include <utility>

namespace harbor {
// Element-count bounded, not allocation-free. try_push consumes its value
// argument even on rejection; resource-owning callers must account for that.
template<class T> class BoundedInbox {
    static_assert(std::is_nothrow_move_constructible_v<T>);
    std::mutex mutex_;

```

```

        std::condition_variable changed_;
        std::deque<T> values_;
        std::size_t capacity_;
        bool closed_ = false;
public:
    explicit BoundedInbox(std::size_t capacity) : capacity_(capacity) {
        if (capacity == 0) throw std::invalid_argument("zero inbox capacity");
    }
    bool try_push(T value) {
        {
            std::lock_guard lock(mutex_);
            if (closed_ || values_.size() == capacity_) return false;
            values_.push_back(std::move(value));
        }
        changed_.notify_one();
        return true;
    }
    std::optional<T> pop() {
        std::unique_lock lock(mutex_);
        changed_.wait(lock, [&] { return closed_ || !values_.empty(); });
        if (values_.empty()) return std::nullopt;
        T value = std::move(values_.front());
        values_.pop_front();
        return value;
    }
    void close() {
        { std::lock_guard lock(mutex_); closed_ = true; }
        changed_.notify_all();
    }
};
} // namespace harbor

```

The capacity is an element limit. Deque insertion can allocate, and the accepting thread temporarily owns one additional connection. With two workers and queue capacity four, count active, queued, listener, and transient owners separately. The source does not claim a universal fixed descriptor total across the whole process.

C.4 Native ownership and operation state

An operation carries one absolute steady-clock deadline and an optional stop-state pointer. The pointer is borrowed from server state that outlives every worker. Native errors preserve a category and, where applicable, the original error number. The descriptor owner closes once under the Linux policy and preserves surrounding `errno` during destructor cleanup.

```

#pragma once
#ifdef __linux__
#error "HarborRelay native adapter requires Linux"
#endif
#include <atomic>
#include <chrono>
#include <cstdint>
#include <optional>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <utility>

namespace harbor {

```

```

class Fd {
    int fd_ = -1;
public:
    explicit Fd(int fd = -1) noexcept : fd_(fd) {}
    ~Fd();
    Fd(const Fd&) = delete;
    Fd& operator=(const Fd&) = delete;
    Fd(Fd&& other) noexcept : fd_(std::exchange(other.fd_, -1)) {}
    Fd& operator=(Fd&& other) noexcept;
    int get() const noexcept { return fd_; }
    int release() noexcept { return std::exchange(fd_, -1); }
    void reset(int fd = -1) noexcept;
};
enum class Failure { io, timeout, cancelled, truncated, oversize };
class TransportError : public std::runtime_error {
public:
    Failure category;
    int native_error;
    TransportError(Failure why, std::string what, int error = 0)
        : std::runtime_error(std::move(what)), category(why),
        native_error(error) {}
};
using Clock = std::chrono::steady_clock;
struct Operation {
    Clock::time_point deadline;
    const std::atomic<bool>* stop = nullptr;
    void check() const;
};
void nonblocking(int fd);
Fd listen_local(unsigned& selected_port);
Fd connect_local(unsigned port, const Operation& operation);
bool read_exact(int fd, std::span<std::byte> bytes, const Operation& operation);
void write_exact(int fd, std::span<const std::byte> bytes, const Operation& operation);
std::optional<std::string> receive_frame(int fd, const Operation& operation);
void send_frame(int fd, std::string_view payload, const Operation& operation);
} // namespace harbor

```

The adapter creates nonblocking sockets, handles partial transfer progress, retries interrupted operations, and waits after temporary unavailability. Polling uses at most twenty-five-millisecond slices to revisit stop and deadline state. Ready events are followed by actual I/O; a readiness flag alone is not interpreted as a complete message. Calls are chunked to a modest maximum rather than passing an arbitrarily large span length directly to the native interface.

```

#include "native.hpp"
#include "frame.hpp"
#include <algorithm>
#include <array>
#include <cerrno>
#include <fcntl.h>
#include <netinet/in.h>
#include <poll.h>
#include <sys/socket.h>
#include <unistd.h>

namespace harbor {
Fd::~Fd() { reset(); }
Fd& Fd::operator=(Fd&& other) noexcept {
    if (this != &other) { reset(); fd_ = other.release(); }
    return *this;
}
void Fd::reset(int fd) noexcept {

```

```

        const int old = std::exchange(fd_, fd);
        if (old >= 0) {
            const int saved = errno;
            ::close(old); // Linux: never retry the released descriptor number
            errno = saved;
        }
    }
}

namespace {
[[noreturn]] void io_error(const char* operation, int error = errno) {
    throw TransportError(Failure::io, std::string(operation) + " (errno " +
        std::to_string(error) + ")", error);
}

void ready(int fd, short events, const Operation& operation) {
    for (;;) {
        operation.check();
        auto left = std::chrono::ceil<std::chrono::milliseconds>(
            operation.deadline - Clock::now());
        const auto ms = std::clamp<long long>(left.count(), 1, 25);
        pollfd item{fd, events, 0};
        const int result = ::poll(&item, 1, static_cast<int>(ms));
        if (result > 0) {
            if (item.revents & POLLNVAL) io_error("poll invalid fd", EBADF);
            operation.check();
            return; // actual I/O interprets readiness, EOF, HUP, or error
        }
        if (result < 0 && errno != EINTR) io_error("poll");
    }
}

Fd new_socket() {
    const int fd = ::socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC | SOCK_NONBLOCK,
0);
    if (fd < 0) io_error("socket");
    return Fd(fd);
}

void Operation::check() const {
    if (stop && stop->load(std::memory_order_acquire))
        throw TransportError(Failure::cancelled, "operation cancelled");
    if (Clock::now() >= deadline)
        throw TransportError(Failure::timeout, "operation deadline exceeded");
}

void nonblocking(int fd) {
    const int flags = ::fcntl(fd, F_GETFL);
    if (flags < 0 || ::fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0)
        io_error("fcntl nonblocking");
}

Fd listen_local(unsigned& selected_port) {
    Fd socket = new_socket();
    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = 0;
    if (::bind(socket.get(), reinterpret_cast<sockaddr*>(&address), sizeof
address) < 0)
        io_error("bind");
    if (::listen(socket.get(), 16) < 0) io_error("listen");
    socklen_t length = sizeof address;
    if (::getsockname(socket.get(), reinterpret_cast<sockaddr*>(&address),
&length) < 0)
        io_error("getsockname");
    selected_port = ntohs(address.sin_port);
    return socket;
}

Fd connect_local(unsigned port, const Operation& operation) {
    if (port == 0 || port > 65535) throw std::invalid_argument("invalid port");
    operation.check();

```

```

    Fd socket = new_socket();
    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons(static_cast<std::uint16_t>(port));
    if (::connect(socket.get(), reinterpret_cast<sockaddr*>(&address), sizeof
address) < 0) {
        if (errno != EINPROGRESS) io_error("connect");
        ready(socket.get(), POLLOUT, operation);
        int error = 0;
        socklen_t size = sizeof error;
        if (::getsockopt(socket.get(), SOL_SOCKET, SO_ERROR, &error, &size) < 0)
            io_error("getsockopt SO_ERROR");
        if (error != 0) io_error("connect completion", error);
    }
    operation.check();
    return socket;
}

bool read_exact(int fd, std::span<std::byte> bytes, const Operation& operation)
{
    std::size_t offset = 0;
    while (offset < bytes.size()) {
        operation.check();
        const auto n = ::recv(fd, bytes.data() + offset,
std::min<std::size_t>(bytes.size() - offset, 1u << 20), 0);
        if (n > 0) { offset += static_cast<std::size_t>(n); continue; }
        if (n == 0) {
            if (offset == 0) return false;
            throw TransportError(Failure::truncated, "EOF inside transfer");
        }
        const int error = errno;
        if (error == EINTR) continue;
        if (error == EAGAIN || error == EWOULDBLOCK) {
            ready(fd, POLLIN, operation); continue;
        }
        io_error("recv", error);
    }
    operation.check();
    return true;
}

void write_exact(int fd, std::span<const std::byte> bytes, const Operation&
operation) {
    std::size_t offset = 0;
    while (offset < bytes.size()) {
        operation.check();
        const auto n = ::send(fd, bytes.data() + offset,
std::min<std::size_t>(bytes.size() - offset, 1u << 20), MSG_NOSIGNAL);
        if (n > 0) { offset += static_cast<std::size_t>(n); continue; }
        if (n == 0) throw TransportError(Failure::io, "send made no progress");
        const int error = errno;
        if (error == EINTR) continue;
        if (error == EAGAIN || error == EWOULDBLOCK) {
            ready(fd, POLLOUT, operation); continue;
        }
        io_error("send", error);
    }
    operation.check();
}

std::optional<std::string> receive_frame(int fd, const Operation& operation) {
    std::array<std::byte, 4> header{};
    if (!read_exact(fd, header, operation)) return std::nullopt;
    std::size_t length = 0;
    try { length = decode_length(header); }
    catch (const std::length_error&) {
        throw TransportError(Failure::oversize, "oversized incoming frame");
    }
}

```

```

        std::string payload(length, '\\0');
        auto bytes = std::as_writable_bytes(std::span(payload.data(),
payload.size()));
        if (!read_exact(fd, bytes, operation))
            throw TransportError(Failure::truncated, "EOF before declared body");
        return payload;
    }
    void send_frame(int fd, std::string_view payload, const Operation& operation) {
        operation.check();
        auto encoded = encode_frame(payload);
        write_exact(fd, encoded, operation);
    }
} // namespace harbor

```

A final progress check can report a deadline after a write has already submitted bytes. That is intentional: the program does not pretend it can retract an external effect. The client may have received some or all of a response even when the server records a late failure. Exactly-once transactional semantics would require another protocol. Time checks are cooperative observations subject to scheduling, not a proof of worst-case operating-system latency.

C.5 Server ownership and supervision

The server owns its listener, stop state, inbox, counters, workers, and acceptor. It cannot be copied or moved because running threads reference its stable address. Only the supervising thread calls joined shutdown or destroys the object. Stop can be requested through the documented thread-safe path; a failure stored by the acceptor is inspected only after that thread has joined.

```

#pragma once
#include "inbox.hpp"
#include "native.hpp"
#include <atomic>
#include <chrono>
#include <cstdint>
#include <exception>
#include <optional>
#include <string>
#include <string_view>
#include <thread>
#include <vector>

namespace harbor {
std::string process_request(std::string_view text);
struct Config {
    unsigned workers = 2;
    std::size_t queue_capacity = 4;
    std::chrono::milliseconds request_budget{1000};
    // Deterministic test seam only; the command-line program never enables it.
    std::optional<unsigned> fail_before_worker;
};
struct Stats {
    std::uint64_t accepted{}, admitted{}, rejected{}, completed{},
        application_errors{}, failures{}, timeouts{}, cancelled{};
};
class RelayServer {
    struct Work { Fd socket; Clock::time_point deadline; };
    struct Counters {
        std::atomic<std::uint64_t> accepted{}, admitted{}, rejected{},

```

```

completed{},
    application_errors{}, failures{}, timeouts{}, cancelled{};
    } stats_;
    Config config_;
    Fd listener_;
    unsigned port_ = 0;
    std::atomic<bool> stop_{false};
    BoundedInbox<Work> inbox_;
    std::exception_ptr accept_failure_; // read only after joining acceptor
    std::vector<std::jthread> workers_;
    std::jthread acceptor_;
    void accept_loop() noexcept;
    void worker_loop() noexcept;
public:
    explicit RelayServer(Config config = {});
    ~RelayServer();
    RelayServer(const RelayServer&) = delete;
    RelayServer& operator=(const RelayServer&) = delete;
    unsigned port() const noexcept { return port_; }
    void request_stop();
    // Only the supervising thread calls stop_and_join/destruction.
    void stop_and_join();
    void rethrow_failure() const;
    Stats snapshot() const noexcept; // coherent accounting only after join
};
std::string request(unsigned port, std::string_view text,
                    std::chrono::milliseconds budget = std::chrono::seconds(3));
} // namespace harbor

```

Construction validates configuration, creates the listener, starts workers, and finally starts the acceptor. A controlled test seam can fail before a selected worker launch. The constructor's failure path closes the inbox and joins workers before their referenced members are destroyed. The ordinary command-line program never enables the seam.

```

#include "relay.hpp"
#include "frame.hpp"
#include <cerrno>
#include <charconv>
#include <poll.h>
#include <locale>
#include <sstream>
#include <stdexcept>
#include <sys/socket.h>

namespace harbor {
std::string process_request(std::string_view text) {
    if (text.size() > max_payload) return "ERR size";
    std::istringstream input{std::string(text)};
    input.imbue(std::locale::classic());
    std::string token;
    if (!(input >> token) || token != "SUM") return "ERR command";
    long long total = 0;
    unsigned count = 0;
    while (input >> token) {
        if (++count > 64) return "ERR count";
        int value = 0;
        auto [end, error] = std::from_chars(token.data(), token.data() +
token.size(), value);
        if (error != std::errc{} || end != token.data() + token.size() ||
            value < -1000000 || value > 1000000) return "ERR value";
        total += value; // <= 64 * 1,000,000 in magnitude
    }
}

```

```

        if (count == 0) return "ERR empty";
        return "OK " + std::to_string(total);
    }
RelayServer::RelayServer(Config config)
    : config_(config), inbox_(config.queue_capacity) {
    if (config.workers == 0 || config.workers > 16 ||
        config.queue_capacity > 64 || config.request_budget.count() < 20 ||
        config.request_budget.count() > 5000)
        throw std::invalid_argument("invalid server configuration");
    listener_ = listen_local(port_);
    try {
        workers_.reserve(config.workers);
        for (unsigned i = 0; i < config.workers; ++i) {
            if (config.fail_before_worker == i)
                throw std::runtime_error("injected worker startup failure");
            workers_.emplace_back([this] { worker_loop(); });
        }
        acceptor_ = std::jthread([this] { accept_loop(); });
    } catch (...) {
        request_stop();
        workers_.clear(); // shared state remains alive; blocked pops wake
        throw;
    }
}

RelayServer::~RelayServer() { stop_and_join(); }
void RelayServer::request_stop() {
    stop_.store(true, std::memory_order_release);
    inbox_.close();
}

void RelayServer::stop_and_join() {
    request_stop();
    if (acceptor_.joinable()) acceptor_.join();
    workers_.clear();
    listener_.reset();
}

void RelayServer::rethrow_failure() const {
    if (accept_failure_) std::rethrow_exception(accept_failure_);
}

void RelayServer::accept_loop() noexcept {
    try {
        while (!stop_.load(std::memory_order_acquire)) {
            const int raw = ::accept4(listener_.get(), nullptr, nullptr,
                                       SOCK_CLOEXEC | SOCK_NONBLOCK);

            if (raw >= 0) {
                ++stats_.accepted;
                Work work{Fd(raw), Clock::now() + config_.request_budget};
                try {
                    if (inbox_.try_push(std::move(work))) ++stats_.admitted;
                    else ++stats_.rejected;
                } catch (...) { ++stats_.rejected; throw; }
                continue;
            }
            const int error = errno;
            if (error == EINTR) continue;
            if (error != EAGAIN && error != EWOULDBLOCK)
                throw TransportError(Failure::io, "accept failed", error);
            pollfd item{listener_.get(), POLLIN, 0};
            const int result = ::poll(&item, 1, 25);
            if (result < 0 && errno != EINTR)
                throw TransportError(Failure::io, "listener poll failed",
                                     errno);
            if (result > 0 && (item.revents & (POLLERR | POLLNVAL | POLLHUP)))
                throw TransportError(Failure::io, "listener became unusable");
        }
    } catch (...) {
        accept_failure_ = std::current_exception();
    }
}

```

```

        request_stop();
    }
    inbox_.close();
}
void RelayServer::worker_loop() noexcept {
    // Queue operations have no user callbacks; fatal synchronization failures
    // are not recoverable application errors. Per-connection errors are
    contained.
    while (auto work = inbox_.pop()) {
        try {
            Operation op{work->deadline, &stop_};
            auto text = receive_frame(work->socket.get(), op);
            if (!text) throw TransportError(Failure::truncated, "no request
frame");

            auto response = process_request(*text);
            send_frame(work->socket.get(), response, op);
            if (response.starts_with("ERR ")) ++stats_.application_errors;
            ++stats_.completed;
        } catch (const TransportError& error) {
            if (error.category == Failure::cancelled) ++stats_.cancelled;
            else {
                if (error.category == Failure::timeout) ++stats_.timeouts;
                ++stats_.failures;
            }
        } catch (...) { ++stats_.failures; }
    }
}
Stats RelayServer::snapshot() const noexcept {
    return {stats_.accepted.load(), stats_.admitted.load(),
stats_.rejected.load(),
        stats_.completed.load(), stats_.application_errors.load(),
        stats_.failures.load(), stats_.timeouts.load(),
stats_.cancelled.load()};
}
std::string request(unsigned port, std::string_view text,
                    std::chrono::milliseconds budget) {
    if (budget.count() < 1 || budget.count() > 30000)
        throw std::invalid_argument("invalid client budget");
    Operation op{Clock::now() + budget};
    auto socket = connect_local(port, op);
    send_frame(socket.get(), text, op);
    auto response = receive_frame(socket.get(), op);
    if (!response) throw TransportError(Failure::truncated, "no response
frame");
    return *response;
}
} // namespace harbor

```

Per-connection failures stay inside the worker loop. Fatal failures of basic synchronization operations are outside the application's recovery model; the `noexcept` worker boundary is not a claim that arbitrary mutex-runtime failure can be recovered. An acceptor failure records its exception, requests stop, and closes the inbox. The supervisor can then join and rethrow that preserved failure.

C.6 Command-line application

The default command runs a small self-test. `--serve SECONDS` runs a loopback listener for a bounded duration and then invokes the same independent stop mechanism used by the tests. This is a timed demonstration, not a full signal-

managed daemon. --request PORT TEXT submits one framed request with a bounded client budget.

```
#include "relay.hpp"
#include <charconv>
#include <iostream>
#include <stdexcept>
#include <string_view>
#include <thread>

namespace {
    unsigned number(std::string_view text, unsigned maximum) {
        unsigned value = 0;
        auto [end, error] = std::from_chars(text.data(), text.data() + text.size(),
        value);
        if (error != std::errc{} || end != text.data() + text.size() ||
            value == 0 || value > maximum) throw std::invalid_argument("invalid
numeric argument");
        return value;
    }
}

void print_stats(harbor::Stats s) {
    std::cout << "accepted=" << s.accepted << " admitted=" << s.admitted
        << " rejected=" << s.rejected << " completed=" << s.completed
        << " app_errors=" << s.application_errors << " failures=" <<
s.failures
        << " timeouts=" << s.timeouts << " cancelled=" << s.cancelled <<
'\n';
}

int main(int argc, char** argv) {
    try {
        if (argc == 1 || (argc == 2 && std::string_view(argv[1]) == "--self-
test")) {
            harbor::RelayServer server;
            if (harbor::request(server.port(), "SUM 7 -2 9") != "OK 14" ||
                harbor::request(server.port(), "SUM nope") != "ERR value")
                throw std::runtime_error("self-test response mismatch");
            server.stop_and_join(); server.rethrow_failure();
            print_stats(server.snapshot());
            std::cout << "PASS\n";
            return 0;
        }
        if (argc == 3 && std::string_view(argv[1]) == "--serve") {
            const auto seconds = number(argv[2], 3600);
            harbor::RelayServer server;
            std::cout << "127.0.0.1:" << server.port() << '\n' << std::flush;
            std::this_thread::sleep_for(std::chrono::seconds(seconds));
            server.stop_and_join(); server.rethrow_failure();
            print_stats(server.snapshot());
            return 0;
        }
        if (argc == 4 && std::string_view(argv[1]) == "--request") {
            std::cout << harbor::request(number(argv[2], 65535), argv[3]) <<
'\n';
            return 0;
        }
        std::cerr << "usage: harbor_relay [--self-test | --serve SECONDS | --
request PORT TEXT]\n";
        return 2;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 1;
    }
}
```

```
}  
}
```

Build and exercise the application from the package root:

```
cmake -S source -B build -DCMAKE_BUILD_TYPE=Debug  
cmake --build build --target harbor_relay harbor_relay_tests -j 2  
./build/bin/harbor_relay --self-test  
ctest --test-dir build --output-on-failure -R '^relay_'
```

For a two-terminal demonstration, start `./build/bin/harbor_relay --serve 60` in the first terminal. Read the actual selected port from its output. In the second terminal, substitute that numeric port in `./build/bin/harbor_relay --request PORT 'SUM 4 -1 8'`. The expected application response is `OK 11`. Do not literally enter the placeholder or expose the listener beyond loopback.

C.7 Complete regression implementation

The test executable selects named cases so each CTest result has a clear scope. Pure grammar and framing tests are distinct from native transfer tests and complete server tests. Socket pairs exercise controlled stream behavior without external networking. Loopback clients exercise the real listener, admission path, workers, processor, and response adapter.

```
#include "relay.hpp"  
#include "frame.hpp"  
#include "check.hpp"  
#include <array>  
#include <cerrno>  
#include <functional>  
#include <iostream>  
#include <map>  
#include <memory>  
#include <span>  
#include <string>  
#include <sys/socket.h>  
#include <thread>  
#include <vector>  
  
using namespace std::chrono_literals;  
namespace {  
std::pair<harbor::Fd, harbor::Fd> pair_sockets() {  
    int descriptors[2];  
    if (::socketpair(AF_UNIX, SOCK_STREAM | SOCK_CLOEXEC | SOCK_NONBLOCK,  
        0, descriptors) < 0) throw  
std::runtime_error("socketpair");  
    return {harbor::Fd(descriptors[0]), harbor::Fd(descriptors[1])};  
}  
harbor::Operation operation(std::chrono::milliseconds budget = 2s) {  
    return {harbor::Clock::now() + budget};  
}  
template<class F> void transport_throws(F action, harbor::Failure expected) {  
    bool matched = false;  
    try { action(); }  
    catch (const harbor::TransportError& e) { matched = e.category ==  
expected; }  
    CHECK(matched);  
}
```

```

template<class Exception, class F> void throws(F action) {
    bool caught = false;
    try { action(); } catch (const Exception&) { caught = true; }
    CHECK(caught);
}

void send_raw(int fd, std::span<const std::byte> bytes) {
    harbor::write_exact(fd, bytes, operation());
}

void grammar() {
    using harbor::process_request;
    CHECK(process_request("SUM 1 -2 3") == "OK 2");
    CHECK(process_request("SUM") == "ERR empty");
    CHECK(process_request("") == "ERR command");
    CHECK(process_request("sum 1") == "ERR command");
    for (auto value : {"1x", "+1", "1000001", "-1000001", "9999999999999999"})
        CHECK(process_request(std::string("SUM ") + value) == "ERR value");
    CHECK(process_request("SUM -1000000 1000000") == "OK 0");
    std::string many = "SUM";
    for (int i = 0; i < 64; ++i) many += " 1";
    CHECK(process_request(many) == "OK 64");
    CHECK(process_request(many + " 1") == "ERR count");
    CHECK(process_request(std::string(4097, 'x')) == "ERR size");
    CHECK(process_request(std::string("SUM 1\\0x", 7)) == "ERR value");
}

void codec_golden() {
    const std::array<std::byte, 7> golden{std::byte{0}, std::byte{0},
std::byte{0},
    std::byte{3}, std::byte{'A'}, std::byte{0}, std::byte{'B'}};
    const std::string text{"A\\0B", 3};
    CHECK(harbor::encode_frame(text) == std::vector<std::byte>(golden.begin(),
golden.end()));
    harbor::FrameDecoder decoder;
    CHECK(decoder.feed(golden) == golden.size());
    CHECK(decoder.finish()); CHECK(decoder.payload() == text);
}

void codec_splits() {
    for (const auto& text : {std::string{}, std::string("SUM 1 2"),
std::string("A\\0B", 3)}) {
        const auto wire = harbor::encode_frame(text);
        for (std::size_t split = 0; split <= wire.size(); ++split) {
            harbor::FrameDecoder decoder;
            CHECK(decoder.feed(std::span(wire).first(split)) == split);
            CHECK(decoder.feed(std::span(wire).subspan(split)) == wire.size() -
split);
            CHECK(decoder.finish()); CHECK(decoder.payload() == text);
        }
    }
    auto wire = harbor::encode_frame("one");
    auto second = harbor::encode_frame("two");
    wire.insert(wire.end(), second.begin(), second.end());
    harbor::FrameDecoder decoder;
    CHECK(decoder.feed(wire) == 7); CHECK(decoder.payload() == "one");
}

void codec_limits() {
    std::string payload(4096, 'x');
    auto wire = harbor::encode_frame(payload);
    harbor::FrameDecoder decoder;
    for (std::size_t i = 0; i < wire.size(); ++i)
        CHECK(decoder.feed(std::span(wire).subspan(i, 1)) == 1);
    CHECK(decoder.payload() == payload);
    throws<std::length_error>([&] { (void)harbor::encode_frame(std::string(4097,
'x')); });
    std::array<std::byte, 4> bad{std::byte{0}, std::byte{0}, std::byte{16},
std::byte{1}};
    harbor::FrameDecoder invalid;
    throws<std::length_error>([&] { invalid.feed(bad); });
}

```

```

        throws<std::logic_error>([&] { invalid.feed({}); });
    }
    void codec_eof() {
        auto wire = harbor::encode_frame("abc");
        for (std::size_t n = 0; n < wire.size(); ++n) {
            harbor::FrameDecoder d;
            d.feed(std::span(wire).first(n));
            if (n == 0) CHECK(!d.finish());
            else throws<std::runtime_error>([&] { d.finish(); });
        }
    }
    void native_roundtrip() {
        auto [a, b] = pair_sockets();
        const std::string text("SUM 7\0 9", 8);
        harbor::send_frame(a.get(), text, operation());
        auto got = harbor::receive_frame(b.get(), operation());
        CHECK(got && *got == text);
    }
    void native_empty() {
        auto [a, b] = pair_sockets();
        harbor::send_frame(a.get(), "", operation());
        auto got = harbor::receive_frame(b.get(), operation());
        CHECK(got && got->empty());
        a.reset();
        CHECK(!harbor::receive_frame(b.get(), operation()));
    }
    void native_truncated_header() {
        auto [a, b] = pair_sockets();
        std::array<std::byte, 2> part{};
        send_raw(a.get(), part); a.reset();
        transport_throws([&] { harbor::receive_frame(b.get(), operation()); },
            harbor::Failure::truncated);
    }
    void native_truncated_body() {
        auto [a, b] = pair_sockets();
        auto wire = harbor::encode_frame("hello");
        send_raw(a.get(), std::span(wire).first(6)); a.reset();
        transport_throws([&] { harbor::receive_frame(b.get(), operation()); },
            harbor::Failure::truncated);
    }
    void native_missing_body() {
        auto [a, b] = pair_sockets();
        auto wire = harbor::encode_frame("hello");
        send_raw(a.get(), std::span(wire).first(4)); a.reset();
        transport_throws([&] { harbor::receive_frame(b.get(), operation()); },
            harbor::Failure::truncated);
    }
    void native_oversize() {
        auto [a, b] = pair_sockets();
        std::array<std::byte, 4> bad{std::byte{255}, std::byte{255}, std::byte{255},
            std::byte{255}};
        send_raw(a.get(), bad);
        transport_throws([&] { harbor::receive_frame(b.get(), operation()); },
            harbor::Failure::oversize);
    }
    void native_deadline() {
        auto [a, b] = pair_sockets();
        transport_throws([&] { harbor::receive_frame(b.get(), operation(60ms)); },
            harbor::Failure::timeout);
        // A previously expired absolute deadline cannot be reset by available data.
        harbor::send_frame(a.get(), "ready", operation());
        harbor::Operation expired{harbor::Clock::now() - 1ms};
        transport_throws([&] { harbor::receive_frame(b.get(), expired); },
            harbor::Failure::timeout);
    }
    void native_cancel() {

```

```

        auto [a, b] = pair_sockets();
        std::atomic<bool> stop{true};
        auto op = operation(); op.stop = &stop;
        transport_throws([&] { harbor::receive_frame(b.get(), op); },
harbor::Failure::cancelled);
        transport_throws([&] { harbor::send_frame(a.get(), "x", op); },
harbor::Failure::cancelled);
    }
    void native_fragmented() {
        auto [a, b] = pair_sockets();
        auto wire = harbor::encode_frame("fragmented data");
        std::exception_ptr error;
        std::jthread writer([&] {
            try { for (std::size_t i = 0; i < wire.size(); ++i)
                send_raw(a.get(), std::span(wire).subspan(i, 1)); }
            catch (...) { error = std::current_exception(); }
        });
        auto got = harbor::receive_frame(b.get(), operation());
        writer.join(); if (error) std::rethrow_exception(error);
        CHECK(got && *got == "fragmented data");
    }
    void inbox_state() {
        harbor::BoundedInbox<std::unique_ptr<int>> q(1);
        CHECK(q.try_push(std::make_unique<int>(7)));
        CHECK(!q.try_push(std::make_unique<int>(8)));
        q.close(); q.close();
        CHECK(!q.try_push(std::make_unique<int>(9)));
        auto item = q.pop(); CHECK(item && **item == 7); CHECK(!q.pop());
        throws<std::invalid_argument>([] { harbor::BoundedInbox<int> bad(0); });
    }
    void accounting(harbor::Stats s) {
        CHECK(s.accepted == s.admitted + s.rejected);
        CHECK(s.admitted == s.completed + s.failures + s.cancelled);
        CHECK(s.timeouts <= s.failures); CHECK(s.application_errors <= s.completed);
    }
    void wait_completed(harbor::RelayServer& server, std::uint64_t expected) {
        const auto until = harbor::Clock::now() + 5s;
        while (server.snapshot().completed < expected) {
            const auto s = server.snapshot();
            if (s.failures || s.cancelled)
                throw std::runtime_error("unexpected server failure before
quiescence");
            if (harbor::Clock::now() >= until)
                throw std::runtime_error("server completion observation deadline");
            std::this_thread::yield();
        }
    }
    void server_smoke() {
        harbor::RelayServer server;
        CHECK(harbor::request(server.port(), "SUM 8 -3") == "OK 5");
        CHECK(harbor::request(server.port(), "SUM bad") == "ERR value");
        wait_completed(server, 2); // receipt can precede worker bookkeeping
        server.stop_and_join(); server.rethrow_failure();
        auto s = server.snapshot(); accounting(s);
        CHECK(s.completed == 2); CHECK(s.application_errors == 1);
    }
    void server_parallel() {
        harbor::Config config; config.workers = 4; config.queue_capacity = 16;
        config.request_budget = 5s;
        harbor::RelayServer server(config);
        std::array<std::string, 8> results;
        std::array<std::exception_ptr, 8> errors;
        std::vector<std::jthread> clients;
        for (std::size_t i = 0; i < results.size(); ++i) clients.emplace_back([&, i]
{
            try { results[i] = harbor::request(server.port(), "SUM " +

```

```

std::to_string(i) + " 10"); }
    catch (...) { errors[i] = std::current_exception(); }
});
clients.clear();
for (std::size_t i = 0; i < results.size(); ++i) {
    if (errors[i]) std::rethrow_exception(errors[i]);
    CHECK(results[i] == "OK " + std::to_string(i + 10));
}
wait_completed(server, 8);
server.stop_and_join(); server.rethrow_failure();
accounting(server.snapshot());
CHECK(server.snapshot().completed == 8);
}

void wait_accepted(harbor::RelayServer& server) {
    const auto until = harbor::Clock::now() + 3s;
    while (server.snapshot().accepted == 0) {
        if (harbor::Clock::now() >= until) throw std::runtime_error("accept
observation deadline");
        std::this_thread::yield();
    }
}

void server_idle_stop() {
    harbor::RelayServer server;
    auto socket = harbor::connect_local(server.port(), operation());
    wait_accepted(server);
    server.stop_and_join(); server.rethrow_failure();
    accounting(server.snapshot());
    CHECK(server.snapshot().completed == 0);
}

void server_no_clients() {
    harbor::RelayServer server;
    server.stop_and_join(); server.stop_and_join(); server.rethrow_failure();
    accounting(server.snapshot()); CHECK(server.snapshot().accepted == 0);
}

void server_timeout() {
    harbor::Config config; config.request_budget = 60ms;
    harbor::RelayServer server(config);
    auto socket = harbor::connect_local(server.port(), operation());
    // Peer closure proves its owned operation finished; no millisecond speed
assertion.
    auto response = harbor::receive_frame(socket.get(), operation(3s));
    CHECK(!response);
    server.stop_and_join(); server.rethrow_failure();
    accounting(server.snapshot());
    CHECK(server.snapshot().timeouts == 1);
}

void server_response_stop_race() {
    harbor::RelayServer server;
    CHECK(harbor::request(server.port(), "SUM 3") == "OK 3");
    // Do not order server bookkeeping by a client-side receive. A final
    // stop/deadline check may classify cancellation after bytes were sent.
    server.stop_and_join(); server.rethrow_failure();
    const auto s = server.snapshot(); accounting(s);
    CHECK(s.accepted == 1); CHECK(s.admitted == 1);
    CHECK(s.completed + s.failures + s.cancelled == 1);
}

void server_start_failure() {
    for (unsigned fail = 0; fail < 2; ++fail) {
        harbor::Config config; config.fail_before_worker = fail;
        throws<std::runtime_error>([&] { harbor::RelayServer server(config); });
    }
    harbor::RelayServer healthy;
    CHECK(harbor::request(healthy.port(), "SUM 1") == "OK 1");
    healthy.stop_and_join(); healthy.rethrow_failure();
}

void invalid_config() {

```

```

    for (unsigned workers : {0u, 17u}) {
        harbor::Config c; c.workers = workers;
        throws<std::invalid_argument>([&] { harbor::RelayServer s(c); });
    }
    for (auto capacity : {std::size_t{0}, std::size_t{65}}) {
        harbor::Config c; c.queue_capacity = capacity;
        throws<std::invalid_argument>([&] { harbor::RelayServer s(c); });
    }
    throws<std::invalid_argument>([] { harbor::connect_local(0,
operation()); });
}
}
int main(int argc, char** argv) {
    const std::map<std::string, std::function<void()>> cases{
        {"grammar", grammar}, {"codec_golden", codec_golden}, {"codec_splits",
codec_splits},
        {"codec_limits", codec_limits}, {"codec_eof", codec_eof},
        {"native_roundtrip", native_roundtrip}, {"native_empty", native_empty},
        {"native_truncated_header", native_truncated_header},
        {"native_truncated_body", native_truncated_body},
        {"native_missing_body", native_missing_body},
        {"native_oversize", native_oversize}, {"native_deadline",
native_deadline},
        {"native_cancel", native_cancel}, {"native_fragmented",
native_fragmented},
        {"inbox_state", inbox_state}, {"server_smoke", server_smoke},
        {"server_parallel", server_parallel}, {"server_idle_stop",
server_idle_stop},
        {"server_no_clients", server_no_clients}, {"server_timeout",
server_timeout},
        {"server_start_failure", server_start_failure},
        {"server_response_stop_race", server_response_stop_race},
        {"invalid_config", invalid_config}
    };
    try {
        if (argc != 2 || !cases.contains(argv[1])) throw
std::invalid_argument("unknown test case");
        cases.at(argv[1])();
        std::cout << "PASS " << argv[1] << '\n';
        return 0;
    } catch (const std::exception& error) {
        std::cerr << "FAIL: " << error.what() << '\n'; return 1;
    }
}

```

The deadline tests check an idle operation and a previously expired absolute deadline even when data is available. They do not assert a particular scheduler-level response time. The parallel-client case uses enough configured queue capacity for its finite batch and verifies response identity after joining clients. The smoke and parallel cases observe server completion before requesting stop: a client can receive bytes before the worker finishes its counter update or final cancellation check. A separate immediate-stop case accepts the documented terminal categories and verifies accounting rather than falsely ordering server bookkeeping through the client receive. The inbox case tests deterministic rejection directly instead of requiring a particular incidental rejection count from a network burst.

C.8 Acceptance review

The implemented contract includes bounded frames, whole-token arithmetic validation, unique descriptor ownership, selected loopback ports, nonblocking transfer progress, shared absolute request deadlines, bounded admission, and joined stop. Tests cover malformed payloads, fragmented and truncated frames, maximum and oversized lengths, empty frames, cancellation, idle clients, no-client shutdown, parallel clients, invalid configuration, and partial worker startup failure.

The remaining production questions are not answered by these tests. Decide how peers are authenticated, how transport is encrypted, how overload is communicated, how the process is supervised, how logs and metrics are retained, how configuration changes are rolled back, and what service-level deadline policy applies across retries. A completed local test suite is the beginning of an operational review, not a substitute for it.

Appendix D — Working Glossary and Final Engineering Review

The definitions below are reminders for applying the chapters, not a replacement for the referenced language and platform contracts. The important habit is to connect each term to an observable state or a specific proof obligation in the implementation.

Address space. The process's virtual-address context. Similar pointer values in different processes do not automatically refer to the same shared object. Mapping and ownership determine what the bytes mean.

Alignment. A requirement on where an object can be placed in storage. Requested alignment can affect object size and layout. A sixty-four-byte alignment request is not a universal statement about every processor's cache-line geometry.

Atomicity. Indivisibility of the specified operation on the relevant atomic object. It does not make several fields one transaction or retain the lifetime of a pointed-to object.

Backpressure. A policy that limits admission or slows producers when downstream capacity is unavailable. Blocking, rejection, and timed admission have different caller-visible contracts.

Borrow. Access to an object or resource without owning its lifetime. A span, string view, raw pointer, or native descriptor argument can be a borrow. Validity depends on the owner's lifetime and mutation rules.

Cancellation. A request or protocol that permits unfinished work to stop. It is not necessarily immediate, does not automatically wake every blocking API, and does not roll back external effects already performed.

Close-on-exec. A descriptor policy controlling whether the descriptor survives a successful process-image replacement. Acquisition-time flags avoid a race between resource creation and later flag assignment.

Condition predicate. The protected state expression that determines whether a waiting participant may proceed. A notification is a wakeup mechanism, not the state or a reserved task.

Critical section. A coherent operation on protected shared state. Its correct extent follows the invariant, not simply the desire to minimize the number of locked lines.

Deadline. An absolute end time for the defined operation. Recreating a timeout after every partial transfer changes the contract from a total budget to a different waiting policy.

Descriptor. A process-local integer reference to an open resource. Copying the integer is not ownership duplication. Duplicating through the native API can create another descriptor referring to the same open file description.

Durability. A promise about persistence under a stated failure model. A successful write, close, unmap, or local in-memory commit does not automatically establish every desired crash-recovery property.

EOF. A termination observation whose meaning depends on the interface and protocol phase. A stream EOF inside a frame is truncation; a zero-length UDP datagram is a legitimate datagram, not stream closure.

Framing. The rule that identifies message boundaries within transported bytes. A length prefix, delimiter protocol, or fixed-size record must define bounds, invalid input, and truncated termination.

Happens-before. A language-level ordering relationship used to reason about visibility and the absence of conflicting unsynchronized accesses. A wall-clock guess or sleep is not a substitute for the required relationship.

Idempotency. A property or protocol under which repeating an operation has an appropriately controlled effect. It matters when a transport failure leaves the caller uncertain whether remote work already occurred.

Invariant. A condition that must hold at the public observation boundaries of a component. Locks protect access, while validation and mutation order preserve logical validity.

Join. Completion synchronization with a represented thread. It also establishes the point after which objects used only by that thread can be reclaimed under the surrounding ownership design.

Mapping. A virtual-memory association with an underlying object or anonymous storage. Mapped bytes still require valid object interpretation, extent checks, and a stability policy for external mutation.

Open file description. Kernel-managed open state that can be shared by duplicated descriptors, including an offset where relevant. Per-descriptor flags are a different layer.

Partial progress. A successful operation that completed only part of the requested transfer. The accepted prefix must be retained in the state machine; replaying from the start can duplicate effects.

Publication. A synchronization protocol that makes prepared data available to another participant. Reusing the storage often needs a reverse ownership handoff after the consumer finishes.

Quiescence. A state in which the relevant participants have stopped accessing an object. Joined shutdown supplies a useful quiescent boundary for resource cleanup and final accounting.

RAII. Associating resource ownership with object lifetime. Correct acquisition, move semantics, release behavior, and declaration order are essential parts of the pattern.

Reaping. Collecting a terminated child process's result through the platform's process-wait mechanism. A numeric PID stored in a C++ variable does not perform this responsibility automatically.

Undefined behavior. A program execution outside the language's prescribed requirements, for which the standard imposes no normal semantic guarantee. Passing tests does not legitimize an invalid lifetime, overflow, or data race.

A final review before expanding the system

Choose one chapter program and explain every acquisition and release without referring to “automatic cleanup” as a black box. Name the owner, the resource's valid state, the possible failure points, the cleanup order, and any borrowed views. Then repeat the exercise for a partially completed construction. The second path often reveals assumptions hidden by normal execution.

Choose one concurrent example and draw all waits. For each wait, identify the predicate, the participant that can change it, and the shutdown path when that participant fails to start or exits early. An unreachable predicate is a liveness defect even when the program contains no data race.

Choose one network operation and identify the accepted-byte prefix at every failure point. Explain whether a retry is safe, uncertain, or requires an operation identifier. Distinguish local resource cleanup from remote effect recovery. A transport adapter cannot invent business semantics that the protocol never defined.

Finally, review the verification report as an engineer rather than a scorecard. Compiler diversity, sanitizer runs, deterministic tests, representative document review, and reproducible source assembly provide different kinds of evidence. None proves every input, schedule, platform, or production environment correct. The strongest conclusion is the specific one supported by the implementation, its reasoning, and the recorded checks.

Primary reference directory

This directory identifies primary language, toolchain, and operating-system documentation. The numbered curriculum and compact examples are retained from the earlier Book IV supplied in this conversation. The expanded explanations, diagnostic scenarios, labs, and extended server are new instructional material for this revision. References distinguish platform contracts from design choices in the fictional HarborWorks examples. No timing or hardware values are presented as universal constants.

The language baseline is C++20. The working-draft links are living documents and can contain later features; a link is not a claim that all displayed facilities belong to C++20. Native examples are Linux-specific where marked. Linux and POSIX do not always agree, notably on interrupted close. Consult the installed manual and platform adapter rather than applying a universal retry macro. Selected native and concurrency documentation was consulted on 19 September 2026; this is not a vendor endorsement or formal certification.

ACCEPT — Linux `accept(2)`

[Linux `accept\(2\)`](#). Accepted-descriptor flags and transient errors.

ASAN — Clang AddressSanitizer

[Clang AddressSanitizer](#). Instrumented memory-error detection, configuration, and limitations.

ATOMICS — Atomic operations and ordering

[Atomic operations and ordering](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

BIT — Bit manipulation utilities

[Bit manipulation utilities](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CLANG — Clang C++ language support

[Clang C++ language support](#). Compiler feature status; check the standard library separately.

CLASS — Classes and special member functions

[Classes and special member functions](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CONDVAR — Condition variables

[Condition variables](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CONNECT — Linux connect(2)

[Linux connect\(2\)](#). Nonblocking connection completion and SO_ERROR inspection.

FORK — Linux fork(2)

[Linux fork\(2\)](#). Process creation, shared open descriptions, and the post-fork restriction in a multithreaded process.

FSYNC — Linux fsync(2)

[Linux fsync\(2\)](#). Durability requests for file data/metadata and the separate directory synchronization requirement.

FUNC — Functions and declarations

[Functions and declarations](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

FUTURE — Futures and promises

[Futures and promises](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

GCC — GCC C++ language support

[GCC C++ language support](#). Compiler support varies by version, language mode, and library.

GETADDRINFO — getaddrinfo(3)

[getaddrinfo\(3\)](#). Address resolution, result ownership, and protocol selection.

GUIDELINES — C++ Core Guidelines

[C++ Core Guidelines](#). Design guidance on ownership, interfaces, safety, and concurrency; recommendations are not language rules.

JTHREAD — C++ working draft — joining threads

[C++ working draft — joining threads](#). jthread lifetime and cooperative stop semantics; consult the C++20 baseline.

LANG — C++ working draft

[C++ working draft](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

LAYOUT — C++ working draft — objects

[C++ working draft — objects](#). Object identity, representation, and storage relationships.

LIFETIME — Object lifetime

[Object lifetime](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

MEMORY — Memory management utilities

[Memory management utilities](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

MEMORYMODEL — Data races and happens-before

[Data races and happens-before](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

MMAP — Linux mmap(2)

[Linux mmap\(2\)](#). File mappings, access modes, and mapping lifetime.

NEW — C++ working draft — new expressions

[C++ working draft — new expressions](#). Allocation, initialization, and object creation are distinct operations.

NUMERIC — Fundamental types and numeric limits

[Fundamental types and numeric limits](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

OPEN — Linux open(2)

[Linux open\(2\)](#). Open file descriptions, creation flags, descriptor inheritance, and close-on-exec.

PIPE — pipe(2)

[pipe\(2\)](#). Pipe creation, descriptor ownership, and EOF behavior.

POLL — Linux poll(2)

[Linux poll\(2\)](#). Readiness events, timeout semantics, and interrupted waits.

POSIX2024CLOSE — The Open Group — close, Issue 8

[The Open Group — close, Issue 8](#). POSIX.1-2024 close/EINTR semantics differ from the Linux behavior used by these native wrappers.

POSIXCLOSE — Linux close(2)

[Linux close\(2\)](#). Descriptor lifetime and close errors; examples follow Linux close behavior.

POSIXIO — Linux read(2) and associated I/O interfaces

[Linux read\(2\) and associated I/O interfaces](#). Partial transfers, interruptions, EOF, and error reporting. See the companion write(2) manual for writes.

PROC — Linux /proc filesystem

[Linux /proc filesystem](#). Linux process and kernel observations; these are not portable C++ interfaces.

SEMAPHORE — sem_init(3)

[sem_init\(3\)](#). Process-shared semaphore initialization and synchronization.

SHUTDOWN — Linux shutdown(2)

[Linux shutdown\(2\)](#). Directional shutdown of sockets, distinct from descriptor ownership release.

SIGNALSAFE — Linux signal-safety(7)

[Linux signal-safety\(7\)](#). Async-signal-safe operations and restrictions; signals are not ordinary C++ callbacks.

SOCKETRECV — Linux recv(2)

[Linux recv\(2\)](#). Receive behavior, stream EOF, datagrams, and MSG_TRUNC.

SOCKETS — socket(2)

[socket\(2\)](#). Socket endpoints; associated send, recv, bind, accept, and connect manuals define operation details.

SOCKETSEND — Linux send(2)

[Linux send\(2\)](#). Stream output, errors, and MSG_NOSIGNAL.

SPAN — Non-owning span

[Non-owning span](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

SPAWN — `posix_spawn(3)`

[`posix\_spawn\(3\)`](#). Spawn attributes, file actions, and direct argument vectors.

STOPTOKEN — C++ working draft — stop tokens

[`C++ working draft — stop tokens`](#). Cooperative cancellation requests and callbacks, not forced interruption of every blocking call.

THREADS — Threads and concurrency

[`Threads and concurrency`](#). Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

TSAN — Clang ThreadSanitizer

[`Clang ThreadSanitizer`](#). Data-race instrumentation and platform limits; run separately from AddressSanitizer.

UBSAN — Clang UndefinedBehaviorSanitizer

[`Clang UndefinedBehaviorSanitizer`](#). Selected runtime undefined-behavior checks; absence of a diagnostic is not a correctness proof.

WAIT — `wait(2)`

[`wait\(2\)`](#). Child status interpretation and process reaping.

WRITE — Linux `write(2)`

[`Linux write\(2\)`](#). Native write return values, partial progress, and delayed error reporting.